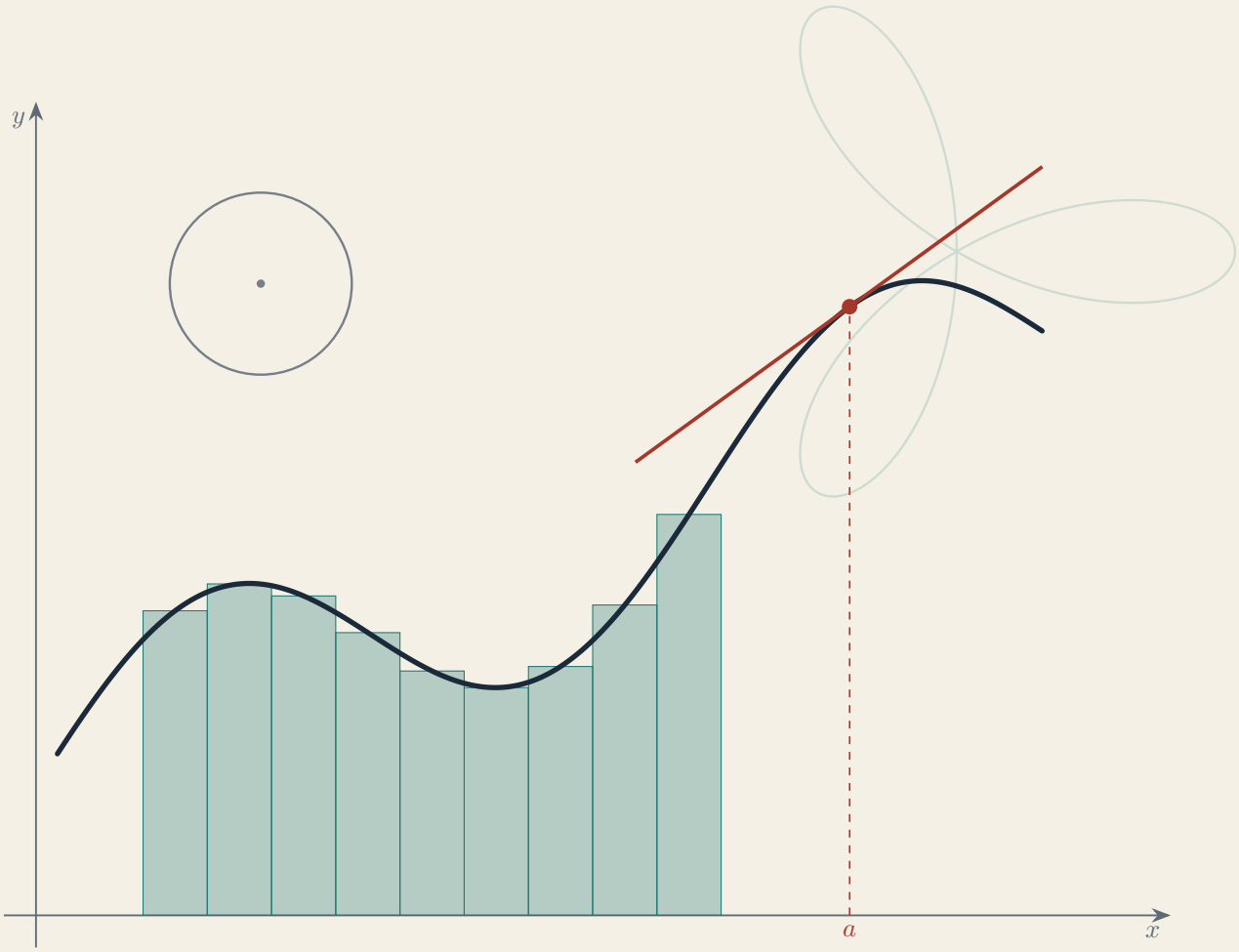
Analytic Geometry and Calculus

with Python



Fourth Edition

Stephen Edward Moore

Department of Mathematics
University of Cape Coast, Ghana

Analytic Geometry and Calculus with Python
Fourth Edition, 2026

Stephen Edward Moore
Department of Mathematics, University of Cape Coast, Ghana
Website: https://moorestephen.info/

First edition 2019 Second edition 2021 Third edition 2024 Fourth edition 2026

Copyright © 2019, 2021, 2024, 2026 Stephen Edward Moore.

Licence for the book. This book is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International Licence (CC BY-NC-ND 4.0). You are free to *share* it (copy and redistribute it in any medium or format), provided that you give appropriate credit and provide a link to the licence. You may not use the material for commercial purposes, and if you remix, transform or build upon the material, you may not distribute the modified material. You may not suggest that the author endorses you or your use. Permission for other uses (for example translations or adapted editions) may be requested from the author. Full licence text: https://creativecommons.org/licenses/by-nc-nd/4.0/.

Licence for the Python code. The Python programs printed in this book and supplied in the accompanying repository are licensed under the MIT Licence. They may be used, copied, modified and distributed freely, provided the copyright notice and the licence text are kept.

Code, notebooks and the latest version of this book:
https://github.com/moorekwesi/analytic-geometry-calculus-with-python

Suggested citation. S. E. Moore, *Analytic Geometry and Calculus with Python*, 4th ed., University of Cape Coast, 2026. Licensed under CC BY-NC-ND 4.0.

Unless a source is given, numerical data in the "In Context" boxes (fares, trip times, tariffs and similar figures) are illustrative and invented for teaching. Typeset with LaTeX; computations and figures produced with Python, NumPy, SymPy, Matplotlib and SciPy.

Dedication

To my students, past, present and future,
whose questions taught me how to teach,

and to my family,
for their patience and constant support.

Preface to the Fourth Edition

These notes began in 2019 as lecture notes for a first-year university course in analytic geometry and calculus. Revised editions followed in 2021 and 2024. This fourth edition has a new title, *Analytic Geometry and Calculus with Python*, because it has a second aim. Students should finish the course understanding the mathematics *and* being able to use a modern programming language to explore it, check it and apply it. The book is written for first-year students in mathematics, science, engineering, economics and education at any university, college or training institution.

Why Python?

Calculus is the mathematics of change, and computers are very good at handling change one small step at a time. A student who has computed a limit by hand and then watched a Python loop creep towards that limit understands the idea more deeply. A student who has found a maximum with the derivative and then confirmed it on a plotted graph has seen the result twice. Python is free, runs on a laptop or (through Google Colab) in a phone browser, and is now the language of data science, engineering and research worldwide. Employers increasingly expect graduates to be able to programme, and a first calculus course is a good place to start.

What is new in this edition

- **Chapter 0, a gentle Python primer.** No programming experience is assumed.
- **Python in every section.** Each topic has worked examples written in Python using `numpy`, `sympy`, `matplotlib` and `scipy`. Every listing in the book was run and its *actual* output printed beneath it. All the code is also supplied as Jupyter notebooks.
- **Examples from everyday life.** The examples use ride-hailing trip times and fares, telecom masts, radio telescopes, lakes and bridges, census figures, savings accounts, farms, rainfall and exam marks. Many are set in Ghana, where these notes were written, so that students meet the ideas in places they know.
- **Highlighted key concepts.** Colour-coded boxes mark key concepts, definitions, theorems, common mistakes and quick self-checks.
- **Historical notes.** Each chapter tells part of the story of how its ideas developed. This includes the large African contribution, from the Ishango bone and the Rhind papyrus to Euclid, Hypatia and Ibn al-Haytham in Alexandria and Cairo.
- **A new structure.** Differentiation and integration each now have two chapters: one on the ideas and techniques (Chapters 4 and 6) and one on their applications (Chapters 5 and 7). A new chapter on parametric equations and polar coordinates (Chapter 8) prepares the

way for conic sections (Chapter 9). The Chapter Map shows how the chapters depend on each other.

- **New topics.** These include the number e, the bisection and Newton methods, l'Hôpital's rule, curve fitting, integration by parts, numerical integration, volumes of revolution, arc length and polar area.
- **More exercises.** Each section ends with short self-check questions. Each chapter ends with a graded set of exercises in three kinds: skills, applications ( ) and programming ( ). Answers to selected exercises are in Appendix B.
- **Corrections.** Several misprints in the worked examples of the third edition have been corrected, and every corrected answer has been checked with SymPy.

Stephen Edward Moore
Department of Mathematics, University of Cape Coast
2026

Acknowledgements

Many people have contributed to these notes since the first edition appeared in 2019.

I thank my teaching assistants, Daniel Kakou and Nabahel Rex-Oneal, who worked tirelessly on the first draft of these notes. I thank the assistant lecturer, Mrs Agnes Adom-Konadu, who inspired that first draft.

I am grateful to my colleagues in the Department of Mathematics at the University of Cape Coast for their encouragement and for many helpful discussions about teaching first-year mathematics.

Above all, I thank the many students who have used earlier editions. Their questions, corrections and puzzled faces have improved every edition. Any errors that remain are my own, and I would be grateful to hear of them.

The book was typeset with LaTeX. The computations and figures were produced with the free, open-source Python libraries NumPy, SymPy, Matplotlib and SciPy, and I thank the many volunteers who develop and maintain them.

Stephen Edward Moore

How to Use These Notes

The boxes

These notes use coloured boxes so you can see at a glance what kind of material you are reading.

Key Concept

The most important ideas, formulas and procedures. If you revise only one thing before an examination, revise these boxes.

Definition. (Name of the idea)

A precise mathematical definition. Definitions and theorems are numbered so that they can be referred to later.

In Context

An application of the mathematics to a real situation. Unless a source is given, the numbers in these boxes are realistic but *illustrative*. For example, the Bolt trip-time models are invented for teaching and are not real company data.

Historical Note

How the ideas were discovered and who discovered them. Mathematics was made by people, in many places, over thousands of years.

Watch Out

Mistakes that students commonly make, and how to avoid them.

Check Your Understanding

Short questions to try *before* reading on. They should take no more than a few minutes.

Python Tip

Small pieces of Python know-how that make the code easier to write or read.

The Python listings

Python code appears in blue-framed boxes, with the file name in the title bar. The box underneath, headed **Output**, shows exactly what Python printed when the code was run. To learn the most, do not only read the code. *Type it in yourself, run it, then change something and run it again.*

You can run the code in either of two ways.

1. **Google Colab** (recommended if you do not have a laptop). Go to `https://colab.research.google.com`, sign in with a Google account, and choose *File* → *Upload notebook* to open the chapter notebook from the `notebooks` folder. Colab runs in a phone or laptop browser and already has every library we use.
2. **Anaconda** (recommended if you have a laptop). Download the free Anaconda Distribution from `https://www.anaconda.com`. It installs Python, Jupyter, Spyder, NumPy, SymPy, Matplotlib and SciPy in one step, and it works offline once installed.

The exercises

Exercises at the end of each chapter are marked as follows.

Unmarked: pen-and-paper exercises, examination style.

Applied problems: the mathematics used in a real situation.

Programming exercises: write or change a Python program.

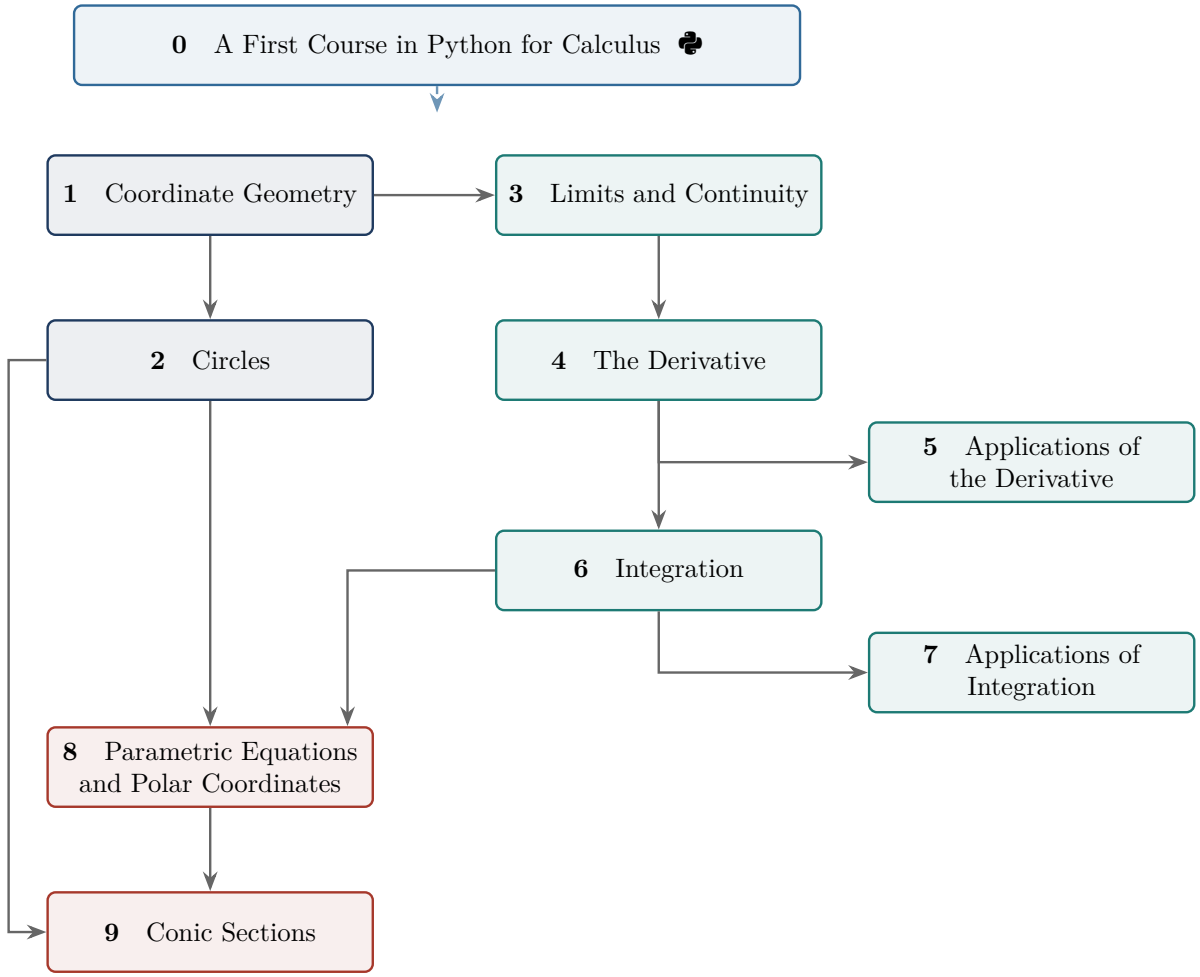
Answers to selected exercises are given in Appendix B. Use Python to check your own answers to the rest.

A suggested weekly rhythm

1. Read the section before the lecture and attempt the *Check Your Understanding* box.
2. After the lecture, run every Python listing for that section and change at least one number in each.
3. Do the pen-and-paper exercises *by hand first*. Then check them with Python.
4. Attempt at least one exercise every week. Programming is learned by doing, not by reading.

Chapter Map

The diagram shows how the chapters depend on each other. An arrow from one chapter to another means that the first should be studied before the second. Chapter 0 (Python) supports every chapter, so it can be read at the start or consulted as needed.



The chapters at a glance

	Chapter	Main ideas
0	Python for Calculus	variables, loops, functions; NumPy, Matplotlib, SymPy
1	Coordinate Geometry	distance, midpoint, gradient, lines, angles, section formula, point–line distance
2	Circles	equations of circles, tangents, line–circle and circle–circle intersection
3	Limits and Continuity	limits, one-sided limits, the number e, squeeze theorem, continuity, IVT
4	The Derivative	definition, rules, chain rule, implicit and logarithmic differentiation
5	Applications of the Derivative	tangents, related rates, extrema, curve sketching, optimisation, Newton's method
6	Integration	antiderivatives, substitution, Riemann sums, the FTC, integration by parts, numerical integration
7	Applications of Integration	areas, accumulation, average value, volumes, arc length, probability
8	Parametric and Polar	parametric curves and their calculus, polar coordinates, polar curves and areas
9	Conic Sections	parabola, ellipse, hyperbola, classification, eccentricity

Suggested routes through the book

- **One-semester course in analytic geometry and differential calculus:** Chapters 0–5, with Chapter 9 if time allows.
- **One-semester course in integral calculus and further geometry:** Chapters 6–9, revising Chapter 4 as needed.
- **Two-semester course:** all chapters in order.
- **Self-study of Python for mathematics:** Chapter 0, then the Python listings and the exercises of any chapter.

Contents

Preface to the Fourth Edition v

Acknowledgements vii

How to Use These Notes viii

Chapter Map x

0 A First Course in Python for Calculus 1

0.1 Getting started . . . 1
0.2 Python as a calculator . . . 2
0.3 Lists, loops and functions . . . 3
0.4 NumPy: computing with arrays . . . 4
0.5 Matplotlib: drawing graphs . . . 5
0.6 SymPy: exact mathematics . . . 6
Exercises . . . 7

1 Coordinate Geometry 9

1.1 The distance between two points . . . 10
1.2 The midpoint of a line segment . . . 12
1.3 The gradient of a line . . . 14
1.4 The equation of a line . . . 16
1.5 The angle between two lines . . . 18
1.6 Division of a line segment in a given ratio . . . 20
1.6.1 Internal division . . . 20
1.6.2 External division . . . 20
1.7 The distance from a point to a line . . . 22
1.8 Bisectors of the angles between two lines . . . 24
1.9 Exercises . . . 25

2 Circles 28

2.1 The equation of a circle . . . 28
2.1.1 Parametric form and plotting . . . 30

2.2 Special circles . . . 31
2.2.1 A circle through the origin . . . 31
2.2.2 The circle on a given diameter . . . 31
2.2.3 The circle through three points . . . 32
2.3 Tangents and normals to a circle . . . 34
2.4 Intersection of a line and a circle . . . 35
2.5 Two circles . . . 37
2.5.1 The common chord . . . 38
2.6 Exercises . . . 41

3 Limits and Continuity **43**
3.1 The idea of a limit . . . 44
3.2 Evaluating limits . . . 45
3.2.1 The laws of limits . . . 45
3.2.2 The indeterminate form $\frac{0}{0}$ . . . 46
3.3 Limits of trigonometric functions . . . 46
3.4 One-sided limits . . . 49
3.5 Infinite limits and limits at infinity . . . 51
3.5.1 The number e . . . 51
3.6 The squeeze (sandwich) theorem . . . 52
3.7 Continuity . . . 53
3.7.1 The Intermediate Value Theorem . . . 55
3.8 The precise definition of a limit . . . 56
3.9 Exercises . . . 57

4 The Derivative **59**
4.1 The derivative as a limit . . . 60
4.2 Rules for differentiation . . . 62
4.3 Derivatives of trigonometric functions . . . 63
4.4 The chain rule . . . 65
4.5 Implicit differentiation . . . 66
4.6 Exponential and logarithmic functions . . . 67
4.6.1 Logarithmic differentiation . . . 68
4.7 Higher-order derivatives . . . 70
4.8 Numerical differentiation . . . 71
4.9 Exercises . . . 73

5 Applications of the Derivative **75**
5.1 Tangents and normals . . . 75
5.2 Related rates . . . 77
5.3 Maximum and minimum values . . . 78

5.3.1 Absolute extrema on a closed interval . . . 79
5.4 Increasing and decreasing functions . . . 81
5.5 Concavity and the second derivative . . . 83
5.6 Optimisation problems . . . 86
5.7 Newton's method . . . 89
5.8 L'Hôpital's rule . . . 90
5.9 Exercises . . . 92

6 Integration **95**
6.1 Antiderivatives and indefinite integrals . . . 96
6.2 Integration by substitution . . . 98
6.3 Area and Riemann sums . . . 99
6.4 The definite integral . . . 101
6.5 The Fundamental Theorem of Calculus . . . 101
6.6 Integration by parts . . . 102
6.7 Numerical integration . . . 104
6.8 Exercises . . . 106

7 Applications of Integration **108**
7.1 Area between curves . . . 108
7.2 Accumulation: distance, rainfall and more . . . 110
7.3 Average value of a function . . . 112
7.4 Volumes of solids of revolution . . . 113
7.5 Arc length . . . 115
7.6 Areas from survey data . . . 117
7.7 Probability . . . 118
7.8 Exercises . . . 119

8 Parametric Equations and Polar Coordinates **121**
8.1 Parametric equations . . . 122
8.1.1 Projectile motion . . . 123
8.2 Calculus with parametric curves . . . 124
8.3 Polar coordinates . . . 126
8.4 Polar curves . . . 127
8.5 Calculus in polar coordinates . . . 128
8.6 Exercises . . . 131

9 Conic Sections **133**
9.1 The parabola . . . 134
9.1.1 The reflective property . . . 136
9.2 The ellipse . . . 138

9.3 The hyperbola 141
9.4 The general second-degree equation 144
9.5 Eccentricity: one definition for all conics 146
9.6 Exercises 147

A Python Quick Reference **149**

B Answers to Selected Exercises **152**

References and Further Reading **155**

Chapter 0

A First Course in Python for Calculus

Learning Objectives

After working through this chapter you should be able to:

- run Python in Google Colab or Anaconda (Jupyter or Spyder);
- use Python as a calculator and store results in variables;
- use lists, `for`-loops, `if`-statements and functions;
- compute with whole arrays of numbers using `numpy`;
- draw graphs with `matplotlib`;
- do exact algebra with `sympy`.

This chapter teaches just enough Python for the rest of the course. You do not need to master it before starting Chapter 1. Come back to it whenever a later listing uses something unfamiliar. Appendix A gives a one-page summary.

Historical Note: Python

Python was created by the Dutch programmer Guido van Rossum and first released in 1991. It is named after the British comedy group *Monty Python*, not after the snake. Van Rossum wanted a language that was easy to read: the indentation (spaces at the start of a line) is part of the grammar, so Python programs tend to look neat. Python is now one of the most widely used programming languages in the world. In Africa, groups such as Data Science Africa, the Deep Learning Indaba and the African Institute for Mathematical Sciences (AIMS) teach it to thousands of students every year. AIMS has a centre in Ghana, at Biriwa in the Central Region, not far from Cape Coast.

0.1 Getting started

The easiest way to run Python is **Google Colab** (`https://colab.research.google.com`). It runs in a web browser, even on a phone, so nothing needs to be installed. Click *New notebook*, type code in the grey *cell*, and press **Shift+Enter** to run it. On a laptop you can instead install the free **Anaconda Distribution**, which provides Python, the Jupyter notebook and the Spyder editor.

Python Tip: Data-saving

A Colab notebook needs only a small amount of mobile data once it has loaded, because the computation happens on Google's computers, not on your phone. A Jupyter notebook on an Anaconda laptop needs no internet connection at all.

0.2 Python as a calculator

The arithmetic operators are `+`, `-`, `*` (multiplication), `/` (division) and `**` (powers). Note that 2^{10} is written `2**10`, *not* `2^10`. A *variable* is a name that stores a value. We create one with the `=` sign.

In Context: Ride-hailing fares

Ride-hailing companies such as Bolt and Uber usually charge a *base fare*, plus an amount per kilometre, plus an amount per minute. The prices in this book are illustrative and are not the companies' actual prices. The mathematics is the same, however: the fare is a *linear function* of distance and time. You will meet linear functions properly in Section 1.4.

Listing 0.1 — A Bolt fare `ex0_01_first_steps.py`

```
# Listing 0.1: Python as a calculator - a Bolt fare in Cape Coast
# Illustrative fare structure (NOT official Bolt prices)
base_fare = 6.00        # GH¢ charged as soon as the trip starts
per_km = 2.50           # GH¢ per kilometre
per_min = 0.40          # GH¢ per minute

distance = 7.2          # km, e.g. UCC Science to Kotokuraba Market
time = 18               # minutes

fare = base_fare + per_km * distance + per_min * time
print("Trip fare in GH¢:", fare)
print(f"Rounded to 2 d.p.: GH¢ {fare:.2f}")

# Powers, integer division and remainder
print(2 ** 10)       # 2 to the power 10
print(17 // 5)       # whole-number division
print(17 % 5)        # remainder
```

Output

```
Trip fare in GH¢: 31.2
Rounded to 2 d.p.: GH¢ 31.20
1024
3
2
```

Watch Out: Python counts from zero

Python ignores everything after a `#` on a line. These are *comments* for the human reader. Also, Python *indexes from zero*: the first item of a list `L` is `L[0]`, not `L[1]`.

0.3 Lists, loops and functions

A *list* is a sequence of values in square brackets. A `for`-loop repeats an indented block of code once for each item in a sequence. `range(1, 11)` produces the whole numbers $1, 2, \ldots, 10$. The end value 11 is *not* included.

Listing 0.2 — Lists and loops `ex0_02_lists_loops.py`

```
# Listing 0.2: Lists and for-loops - populations of some regions of Ghana
regions = ["Greater Accra", "Ashanti", "Eastern", "Central", "Northern", "Volta"]
# Populations in thousands, rounded, from the 2021 Population and Housing Census
pop = [5456, 5440, 2926, 2860, 2311, 1659]

for region, p in zip(regions, pop):
    print(f"{region:<14} {p:>6} thousand")

print("Number of regions:", len(regions))
print("Total:", sum(pop), "thousand")
print("Largest:", regions[pop.index(max(pop))])

# A loop that builds a new list: square numbers 1..10
squares = []
for n in range(1, 11):
    squares.append(n ** 2)
print(squares)

# The same thing as a one-line "list comprehension"
print([n ** 2 for n in range(1, 11)])
```

Output

```
Greater Accra    5456 thousand
Ashanti          5440 thousand
Eastern          2926 thousand
Central          2860 thousand
Northern         2311 thousand
Volta            1659 thousand
Number of regions: 6
Total: 20652 thousand
Largest: Greater Accra
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

A *function* packages a calculation so it can be reused. A Python function is defined with `def`, much as a mathematician writes $f(x) = x^2 - 4x + 3$.

Listing 0.3 — Functions and if-statements ex0_03_functions.py

```python
# Listing 0.3: Functions and if-statements - a fare with a minimum charge
def bolt_fare(distance_km, minutes, base=6.0, per_km=2.5, per_min=0.4, minimum=15.0):
    """Return an (illustrative) ride-hailing fare in GH¢."""
    fare = base + per_km * distance_km + per_min * minutes
    if fare < minimum:
        fare = minimum
    return round(fare, 2)

print(bolt_fare(1.0, 4))     # a very short trip pays the minimum
print(bolt_fare(7.2, 18))
print(bolt_fare(25, 55))     # Kasoa to Accra Mall, say

# A function of x, written exactly as in mathematics
def f(x):
    return x**2 - 4*x + 3

for x in [0, 1, 2, 3, 4]:
    print("f(", x, ") =", f(x))
```

Output

```
15.0
31.2
90.5
f( 0 ) = 3
f( 1 ) = 0
f( 2 ) = -1
f( 3 ) = 0
f( 4 ) = 3
```

Key Concept: Indentation

The body of a `for`-loop, an `if`-statement or a function `def` is the block of lines *indented* beneath it, normally by four spaces. A colon `:` always comes before an indented block. Wrong indentation is the most common error for beginners: Python reports it as `IndentationError`.

0.4 NumPy: computing with arrays

The library `numpy` (imported as `np`) lets us apply a formula to thousands of numbers in one line. Two functions we use constantly are `np.linspace(a, b, n)`, which gives n equally spaced points from a to b, and `np.array([...])`, which makes an array from a list. NumPy also provides π, e, $\sqrt{}$, sin, cos, exp and ln (written `np.log`).

Listing 0.4 — NumPy arrays ex0_04_numpy.py

```python
# Listing 0.4: NumPy arrays - computing many values at once
import numpy as np

x = np.array([0, 1, 2, 3, 4])
```

```
print("x       =", x)
print("x**2    =", x**2)            # squares every entry
print("2*x + 1 =", 2*x + 1)

# 5 equally spaced points from 0 to 1 (inclusive)
print(np.linspace(0, 1, 5))

# Built-in mathematical functions act on whole arrays
t = np.linspace(0, np.pi, 5)
print("sin(t) =", np.round(np.sin(t), 4))
print("pi =", np.pi, " e =", np.e, " sqrt(2) =", np.sqrt(2))

# Summary statistics of daily Bolt trips for a driver (illustrative)
trips = np.array([14, 17, 12, 20, 22, 25, 9])   # Mon..Sun
print("mean =", trips.mean(), " max =", trips.max(), " total =", trips.sum())
```

Output

```
x       = [0 1 2 3 4]
x**2    = [ 0  1  4  9 16]
2*x + 1 = [1 3 5 7 9]
[0.   0.25 0.5  0.75 1.  ]
sin(t) = [0.     0.7071 1.     0.7071 0.    ]
pi = 3.141592653589793  e = 2.718281828459045  sqrt(2) = 1.4142135623730951
mean = 17.0  max = 25  total = 119
```

Watch Out: `np.log` is the natural logarithm

In NumPy, `np.log(x)` is $\ln x$. For $\log_{10} x$ use `np.log10(x)`. Trigonometric functions always work in **radians**. Use `np.radians(30)` to convert $30°$ to radians and `np.degrees(...)` to convert back.

0.5 Matplotlib: drawing graphs

To draw the graph of a function, we compute many points with NumPy and join them with `plt.plot`.

Listing 0.5 — Plotting two fare plans `ex0_05_matplotlib.py`

```
# Listing 0.5: Plotting with Matplotlib - comparing two fare plans
import numpy as np
import matplotlib.pyplot as plt

d = np.linspace(0, 30, 200)          # distance in km
plan_A = 6.0 + 2.5 * d               # higher per-km rate
plan_B = 12.0 + 2.0 * d              # higher base fare, cheaper per km

plt.figure(figsize=(6, 3.6))
plt.plot(d, plan_A, label="Plan A: 6 + 2.5d")
plt.plot(d, plan_B, "--", label="Plan B: 12 + 2.0d")
plt.plot(12, 36, "ko")               # the plans cost the same at d = 12 km
```

```
plt.annotate("equal cost at 12 km", (12, 36), (15, 25), arrowprops=dict(arrowstyle="->"))
plt.xlabel("distance d (km)")
plt.ylabel("fare (GH¢)")
plt.title("Two illustrative fare plans")
plt.grid(alpha=0.3)
plt.legend()
plt.show()
```

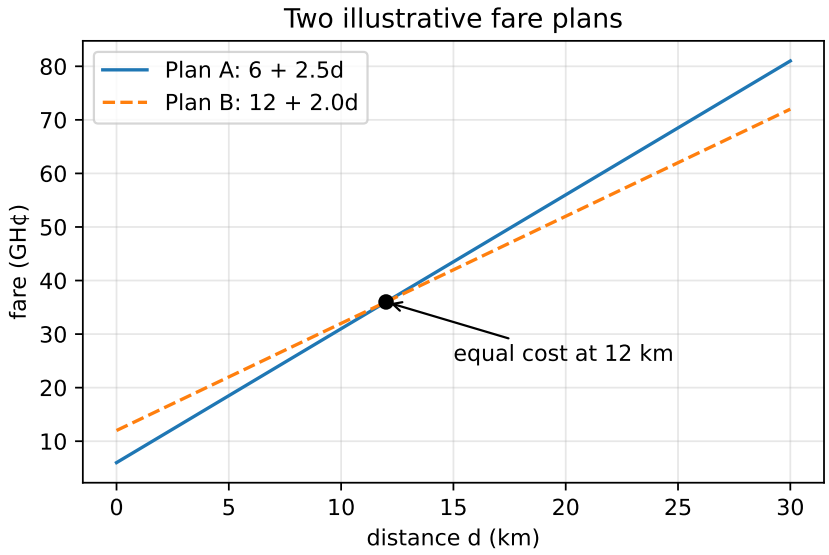


Figure 1: Output of Listing 0.5. The two lines cross where $6 + 2.5d = 12 + 2d$, that is, at $d = 12\,\text{km}$.

0.6 SymPy: exact mathematics

NumPy works with decimal approximations. `sympy` (imported as `sp`) works with *symbols*, as we do with pen and paper. It can expand, factorise, solve equations, and find limits, derivatives and integrals exactly. We use it throughout the book to *check* answers worked by hand.

Listing 0.6 — SymPy `ex0_06_sympy.py`

```
# Listing 0.6: SymPy - exact (symbolic) mathematics
import sympy as sp

x = sp.symbols("x")
f = x**2 - 4*x + 3

print("f(x)       =", f)
print("factorised =", sp.factor(f))
print("roots      =", sp.solve(f, x))
print("(x+1)^3    =", sp.expand((x + 1)**3))
print("f(5)       =", f.subs(x, 5))

# Exact arithmetic: no rounding errors
print(sp.Rational(1, 3) + sp.Rational(1, 6))
```

```python
print(sp.sqrt(8))

# A preview of the calculus to come
print("limit of sin(x)/x as x->0:", sp.limit(sp.sin(x)/x, x, 0))
print("derivative of f:", sp.diff(f, x))
print("integral of f:", sp.integrate(f, x))
```

Output

```
f(x)       = x**2 - 4*x + 3
factorised = (x - 3)*(x - 1)
roots      = [1, 3]
(x+1)^3    = x**3 + 3*x**2 + 3*x + 1
f(5)       = 8
1/2
2*sqrt(2)
limit of sin(x)/x as x->0: 1
derivative of f: 2*x - 4
integral of f: x**3/3 - 2*x**2 + 3*x
```

Key Concept: NumPy or SymPy?

- Use **NumPy** for numbers: tables of values, data and graphs. It is fast but approximate (about 16 significant figures).
- Use **SymPy** for algebra: formulas, exact answers and checking working. It is exact but slower.

Check Your Understanding

1. Use Python to evaluate $\dfrac{3^5 - 2^7}{\sqrt{49}}$.
2. Write a function `area(r)` that returns the area of a circle of radius r. Use it to find the area of a circular farm of radius 120 m.
3. Use SymPy to factorise $x^3 - 6x^2 + 11x - 6$.

Exercises

1. Use Python to evaluate (a) 7^{12}, (b) $1000 \bmod 7$, (c) $\sqrt{2}+\sqrt{3}$ to 6 decimal places, (d) $\sin(\pi/6)$ and $\sin 30°$.

2. A trotro fare from Kotokuraba to UCC is GH₵ 5.50. Write a program that prints a table of the total fare paid by 1 to 10 passengers.

3. Write a function `bmi(mass, height)` that returns body-mass index, mass/height^2. Add an `if`-statement so that it also prints "underweight" when BMI < 18.5.

4. Use `np.linspace` and `plt.plot` to draw the graph of $y = x^3 - 3x$ for $-2.5 \le x \le 2.5$. Where does the graph seem to turn around? (You will learn in Chapter 5 how to find these points exactly.)

5. Use SymPy to solve $2x^2 - 7x + 3 = 0$ exactly. Then use NumPy to evaluate the quadratic formula numerically, and compare.

6. Change Listing 0.5 so that Plan B charges a base fare of GH₵ 10 and GH₵ 2.20 per km. Find the new crossing point (a) from the graph and (b) by solving an equation with `sp.solve`.

7. A mobile-money wallet earns interest of 1% per month, credited monthly. Write a `for`-loop that starts with GH₵ 500 and prints the balance at the end of each month for a year. (We return to this example when we study the number e in Chapter 3.)

Chapter 1

Coordinate Geometry

Learning Objectives

After working through this chapter you should be able to:

- find the distance between two points and the midpoint of a segment;
- find the gradient of a line and recognise parallel and perpendicular lines;
- write the equation of a line in several equivalent forms;
- find the angle between two lines;
- divide a line segment internally and externally in a given ratio;
- find the distance from a point to a line and the bisectors of the angles between two lines;
- write short Python functions for each of these, and use SymPy to check your working.

Points on a line can be identified with real numbers to form the *coordinate line*. In the same way, points of a plane can be identified with *ordered pairs* of real numbers to form the *Cartesian plane*. Any point P of the plane is located by a unique ordered pair (a, b). The first number, a, is the *x-coordinate* (or *abscissa*) of P, and the second, b, is the *y-coordinate* (or *ordinate*).

Historical Note: From Egyptian rope-stretchers to Descartes

Geometry began as a practical art in Africa. Every year the Nile flood washed away the boundary marks of farms. Egyptian surveyors, whom the Greeks called *harpedonaptai* ("rope-stretchers"), re-measured the fields with knotted ropes. The Rhind Mathematical Papyrus, copied by the scribe Ahmes around 1550 BCE, contains problems on the areas of fields and the slopes of pyramids. The Egyptians measured the slope of a pyramid face by a quantity they called the *seked*, which is close to our gradient.

Around 300 BCE, Euclid, working in the great library of **Alexandria** in Egypt, organised Greek geometry into the *Elements*. It is perhaps the most influential textbook ever written. Euclid's geometry used no numbers for points, only figures and logic.

The decisive step of joining geometry to algebra was taken independently by two Frenchmen. Pierre de Fermat circulated his ideas in manuscript around 1636. René Descartes published *La Géométrie* in 1637 as an appendix to his *Discourse on Method*. Their idea was that a curve can be described by an *equation* relating the coordinates of its points. Geometry problems could then be solved with algebra, and algebra could be pictured with geometry. This is *analytic geometry*, and it led directly to the calculus of Newton and Leibniz (Chapter 4). The "Cartesian" plane is named after Descartes (in Latin, *Cartesius*).

1.1 The distance between two points

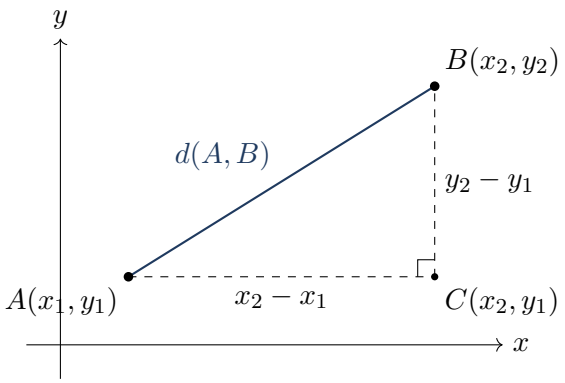


Figure 1.1: The distance formula comes from Pythagoras' theorem applied to triangle ABC.

Consider the right-angled triangle ABC in Fig. 1.1. The horizontal side has length $|x_2 - x_1|$ and the vertical side has length $|y_2 - y_1|$. By Pythagoras' theorem,

$$d(A,B)^2 = |AC|^2 + |BC|^2 = (x_2 - x_1)^2 + (y_2 - y_1)^2.$$

Key Concept: The distance formula

The distance between $A(x_1, y_1)$ and $B(x_2, y_2)$ is

$$d(A,B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}. \tag{1.1}$$

It does not matter which point is called A: the differences are squared, so their signs disappear.

Example 1.1 Which of the points $P(-1,-2)$ and $Q(8,9)$ is closer to $A(3,5)$?

Solution. By (1.1),

$$d(P,A) = \sqrt{(3-(-1))^2 + (5-(-2))^2} = \sqrt{4^2 + 7^2} = \sqrt{65},$$
$$d(Q,A) = \sqrt{(3-8)^2 + (5-9)^2} = \sqrt{25 + 16} = \sqrt{41}.$$

Since $\sqrt{41} < \sqrt{65}$, the point Q is closer to A. ■

In Python we write the formula once, as a function, and then reuse it. Listing 1.1 checks Example 1.1. It then computes straight-line ("as the crow flies") distances between some landmarks in Cape Coast.

Listing 1.1 — The distance formula ex1_01_distance.py

```
# Listing 1.1: The distance formula in Python
import math

def distance(A, B):
    """Distance between points A=(x1, y1) and B=(x2, y2)."""
    (x1, y1), (x2, y2) = A, B
    return math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
```

```python

# Checking the worked example: which of P or Q is closer to A?
A, P, Q = (3, 5), (-1, -2), (8, 9)
print(f"d(P,A) = {distance(P, A):.4f}  (sqrt(65) = {math.sqrt(65):.4f})")
print(f"d(Q,A) = {distance(Q, A):.4f}  (sqrt(41) = {math.sqrt(41):.4f})")

# Approximate positions (km) on a simple grid of Cape Coast:
# origin at Cape Coast Castle, x-axis pointing east, y-axis pointing north.
places = {
    "Cape Coast Castle": (0.0, 0.0),
    "Kotokuraba Market": (-0.4, 0.5),
    "UCC Science":       (-4.4, 1.5),
    "Abura":             (-1.6, 2.4),
}
start = "UCC Science"
for name, pt in places.items():
    if name != start:
        print(f"{start} -> {name:<18}: {distance(places[start], pt):5.2f} km")
```

Output

```
d(P,A) = 8.0623  (sqrt(65) = 8.0623)
d(Q,A) = 6.4031  (sqrt(41) = 6.4031)
UCC Science -> Cape Coast Castle :  4.65 km
UCC Science -> Kotokuraba Market :  4.12 km
UCC Science -> Abura             :  2.94 km
```

In Context: How far is Kumasi from Accra?

Map positions are given by *latitude* (degrees north of the equator) and *longitude* (degrees east or west of Greenwich). Accra lies at about 5.60°N, 0.19°W, and Kumasi at about 6.69°N, 1.62°W. Is the distance formula good enough to find the distance between them? The Earth is a sphere, not a flat sheet of paper.

Listing 1.2 compares two answers. The first converts degrees to kilometres and uses the distance formula. The second uses the *haversine formula* for distances on a sphere. For distances within Ghana the two agree to within 0.1 km. This is why maps of a country can safely be drawn flat. The road distance from Accra to Kumasi is about 250 km, longer than the 200 km straight line, because roads bend.

Listing 1.2 — Flat map versus sphere `ex1_02_great_circle.py`

```python
# Listing 1.2: Flat-map distance versus true (great-circle) distance in Ghana
import math

R = 6371.0   # mean radius of the Earth in km
towns = {    # (latitude, longitude) in degrees; west is negative
    "Accra":      (5.6037, -0.1870),
    "Kumasi":     (6.6885, -1.6244),
    "Cape Coast": (5.1053, -1.2466),
    "Tamale":     (9.4008, -0.8393),
}

```

```python
def flat_distance(P, Q):
    """Treat (lat, lon) as x-y coordinates on a flat map.
    East-west distances are shrunk by cos(latitude) because meridians converge."""
    lat0 = math.radians((P[0] + Q[0]) / 2)
    dx = math.radians(Q[1] - P[1]) * R * math.cos(lat0)
    dy = math.radians(Q[0] - P[0]) * R
    return math.sqrt(dx**2 + dy**2)        # the distance formula!

def great_circle(P, Q):
    """Haversine formula: distance along the surface of a sphere."""
    p1, p2 = math.radians(P[0]), math.radians(Q[0])
    dphi = p2 - p1
    dlam = math.radians(Q[1] - P[1])
    a = math.sin(dphi/2)**2 + math.cos(p1)*math.cos(p2)*math.sin(dlam/2)**2
    return 2 * R * math.asin(math.sqrt(a))

for B in ["Kumasi", "Cape Coast", "Tamale"]:
    f = flat_distance(towns["Accra"], towns[B])
    g = great_circle(towns["Accra"], towns[B])
    print(f"Accra -> {B:<10}  flat map: {f:6.1f} km   sphere: {g:6.1f} km")
```

Output

```
Accra -> Kumasi      flat map:  199.5 km   sphere:  199.5 km
Accra -> Cape Coast  flat map:  129.7 km   sphere:  129.7 km
Accra -> Tamale      flat map:  428.3 km   sphere:  428.3 km
```

Historical Note: Eratosthenes measures the Earth

Around 240 BCE Eratosthenes, the chief librarian at Alexandria, heard that at noon on midsummer's day the Sun shone straight down a well at Syene (modern Aswan). At the same moment in Alexandria, about 5000 *stadia* to the north, a vertical stick cast a shadow at an angle of about 7.2°, which is $\frac{1}{50}$ of a full circle. He concluded that the Earth's circumference is about $50 \times 5000 = 250\,000$ stadia, remarkably close to the modern value of about 40 000 km. He did all this in Africa, using only geometry.

Check Your Understanding

1. Find the distance between $(-3, 5)$ and $(5, -10)$.
2. Show that the triangle with vertices $(1, 1)$, $(4, 5)$ and $(8, 2)$ is isosceles. Is it also right-angled?

1.2 The midpoint of a line segment

Let $M(x, y)$ be the midpoint of the segment from $A(x_1, y_1)$ to $B(x_2, y_2)$. Going from A to M must cover the same horizontal distance as going from M to B, so $x - x_1 = x_2 - x$, that is $2x = x_1 + x_2$. The same argument works vertically.

Key Concept: The midpoint formula

The midpoint of the segment joining $A(x_1, y_1)$ and $B(x_2, y_2)$ is

$$M = \left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2} \right). \tag{1.2}$$

The midpoint is the *average* of the endpoints.

Example 1.2 Show that the quadrilateral with vertices $P(1, 2)$, $Q(4, 4)$, $R(5, 9)$ and $S(2, 7)$ is a parallelogram by proving that its diagonals bisect each other.

Solution. The midpoint of diagonal PR is $\left(\frac{1+5}{2}, \frac{2+9}{2}\right) = \left(3, \frac{11}{2}\right)$. The midpoint of diagonal QS is $\left(\frac{4+2}{2}, \frac{4+7}{2}\right) = \left(3, \frac{11}{2}\right)$. The diagonals have the same midpoint, so they bisect each other, and $PQRS$ is a parallelogram. ■

Listing 1.3 — Checking and drawing the parallelogram `ex1_03_midpoint.py`

```python
# Listing 1.3: Midpoints and the parallelogram PQRS
import matplotlib.pyplot as plt

def midpoint(A, B):
    return ((A[0] + B[0]) / 2, (A[1] + B[1]) / 2)

P, Q, R, S = (1, 2), (4, 4), (5, 9), (2, 7)
M1, M2 = midpoint(P, R), midpoint(Q, S)
print("Midpoint of diagonal PR:", M1)
print("Midpoint of diagonal QS:", M2)
print("Diagonals bisect each other:", M1 == M2)

xs = [P[0], Q[0], R[0], S[0], P[0]]
ys = [P[1], Q[1], R[1], S[1], P[1]]
plt.figure(figsize=(4, 4.5))
plt.plot(xs, ys, "b-o")
plt.plot([P[0], R[0]], [P[1], R[1]], "g--")
plt.plot([Q[0], S[0]], [Q[1], S[1]], "g--")
plt.plot(*M1, "rs")
for name, pt in zip("PQRS", [P, Q, R, S]):
    plt.annotate(name, pt, textcoords="offset points", xytext=(6, -10))
plt.annotate("M(3, 5.5)", M1, textcoords="offset points", xytext=(8, 0))
plt.gca().set_aspect("equal")
plt.grid(alpha=0.3)
plt.show()
```

Output

```
Midpoint of diagonal PR: (3.0, 5.5)
Midpoint of diagonal QS: (3.0, 5.5)
Diagonals bisect each other: True
```

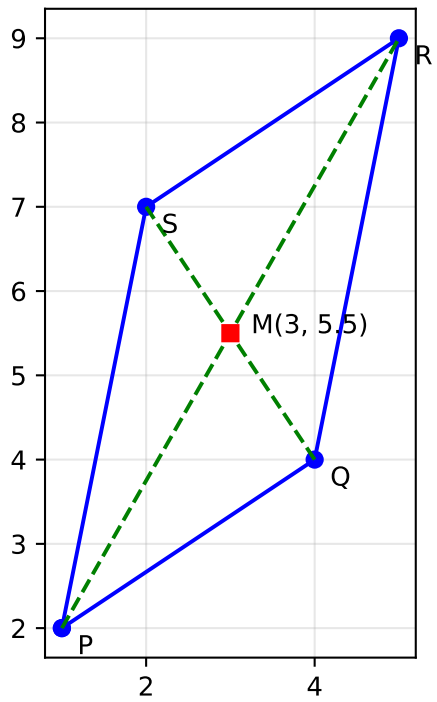


Figure 1.2: The parallelogram $PQRS$ of Example 1.2. The diagonals (dashed) meet at their common midpoint M.

In Context: Meeting halfway

Two friends, one at Kotokuraba Market $(-0.4,\ 0.5)$ and one at UCC Science $(-4.4,\ 1.5)$, agree to meet halfway in a straight line. On the grid of Listing 1.1 the meeting point is $\left(\frac{-0.4-4.4}{2}, \frac{0.5+1.5}{2}\right) = (-2.4,\ 1.0)$. Midpoints are used in the same way to place a health post or a bus stop fairly between two communities.

Check Your Understanding

$M(2,-1)$ is the midpoint of AB, and $A=(-3,4)$. Find B. (*Hint:* solve $\frac{-3+x}{2}=2$ and a similar equation for y.)

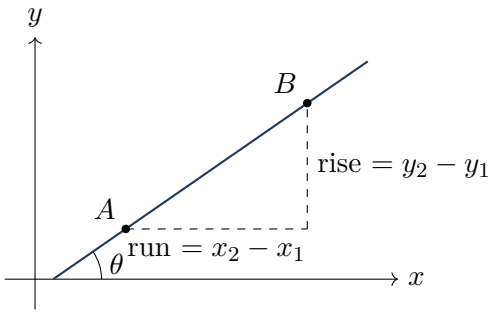
1.3 The gradient of a line

Definition 1.1. (Gradient)

The *gradient* (or *slope*) of the line through $A(x_1,y_1)$ and $B(x_2,y_2)$, where $x_1 \neq x_2$, is

$$m = \frac{\text{rise}}{\text{run}} = \frac{y_2-y_1}{x_2-x_1} = \tan\theta,$$

where θ is the *angle of inclination*: the angle the line makes with the positive x-axis. A vertical line ($x_1=x_2$) has no gradient (it is undefined).



Key Concept: Parallel and perpendicular lines

Let lines L_1 and L_2 have gradients m_1 and m_2.

- $L_1 \parallel L_2 \iff m_1 = m_2$ (parallel lines have equal gradients).
- $L_1 \perp L_2 \iff m_1 m_2 = -1$, that is, $m_2 = -\dfrac{1}{m_1}$ (the gradients are *negative reciprocals*).

Example 1.3 Let $A(1,2)$, $B(4,8)$, $C(0,5)$ and $D(2,4)$. Then $m_{AB} = \frac{8-2}{4-1} = 2$ and $m_{CD} = \frac{4-5}{2-0} = -\frac{1}{2}$. Since $m_{AB}\, m_{CD} = -1$, $AB \perp CD$.

In Context: Gradients on the road

Road signs in Ghana and elsewhere often show a gradient as a percentage. A "7.5% grade" means a rise of 7.5 m for every 100 m travelled horizontally, so $m = 0.075$. The mountain road from Accra up to Aburi has steep sections like this. Building codes ask that a wheelchair ramp have a gradient of no more than 1:12 $\approx$ 8.3%.

Listing 1.4 — Gradients, parallel and perpendicular lines `ex1_04_gradient.py`

```python
# Listing 1.4: Gradients - parallel and perpendicular lines, and road slopes
from fractions import Fraction
import math

def gradient(A, B):
    (x1, y1), (x2, y2) = A, B
    return Fraction(y2 - y1, x2 - x1)   # exact fraction, no rounding

A, B, C, D = (1, 2), (4, 8), (0, 5), (2, 4)
mAB, mCD = gradient(A, B), gradient(C, D)
print("m_AB =", mAB, "  m_CD =", mCD)
print("Parallel?      ", mAB == mCD)
print("Perpendicular? ", mAB * mCD == -1)

# A road rises 45 m over a horizontal distance of 600 m
rise, run = 45, 600
m = rise / run
print(f"gradient = {m:.3f}, i.e. a {100*m:.1f}% grade")
print(f"angle of inclination = {math.degrees(math.atan(m)):.2f} degrees")
```

Output

```
m_AB = 2   m_CD = -1/2
Parallel?      False
Perpendicular?  True
gradient = 0.075, i.e. a 7.5% grade
angle of inclination = 4.29 degrees
```

Python Tip: Exact fractions

`from fractions import Fraction` lets Python compute with exact fractions such as $-\frac{1}{2}$ instead of the decimal `-0.5`. Exact arithmetic is useful when testing whether $m_1 m_2$ is *exactly* -1. Decimal arithmetic can give answers like `-0.9999999999999999`.

1.4 The equation of a line

Let a line pass through $A(x_1, y_1)$ with gradient m, and let $P(x, y)$ be any other point on it. The gradient of AP must equal m:

$$\frac{y - y_1}{x - x_1} = m \quad \Longrightarrow \quad y - y_1 = m(x - x_1).$$

Key Concept: Forms of the equation of a straight line

Point–gradient form	$y - y_1 = m(x - x_1)$
Gradient–intercept form	$y = mx + c$ (m = gradient, c = y-intercept)
Two-point form	$\dfrac{y - y_1}{y_2 - y_1} = \dfrac{x - x_1}{x_2 - x_1}$
Intercept form	$\dfrac{x}{a} + \dfrac{y}{b} = 1$ (intercepts a on the x-axis, b on the y-axis)
General form	$ax + by + c = 0$ (gradient $-a/b$ when $b \neq 0$)
Vertical / horizontal	$x = k$ / $y = k$

Example 1.4 Find the equation of the line through $A(1, 3)$ with gradient 5.

Solution. $y - 3 = 5(x - 1)$, so $y = 5x - 2$. ■

Example 1.5 Find the equation of the line with x-intercept 3 and y-intercept -5.

Solution. By the intercept form, $\frac{x}{3} + \frac{y}{-5} = 1$. Multiplying by 15 gives $5x - 3y = 15$, or $5x - 3y - 15 = 0$. ■

In Context: A fare is a straight line

Suppose a ride-hailing driver charged GH₵ 18 for a 4 km trip and GH₵ 33 for a 10 km trip, at similar traffic times. If the fare is a linear function of distance, the gradient is the price per km, $m = \frac{33-18}{10-4} = 2.5$, and the y-intercept is the base fare: $y - 18 = 2.5(x - 4)$, so $y = 2.5x + 8$. A 15 km trip should cost $2.5(15) + 8 =$ GH₵ 45.50. Listing 1.5 does this with

SymPy.

Listing 1.5 — Finding a fare formula from two trips ex1_05_line_equation.py

```python
# Listing 1.5: Finding a fare formula (a straight line) from two trips
import sympy as sp

x, y = sp.symbols("x y")

def line_through(P, Q):
    """Return the line through P and Q in the form y = m x + c."""
    m = sp.Rational(Q[1] - P[1], Q[0] - P[0])
    return sp.Eq(y, sp.expand(m * (x - P[0]) + P[1]))

# Checking the worked example: line through (1, 3) with gradient 5
print(sp.Eq(y, sp.expand(5 * (x - 1) + 3)))

# Two trips with the same driver (distance km, fare GH¢), illustrative
trip1, trip2 = (4, 18), (10, 33)
L = line_through(trip1, trip2)
print("Fare line:", L)
m = L.rhs.coeff(x)
c = L.rhs.subs(x, 0)
print("per-km rate =", m, "  base fare =", c)
print("Predicted fare for a 15 km trip:", L.rhs.subs(x, 15))

# The same line in general form ax + by + c = 0 with integer coefficients
num, den = sp.fraction(sp.together(L.rhs - y))
print("General form:", sp.Eq(sp.expand(num), 0))
```

Output

```
Eq(y, 5*x - 2)
Fare line: Eq(y, 5*x/2 + 8)
per-km rate = 5/2   base fare = 8
Predicted fare for a 15 km trip: 91/2
General form: Eq(5*x - 2*y + 16, 0)
```

Watch Out: Read the gradient from $y = mx + c$ only

The gradient of $2y = 6x + 5$ is 3, not 6. First divide through so that y stands alone. For $ax + by + c = 0$, the gradient is $-a/b$.

Check Your Understanding

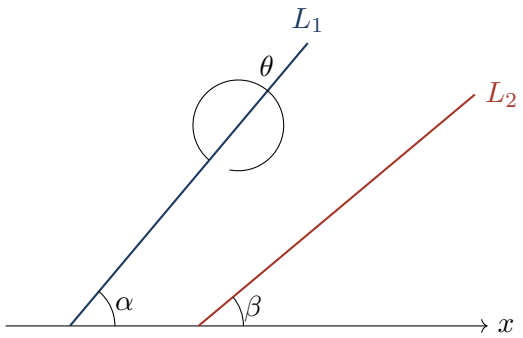
1. Find the equation of the line through $(2, 3)$ that is perpendicular to $4x - y + 7 = 0$.
2. Write the line through $(-3, 5)$ and $(5, -10)$ in general form.

1.5 The angle between two lines

Let L_1 and L_2 have angles of inclination α and β, so that $m_1 = \tan\alpha$ and $m_2 = \tan\beta$. Suppose $\alpha > \beta$. The angle θ between the lines satisfies $\theta = \alpha - \beta$ (an exterior angle of a triangle equals the sum of the two opposite interior angles). Hence

$$\tan\theta = \tan(\alpha-\beta) = \frac{\tan\alpha - \tan\beta}{1+\tan\alpha\tan\beta} = \frac{m_1 - m_2}{1 + m_1 m_2}.$$

Two lines make two angles, θ and $180^\circ - \theta$, and $\tan(180^\circ - \theta) = -\tan\theta$.



Key Concept: Angle between two lines

If lines with gradients m_1 and m_2 meet at an acute angle θ, then

$$\tan\theta = \left|\frac{m_1 - m_2}{1 + m_1 m_2}\right|. \tag{1.3}$$

If $1 + m_1 m_2 = 0$, the lines are perpendicular ($\theta = 90^\circ$).

Example 1.6 Find the acute angle between the lines $y = 2x + 1$ and $y = 3x + 2$.

Solution. With $m_1 = 3$ and $m_2 = 2$, $\tan\theta = \frac{3-2}{1+6} = \frac{1}{7}$, so $\theta = \tan^{-1}\frac{1}{7} \approx 8.13^\circ$ (more precisely 8.1301°; the third edition's value of 8.1325° was a misprint). ■

Listing 1.6 — Angle between two lines `ex1_06_angle.py`

```python
# Listing 1.6: Angle between two lines
import math

def acute_angle(m1, m2):
    """Acute angle (degrees) between lines with gradients m1 and m2."""
    if 1 + m1 * m2 == 0:
        return 90.0
    return math.degrees(math.atan(abs((m1 - m2) / (1 + m1 * m2))))

print(f"y=2x+1 and y=3x+2: {acute_angle(3, 2):.4f} degrees")

# Two roads meet at a junction: one along y = 0.5x, the other along y = -3x + 14.
print(f"Junction angle: {acute_angle(0.5, -3):.1f} degrees")

# Check with angles of inclination: theta = atan(m)
t1, t2 = math.atan(3), math.atan(2)
```

```
print(f"Check: {math.degrees(t1 - t2):.4f} degrees")
```

Output

```
y=2x+1 and y=3x+2: 8.1301 degrees
Junction angle: 81.9 degrees
Check: 8.1301 degrees
```

Example 1.7 The point $A(1,2)$ is a vertex of a parallelogram $ABCD$. Side AB lies on the line $2y = x + 3$ and the diagonal BD lies on $3y + x = 17$. If $|AD| = |BD|$, find the coordinates of C and the equation of the perpendicular bisector of AB.

Solution. Solving $2y = x + 3$ and $3y = -x + 17$ simultaneously gives $B(5,4)$. Let θ be the angle $\angle DBA$. The gradients of AB and BD are $\frac{1}{2}$ and $-\frac{1}{3}$, so by (1.3), $\tan\theta = \left|\frac{\frac{1}{2}+\frac{1}{3}}{1-\frac{1}{6}}\right| = 1$. Since $|AD| = |BD|$, triangle ABD is isosceles with $\angle DAB = \angle DBA = \theta$. Line AD therefore makes angle θ with AB, on the other side from BD. If m is the gradient of AD, then $\frac{m-\frac{1}{2}}{1+\frac{1}{2}m} = 1$, which gives $m = 3$. The equation of AD is $y - 2 = 3(x - 1)$, that is $y = 3x - 1$. Solving this with $x + 3y = 17$ gives $D(2,5)$.

The diagonals of a parallelogram bisect each other. The midpoint of BD is $M\left(\frac{7}{2}, \frac{9}{2}\right)$, which must also be the midpoint of AC. If $C = (p,q)$, then $\frac{1+p}{2} = \frac{7}{2}$ and $\frac{2+q}{2} = \frac{9}{2}$, so $C = (6,7)$.

The midpoint of AB is $(3,3)$ and the perpendicular gradient is -2, so the perpendicular bisector of AB is $y - 3 = -2(x - 3)$, that is, $y = -2x + 9$. ■

Listing 1.7 reaches the same answers using SymPy's geometry objects `Point` and `Line`. It finds D directly from the condition $|AD| = |BD|$, without any angles.

Listing 1.7 — Example 1.7 with SymPy geometry `ex1_07_parallelogram.py`

```
# Listing 1.7: The parallelogram ABCD solved with SymPy
import sympy as sp

x, y, t = sp.symbols("x y t")
A = sp.Point(1, 2)
AB = sp.Line(sp.Point(-3, 0), sp.Point(1, 2))      # 2y = x + 3 passes through these
BD = sp.Line(sp.Point(17, 0), sp.Point(2, 5))      # 3y + x = 17 passes through these

B = AB.intersection(BD)[0]
print("B =", B)

# D is the point of BD (other than B) with |AD| = |BD|.
D_general = sp.Point(17 - 3*t, t)                  # every point of BD: x = 17 - 3y
sols = sp.solve(sp.Eq(A.distance(D_general)**2, B.distance(D_general)**2), t)
D = D_general.subs(t, sols[0])
print("D =", D)

M = B.midpoint(D)                                  # diagonals bisect each other
C = sp.Point(2*M.x - A.x, 2*M.y - A.y)
print("M =", M, "  C =", C)

mid_AB = A.midpoint(B)
```

```
perp = AB.perpendicular_line(mid_AB)
print("Perpendicular bisector of AB:", sp.expand(perp.equation(x, y)), "= 0")
```

Output

```
B = Point2D(5, 4)
D = Point2D(2, 5)
M = Point2D(7/2, 9/2)   C = Point2D(6, 7)
Perpendicular bisector of AB: -4*x - 2*y + 18 = 0
```

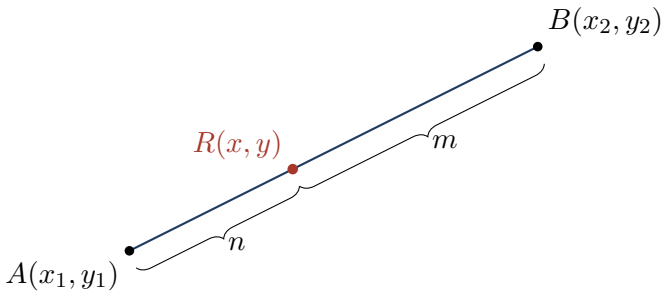
1.6 Division of a line segment in a given ratio

1.6.1 Internal division

Let $R(x, y)$ lie on the segment AB, where $A = (x_1, y_1)$ and $B = (x_2, y_2)$, with $|AR| : |RB| = n : m$. The triangles formed with horizontal and vertical lines are similar, so

$$\frac{x - x_1}{x_2 - x} = \frac{n}{m} \implies m(x - x_1) = n(x_2 - x) \implies x = \frac{mx_1 + nx_2}{m + n},$$

and similarly $y = \dfrac{my_1 + ny_2}{m + n}$.



1.6.2 External division

If R lies on the line AB *outside* the segment, with $|AR| : |RB| = n : m$, we say R divides AB *externally*. Equivalently, it divides AB in the "negative" ratio $n : (-m)$. Replacing m by $-m$ in the internal formula gives the external one.

Key Concept: The section formulae

The point dividing $A(x_1, y_1)B(x_2, y_2)$ in the ratio $n : m$ is

$$\text{internally: } \left(\frac{mx_1 + nx_2}{m + n}, \frac{my_1 + ny_2}{m + n}\right), \qquad \text{externally: } \left(\frac{mx_1 - nx_2}{m - n}, \frac{my_1 - ny_2}{m - n}\right).$$

The midpoint formula is the case $n = m = 1$ of internal division.

Example 1.8 Find the points dividing the join of $A(1, 2)$ and $B(3, 1)$ (a) internally in the ratio $1 : 2$; (b) externally in the ratio $1 : 2$.

Solution. (a) With $n = 1$, $m = 2$: $\left(\frac{2(1)+1(3)}{3}, \frac{2(2)+1(1)}{3}\right) = \left(\frac{5}{3}, \frac{5}{3}\right)$.

(b) $\left(\frac{2(1)-1(3)}{2-1}, \frac{2(2)-1(1)}{2-1}\right) = (-1, 3)$. ■

In Context: Placing a booster pump

Water reaches many Ghanaian towns through long pipelines from treatment plants. Booster pumps keep the pressure up along the way. Suppose a pipeline runs straight from a plant at $W(0,0)$ to a town at $T(12,9)$, measured in km, and engineers want a pump at P with $WP : PT = 2 : 3$. The section formula gives $P = \left(\frac{3(0)+2(12)}{5}, \frac{3(0)+2(9)}{5}\right) = (4.8,\ 3.6)$.

Listing 1.8 — The section formula ex1_08_section.py

```python
# Listing 1.8: Dividing a line segment in a given ratio
from fractions import Fraction as F

def divide(A, B, n, m, external=False):
    """Point R with AR:RB = n:m (internal) or n:-m (external)."""
    (x1, y1), (x2, y2) = A, B
    if external:
        m = -m
    return (F(m*x1 + n*x2, m + n), F(m*y1 + n*y2, m + n))

A, B = (1, 2), (3, 1)
R = divide(A, B, 1, 2)
print("Internal 1:2 ->", tuple(str(v) for v in R))
S = divide(A, B, 1, 2, external=True)
print("External 1:2 ->", tuple(str(v) for v in S))

# A water pipeline runs from a treatment plant W(0, 0) to a town T(12, 9) (km).
# A booster pump must be placed at P with WP : PT = 2 : 3.
P = divide((0, 0), (12, 9), 2, 3)
print("Pump position:", tuple(float(v) for v in P))
```

Output

```
Internal 1:2 -> ('5/3', '5/3')
External 1:2 -> ('-1', '3')
Pump position: (4.8, 3.6)
```

Watch Out: Which is n and which is m?

In "R divides AB in the ratio $n : m$", the number n goes with the part next to A, but in the formula n multiplies the coordinates of B. Check your answer: an internal point must lie *between* A and B, and nearer to A when $n < m$.

1.7 The distance from a point to a line

Theorem 1.1. (Perpendicular distance)

The perpendicular distance from $P(x_1, y_1)$ to the line $ax + by + c = 0$ is

$$d = \frac{|ax_1 + by_1 + c|}{\sqrt{a^2 + b^2}}. \tag{1.4}$$

Proof (sketch). The line has gradient $-a/b$, so the perpendicular through P has gradient b/a and equation $y - y_1 = \frac{b}{a}(x - x_1)$. Solving this simultaneously with $ax + by + c = 0$ gives the foot of the perpendicular,

$$D = (x_1 - ak,\ y_1 - bk)\,, \qquad \text{where } k = \frac{ax_1 + by_1 + c}{a^2 + b^2}.$$

Then $|PD|^2 = a^2k^2 + b^2k^2 = (a^2 + b^2)k^2 = \dfrac{(ax_1 + by_1 + c)^2}{a^2 + b^2}$. Taking square roots gives (1.4). □

Example 1.9 Find the distance of $P(3, 2)$ from the line $12x + 5y - 11 = 0$.

Solution. $d = \dfrac{|12(3) + 5(2) - 11|}{\sqrt{12^2 + 5^2}} = \dfrac{35}{13} \approx 2.69.$ ■

Example 1.10 Find the set of points at distance $\sqrt{5}$ from the line $2x - y - 4 = 0$.

Solution. A point (h, k) is at distance $\sqrt{5}$ when $\dfrac{|2h - k - 4|}{\sqrt{5}} = \sqrt{5}$, that is, $2h - k - 4 = \pm 5$. The set is the pair of parallel lines $2x - y - 9 = 0$ and $2x - y + 1 = 0$. ■

In Context: The shortest feeder road

A village V lies at $(6, 8)$ on a km grid. A straight highway follows the line $3x + 4y - 12 = 0$. The shortest feeder road from V meets the highway at a right angle. Its length is $\dfrac{|3(6) + 4(8) - 12|}{5} = \dfrac{38}{5} = 7.6\,\text{km}$. The same calculation tells a farmer the shortest pipe from a borehole to a straight irrigation channel, and tells an engineer how close a building stands to the centre-line of a road.

Listing 1.9 — Shortest feeder road `ex1_09_point_line.py`

```python
# Listing 1.9: Shortest feeder road from a village to a straight highway
import math
import matplotlib.pyplot as plt
import numpy as np

def point_line_distance(P, a, b, c):
    """Distance from P=(x1, y1) to the line ax + by + c = 0."""
    x1, y1 = P
    return abs(a*x1 + b*y1 + c) / math.sqrt(a**2 + b**2)

def foot_of_perpendicular(P, a, b, c):
    x1, y1 = P
    k = (a*x1 + b*y1 + c) / (a**2 + b**2)
```

```python
    return (x1 - a*k, y1 - b*k)

# Checking the worked example: P(3, 2) and 12x + 5y - 11 = 0
print("Distance =", point_line_distance((3, 2), 12, 5, -11))

# Highway: 3x + 4y - 12 = 0 (km grid).  Village V(6, 8).
V = (6, 8)
d = point_line_distance(V, 3, 4, -12)
F = foot_of_perpendicular(V, 3, 4, -12)
print(f"Feeder road length = {d:.2f} km, meeting the highway at ({F[0]:.2f}, {F[1]:.2f})")

x = np.linspace(-2, 8, 50)
plt.figure(figsize=(5, 4.5))
plt.plot(x, (12 - 3*x)/4, "k", lw=3, label="highway 3x + 4y = 12")
plt.plot([V[0], F[0]], [V[1], F[1]], "g--o", label=f"feeder road, {d:.1f} km")
plt.annotate("village V", V, textcoords="offset points", xytext=(5, 5))
plt.gca().set_aspect("equal")
plt.grid(alpha=0.3)
plt.legend(loc="lower left")
plt.show()
```

Output

```
Distance = 2.6923076923076925
Feeder road length = 7.60 km, meeting the highway at (1.44, 1.92)
```

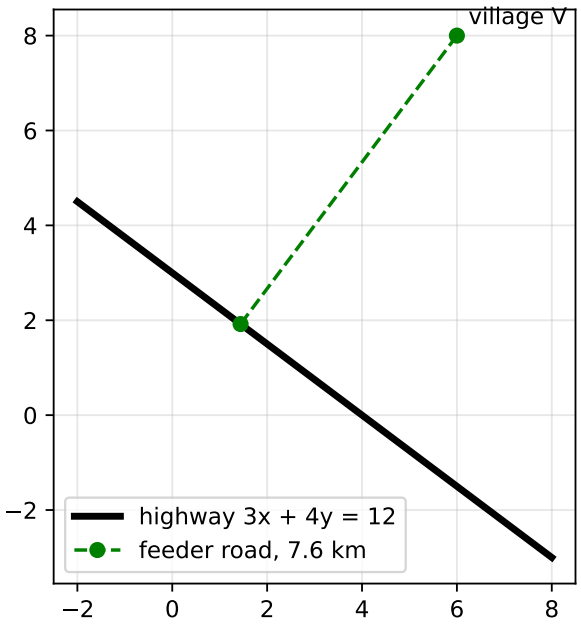


Figure 1.3: The feeder road (dashed) meets the highway at a right angle. Any other route to the highway is longer.

1.8 Bisectors of the angles between two lines

Let L_1 and L_2 be two intersecting lines. The two *bisectors* B_1 and B_2 of the angles between them are the locus of points *equidistant* from L_1 and L_2. The two bisectors are perpendicular to each other.

Key Concept: Angle bisectors

The bisectors of the angles between $a_1x + b_1y + c_1 = 0$ and $a_2x + b_2y + c_2 = 0$ are

$$\frac{a_1x + b_1y + c_1}{\sqrt{a_1^2 + b_1^2}} = \pm\frac{a_2x + b_2y + c_2}{\sqrt{a_2^2 + b_2^2}}.$$

Example 1.11 Find the equations of the bisectors of the angles between $x + y + 2 = 0$ and $x + 7y + 26 = 0$.

Solution. $\dfrac{x+y+2}{\sqrt{2}} = \pm\dfrac{x+7y+26}{\sqrt{50}} = \pm\dfrac{x+7y+26}{5\sqrt{2}}$, so $5(x+y+2) = \pm(x+7y+26)$. The "+" sign gives $4x - 2y - 16 = 0$, that is $2x - y - 8 = 0$. The "−" sign gives $6x + 12y + 36 = 0$, that is $x + 2y + 6 = 0$. (Check: the gradients 2 and $-\frac{1}{2}$ multiply to -1, so the bisectors are perpendicular.) ■

In Context: Siting a borehole

Two straight roads leave a junction O: one along $y = 0$ and the other along $4x - 3y = 0$. A community wants a borehole that is the same distance from both roads, lies in the angle between them, and is 5 km from the junction. The borehole must lie on the bisector $y = \frac{4x-3y}{5}$, that is $y = \frac{x}{2}$. Setting $x^2 + y^2 = 25$ gives $(2\sqrt{5}, \sqrt{5}) \approx (4.47, 2.24)$.

Listing 1.10 — Angle bisectors ex1_10_bisectors.py

```python
# Listing 1.10: Bisectors of the angles between two lines (with SymPy)
import sympy as sp

x, y = sp.symbols("x y")
L1 = x + y + 2
L2 = x + 7*y + 26

# A point on a bisector is equally far from both lines:
#     L1 / sqrt(1**2 + 1**2) = +/- L2 / sqrt(1**2 + 7**2)
for sign in (+1, -1):
    eq = sp.expand(L1 / sp.sqrt(2) - sign * L2 / sp.sqrt(50))
    eq = sp.expand(eq * 5 * sp.sqrt(2))      # clear the surds
    print("Bisector:", eq, "= 0   i.e.", sp.factor_terms(eq), "= 0")

# Borehole problem: two straight roads y = 0 and 4x - 3y = 0 meet at a junction O.
# The borehole must be equidistant from both roads, inside the angle between them
# (x > 0, y > 0), and 5 km from the junction.
bisector = sp.Eq(y, (4*x - 3*y) / 5)         # distances y and (4x-3y)/5 are equal
sol = sp.solve([bisector, sp.Eq(x**2 + y**2, 25)], [x, y], dict=True)
for s in sol:
    if s[x] > 0:
        print("Borehole at", (s[x], s[y]),
```

```
23              "~", (round(float(s[x]), 3), round(float(s[y]), 3)))
```

Output

```
Bisector: 4*x - 2*y - 16 = 0   i.e. 2*(2*x - y - 8) = 0
Bisector: 6*x + 12*y + 36 = 0   i.e. 6*(x + 2*y + 6) = 0
Borehole at (2*sqrt(5), sqrt(5)) ~ (4.472, 2.236)
```

Chapter Summary

- Distance: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. Midpoint: $\left(\frac{x_1+x_2}{2}, \frac{y_1+y_2}{2}\right)$.
- Gradient: $m = \frac{y_2-y_1}{x_2-x_1} = \tan\theta$. Parallel lines: $m_1 = m_2$. Perpendicular lines: $m_1 m_2 = -1$.
- Line: $y - y_1 = m(x - x_1)$, $y = mx + c$ or $ax + by + c = 0$.
- Angle between lines: $\tan\theta = \left|\frac{m_1-m_2}{1+m_1m_2}\right|$.
- Section formula (internal): $\left(\frac{mx_1+nx_2}{m+n}, \frac{my_1+ny_2}{m+n}\right)$; for external division, replace m by $-m$.
- Point to line: $d = \frac{|ax_1+by_1+c|}{\sqrt{a^2+b^2}}$. Bisectors: equal distances with a $\pm$ sign.
- Python: write each formula once as a function; check algebra with SymPy's `Point`, `Line` and `solve`.

1.9 Exercises

A. Skills

1. Given $A(-3, 5)$ and $B(5, -10)$, find: (a) the distance AB; (b) the midpoint P of AB; (c) the point Q that divides AB internally in the ratio $2 : 5$; (d) the point R that divides AB externally in the ratio $3 : 1$; (e) the gradient of AB; (f) the equation of the line AB; (g) the equation of the perpendicular bisector of AB; (h) the perpendicular distance from $S(2, 4)$ to AB.

2. Find all values of r such that the gradient of the line through $(r, 4)$ and $(1, 3 - 2r)$ is less than 5.

3. M is the midpoint of AB, where $A = (2, 3)$ and $B = (18, 20)$. The points P and Q divide AB internally and externally, respectively, in the ratio $2 : 3$. Show that $|MP| \cdot |MQ| = |MB|^2$.

4. Two sides of a parallelogram lie on $y - x = 2$ and $2x + y = 4$. Find the equations of the other two sides if they intersect at $(0, -4)$.

5. Find the gradients of the lines joining the following pairs of points: (a) $\left(cp, \frac{c}{p}\right)$, $\left(cq, \frac{c}{q}\right)$; (b) $(ap^2, 2ap)$, $(aq^2, 2aq)$; (c) $(a\cos\alpha, b\sin\alpha)$, $(a\cos\beta, b\sin\beta)$.

6. Find the equation of the line parallel to the x-axis through the point of intersection of $4x + 3y - 6 = 0$ and $x - 2y - 7 = 0$.

7. Find the equation of the line through $(2, 3)$ and the point of intersection of $3x + 2y = 2$ and $4x + 3y = 7$.

8. Let l be the line $ax + by + c = 0$ and $P = (x_1, y_1)$. Show that: (a) the line through P parallel to l is $ax + by = ax_1 + by_1$; (b) the line through P perpendicular to l is $bx - ay = bx_1 - ay_1$.

9. Let $A(-2, 1)$, $B(2, 3)$ and $C(-2, -4)$. (a) Find the angle between AB and BC. (b) Are A, B and C collinear?

10. At what angle are the lines $ax + by + c = 0$ and $(a - b)x + (a + b)y + d = 0$ inclined to each other, where $a, b > 0$?

11. One side of a square lies along $4x + 3y = 26$, and the diagonals of the square intersect at $(-2, 3)$. Find (a) the vertices of the square; (b) the equations of the sides perpendicular to the given line.

12. Find the bisectors of the angles between $3x + 4y - 7 = 0$ and $12x - 5y + 3 = 0$.

B. Applications

13. ⚙ On a km grid with origin at Kejetia, Kumasi, the Komfo Anokye Teaching Hospital is at $(-1.2,\ 0.6)$ and KNUST is at $(4.1,\ -3.0)$. Find the straight-line distance between them, and the point exactly halfway.

14. ⚙ A trotro driver records fares of GH₵ 7 for a 5 km trip and GH₵ 12.40 for a 14 km trip. Assuming a linear fare rule, find (a) the per-km charge; (b) the fixed charge; (c) the fare for a 20 km trip.

15. ⚙ A straight stretch of the Accra–Kumasi highway climbs 18 m over a horizontal distance of 400 m. Find the gradient as a percentage, and the angle of inclination in degrees.

16. ⚙ A cocoa farmer's plot near Bibiani is bounded by a straight stream along $x - 2y + 4 = 0$, with distances in hundreds of metres. The farmhouse is at $(5, 6)$. How far is the farmhouse from the stream, and where is the nearest point of the stream?

17. ⚙ Three towns lie at $A(0, 0)$, $B(30, 0)$ and $C(12, 24)$ (km). A district hospital is to be built at the *centroid* of triangle ABC, which divides each median in the ratio $2 : 1$ from the vertex. Use the section formula to locate the hospital, and check that it is the average of the three vertices.

C. Programming

18. Write a function `line_from_points(P, Q)` that returns the tuple (a, b, c) of the general form $ax + by + c = 0$, with integer coefficients, for the line through P and Q. Test it on Exercise 1(f).

19. Write a function `is_collinear(A, B, C)` that returns `True` when three points lie on one straight line. Use it to answer Exercise 9(b). (*Hint:* compare gradients, or test whether the area of triangle ABC is zero.)

20. Use `matplotlib` to draw the two lines of Example 1.11 together with both bisectors. Use `plt.gca().set_aspect("equal")`, then check by eye that the bisectors are perpendicular and that they bisect the angles.

21. Extend Listing 1.2 by adding Takoradi (4.90°N, 1.76°W) and Bolgatanga (10.79°N, 0.85°W). Print a complete distance table (a "matrix") between all pairs of towns.

22. A drone delivering medical supplies flies in straight lines from a depot at $(0, 0)$ to clinics at $(12, 5)$, $(-8, 15)$ and $(3, -9)$ (km), returning to the depot after each delivery. Write a program that finds the total distance flown. (Drone delivery of blood and medicines has operated in Ghana since 2019.)

Chapter 2

Circles

Learning Objectives

After working through this chapter you should be able to:

- write the equation of a circle in standard and general form, and move between them by completing the square;
- find the circle on a given diameter and the circle through three points;
- find tangents and normals to a circle;
- decide whether a line cuts, touches or misses a circle;
- describe how two circles meet, and find their common chord;
- use Python to draw circles, solve the resulting equations, and locate a phone by trilateration.

Definition 2.1. (Circle)

A *circle* is the locus (set) of all points $P(x, y)$ in a plane that are at a constant distance r, the *radius*, from a fixed point $C(a, b)$, the *centre*.

Historical Note: The circle and the number π

Every circle has the same ratio of circumference to diameter. This ratio is the number $\pi \approx 3.14159$. Problem 50 of the Egyptian *Rhind Papyrus* (c. 1550 BCE) finds the area of a round field of diameter 9 by squaring 8, which is $\frac{8}{9}$ of the diameter. This amounts to taking $\pi \approx \left(\frac{16}{9}\right)^2 \approx 3.1605$, within 1% of the true value. Around 250 BCE, Archimedes of Syracuse, who studied in Alexandria, trapped the circle between regular polygons with up to 96 sides. He proved that $\frac{223}{71} < \pi < \frac{22}{7}$. In the 2nd century CE, Claudius Ptolemy of Alexandria produced a table of chords of a circle, the forerunner of our trigonometric tables. Listing 2.9 at the end of this chapter repeats Archimedes' calculation in a few lines of Python.

2.1 The equation of a circle

Let $P(x, y)$ be any point on the circle with centre $C(a, b)$ and radius r. Then $|CP| = r$. By the distance formula (1.1),

$$\sqrt{(x-a)^2 + (y-b)^2} = r \quad \Longleftrightarrow \quad (x-a)^2 + (y-b)^2 = r^2.$$

Expanding gives $x^2 + y^2 - 2ax - 2by + (a^2 + b^2 - r^2) = 0$. Writing $g = -a$, $f = -b$ and $c = a^2 + b^2 - r^2$ gives the *general form*.

Key Concept: Equation of a circle

Standard form: centre (a, b), radius r:

$$(x - a)^2 + (y - b)^2 = r^2. \tag{2.1}$$

General form:

$$x^2 + y^2 + 2gx + 2fy + c = 0, \qquad \text{centre } (-g, -f), \quad r = \sqrt{g^2 + f^2 - c}. \tag{2.2}$$

The general form represents a real circle only when $g^2 + f^2 - c > 0$.

Key Concept: How to recognise the equation of a circle

A second-degree equation in x and y is a circle when

1. the coefficients of x^2 and y^2 are *equal* (divide through so that both are 1);
2. there is *no* xy term; and
3. $g^2 + f^2 - c > 0$.

Example 2.1 Find the centre and radius of the circle $x^2 + y^2 + 3x - 6y - 1 = 0$.

Solution. Comparing with (2.2): $2g = 3$, $2f = -6$ and $c = -1$, so $g = \frac{3}{2}$ and $f = -3$. The centre is $\left(-\frac{3}{2}, 3\right)$ and $r = \sqrt{\frac{9}{4} + 9 + 1} = \sqrt{\frac{49}{4}} = \frac{7}{2}$.

Alternatively, complete the square: $\left(x + \frac{3}{2}\right)^2 - \frac{9}{4} + (y-3)^2 - 9 - 1 = 0$, so $\left(x + \frac{3}{2}\right)^2 + (y-3)^2 = \frac{49}{4}$. ■

Watch Out: Signs of the centre

The centre of $x^2 + y^2 + 2gx + 2fy + c = 0$ is $(-g, -f)$, *with minus signs*. For example, the circle $x^2 + y^2 - 4x + 6y = 0$ has centre $(2, -3)$, not $(-2, 3)$. Also, if the coefficient of x^2 is not 1 (for example $3x^2 + 3y^2 - 5x - 7y + 2 = 0$), divide through *first*.

Listing 2.1 — Centre and radius from the general form `ex2_01_centre_radius.py`

```python
# Listing 2.1: Centre and radius from the general equation of a circle
import sympy as sp

def centre_radius(g2, f2, c):
    """For x^2 + y^2 + (2g)x + (2f)y + c = 0, given 2g, 2f and c."""
    g, f = sp.Rational(g2) / 2, sp.Rational(f2) / 2
    r2 = g**2 + f**2 - c
    if r2 < 0:
        return (-g, -f), None          # no real circle
    return (-g, -f), sp.sqrt(r2)

# Checking the worked example: x^2 + y^2 + 3x - 6y - 1 = 0
print(centre_radius(3, -6, -1))
```

```
14
15 # Exercise-style checks
16 for coeffs in [(6, -10, -9), (0, 4, 0), (2, 2, 10)]:
17     centre, r = centre_radius(*coeffs)
18     print(f"2g={coeffs[0]:>3}, 2f={coeffs[1]:>3}, c={coeffs[2]:>3}  ->  centre {centre},
       radius {r}")
19
20 # SymPy can complete the square for us too
21 x, y = sp.symbols("x y")
22 expr = x**2 + y**2 + 3*x - 6*y - 1
23 print(sp.expand((x + sp.Rational(3, 2))**2 + (y - 3)**2 - sp.Rational(49, 4)) == expr)
```

Output

```
((-3/2, 3), 7/2)
2g=  6, 2f=-10, c= -9  ->  centre (-3, 5), radius sqrt(43)
2g=  0, 2f=  4, c=  0  ->  centre (0, -2), radius 2
2g=  2, 2f=  2, c= 10  ->  centre (-1, -1), radius None
True
```

The third circle in the output, $x^2 + y^2 + 2x + 2y + 10 = 0$, gives $g^2 + f^2 - c = 1 + 1 - 10 < 0$. No real points satisfy it, so it is not a real circle.

2.1.1 Parametric form and plotting

Every point of the circle (2.1) can be written as

$$x = a + r\cos t, \qquad y = b + r\sin t, \qquad 0 \le t < 2\pi,$$

since then $(x-a)^2 + (y-b)^2 = r^2(\cos^2 t + \sin^2 t) = r^2$. This *parametric form* is the easiest way to draw a circle on a computer.

In Context: Lake Bosomtwe

Lake Bosomtwe, about 30 km south-east of Kumasi, is Ghana's only natural lake. It fills a crater made by a meteorite impact roughly one million years ago, and it is almost circular, about 8 km across. Modelling it as a circle of radius 4 km gives a surface area of about $\pi(4)^2 \approx 50\,\text{km}^2$, close to the measured value.

Listing 2.2 — Plotting circles parametrically `ex2_02_plot_circle.py`

```
1 # Listing 2.2: Drawing circles with the parametric form x = a + r cos t, y = b + r sin t
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 t = np.linspace(0, 2*np.pi, 300)
6
7 plt.figure(figsize=(5, 5))
8 # Lake Bosomtwe is roughly circular, about 8 km across (radius ~ 4 km).
9 a, b, r = 0, 0, 4
10 plt.fill(a + r*np.cos(t), b + r*np.sin(t), color="tab:blue", alpha=0.25,
11          label="Lake Bosomtwe (approx.), r = 4 km")
12
```

```python
# Circle of Example 2.1: centre (-3/2, 3), radius 7/2
a, b, r = -1.5, 3, 3.5
plt.plot(a + r*np.cos(t), b + r*np.sin(t), "r", label="x²+y²+3x−6y−1=0")
plt.plot(a, b, "r+")

plt.gca().set_aspect("equal")
plt.grid(alpha=0.3)
plt.legend(loc="lower right", fontsize=8)
plt.show()

print(f"Area of a circular lake of radius 4 km: {np.pi*4**2:.1f} km^2")
```

Output

```
Area of a circular lake of radius 4 km: 50.3 km^2
```

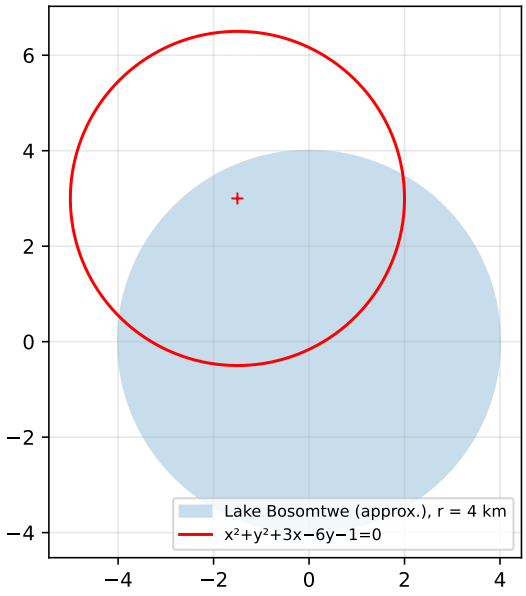


Figure 2.1: Circles drawn with $x = a + r\cos t,\ y = b + r\sin t$.

2.2 Special circles

2.2.1 A circle through the origin

If $x^2 + y^2 + 2gx + 2fy + c = 0$ passes through $O(0,0)$, then substituting gives $c = 0$. So a circle through the origin has the form $x^2 + y^2 + 2gx + 2fy = 0$.

2.2.2 The circle on a given diameter

Let $P_1(x_1, y_1)$ and $P_2(x_2, y_2)$ be the ends of a diameter, and let $P(x, y)$ be any other point on the circle. The angle in a semicircle is a right angle, so $PP_1 \perp PP_2$:

$$\frac{y - y_1}{x - x_1} \cdot \frac{y - y_2}{x - x_2} = -1.$$

Key Concept: Circle on a diameter

The circle with diameter P_1P_2 is

$$(x - x_1)(x - x_2) + (y - y_1)(y - y_2) = 0. \tag{2.3}$$

Equivalently, its centre is the midpoint of P_1P_2 and its radius is $\frac{1}{2}\,|P_1P_2|$.

Example 2.2 Find the equation of the circle on AB as diameter, where $A(10, 2)$ and $B(3, 8)$.

Solution. By (2.3), $(x - 10)(x - 3) + (y - 2)(y - 8) = 0$, that is, $x^2 + y^2 - 13x - 10y + 46 = 0$.

Check (method 2): the centre is the midpoint $\left(\frac{13}{2}, 5\right)$, and $r^2 = \left(\frac{13}{2} - 3\right)^2 + (5 - 8)^2 = \frac{49}{4} + 9 = \frac{85}{4}$. Expanding $\left(x - \frac{13}{2}\right)^2 + (y - 5)^2 = \frac{85}{4}$ gives the same equation. ■

Listing 2.3 — Circle on a diameter, two ways `ex2_03_diameter.py`

```python
# Listing 2.3: The circle on a given diameter - two methods
import sympy as sp

x, y = sp.symbols("x y")
A, B = (10, 2), (3, 8)

# Method 1: angle in a semicircle is 90 degrees -> (x-x1)(x-x2) + (y-y1)(y-y2) = 0
eq1 = sp.expand((x - A[0])*(x - B[0]) + (y - A[1])*(y - B[1]))
print("Method 1:", eq1, "= 0")

# Method 2: centre = midpoint, radius = half the diameter
C = sp.Point(A).midpoint(sp.Point(B))
r2 = C.distance(sp.Point(B))**2
eq2 = sp.expand((x - C.x)**2 + (y - C.y)**2 - r2)
print("Method 2:", eq2, "= 0   centre", C, " radius^2 =", r2)
print("Same circle?", sp.simplify(eq1 - eq2) == 0)
```

Output

```
Method 1: x**2 - 13*x + y**2 - 10*y + 46 = 0
Method 2: x**2 - 13*x + y**2 - 10*y + 46 = 0   centre Point2D(13/2, 5)  radius^2 = 85/4
Same circle? True
```

2.2.3 The circle through three points

Three points that do not lie on one line determine exactly one circle. To find it:

1. assume the equation $x^2 + y^2 + 2gx + 2fy + c = 0$;
2. substitute the three points in turn, giving three *linear* equations in g, f and c;
3. solve for g, f and c, and write down the circle.

Example 2.3 Find the equation of the circle through $(2, 1)$, $(0, 2)$ and $(1, 0)$.

Solution. Substituting the three points gives

$$(2,1): \quad 5 + 4g + 2f + c = 0, \tag{i}$$
$$(0,2): \quad 4 + 4f + c = 0, \tag{ii}$$
$$(1,0): \quad 1 + 2g + c = 0. \tag{iii}$$

Subtracting (iii) from (i) gives $4 + 2g + 2f = 0$, that is $g + f = -2$. Subtracting (iii) from (ii) gives $3 - 2g + 4f = 0$. Solving these two equations gives $f = -\frac{7}{6}$ and $g = -\frac{5}{6}$. Then (iii) gives $c = -1 - 2g = \frac{2}{3}$. The circle is

$$x^2 + y^2 - \tfrac{5}{3}x - \tfrac{7}{3}y + \tfrac{2}{3} = 0 \quad \Longleftrightarrow \quad 3x^2 + 3y^2 - 5x - 7y + 2 = 0.$$

■

The three equations in step 2 form a *linear system*, which a computer solves instantly. We write it in matrix form as

$$\begin{pmatrix} 2x_1 & 2y_1 & 1 \\ 2x_2 & 2y_2 & 1 \\ 2x_3 & 2y_3 & 1 \end{pmatrix} \begin{pmatrix} g \\ f \\ c \end{pmatrix} = - \begin{pmatrix} x_1^2 + y_1^2 \\ x_2^2 + y_2^2 \\ x_3^2 + y_3^2 \end{pmatrix}$$

and call `np.linalg.solve`.

In Context: Where should the mast go?

A telecom company wants to place a mast so that three towns, at $A(0,0)$, $B(10,2)$ and $C(4,8)$ km, are all the same distance from it. That point is the centre of the circle through A, B and C, called the *circumcentre* of triangle ABC. Listing 2.4 finds it at about (4.67, 2.67), with every town 5.38 km away. This is the least coverage radius that reaches all three towns from a single point equidistant from them.

Listing 2.4 — The circle through three points `ex2_04_three_points.py`

```python
# Listing 2.4: The circle through three points - a linear system for g, f, c
import numpy as np
import sympy as sp

def circle_through(P1, P2, P3):
    """Solve x^2+y^2+2gx+2fy+c=0 through three points. Returns (g, f, c)."""
    M, rhs = [], []
    for (x, y) in (P1, P2, P3):
        M.append([2*x, 2*y, 1])          # unknowns g, f, c
        rhs.append(-(x**2 + y**2))
    return np.linalg.solve(np.array(M, float), np.array(rhs, float))

# Checking the worked example: (2,1), (0,2), (1,0)
g, f, c = circle_through((2, 1), (0, 2), (1, 0))
print(f"g = {g:.4f}, f = {f:.4f}, c = {c:.4f}")

# The same, exactly, with SymPy
G, F, Cc, x, y = sp.symbols("g f c x y")
circle = x**2 + y**2 + 2*G*x + 2*F*y + Cc
sol = sp.solve([circle.subs({x: px, y: py}) for px, py in [(2, 1), (0, 2), (1, 0)]], [G, F,
      Cc])
print("Exact:", sol, " ->", sp.expand(3*circle.subs(sol)), "= 0")
```

```python

# A telecom mast equidistant from three towns A(0,0), B(10,2), C(4,8)  (km)
g, f, c = circle_through((0, 0), (10, 2), (4, 8))
centre = (-g, -f)
radius = np.sqrt(g**2 + f**2 - c)
print(f"Mast at ({centre[0]:.3f}, {centre[1]:.3f}); each town is {radius:.3f} km away")
```

Output

```
g = -0.8333, f = -1.1667, c = 0.6667
Exact: {c: 2/3, f: -7/6, g: -5/6}  -> 3*x**2 - 5*x + 3*y**2 - 7*y + 2 = 0
Mast at (4.667, 2.667); each town is 5.375 km away
```

Check Your Understanding

1. Find the circle through $(0,6)$, $(0,0)$ and $(8,0)$. (*Hint:* because of the right angle at the origin, the hypotenuse is a diameter.)
2. Find the circle that passes through the origin and has intercepts 1 and 2 on the x- and y-axes.

2.3 Tangents and normals to a circle

A *tangent* to a circle touches it at exactly one point, and is perpendicular to the radius at that point. The *normal* at a point is the line through the point and the centre.

Key Concept: Tangent at a point of a circle

The tangent to $x^2+y^2+2gx+2fy+c=0$ at the point (x_1,y_1) on the circle is

$$xx_1+yy_1+g(x+x_1)+f(y+y_1)+c=0.$$

(*Rule:* replace x^2 by xx_1, y^2 by yy_1, $2x$ by $x+x_1$ and $2y$ by $y+y_1$.) For the circle $x^2+y^2=r^2$ this gives $xx_1+yy_1=r^2$.

Example 2.4 Find the tangent and normal to $x^2+y^2-4x+2y=0$ at the origin.

Solution. The centre is $(2,-1)$, so the radius to O has gradient $-\tfrac12$. The tangent has gradient 2: $y=2x$. The normal is the line through O and the centre: $y=-\tfrac12 x$, that is, $x+2y=0$. The rule gives the same tangent: $0+0-2(x+0)+1(y+0)+0=0$, so $y=2x$. ∎

Listing 2.5 — Tangent and normal to a circle ex2_05_tangent.py

```python
# Listing 2.5: Tangent and normal to a circle at a point
import sympy as sp

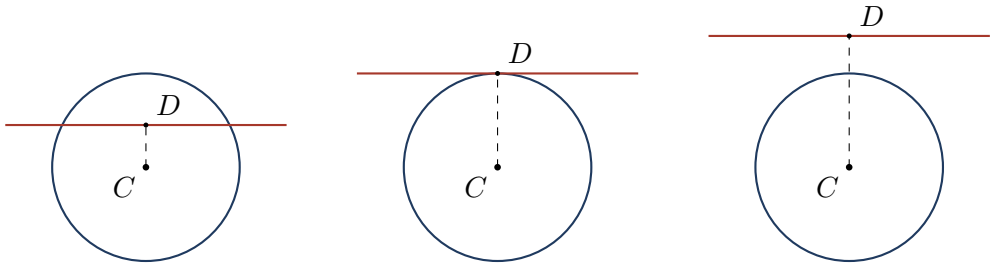
x, y = sp.symbols("x y")
S = x**2 + y**2 - 4*x + 2*y            # circle x^2 + y^2 - 4x + 2y = 0
P = (0, 0)
print("P on circle?", S.subs({x: P[0], y: P[1]}) == 0)
```

```
8
9  # The normal at P passes through the centre (-g, -f) = (2, -1)
10 centre = (2, -1)
11 m_radius = sp.Rational(centre[1] - P[1], centre[0] - P[0])
12 m_tangent = -1 / m_radius
13 tangent = sp.expand(y - P[1] - m_tangent*(x - P[0]))
14 normal = sp.expand(y - P[1] - m_radius*(x - P[0]))
15 print("gradient of radius =", m_radius, ",  gradient of tangent =", m_tangent)
16 print("tangent:", tangent, "= 0")
17 print("normal :", normal, "= 0")
18
19 # The 'replace one x by x1' rule: x x1 + y y1 + g(x + x1) + f(y + y1) + c = 0
20 g, f, c = -2, 1, 0
21 rule = sp.expand(x*P[0] + y*P[1] + g*(x + P[0]) + f*(y + P[1]) + c)
22 print("by the rule:", rule, "= 0")
```

Output

```
P on circle? True
gradient of radius = -1/2 ,  gradient of tangent = 2
tangent: -2*x + y = 0
normal : x/2 + y = 0
by the rule: -2*x + y = 0
```

2.4 Intersection of a line and a circle



$|CD| < r$: two points $\quad$ $|CD| = r$: tangent $\quad$ $|CD| > r$: no point

Let D be the foot of the perpendicular from the centre C to a line. Geometrically, the line cuts the circle in two points, touches it, or misses it, according as $|CD| < r$, $|CD| = r$ or $|CD| > r$. The distance $|CD|$ can be found with Theorem 1.1.

Algebraically, substitute $y = mx + k$ into $x^2 + y^2 + 2gx + 2fy + c = 0$:

$$(1 + m^2)x^2 + 2(g + mf + mk)x + (k^2 + 2fk + c) = 0.$$

This is a quadratic in x. Its discriminant $\Delta = B^2 - 4AC$ decides the case.

Key Concept: Line and circle

$\Delta > 0$	two distinct points of intersection	(the line is a *secant*)
$\Delta = 0$	one repeated point	(the line is a *tangent*)
$\Delta < 0$	no real point of intersection	(the line misses the circle)

Example 2.5 Prove that $y = mx + c$ is a tangent to $x^2 + y^2 = 25$ if and only if $c^2 = 25(1 + m^2)$.

Solution. Substituting gives $x^2 + (mx + c)^2 = 25$, that is, $(1 + m^2)x^2 + 2mcx + (c^2 - 25) = 0$. The line is a tangent exactly when the discriminant is zero:

$$(2mc)^2 - 4(1 + m^2)(c^2 - 25) = 0 \iff m^2c^2 - c^2 + 25 - m^2c^2 + 25m^2 = 0 \iff c^2 = 25(1 + m^2).$$

■

Example 2.6 (Past examination question) Find the points of intersection of the line $x - 2y - 1 = 0$ and the circle $x^2 + y^2 + 4x - 2y - 5 = 0$.

Solution. From the line, $x = 2y + 1$. Substituting gives $(2y + 1)^2 + y^2 + 4(2y + 1) - 2y - 5 = 0$, that is, $5y^2 + 10y = 0$, so $y = 0$ or $y = -2$. The points are $(1, 0)$ and $(-3, -2)$. ■

In Context: Is the road covered?

A mobile-network mast at the origin covers a disc of radius 5 km. Three straight roads pass nearby. Listing 2.6 uses the discriminant to decide which roads enter the coverage area. For a road that does, it finds how many kilometres of that road have signal (the length of the *chord*).

Listing 2.6 — A road and a coverage circle `ex2_06_line_circle.py`

```python
# Listing 2.6: Where does a road cross a mast's coverage circle?
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

x = sp.symbols("x")
R = 5                                   # coverage radius (km), mast at the origin

def crossing(m, c):
    """Intersect y = m x + c with x^2 + y^2 = R^2; classify with the discriminant."""
    quad = sp.expand(x**2 + (m*x + c)**2 - R**2)
    A, B, C = [quad.coeff(x, k) for k in (2, 1, 0)]
    disc = B**2 - 4*A*C
    xs = sp.solve(quad, x)
    pts = [(float(xi), float(m*xi + c)) for xi in xs if xi.is_real]
    kind = "2 points" if disc > 0 else ("tangent" if disc == 0 else "misses")
    return kind, disc, pts

roads = {"Road 1: y = x/2 + 1": (sp.Rational(1, 2), 1),
         "Road 2: y = 5":       (0, 5),
         "Road 3: y = -x + 9":  (-1, 9)}
plt.figure(figsize=(5, 5))
t = np.linspace(0, 2*np.pi, 300)
plt.fill(R*np.cos(t), R*np.sin(t), alpha=0.15, label="coverage")
xx = np.linspace(-7, 9, 50)
for name, (m, c) in roads.items():
    kind, disc, pts = crossing(m, c)
    print(f"{name:<20} discriminant = {float(disc):8.2f}  -> {kind}", [tuple(round(v, 3)
     for v in p) for p in pts])
    if len(pts) == 2:
        L = np.hypot(pts[0][0] - pts[1][0], pts[0][1] - pts[1][1])
        print(f"{'':20} length of road inside coverage = {L:.3f} km")
```

```
    plt.plot(xx, float(m)*xx + c, label=name)
plt.plot(0, 0, "k^", ms=10)
plt.xlim(-7, 9); plt.ylim(-7, 9)
plt.gca().set_aspect("equal"); plt.grid(alpha=0.3); plt.legend(fontsize=7, loc="lower left")
plt.show()

# Exam question (2021): x - 2y - 1 = 0 and x^2 + y^2 + 4x - 2y - 5 = 0
X, Y = sp.symbols("X Y")
print(sp.solve([X - 2*Y - 1, X**2 + Y**2 + 4*X - 2*Y - 5], [X, Y]))
```

Output

```
Road 1: y = x/2 + 1  discriminant =   121.00  -> 2 points [(-4.8, -1.4), (4.0, 3.0)]
                     length of road inside coverage = 9.839 km
Road 2: y = 5        discriminant =     0.00  -> tangent [(0.0, 5.0)]
Road 3: y = -x + 9   discriminant =  -124.00  -> misses []
[(-3, -2), (1, 0)]
```

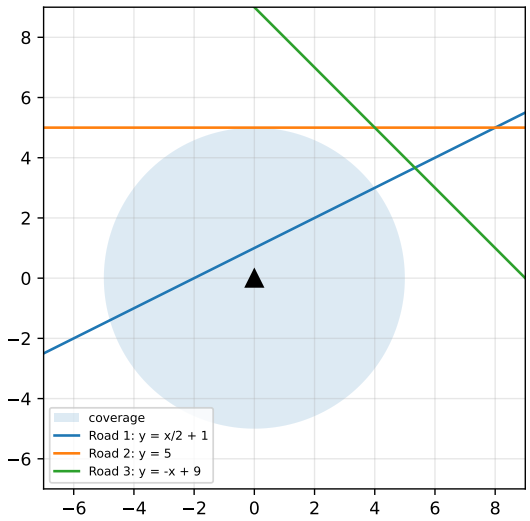


Figure 2.2: Road 1 is a secant, Road 2 a tangent, and Road 3 misses the coverage disc.

2.5 Two circles

Let two circles have centres C_1 and C_2, radii $r_1 \geq r_2$, and let $d = |C_1C_2|$ be the distance between their centres.

Key Concept: Relative positions of two circles

$d > r_1 + r_2$	separate: no common point
$d = r_1 + r_2$	touch externally (one common point)
$r_1 - r_2 < d < r_1 + r_2$	intersect at two distinct points
$d = r_1 - r_2$	touch internally (one common point)
$d < r_1 - r_2$	one circle lies inside the other
$d^2 = r_1^2 + r_2^2$	the circles cut *orthogonally* (their tangents are perpendicular where they meet)

2.5.1 The common chord

Let

$$S_1 : \; x^2 + y^2 + 2g_1x + 2f_1y + c_1 = 0, \qquad S_2 : \; x^2 + y^2 + 2g_2x + 2f_2y + c_2 = 0.$$

Any point on both circles satisfies $S_1 = 0$ and $S_2 = 0$, and hence $S_1 - S_2 = 0$. The squared terms cancel, so this is a straight line.

Key Concept: Common chord

If two circles meet at two points, the line through those points (the *common chord*) is

$$S_1 - S_2 = 0 : \qquad 2(g_1 - g_2)x + 2(f_1 - f_2)y + (c_1 - c_2) = 0.$$

In Context: Overlapping coverage

Mast A at $(0, 0)$ covers radius 5 km, and mast B at $(6, 2)$ covers radius 4 km. Since $d = \sqrt{40} \approx 6.32$ lies between $5 - 4 = 1$ and $5 + 4 = 9$, the coverage areas overlap. The common chord, $12x + 4y - 49 = 0$, marks where the signals from the two masts are received at the edge of both coverage areas. Network engineers use it when deciding where a phone should "hand over" from one mast to the other.

Listing 2.7 — Two circles and their common chord `ex2_07_two_circles.py`

```python
# Listing 2.7: Two coverage circles - how do they meet, and where is the common chord?
import math
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

def relative_position(c1, r1, c2, r2):
    d = math.dist(c1, c2)
    if d > r1 + r2:            return d, "separate (no common point)"
    if d == r1 + r2:           return d, "touch externally"
    if d == abs(r1 - r2):      return d, "touch internally"
    if d < abs(r1 - r2):       return d, "one inside the other"
    return d, "intersect at two points"

print(relative_position((0, 0), 5, (6, 2), 4))
print(relative_position((0, 0), 5, (8, 6), 5))
print(relative_position((0, 0), 5, (1, 1), 2))

```

```
# Common chord: subtract the two equations
x, y = sp.symbols("x y")
S1 = x**2 + y**2 - 25                        # mast A at (0,0), radius 5 km
S2 = (x - 6)**2 + (y - 2)**2 - 16            # mast B at (6,2), radius 4 km
chord = sp.expand(S1 - S2)
print("Common chord:", chord, "= 0")
pts = sp.solve([S1, S2], [x, y])
pts = [(float(a), float(b)) for a, b in pts]
print("Intersection points:", [tuple(round(v, 4) for v in p) for p in pts])

t = np.linspace(0, 2*np.pi, 300)
plt.figure(figsize=(5.5, 4.5))
plt.fill(5*np.cos(t), 5*np.sin(t), alpha=0.2, label="mast A")
plt.fill(6 + 4*np.cos(t), 2 + 4*np.sin(t), alpha=0.2, label="mast B")
plt.plot(*zip(*pts), "k-o", label="common chord")
plt.gca().set_aspect("equal"); plt.grid(alpha=0.3); plt.legend(fontsize=8)
plt.show()
```

Output

```
(6.324555320336759, 'intersect at two points')
(10.0, 'touch externally')
(1.4142135623730951, 'one inside the other')
Common chord: 12*x + 4*y - 49 = 0
Intersection points: [(2.6753, 4.2241), (4.6747, -1.7741)]
```

In Context: How does your phone know where it is?

GPS receivers, and mobile networks that locate phones, use *trilateration*. If you know that you are d_1 from mast 1, you lie on a circle around it. Two masts give two circles, which meet in up to two points. A third mast decides between them. Subtracting the circle equations in pairs gives two common-chord *lines*, and these lines meet at the phone. Listing 2.8 carries this out. (Real GPS works in three dimensions with spheres, but the idea is the same.)

Listing 2.8 — Trilateration `ex2_08_trilateration.py`

```
# Listing 2.8: Locating a phone from its distances to three masts (trilateration)
import numpy as np

masts = np.array([[0.0, 0.0], [10.0, 0.0], [4.0, 9.0]])   # km
d = np.array([5.000, 8.062, 5.099])                        # measured distances, km

# Circle i:  x^2 + y^2 - 2 a_i x - 2 b_i y + (a_i^2 + b_i^2 - d_i^2) = 0
# Subtracting circle 1 from circles 2 and 3 removes x^2 + y^2 and leaves two LINES
# (the common chords), whose intersection is the phone.
(a1, b1), (a2, b2), (a3, b3) = masts
M = np.array([[2*(a2 - a1), 2*(b2 - b1)],
              [2*(a3 - a1), 2*(b3 - b1)]])
rhs = np.array([d[0]**2 - d[1]**2 + a2**2 - a1**2 + b2**2 - b1**2,
                d[0]**2 - d[2]**2 + a3**2 - a1**2 + b3**2 - b1**2])
phone = np.linalg.solve(M, rhs)
```

```
print("Estimated position of the phone:", np.round(phone, 3))
print("Check distances:", np.round(np.linalg.norm(masts - phone, axis=1), 3))
```

Output

```
Estimated position of the phone: [3. 4.]
Check distances: [5.    8.062 5.099]
```

Listing 2.9 — Approximating π like Archimedes `ex2_09_archimedes_pi.py`

```
# Listing 2.9: From the Rhind papyrus to Archimedes - approximating pi
import math

print(f"Rhind papyrus (c. 1550 BCE): (16/9)^2 = {(16/9)**2:.6f}")
print(f"Archimedes' bounds: 223/71 = {223/71:.6f} < pi < 22/7 = {22/7:.6f}")

# Archimedes: perimeters of regular polygons inside and outside a circle of diameter 1.
# Start with hexagons and keep doubling the number of sides.
n = 6
inner = 3.0                       # inscribed hexagon: perimeter 6 * (1/2)
outer = 2 * math.sqrt(3)          # circumscribed hexagon
print(f"{'sides':>6} {'inscribed':>12} {'circumscribed':>14}")
for _ in range(5):
    print(f"{n:>6} {inner:>12.6f} {outer:>14.6f}")
    outer = 2 * inner * outer / (inner + outer)   # harmonic mean
    inner = math.sqrt(inner * outer)              # geometric mean
    n *= 2
print(f"{n:>6} {inner:>12.6f} {outer:>14.6f}")
print(f"{'pi':>6} {math.pi:>12.6f}")
```

Output

```
Rhind papyrus (c. 1550 BCE): (16/9)^2 = 3.160494
Archimedes' bounds: 223/71 = 3.140845 < pi < 22/7 = 3.142857
 sides    inscribed  circumscribed
     6     3.000000       3.464102
    12     3.105829       3.215390
    24     3.132629       3.159660
    48     3.139350       3.146086
    96     3.141032       3.142715
   192     3.141452       3.141873
    pi     3.141593
```

Chapter Summary

- $(x-a)^2+(y-b)^2=r^2$, or $x^2+y^2+2gx+2fy+c=0$ with centre $(-g,-f)$ and $r=\sqrt{g^2+f^2-c}$.
- Circle on a diameter: $(x-x_1)(x-x_2)+(y-y_1)(y-y_2)=0$.
- Circle through three points: solve a linear system for g, f and c.

- The tangent is perpendicular to the radius. Tangent at (x_1, y_1): $xx_1 + yy_1 + g(x + x_1) + f(y + y_1) + c = 0$.
- Line and circle: substitute, then use the discriminant (or compare the distance from the centre with r).
- Two circles: compare d with $r_1 + r_2$ and $|r_1 - r_2|$. Common chord: $S_1 - S_2 = 0$.

2.6 Exercises

A. Skills

1. A circle has centre $(1, 2)$ and radius $\sqrt{5}$. (a) Find the perpendicular distance from its centre to the line $x + 2y - 10 = 0$, and hence show that this line is a tangent to the circle. (b) Show similarly that the line $x + 2y - 12 = 0$ does not meet the circle.

2. For what values of k is the point $(k, 2k)$ on the circle $x^2 + y^2 = 5$?

3. Is the point $(3, 5)$ inside, outside or on the circle $x^2 + y^2 = 9$?

4. For what value of k is the line $x = 6$ a tangent to the circle $x^2 + y^2 = k$?

5. Find the equation of the circle through the origin with intercepts 1 and 2 on the x- and y-axes respectively.

6. Find the equation of the tangent at $(0, 2)$ to the circle $x^2 + y^2 - 4x + 2y = 8$.

7. Find the equation of the circle through $(0, 6)$, $(0, 0)$ and $(8, 0)$.

8. Find the points of intersection of $x^2 + y^2 = 4$ and $(x - 2)^2 + (y - 2)^2 = 4$.

9. Find the equation of the circle with a diameter whose ends are $(-6, 1)$ and $(2, -5)$.

10. Find the centre and radius of each circle: (a) $x^2 + y^2 + 6x - 10y = 9$; (b) $x^2 + y^2 + 4y = 0$; (c) $-x^2 - y^2 + 8x = 0$;
(d) $(-4 - x)^2 + (-y + 11)^2 = 9$; (e) $(5 - x)^2 + (y - 1)^2 = 4$.

11. Find the equation of the circle with centre $(1, 3)$ that has the line $3x + 4y + 10 = 0$ as a tangent. (*Past examination question.*)

12. Find the equation of the normal to $x^2 + y^2 - 4x + 6y - 104 = 0$ at $(-4, 6)$. (*Past examination question.*)

13. Show that the circles $x^2 + y^2 - 2x - 4y = 0$ and $x^2 + y^2 + 4x + 2y - 20 = 0$ intersect, and find their common chord and its length.

14. Show that the circles $x^2 + y^2 = 9$ and $x^2 + y^2 - 10x + 9 = 0$ cut orthogonally.

B. Applications

15. A circular roundabout is to be built at a junction. On the engineers' plan (in metres) its kerb passes through the points $(0, 0)$, $(40, 0)$ and $(10, 30)$. Find the centre and radius of the roundabout.

16. An FM radio station in Cape Coast has a transmission radius of 60 km. A straight section of road is modelled by $y = 0.5x + 40$, with the transmitter at the origin and distances in km. For how many kilometres of this road can the station be received?

17. A circular cassava farm has the equation $x^2 + y^2 - 60x - 80y + 2100 = 0$ (metres). Find its centre, its radius and its area in hectares ($1\,\text{ha} = 10\,000\,\text{m}^2$).

18. A sprinkler at $(3, 4)$ waters a circle of radius 5 m, and a second sprinkler at $(10, 4)$ waters a circle of radius 4 m. Do the watered regions overlap? If so, find the common chord.

C. Programming

19. Write a function `circle_info(A, B, D, E, F)` for the equation $Ax^2 + By^2 + Dx + Ey + F = 0$. It should return the centre and radius, or print a message when the equation is not a real circle (for example when $A \neq B$, or when the "radius squared" is negative).

20. Write a program that takes three points, tests whether they are collinear, and if not, draws the triangle together with its circumcircle.

21. Modify Listing 2.8 so that the measured distances contain small random errors (use `np.random.normal(0, 0.05, 3)`). Repeat the location calculation 1000 times and plot all the estimated positions. How large is the scatter?

22. Use Listing 2.9 to find how many sides the polygons need before the two bounds agree with π to 8 decimal places.

Chapter 3

Limits and Continuity

Learning Objectives

After working through this chapter you should be able to:

- explain in words, and with a table of values, what $\lim_{x \to a} f(x) = L$ means;
- evaluate limits by substitution, factorising, rationalising and the laws of limits;
- use $\lim_{x \to 0} \frac{\sin x}{x} = 1$ to evaluate trigonometric limits;
- find one-sided limits, limits at infinity and infinite limits;
- apply the squeeze theorem;
- decide whether a function is continuous, and use the Intermediate Value Theorem;
- explore limits numerically and symbolically in Python.

The limit describes what happens to the values $f(x)$ of a function as x gets closer and closer to a number a. This is different from $f(a)$, the value *at* a, which may not even exist. Limits are the foundation of calculus. The derivative (Chapter 4) and the integral (Chapter 6) are both defined as limits.

Historical Note: Two thousand years to define a limit

Around 450 BCE the Greek philosopher Zeno of Elea argued that swift Achilles can never overtake a tortoise. Each time Achilles reaches the place where the tortoise was, the tortoise has moved a little further on. The puzzle turns on adding up infinitely many ever-smaller distances, which is a limit. Archimedes later used the "method of exhaustion", filling a shape with more and more triangles, to compute areas and the value of π (Listing 2.9).
Newton and Leibniz built calculus in the 1660s–1680s on "infinitely small" quantities, but they could not say precisely what these were. In 1734 Bishop George Berkeley mocked them as "the ghosts of departed quantities". The precise definition came in the 19th century. Bernard Bolzano (1817) and Augustin-Louis Cauchy, in his *Cours d'analyse* (1821), defined limits and continuity carefully. Karl Weierstrass in the 1860s gave the ε–δ definition used today (Section 3.8).

3.1 The idea of a limit

Definition 3.1. (Limit (informal))

We write

$$\lim_{x \to a} f(x) = L$$

and say "the limit of $f(x)$ as x approaches a is L" if $f(x)$ can be made as close as we like to L by taking x sufficiently close to a, *but not equal to* a.

The function $f(x) = \dfrac{x^2 - 16}{x - 4}$ is not defined at $x = 4$, because it gives $\frac{0}{0}$ there. However, we can compute $f(x)$ for x *near* 4. Listing 3.1 also tabulates $g(x) = \frac{\sin x}{x}$ near 0.

Listing 3.1 — Guessing a limit from a table `ex3_01_numerical_limit.py`

```python
# Listing 3.1: Guessing a limit from a table of values
import math

def f(x):
    return (x**2 - 16) / (x - 4)      # undefined at x = 4

def g(x):
    return math.sin(x) / x            # undefined at x = 0 (x in radians)

print(f"{'h':>10} {'f(4-h)':>12} {'f(4+h)':>12} {'g(-h)':>12} {'g(h)':>12}")
for k in range(1, 7):
    h = 10**(-k)
    print(f"{h:>10.0e} {f(4-h):>12.6f} {f(4+h):>12.6f} {g(-h):>12.8f} {g(h):>12.8f}")
```

Output

```
         h       f(4-h)       f(4+h)         g(-h)         g(h)
     1e-01     7.900000     8.100000   0.99833417   0.99833417
     1e-02     7.990000     8.010000   0.99998333   0.99998333
     1e-03     7.999000     8.001000   0.99999983   0.99999983
     1e-04     7.999900     8.000100   1.00000000   1.00000000
     1e-05     7.999990     8.000010   1.00000000   1.00000000
     1e-06     7.999999     8.000001   1.00000000   1.00000000
```

The table suggests $\lim_{x\to 4} \frac{x^2-16}{x-4} = 8$ and $\lim_{x\to 0} \frac{\sin x}{x} = 1$. A table is evidence, not proof, so next we learn to compute limits exactly.

In Context: What does a speedometer measure?

A Bolt car pulls away from the traffic lights at Kotokuraba. After t seconds it has travelled $s(t) = 1.5t^2$ metres. Between $t = 4$ and $t = 4 + h$, its *average* speed is $\dfrac{s(4+h) - s(4)}{h}$. What does the speedometer read at the *instant* $t = 4$? It reads the *limit* of the average speed as $h \to 0$. Listing 3.2 shows the averages closing in on 12 m/s (43.2 km/h). In Chapter 4 this limit becomes the *derivative*.

Listing 3.2 — Average speed to instantaneous speed ex3_02_average_speed.py

```python
# Listing 3.2: From average speed to instantaneous speed
# A Bolt car pulls away from the traffic lights at Kotokuraba.
# Distance travelled (metres) after t seconds:  s(t) = 1.5 t^2   (0 <= t <= 10)

def s(t):
    return 1.5 * t**2

t0 = 4.0
print(f"{'interval':>22} {'average speed (m/s)':>20}")
for h in [1, 0.5, 0.1, 0.01, 0.001, 0.0001]:
    avg = (s(t0 + h) - s(t0)) / h
    print(f"[{t0}, {t0 + h:<9}] {avg:>20.6f}")
print("The averages approach 12 m/s =", 12 * 3.6, "km/h: the speedometer reading at t = 4
    s.")
```

Output

```
              interval  average speed (m/s)
[4.0, 5.0      ]            13.500000
[4.0, 4.5      ]            12.750000
[4.0, 4.1      ]            12.150000
[4.0, 4.01     ]            12.015000
[4.0, 4.001    ]            12.001500
[4.0, 4.0001   ]            12.000150
The averages approach 12 m/s = 43.2 km/h: the speedometer reading at t = 4 s.
```

3.2 Evaluating limits

3.2.1 The laws of limits

Theorem 3.1. (Laws of limits)

Suppose $\lim_{x\to a} f(x) = L$ and $\lim_{x\to a} g(x) = M$ both exist, and let c be a constant. Then

(i) $\lim_{x\to a}[f(x) \pm g(x)] = L \pm M$ (sum/difference law);

(ii) $\lim_{x\to a}[cf(x)] = cL$ (constant multiple law);

(iii) $\lim_{x\to a}[f(x)g(x)] = LM$ (product law);

(iv) $\lim_{x\to a} \dfrac{f(x)}{g(x)} = \dfrac{L}{M}$, provided $M \neq 0$ (quotient law);

(v) $\lim_{x\to a}[f(x)]^{1/n} = L^{1/n}$ for positive integers n (with $L > 0$ when n is even) (root law).

A consequence of these laws is the most useful rule of all.

Key Concept: Direct substitution

If f is a polynomial, or a rational function with a in its domain, then

$$\lim_{x\to a} f(x) = f(a).$$

The same is true for sin, cos, e^x, $\ln x$ and $\sqrt{x}$ at every point of their domains. (These functions are *continuous*: see Section 3.7.)

Example 3.1 $\lim_{x\to 1}(2x^2+1) = 2(1)^2+1 = 3.$

Example 3.2 $\lim_{x\to 3}\dfrac{x^2-2x+1}{3x-2} = \dfrac{9-6+1}{9-2} = \dfrac{4}{7}$. Here 3 *is* in the domain (the denominator is $7 \neq 0$), so direct substitution works.

3.2.2 The indeterminate form $\frac{0}{0}$

If substitution gives $\frac{0}{0}$, the limit may still exist, but we must do some algebra first. The usual strategies are to **factorise and cancel**, or to **rationalise** by multiplying by a conjugate.

Example 3.3 $\lim_{x\to 4}\dfrac{x^2-16}{x-4} = \lim_{x\to 4}\dfrac{(x-4)(x+4)}{x-4} = \lim_{x\to 4}(x+4) = 8$. Cancelling $x-4$ is allowed because $x \neq 4$ while $x \to 4$.

Example 3.4 Evaluate $\lim_{x\to 0}\dfrac{\sqrt{x+9}-3}{x}$.

Solution. Substitution gives $\frac{0}{0}$. Multiply above and below by the conjugate $\sqrt{x+9}+3$:

$$\frac{\sqrt{x+9}-3}{x}\cdot\frac{\sqrt{x+9}+3}{\sqrt{x+9}+3} = \frac{(x+9)-9}{x(\sqrt{x+9}+3)} = \frac{1}{\sqrt{x+9}+3} \to \frac{1}{6}.$$

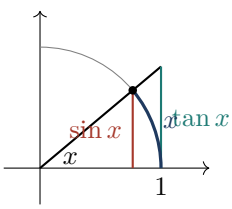
■

Watch Out: $\frac{0}{0}$ is not an answer

"$\frac{0}{0}$" does not mean the limit is 0, or 1, or that it does not exist. It means *more work is needed.* In contrast, $\frac{5}{0}$ (a non-zero number over zero) means the function grows without bound near a. Check the one-sided behaviour (Section 3.5).

3.3 Limits of trigonometric functions

Calculus with trigonometric functions always measures angles in **radians**. The radian measure of an angle is the length of the arc it cuts off on the *unit circle* (radius 1). So $360° = 2\pi$ radians and $1° = \frac{\pi}{180}$ radians.



For $0 < x < \frac{\pi}{2}$, the figure shows $\sin x < x < \tan x$. Dividing by $\sin x$ gives $1 < \frac{x}{\sin x} < \frac{1}{\cos x}$. Taking reciprocals, $\cos x < \frac{\sin x}{x} < 1$. As $x \to 0$, $\cos x \to 1$, so $\frac{\sin x}{x}$ is squeezed to 1 (see Section 3.6).

Theorem 3.2. (The fundamental trigonometric limit)

If x is measured in radians, then

$$\lim_{x\to 0} \frac{\sin x}{x} = 1 \qquad \text{and} \qquad \lim_{x\to 0} \frac{1-\cos x}{x} = 0.$$

Example 3.5 $\lim_{x\to 0} \frac{\tan x}{x} = \lim_{x\to 0} \frac{\sin x}{x} \cdot \frac{1}{\cos x} = 1 \cdot \frac{1}{1} = 1.$

Example 3.6 $\lim_{x\to 0} \frac{\sin 4x}{x} = \lim_{x\to 0} 4 \cdot \frac{\sin 4x}{4x} = 4(1) = 4$, putting $u = 4x \to 0$.

Example 3.7 Evaluate $\lim_{x\to 0} \frac{\cos 3x - \cos x}{x^2}$.

Solution. By the sum-to-product formula, $\cos 3x - \cos x = -2\sin\frac{3x+x}{2}\sin\frac{3x-x}{2} = -2\sin 2x \sin x$. Hence

$$\lim_{x\to 0} \frac{-2\sin 2x \sin x}{x^2} = -2 \lim_{x\to 0} \frac{\sin 2x}{x} \cdot \lim_{x\to 0} \frac{\sin x}{x} = -2(2)(1) = -4.$$

■

Key Concept: A useful pattern

For any constant $k \neq 0$: $\lim_{x\to 0} \frac{\sin kx}{x} = k$ and $\lim_{x\to 0} \frac{\sin ax}{\sin bx} = \frac{a}{b}$ $(b \neq 0)$.

Listing 3.3 — Checking limits with SymPy ex3_03_sympy_limits.py

```python
# Listing 3.3: Checking the worked examples with sp.limit
import sympy as sp

x, t = sp.symbols("x t")
examples = [
    ("2x^2 + 1, x->1",              2*x**2 + 1,                      x, 1),
    ("(x^2-2x+1)/(3x-2), x->3",     (x**2 - 2*x + 1)/(3*x - 2),      x, 3),
    ("(x^2-16)/(x-4), x->4",        (x**2 - 16)/(x - 4),             x, 4),
    ("(sqrt(x+9)-3)/x, x->0",       (sp.sqrt(x + 9) - 3)/x,          x, 0),
    ("tan(x)/x, x->0",              sp.tan(x)/x,                     x, 0),
    ("sin(4x)/x, x->0",             sp.sin(4*x)/x,                   x, 0),
    ("(cos3x - cos x)/x^2, x->0",   (sp.cos(3*x) - sp.cos(x))/x**2,  x, 0),
    ("(1 - cos x)/x^2, x->0",       (1 - sp.cos(x))/x**2,            x, 0),
]
for name, expr, var, a in examples:
    print(f"{name:<28} = {sp.limit(expr, var, a)}")

# One-sided limits
expr = sp.Abs(x - 8)/(x - 8)
print("|x-8|/(x-8): left =", sp.limit(expr, x, 8, dir="-"), ", right =", sp.limit(expr, x,
      8, dir="+"))
```

Output

```
2x^2 + 1, x->1               = 3
(x^2-2x+1)/(3x-2), x->3      = 4/7
(x^2-16)/(x-4), x->4         = 8
(sqrt(x+9)-3)/x, x->0        = 1/6
tan(x)/x, x->0               = 1
sin(4x)/x, x->0              = 4
(cos3x - cos x)/x^2, x->0    = -4
(1 - cos x)/x^2, x->0        = 1/2
|x-8|/(x-8): left = -1 , right = 1
```

Listing 3.4 — $\sin x/x$ near 0, in radians and degrees `ex3_04_sinx_over_x.py`

```python
# Listing 3.4: sin(x)/x near 0 - and why radians matter
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-10, 10, 1000)
x = x[x != 0]                                # avoid dividing by zero
plt.figure(figsize=(6, 3.2))
plt.plot(x, np.sin(x)/x, label="sin(x)/x")
plt.plot(0, 1, "o", mfc="white", mec="C0")   # the 'hole' at x = 0
plt.axhline(0, color="k", lw=0.5)
plt.grid(alpha=0.3); plt.legend()
plt.show()

# If x is measured in DEGREES the limit is not 1:
for d in [10, 1, 0.1, 0.01]:
    print(f"x = {d:>5} degrees:  sin(x)/x = {np.sin(np.radians(d))/d:.8f}")
print("pi/180 =", np.pi/180)
```

Output

```
x =    10 degrees:  sin(x)/x = 0.01736482
x =     1 degrees:  sin(x)/x = 0.01745241
x =   0.1 degrees:  sin(x)/x = 0.01745328
x =  0.01 degrees:  sin(x)/x = 0.01745329
pi/180 = 0.017453292519943295
```

Watch Out: Degrees spoil the limit

The output of Listing 3.4 shows that when x is measured in degrees, $\frac{\sin x}{x} \to \frac{\pi}{180} \approx 0.01745$, not 1. Calculus formulas such as $\frac{\mathrm{d}}{\mathrm{d}x}\sin x = \cos x$ hold *only in radians*. In Python, `np.sin` always expects radians.

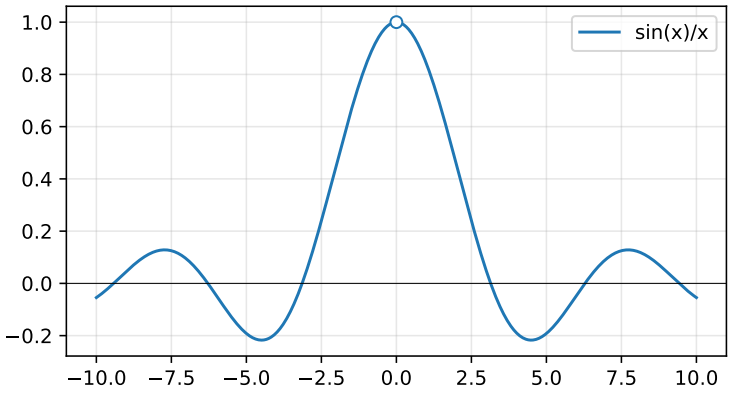


Figure 3.1: $\frac{\sin x}{x}$ has a "hole" at $x = 0$, but its limit there is 1.

3.4 One-sided limits

Definition 3.2. (One-sided limits)

We write $\lim_{x\to a^+} f(x) = L$ (the *right-hand limit*) if $f(x) \to L$ as x approaches a through values *greater* than a. We write $\lim_{x\to a^-} f(x) = L$ (the *left-hand limit*) if $f(x) \to L$ as x approaches a through values *less* than a.

Key Concept: Two-sided and one-sided limits

$$\lim_{x\to a} f(x) = L \iff \lim_{x\to a^-} f(x) = L \quad \text{and} \quad \lim_{x\to a^+} f(x) = L.$$

If the two one-sided limits differ, the (two-sided) limit **does not exist**.

Example 3.8 Let $f(x) = |x| = \begin{cases} x, & x \geq 0 \\ -x, & x < 0. \end{cases}$ Then $\lim_{x\to 0^+} |x| = \lim_{x\to 0^+} x = 0$ and $\lim_{x\to 0^-} |x| = \lim_{x\to 0^-} (-x) = 0$. So $\lim_{x\to 0} |x| = 0$.

Example 3.9 Evaluate $\lim_{x\to 8} \dfrac{|x-8|}{x-8}$.

Solution. For $x > 8$, $|x-8| = x-8$ and the fraction is 1. For $x < 8$, $|x-8| = -(x-8)$ and the fraction is -1. Thus the right-hand limit is 1 and the left-hand limit is -1. These differ, so the limit does not exist. ■

In Context: A jump in the electricity bill

Electricity tariffs often have *bands*. Here is an illustrative example (not the actual PURC tariff). Up to 30 kWh a month is billed at a "lifeline" rate of GH₵ 0.65 per unit. Above 30 kWh, *every* unit costs GH₵ 1.20, plus a service charge of GH₵ 10. Then $\lim_{u\to 30^-} B(u) = 19.50$ but $\lim_{u\to 30^+} B(u) = 46.00$. Using one extra unit costs over GH₵ 26! The bill has a *jump discontinuity*, and households near the boundary have a strong incentive to save power.

Listing 3.5 — A tariff with a jump ex3_05_one_sided.py

```python
# Listing 3.5: One-sided limits of a tariff with a jump (illustrative prices)
# Monthly electricity bill B(u) in GH¢ for u units (kWh):
#   up to 30 units: 0.65 per unit (a "lifeline" band, no service charge)
#   above 30 units: 1.20 per unit plus a fixed service charge of GH¢ 10
import numpy as np
import matplotlib.pyplot as plt

def bill(u):
    return np.where(u <= 30, 0.65*u, 10 + 1.20*u)

for u in [29.9, 29.99, 29.999, 30, 30.001, 30.01, 30.1]:
    print(f"B({u}) = {float(bill(u)):.4f}")
print("Left-hand limit at 30  :", 0.65*30)
print("Right-hand limit at 30 :", 10 + 1.20*30)

u1 = np.linspace(0, 30, 50); u2 = np.linspace(30.0001, 60, 50)
plt.figure(figsize=(5.5, 3.4))
plt.plot(u1, bill(u1), "C0"); plt.plot(u2, bill(u2), "C0")
plt.plot(30, 19.5, "o", color="C0"); plt.plot(30, 46, "o", mfc="white", mec="C0")
plt.xlabel("units used (kWh)"); plt.ylabel("bill (GH¢)"); plt.grid(alpha=0.3)
plt.show()
```

Output

```
B(29.9) = 19.4350
B(29.99) = 19.4935
B(29.999) = 19.4993
B(30) = 19.5000
B(30.001) = 46.0012
B(30.01) = 46.0120
B(30.1) = 46.1200
Left-hand limit at 30  : 19.5
Right-hand limit at 30 : 46.0
```

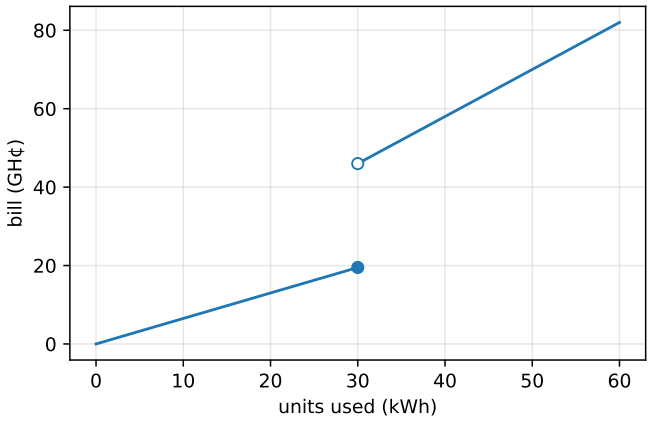


Figure 3.2: The bill jumps at $u = 30$, so the one-sided limits differ there.

3.5 Infinite limits and limits at infinity

Key Concept: Infinity in limits

- $\lim_{x\to a} f(x) = \infty$ means $f(x)$ becomes arbitrarily large as $x \to a$. The line $x = a$ is then a **vertical asymptote**. Example: $\lim_{x\to 0} \frac{1}{x^2} = \infty$.
- $\lim_{x\to\infty} f(x) = L$ means $f(x) \to L$ as x grows without bound. The line $y = L$ is then a **horizontal asymptote**. Example: $\lim_{x\to\infty} \frac{1}{x} = 0$.

∞ is not a number. Writing $\lim f(x) = \infty$ is a short way of saying that the limit does not exist *because the values grow without bound.*

To find a limit at infinity of a rational function, divide the top and bottom by the highest power of x in the denominator:

$$\lim_{x\to\infty} \frac{3x^2 - 5x + 1}{2x^2 + 7} = \lim_{x\to\infty} \frac{3 - \frac{5}{x} + \frac{1}{x^2}}{2 + \frac{7}{x^2}} = \frac{3}{2}.$$

3.5.1 The number e

In Context: Mobile-money savings and the number e

Suppose GH₵ 1000 is saved at an annual interest rate of 12% (an illustrative rate). Paid once a year, the interest gives GH₵ 1120. Paid monthly at 1% per month, it gives $1000(1.01)^{12} \approx$ GH₵ 1126.83. What if the interest is compounded weekly, daily, every minute? Listing 3.6 shows that the balance does *not* grow without bound. It approaches a limit, $1000\, e^{0.12} \approx$ GH₵ 1127.50.

Key Concept: The number e

$$e = \lim_{n\to\infty} \left(1 + \frac{1}{n}\right)^n \approx 2.718281828, \qquad \lim_{n\to\infty} \left(1 + \frac{r}{n}\right)^n = e^r.$$

Listing 3.6 — Compound interest and e ex3_06_compound_interest.py

```
# Listing 3.6: Compounding more and more often - the number e
import math

P, r = 1000, 0.12            # GH₵ 1000 at 12% per year (illustrative)
print(f"{'compounded':>14} {'n':>8} {'balance after 1 year':>22}")
for label, n in [("yearly", 1), ("monthly", 12), ("weekly", 52), ("daily", 365),
                 ("hourly", 8760), ("every minute", 525600)]:
    print(f"{label:>14} {n:>8} {P*(1 + r/n)**n:>22.4f}")
print(f"{'continuously':>14} {'∞':>8} {P*math.exp(r):>22.4f}")

print()
for n in [1, 10, 100, 1000, 10**6]:
    print(f"(1 + 1/n)^n with n = {n:>7}: {(1 + 1/n)**n:.8f}")
print("e =", math.e)
```

Output

```
    compounded         n   balance after 1 year
        yearly         1              1120.0000
       monthly        12              1126.8250
        weekly        52              1127.3410
         daily       365              1127.4746
        hourly      8760              1127.4959
  every minute    525600              1127.4968
  continuously         ∞               1127.4969

(1 + 1/n)^n with n =       1: 2.00000000
(1 + 1/n)^n with n =      10: 2.59374246
(1 + 1/n)^n with n =     100: 2.70481383
(1 + 1/n)^n with n =    1000: 2.71692393
(1 + 1/n)^n with n = 1000000: 2.71828047
e = 2.718281828459045
```

Historical Note: Bernoulli, Euler and e

Jacob Bernoulli met the limit $\left(1+\frac{1}{n}\right)^n$ in 1683 while studying exactly this compound-interest problem. Leonhard Euler named the number e, proved many of its properties, and calculated it to 23 decimal places. We meet e again as the base of the *natural* exponential function in Section 4.6.

3.6 The squeeze (sandwich) theorem

Theorem 3.3. (Squeeze theorem)

Suppose $f(x) \le g(x) \le h(x)$ for all $x \neq a$ in an open interval containing a, and

$$\lim_{x\to a} f(x) = \lim_{x\to a} h(x) = L.$$

Then $\lim_{x\to a} g(x) = L$.

Example 3.10 Show that $\lim_{x\to 0} x^2 \sin\dfrac{1}{x} = 0$.

Solution. We cannot use the product law, because $\lim_{x\to 0}\sin\frac{1}{x}$ does not exist (it oscillates between -1 and 1). However, $-1 \le \sin\frac{1}{x} \le 1$, so $-x^2 \le x^2\sin\frac{1}{x} \le x^2$ for $x \neq 0$. Since $\lim_{x\to 0}(-x^2) = \lim_{x\to 0} x^2 = 0$, the squeeze theorem gives $\lim_{x\to 0} x^2\sin\frac{1}{x} = 0$. ■

Example 3.11 Since $-1 \le \cos x \le 1$, we have $-\frac{x^2}{x^2+1} \le \frac{x^2\cos x}{x^2+1} \le \frac{x^2}{x^2+1}$. Both bounds tend to 0 as $x \to 0$, so $\lim_{x\to 0}\frac{x^2\cos x}{x^2+1} = 0$. (Here direct substitution also works. Try $\lim_{x\to\infty}\frac{\cos x}{x} = 0$ for a case where it does not.)

Listing 3.7 — The squeeze theorem in a picture ex3_07_squeeze.py

```python
# Listing 3.7: The squeeze (sandwich) theorem: -x^2 <= x^2 sin(1/x) <= x^2
import numpy as np
import matplotlib.pyplot as plt
```

```
x = np.linspace(-0.4, 0.4, 4001)
x = x[x != 0]
plt.figure(figsize=(5.5, 3.4))
plt.plot(x, x**2, "k--", lw=1, label="$x^2$")
plt.plot(x, -x**2, "k--", lw=1, label="$-x^2$")
plt.plot(x, x**2*np.sin(1/x), "C3", lw=0.8, label="$x^2\\sin(1/x)$")
plt.legend(); plt.grid(alpha=0.3)
plt.show()

for h in [0.1, 0.01, 0.001]:
    print(f"x = {h}:  x^2 sin(1/x) = {h**2*np.sin(1/h):+.2e}  (between ±{h**2:.0e})")
```

Output

```
x = 0.1:  x^2 sin(1/x) = -5.44e-03  (between ±1e-02)
x = 0.01:  x^2 sin(1/x) = -5.06e-05  (between ±1e-04)
x = 0.001:  x^2 sin(1/x) = +8.27e-07  (between ±1e-06)
```

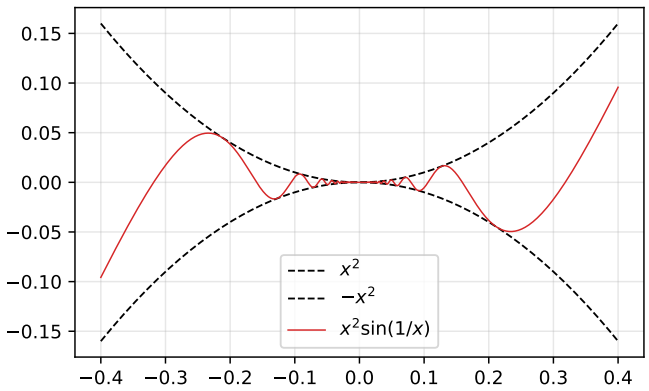


Figure 3.3: $x^2 \sin(1/x)$ oscillates infinitely often, but it is trapped between $-x^2$ and x^2, so it is forced to 0.

3.7 Continuity

In everyday language, "continuous" means without a break. A function is continuous if its graph can be drawn without lifting the pen.

Definition 3.3. (Continuity)

A function f is *continuous at* a if

(i) $f(a)$ is defined;

(ii) $\lim_{x \to a} f(x)$ exists; and

(iii) $\lim_{x \to a} f(x) = f(a)$.

If any of these fails, f is *discontinuous* at a. A function is continuous on an interval if it is continuous at every point of the interval.

Listing 3.8 — Three kinds of discontinuity `ex3_08_discontinuities.py`

```python
# Listing 3.8: Three ways a function can fail to be continuous
import numpy as np
import matplotlib.pyplot as plt

fig, ax = plt.subplots(1, 3, figsize=(9, 2.8))

# (a) a removable discontinuity (hole): (x^2 - 4)/(x - 2)
x = np.linspace(0, 4, 200); x = x[np.abs(x - 2) > 1e-9]
ax[0].plot(x, (x**2 - 4)/(x - 2)); ax[0].plot(2, 4, "o", mfc="white", mec="C0")
ax[0].set_title("(a) hole at x = 2")

# (b) a jump: f(x) = x for x < 1, x + 2 for x >= 1
x1 = np.linspace(-1, 1, 50, endpoint=False); x2 = np.linspace(1, 3, 50)
ax[1].plot(x1, x1, "C0"); ax[1].plot(x2, x2 + 2, "C0")
ax[1].plot(1, 1, "o", mfc="white", mec="C0"); ax[1].plot(1, 3, "o", color="C0")
ax[1].set_title("(b) jump at x = 1")

# (c) an infinite discontinuity (vertical asymptote): 1/(x - 1)^2
x = np.linspace(-1, 3, 400); x = x[np.abs(x - 1) > 0.08]
y = 1/(x - 1)**2
ax[2].plot(x[x < 1], y[x < 1], "C0"); ax[2].plot(x[x > 1], y[x > 1], "C0")
ax[2].axvline(1, ls="--", color="gray"); ax[2].set_ylim(0, 40)
ax[2].set_title("(c) asymptote at x = 1")

for a in ax:
    a.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

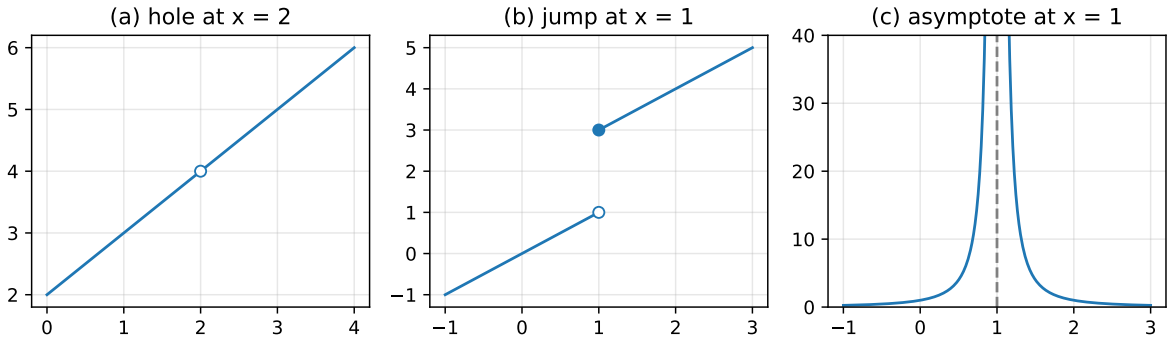


Figure 3.4: (a) A removable discontinuity (a hole); (b) a jump; (c) an infinite discontinuity.

Example 3.12 Is $f(x) = \dfrac{x^2 - 4}{x^2 - 3x + 2}$ continuous at $a = 2$?

Solution. $f(2) = \frac{0}{0}$ is undefined, so condition (i) fails and f is discontinuous at 2. The discontinuity is *removable*: $\lim_{x\to 2} \frac{(x-2)(x+2)}{(x-2)(x-1)} = \lim_{x\to 2} \frac{x+2}{x-1} = 4$. Defining $f(2) = 4$ would make f continuous at 2. ■

Example 3.13 Find the constant c that makes $f(x) = \begin{cases} \dfrac{x^2 - 1}{x - 1}, & x \neq 1, \\ c, & x = 1 \end{cases}$ continuous at $x = 1$.

Solution. $\lim_{x\to 1}\frac{(x-1)(x+1)}{x-1} = \lim_{x\to 1}(x+1) = 2$. Continuity requires $f(1) = 2$, so $c = 2$. ■

Key Concept: Continuous functions

Polynomials, rational functions (where defined), $\sqrt[n]{x}$, $\sin x$, $\cos x$, e^x and $\ln x$ are continuous on their domains. Sums, products, quotients (where the denominator is non-zero) and compositions of continuous functions are continuous.

3.7.1 The Intermediate Value Theorem

Theorem 3.4. (Intermediate Value Theorem (IVT))

If f is continuous on $[a, b]$ and N is any number between $f(a)$ and $f(b)$, then there is at least one $c \in (a, b)$ with $f(c) = N$. In particular, if $f(a)$ and $f(b)$ have opposite signs, then f has a root in (a, b).

In Context: Temperature in Tamale

If the temperature in Tamale is 24°C at 6 am and 38°C at 2 pm, then at some moment in between it was exactly 30°C. Temperature changes continuously; it cannot jump from 29° to 31° without passing through 30°. This is the IVT in everyday life.

The IVT gives a simple and reliable way to solve equations, the **bisection method**. If f changes sign on $[a, b]$, test the midpoint m. The sign change, and so a root, is in either $[a, m]$ or $[m, b]$. Keep the half that contains it and repeat. Each step halves the interval.

Listing 3.9 — The bisection method `ex3_09_bisection.py`

```python
# Listing 3.9: The Intermediate Value Theorem at work - the bisection method
def bisection(f, a, b, tol=1e-6):
    """Find a root of a continuous f on [a, b], where f(a) and f(b) differ in sign."""
    if f(a) * f(b) > 0:
        raise ValueError("f(a) and f(b) must have opposite signs")
    steps = 0
    while b - a > tol:
        m = (a + b) / 2
        if f(a) * f(m) <= 0:      # sign change in the left half
            b = m
        else:                      # sign change in the right half
            a = m
        steps += 1
    return (a + b) / 2, steps

f = lambda x: x**3 - x - 2
root, n = bisection(f, 1, 2)
print(f"root of x^3 - x - 2 = 0: {root:.6f}  ({n} halvings)")

# A kenkey seller's daily profit (GH¢) when she makes x hundred balls (illustrative):
#   P(x) = -4x^3 + 30x^2 - 20x - 15.  When does she first break even (P = 0)?
P = lambda x: -4*x**3 + 30*x**2 - 20*x - 15
print("P(0) =", P(0), ", P(2) =", P(2))
root, n = bisection(P, 0, 2)
```

```
print(f"break-even at about {100*root:.0f} balls")
```

Output

```
root of x^3 - x - 2 = 0: 1.521380  (20 halvings)
P(0) = -15 , P(2) = 33
break-even at about 128 balls
```

3.8 The precise definition of a limit ⋆

This section is optional on a first reading.

Definition 3.4. (Limit (ε–δ))

$\lim_{x\to a} f(x) = L$ means: for every $\varepsilon > 0$ there is a $\delta > 0$ such that

$$0 < |x - a| < \delta \quad \Longrightarrow \quad |f(x) - L| < \varepsilon.$$

Think of ε as a *tolerance* demanded by a customer, for example "the output must be within 0.01 of L". You must reply with a δ: "then keep the input within δ of a". The limit exists if you can always reply, however small ε is.

Example 3.14 Prove that $\lim_{x\to 2} x^2 = 4$.

Solution. We have $|x^2 - 4| = |x - 2|\,|x + 2|$. If $|x - 2| < 1$, then $1 < x < 3$ and $|x + 2| < 5$, so $|x^2 - 4| < 5\,|x - 2|$. Given $\varepsilon > 0$, choose $\delta = \min\left(1, \frac{\varepsilon}{5}\right)$. Then $0 < |x - 2| < \delta$ implies $|x^2 - 4| < 5 \cdot \frac{\varepsilon}{5} = \varepsilon$. ■

Listing 3.10 — Searching for δ numerically ex3_10_epsilon_delta.py

```
# Listing 3.10: Epsilon and delta - how close is close enough?
# lim_{x->2} x^2 = 4. Given a tolerance eps, find delta so that
# |x - 2| < delta guarantees |x^2 - 4| < eps.
import numpy as np

def works(delta, eps, n=100001):
    x = np.linspace(2 - delta, 2 + delta, n)[1:-1]
    return np.all(np.abs(x**2 - 4) < eps)

for eps in [1, 0.1, 0.01, 0.001]:
    delta = 1.0
    while not works(delta, eps):
        delta /= 2                      # halve delta until it works
    rule = min(1, eps/5)                # the textbook choice, proved in the text
    print(f"eps = {eps:<6} found delta = {delta:<12.6g} textbook delta = min(1, eps/5) = {
    rule:g}")
```

Output

```
eps = 1      found delta = 0.125        textbook delta = min(1, eps/5) = 0.2
eps = 0.1    found delta = 0.015625     textbook delta = min(1, eps/5) = 0.02
```

```
eps = 0.01    found delta = 0.00195312    textbook delta = min(1, eps/5) = 0.002
eps = 0.001   found delta = 0.000244141   textbook delta = min(1, eps/5) = 0.0002
```

Chapter Summary

- $\lim_{x\to a} f(x) = L$: $f(x)$ gets close to L as x gets close to (but not equal to) a.
- Try direct substitution first. If it gives $\frac{0}{0}$, factorise, rationalise or use a trigonometric identity.
- $\lim_{x\to 0} \frac{\sin x}{x} = 1$ (radians!) and $\lim_{n\to\infty} \left(1 + \frac{1}{n}\right)^n = e$.
- The limit exists exactly when the left and right limits exist and are equal.
- Squeeze theorem: if $f \le g \le h$ and $f, h \to L$, then $g \to L$.
- f is continuous at a when $\lim_{x\to a} f(x) = f(a)$. The IVT gives the bisection method for solving equations.
- Python: `sp.limit(expr, x, a)`, with `dir="+"` or `dir="-"` for one-sided limits.

3.9 Exercises

A. Skills

1. Find the following limits, if they exist.

(i) $\lim\limits_{x\to 1} \dfrac{x^4 - 1}{x - 1}$

(ii) $\lim\limits_{x\to 0} \dfrac{2 - \cos 3x - \cos 4x}{x}$

(iii) $\lim\limits_{x\to 8} \dfrac{x - 8}{\sqrt[3]{x} - 2}$

(iv) $\lim\limits_{\theta\to 0} \dfrac{\theta^2}{1 - \cos^2\theta}$

(v) $\lim\limits_{x\to 1} \dfrac{x\,|x - 1|}{x - 1}$

(vi) $\lim\limits_{\theta\to 0} \dfrac{1 - \cos 4\theta}{1 - \cos 6\theta}$

(vii) $\lim\limits_{t\to 0} \dfrac{\sqrt{2 + t} - \sqrt{2}}{t}$

(viii) $\lim\limits_{x\to 0} \dfrac{\sin(\sin x)}{x}$

(ix) $\lim\limits_{x\to 1.5} \dfrac{2x^2 - 3x}{|2x - 3|}$

(x) $\lim\limits_{x\to 0} \dfrac{1 - \cos x}{x}$

(xi) $\lim\limits_{x\to 0} \dfrac{x^2 - 3\sin x}{x}$

(xii) $\lim\limits_{x\to 4} \left|10 - 3x^2\right|$

2. Let $f(x) = \begin{cases} -x - 2, & x \le -1, \\ x, & -1 < x < 1, \\ x^2 - 2x, & x \ge 1. \end{cases}$ Determine whether $\lim_{x\to -1} f(x)$ and $\lim_{x\to 1} f(x)$ exist.

3. For what value of the constant c is $f(x) = \begin{cases} x + c, & x < 2, \\ cx^2 + 1, & x \ge 2 \end{cases}$ continuous everywhere?

4. If $\lim_{x\to 4} f(x) = 4$, evaluate $\lim_{x\to 4} [f(x) + 3x]$.

5. Find a so that $\lim_{x\to 1} f(x)$ exists, where $f(x) = \begin{cases} 3x + 5, & x \le 1, \\ 2x + a, & x > 1. \end{cases}$

6. Evaluate $\displaystyle\lim_{x\to 3}\frac{\sqrt{12-x}-x}{\sqrt{6+x}-3}$.

7. Evaluate $\displaystyle\lim_{x\to \pi/2}\frac{1+\cos 2x}{(\pi-2x)^2}$.

8. For what values of a and b is

$$f(x)=\begin{cases}\dfrac{x^2-4}{x-2}, & x<2,\\ ax^2-bx+3, & 2\le x<3,\\ 2x-a+b, & x\ge 3\end{cases}$$

continuous on $\mathbb{R}$?

9. Evaluate (a) $\displaystyle\lim_{x\to\infty}\frac{5x^3-x}{2x^3+4x^2+1}$; (b) $\displaystyle\lim_{x\to\infty}\frac{4x+1}{x^2+3}$; (c) $\displaystyle\lim_{x\to\infty}\left(1+\frac{2}{x}\right)^x$.

10. Use the squeeze theorem to show that $\lim_{x\to 0} x\cos\frac{1}{x^2}=0$.

11. Show that $x^3+x-1=0$ has a root between 0 and 1.

B. Applications

12. ⚙ A trotro's distance from Tema Station after t minutes is $s(t)=0.02t^2+0.3t$ km. Find its average speed over $[10,10+h]$ as a formula in h, and hence its speed at the instant $t=10$ min.

13. ⚙ A parking lot at Kotoka Airport charges GH₵ 10 for the first hour or part of it, and GH₵ 5 for each further hour or part of an hour (illustrative prices). Sketch the charge $C(t)$ for $0<t\le 5$ hours. At which values of t is C discontinuous? Find $\lim_{t\to 2^-}C(t)$ and $\lim_{t\to 2^+}C(t)$.

14. ⚙ Ghana's population was 24.66 million in 2010 and 30.83 million in 2021. A model of the form $P(t)=24.66\,e^{rt}$ (millions, with t in years after 2010) fits both figures. Find r, and find $\lim_{h\to 0}\frac{P(11+h)-P(11)}{h}$ numerically. What does this number mean?

15. ⚙ A susu collector offers 10% annual interest compounded n times per year (illustrative). How much more does GH₵ 5000 earn in a year under continuous compounding than under monthly compounding?

C. Programming

16. Write a function `limit_table(f, a)` that prints $f(a-h)$ and $f(a+h)$ for $h=10^{-1},\dots,10^{-8}$. Use it on $\frac{1-\cos x}{x^2}$ at $a=0$. What happens for very small h? (Look up *floating-point cancellation*.)

17. Use `sp.limit` to check all your answers to Exercise 1.

18. Use the bisection method to solve $\cos x = x$ to 8 decimal places. How many halvings are needed? Predict this number before running the code.

19. Plot $\left(1+\frac{1}{n}\right)^n$ against n for $n=1,\dots,200$, together with the horizontal line $y=e$.

Chapter 4

The Derivative

Learning Objectives

After working through this chapter you should be able to:

- interpret the derivative as the gradient of a tangent and as an instantaneous rate of change;
- differentiate from first principles;
- use the power, sum, product, quotient and chain rules;
- differentiate trigonometric, exponential and logarithmic functions;
- use implicit and logarithmic differentiation;
- find higher-order derivatives and interpret them as velocity and acceleration;
- differentiate symbolically with `sp.diff` and numerically from data.

The gradient of a straight line is the same everywhere: divide the change in y by the change in x. A curve has no single gradient. Its steepness changes from point to point. The *derivative* gives the gradient of the curve, that is, the gradient of its tangent line, at each point. Equivalently, it gives the *instantaneous rate of change* of one quantity with respect to another.

Historical Note: Newton, Leibniz and the great dispute

In the 1630s Pierre de Fermat had already found tangents to curves and located maxima and minima, using a method he called *adequality*. Isaac Barrow, Newton's teacher at Cambridge, studied tangents and areas geometrically.

In 1665–66 an outbreak of plague closed Cambridge University. The young Isaac Newton, working at his mother's farm, developed his "method of fluxions". A *fluent* was a quantity that flows in time, and its *fluxion* (written $\dot{x}$) was its rate of flow. Newton did not publish this work for decades.

Gottfried Wilhelm Leibniz, in Germany, developed calculus independently. He published first, in 1684, in a paper called *Nova Methodus pro Maximis et Minimis* ("A new method for maxima and minima"). Leibniz invented the notation $\frac{\mathrm{d}y}{\mathrm{d}x}$ and the integral sign $\int$ that we still use.

A bitter dispute over who deserved the credit followed. In 1712 the Royal Society, whose president was Newton himself, ruled for Newton. Historians today agree that the two men discovered calculus independently. Leibniz's notation proved so much better that it is now used everywhere.

4.1 The derivative as a limit

Let $P(a, f(a))$ be a point on the curve $y = f(x)$, and let $Q(a+h, f(a+h))$ be a nearby point. The *secant* line PQ has gradient

$$m_{PQ} = \frac{f(a+h) - f(a)}{h}.$$

As $h \to 0$, Q slides along the curve towards P, and the secant turns into the *tangent* at P.

Listing 4.1 — Secants approaching the tangent ex4_01_secant_to_tangent.py

```python
# Listing 4.1: Secant lines turning into the tangent line
import numpy as np
import matplotlib.pyplot as plt

f = lambda x: x**2
a = 1.0
x = np.linspace(-0.5, 3, 200)

plt.figure(figsize=(5.5, 4))
plt.plot(x, f(x), "k", lw=2, label="$y = x^2$")
for h, c in zip([1.5, 1.0, 0.5], ["C1", "C2", "C4"]):
    m = (f(a + h) - f(a)) / h                  # gradient of the secant
    plt.plot(x, f(a) + m*(x - a), c, lw=1, label=f"secant, h = {h}: m = {m:.2f}")
    plt.plot(a + h, f(a + h), "o", color=c)
plt.plot(x, f(a) + 2*(x - a), "C3--", lw=2, label="tangent: m = 2")
plt.plot(a, f(a), "ko")
plt.ylim(-1, 9); plt.grid(alpha=0.3); plt.legend(fontsize=8)
plt.show()

for h in [1, 0.1, 0.01, 0.001]:
    print(f"h = {h:<6} secant gradient = {(f(a + h) - f(a))/h:.4f}")
```

Output

```
h = 1      secant gradient = 3.0000
h = 0.1    secant gradient = 2.1000
h = 0.01   secant gradient = 2.0100
h = 0.001  secant gradient = 2.0010
```

Definition 4.1. (Derivative)

The *derivative* of f is the function f' defined by

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}, \tag{4.1}$$

wherever this limit exists. If $f'(a)$ exists, f is *differentiable* at a. Other notations for the derivative of $y = f(x)$ are

$$\frac{\mathrm{d}y}{\mathrm{d}x}, \qquad y', \qquad \frac{\mathrm{d}}{\mathrm{d}x} f(x), \qquad D_x f(x), \qquad \dot{y} \ \text{(for time derivatives)}.$$

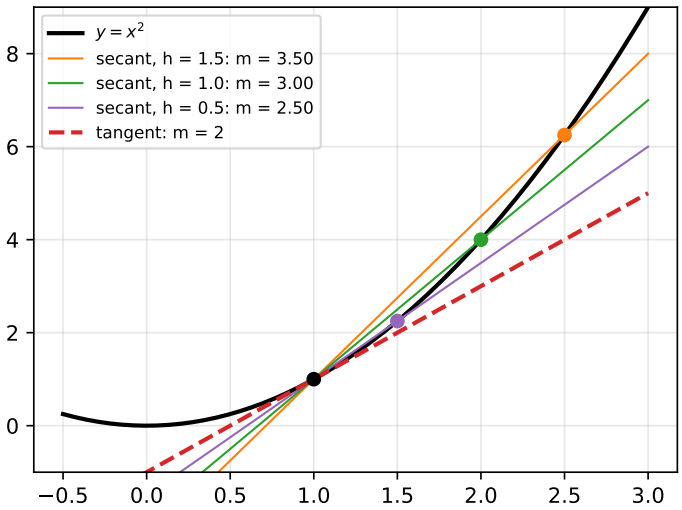


Figure 4.1: Secants of $y = x^2$ through $(1, 1)$. As $h \to 0$ their gradients $3.5,\ 3,\ 2.5, \ldots$ approach 2, the gradient of the tangent.

Key Concept: Two meanings of the derivative

1. **Geometric:** $f'(a)$ is the gradient of the tangent to $y = f(x)$ at $x = a$.
2. **Physical:** $f'(a)$ is the *instantaneous rate of change* of f with respect to x at $x = a$. If $s(t)$ is position, $s'(t)$ is velocity. If $C(q)$ is cost, $C'(q)$ is the marginal cost. If $P(t)$ is a population, $P'(t)$ is its growth rate.

Example 4.1 Use the definition to find $f'(x)$ if $f(x) = 4x - 2$.

Solution. $f'(x) = \lim_{h\to 0} \dfrac{4(x+h) - 2 - (4x-2)}{h} = \lim_{h\to 0} \dfrac{4h}{h} = 4$. This is the gradient of the straight line, as expected. ■

Example 4.2 Using first principles, find $f'(x)$ if $f(x) = \dfrac{1}{x^2}$.

Solution.

$$f'(x) = \lim_{h\to 0} \frac{\frac{1}{(x+h)^2} - \frac{1}{x^2}}{h} = \lim_{h\to 0} \frac{x^2 - (x+h)^2}{hx^2(x+h)^2} = \lim_{h\to 0} \frac{-h(2x+h)}{hx^2(x+h)^2}$$
$$= \lim_{h\to 0} \frac{-(2x+h)}{x^2(x+h)^2} = \frac{-2x}{x^4} = -\frac{2}{x^3}.$$

■

Listing 4.2 — First principles with SymPy ex4_02_first_principles.py

```
# Listing 4.2: Differentiation from first principles with SymPy
import sympy as sp

x, h = sp.symbols("x h")

def derivative_from_definition(f):
    return sp.simplify(sp.limit((f.subs(x, x + h) - f) / h, h, 0))

```

```python
for f in [4*x - 2, 1/x**2, sp.sqrt(x), x**3, sp.sin(x), 4*x/(x + 1)]:
    print(f"f(x) = {str(f):<12}  f'(x) = {derivative_from_definition(f)}")

# Not differentiable: |x| at 0. The one-sided limits of the difference quotient differ.
q = sp.Abs(h) / h
print("|x| at 0: left limit =", sp.limit(q, h, 0, "-"), ", right limit =", sp.limit(q, h,
    0, "+"))
```

Output

```
f(x) = 4*x - 2       f'(x) = 4
f(x) = x**(-2)       f'(x) = -2/x**3
f(x) = sqrt(x)       f'(x) = 1/(2*sqrt(x))
f(x) = x**3          f'(x) = 3*x**2
f(x) = sin(x)        f'(x) = cos(x)
f(x) = 4*x/(x + 1)   f'(x) = 4/(x**2 + 2*x + 1)
|x| at 0: left limit = -1 , right limit = 1
```

Key Concept: Differentiability and continuity

If f is differentiable at a, then f is continuous at a. The converse is *false*. For example, $f(x) = |x|$ is continuous at 0, but its graph has a corner there. The difference quotient $\frac{|h|}{h}$ has left limit -1 and right limit 1, so $f'(0)$ does not exist (see the last line of the output above).

4.2 Rules for differentiation

Working from the definition every time would be slow. Instead, we prove a handful of rules once and then combine them.

Key Concept: Basic rules

Constant	$\frac{\mathrm{d}}{\mathrm{d}x}[c] = 0$
Power rule	$\frac{\mathrm{d}}{\mathrm{d}x}[x^n] = nx^{n-1}$ (any real n)
Constant multiple	$\frac{\mathrm{d}}{\mathrm{d}x}[cf(x)] = cf'(x)$
Sum/difference	$\frac{\mathrm{d}}{\mathrm{d}x}[f(x) \pm g(x)] = f'(x) \pm g'(x)$
Product rule	$\frac{\mathrm{d}}{\mathrm{d}x}[f(x)g(x)] = f(x)g'(x) + g(x)f'(x)$
Quotient rule	$\frac{\mathrm{d}}{\mathrm{d}x}\left[\frac{f(x)}{g(x)}\right] = \frac{g(x)f'(x) - f(x)g'(x)}{[g(x)]^2}$

Example 4.3

(a) $f(x) = 200 \Rightarrow f'(x) = 0$.

(b) $f(x) = x^5 \Rightarrow f'(x) = 5x^4$.

(c) $f(x) = \sqrt{x} = x^{1/2} \Rightarrow f'(x) = \frac{1}{2}x^{-1/2} = \dfrac{1}{2\sqrt{x}}$.

(d) $f(x) = \dfrac{1}{\sqrt[3]{x}} = x^{-1/3} \Rightarrow f'(x) = -\frac{1}{3}x^{-4/3}$.

(e) $f(x) = x^7 + \sqrt{x} \Rightarrow f'(x) = 7x^6 + \frac{1}{2}x^{-1/2}$.

(f) $f(x) = 3\sqrt{x} + 5x^2 \Rightarrow f'(x) = \frac{3}{2}x^{-1/2} + 10x$.

Example 4.4 Find $\frac{dy}{dx}$ if $y = (x^2 + x - 1)(2x + 4)$.

Solution. By the product rule, $\frac{dy}{dx} = (x^2+x-1)(2)+(2x+4)(2x+1) = 2x^2+2x-2+4x^2+10x+4 = 6x^2 + 12x + 2$. ■

Example 4.5 Find $f'(x)$ if $f(x) = \dfrac{x^4 + 2x - 1}{3x^2 + 5}$.

Solution. By the quotient rule,

$$f'(x) = \frac{(3x^2 + 5)(4x^3 + 2) - (x^4 + 2x - 1)(6x)}{(3x^2 + 5)^2} = \frac{6x^5 + 20x^3 - 6x^2 + 6x + 10}{(3x^2 + 5)^2}.$$

■

Watch Out: The quotient rule order

The numerator of the quotient rule is "*bottom times derivative of top, minus top times derivative of bottom*". Getting the order wrong changes the sign of the answer. Also, $\frac{d}{dx}[f(x)g(x)]$ is **not** $f'(x)g'(x)$. Check: $x \cdot x = x^2$ has derivative $2x$, not $1 \cdot 1 = 1$.

4.3 Derivatives of trigonometric functions

Using $\lim_{h\to 0} \frac{\sin h}{h} = 1$ and $\lim_{h\to 0} \frac{1-\cos h}{h} = 0$ (Theorem 3.2),

$$\frac{d}{dx}\sin x = \lim_{h\to 0} \frac{\sin x \cos h + \cos x \sin h - \sin x}{h} = \sin x \cdot \lim_{h\to 0} \frac{\cos h - 1}{h} + \cos x \cdot \lim_{h\to 0} \frac{\sin h}{h} = \cos x.$$

The other derivatives follow from the quotient rule.

Key Concept: Trigonometric derivatives (radians)

$\frac{d}{dx}\sin x = \cos x$ $\quad$ $\frac{d}{dx}\cos x = -\sin x$ $\quad$ $\frac{d}{dx}\tan x = \sec^2 x$

$\frac{d}{dx}\cot x = -\csc^2 x$ $\quad$ $\frac{d}{dx}\sec x = \sec x \tan x$ $\quad$ $\frac{d}{dx}\csc x = -\csc x \cot x$

Memory aid: the derivatives of the "co-" functions (cos, cot, csc) all carry a minus sign.

Example 4.6 If $f(x) = x\sin x - \cos x$, then $f'(x) = (x\cos x + \sin x) - (-\sin x) = x\cos x + 2\sin x$.

Example 4.7 If $y = \dfrac{x^2 + 2}{1 + \sin x}$, then $\frac{dy}{dx} = \dfrac{(1 + \sin x)(2x) - (x^2 + 2)\cos x}{(1 + \sin x)^2}$.

Example 4.8 Find $f'(x)$ if $f(x) = \dfrac{\tan x}{1 + 2\sec x}$.

Solution.

$$f'(x) = \frac{(1+2\sec x)\sec^2 x - \tan x\,(2\sec x\tan x)}{(1+2\sec x)^2}$$
$$= \frac{\sec^2 x + 2\sec x(\sec^2 x - \tan^2 x)}{(1+2\sec x)^2} = \frac{\sec^2 x + 2\sec x}{(1+2\sec x)^2}.$$

Multiplying top and bottom by $\cos^2 x$ gives the neater form $\dfrac{1+2\cos x}{(\cos x+2)^2}$, which is what SymPy prints in Listing 4.3. ■

Example 4.9 Show that $\frac{\mathrm{d}}{\mathrm{d}x}\cot x = -\csc^2 x$.

Solution. $\dfrac{\mathrm{d}}{\mathrm{d}x}\dfrac{\cos x}{\sin x} = \dfrac{\sin x(-\sin x) - \cos x(\cos x)}{\sin^2 x} = \dfrac{-(\sin^2 x + \cos^2 x)}{\sin^2 x} = -\csc^2 x.$ ■

Listing 4.3 — The rules checked with `sp.diff` `ex4_03_rules.py`

```python
# Listing 4.3: The rules of differentiation, checked with sp.diff
import sympy as sp

x = sp.symbols("x")
examples = {
    "x^5":                          x**5,
    "1/cbrt(x)":                    1/sp.cbrt(x),
    "x^7 + sqrt(x)":                x**7 + sp.sqrt(x),
    "3 sqrt(x) + 5x^2":             3*sp.sqrt(x) + 5*x**2,
    "(x^2+x-1)(2x+4)":              (x**2 + x - 1)*(2*x + 4),
    "(x^4+2x-1)/(3x^2+5)":          (x**4 + 2*x - 1)/(3*x**2 + 5),
    "x sin x - cos x":              x*sp.sin(x) - sp.cos(x),
    "(x^2+2)/(1+sin x)":            (x**2 + 2)/(1 + sp.sin(x)),
    "tan x/(1 + 2 sec x)":          sp.tan(x)/(1 + 2*sp.sec(x)),
}
for name, f in examples.items():
    d = sp.diff(f, x)
    print(f"d/dx[{name}] = {sp.simplify(d)}")
```

Output

```
d/dx[x^5] = 5*x**4
d/dx[1/cbrt(x)] = -1/(3*x**(4/3))
d/dx[x^7 + sqrt(x)] = 7*x**6 + 1/(2*sqrt(x))
d/dx[3 sqrt(x) + 5x^2] = 10*x + 3/(2*sqrt(x))
d/dx[(x^2+x-1)(2x+4)] = 6*x**2 + 12*x + 2
d/dx[(x^4+2x-1)/(3x^2+5)] = 2*(3*x**5 + 10*x**3 - 3*x**2 + 3*x + 5)/(9*x**4 + 30*x**2 + 25)
d/dx[x sin x - cos x] = x*cos(x) + 2*sin(x)
d/dx[(x^2+2)/(1+sin x)] = (2*x*(sin(x) + 1) - (x**2 + 2)*cos(x))/(sin(x) + 1)**2
d/dx[tan x/(1 + 2 sec x)] = (2*cos(x) + 1)/(cos(x) + 2)**2
```

4.4 The chain rule

Theorem 4.1. (Chain rule)

If g is differentiable at x and f is differentiable at $g(x)$, then $f \circ g$ is differentiable at x and

$$\frac{\mathrm{d}}{\mathrm{d}x} f(g(x)) = f'(g(x))\, g'(x).$$

In Leibniz notation, if $y = f(u)$ and $u = g(x)$, then

$$\frac{\mathrm{d}y}{\mathrm{d}x} = \frac{\mathrm{d}y}{\mathrm{d}u} \cdot \frac{\mathrm{d}u}{\mathrm{d}x}. \tag{4.2}$$

Key Concept: Chain rule in words

Differentiate the *outside* function, leaving the inside alone. Then multiply by the derivative of the *inside*. A special case is the **general power rule**:

$$\frac{\mathrm{d}}{\mathrm{d}x}[f(x)]^n = n[f(x)]^{n-1} f'(x).$$

Example 4.10 Differentiate $y = (x^2 + x)^{-5}$.

Solution. Let $u = x^2 + x$. Then $y = u^{-5}$, so $\frac{\mathrm{d}y}{\mathrm{d}x} = -5u^{-6}(2x+1) = -5(2x+1)(x^2+x)^{-6}$. ■

Example 4.11

(a) $y = \sin^5 x \Rightarrow \frac{\mathrm{d}y}{\mathrm{d}x} = 5\sin^4 x \cos x$.

(b) $y = \tan(3x^2 + 1) \Rightarrow \frac{\mathrm{d}y}{\mathrm{d}x} = 6x \sec^2(3x^2+1)$.

(c) $y = \cos^3(x^2+1) \Rightarrow y' = 3\cos^2(x^2+1) \cdot [-\sin(x^2+1)] \cdot 2x = -6x\cos^2(x^2+1)\sin(x^2+1)$.

In Context: Rates multiply

Equation (4.2) says that *rates multiply*. Suppose a Bolt driver's car uses 0.08 litres of fuel per km, fuel costs GH₵ 14 per litre, and the car averages 40 km/h in town (illustrative figures). Then the fuel cost per hour is

$$\frac{\mathrm{d}C}{\mathrm{d}t} = \frac{\mathrm{d}C}{\mathrm{d}F} \cdot \frac{\mathrm{d}F}{\mathrm{d}x} \cdot \frac{\mathrm{d}x}{\mathrm{d}t} = 14 \times 0.08 \times 40 = \text{GH₵ } 44.80 \text{ per hour}.$$

Notice how the units cancel, just as the "fractions" in Leibniz's notation seem to.

Listing 4.4 — The chain rule `ex4_04_chain_rule.py`

```python
# Listing 4.4: The chain rule - symbolic examples and a fuel-cost calculation
import sympy as sp

x = sp.symbols("x")
for f in [(x**2 + x)**-5, sp.sin(x)**5, sp.tan(3*x**2 + 1), sp.cos(x**2 + 1)**3]:
    print(f"d/dx[{f}] = {sp.factor(sp.diff(f, x))}")

```

```python
# Fuel cost of a Bolt driver (illustrative numbers):
#   price  p = 14 GH¢ per litre,  consumption = 0.08 litres per km,  speed = 40 km/h
dC_dF = 14          # GH¢ per litre
dF_dx = 0.08        # litres per km
dx_dt = 40          # km per hour
dC_dt = dC_dF * dF_dx * dx_dt
print(f"Fuel cost rate dC/dt = {dC_dF} x {dF_dx} x {dx_dt} = GH¢ {dC_dt:.2f} per hour")
```

Output

```
d/dx[(x**2 + x)**(-5)] = -5*(2*x + 1)/(x**6*(x + 1)**6)
d/dx[sin(x)**5] = 5*sin(x)**4*cos(x)
d/dx[tan(3*x**2 + 1)] = 6*x*(tan(3*x**2 + 1)**2 + 1)
d/dx[cos(x**2 + 1)**3] = -6*x*sin(x**2 + 1)*cos(x**2 + 1)**2
Fuel cost rate dC/dt = 14 x 0.08 x 40 = GH¢ 44.80 per hour
```

4.5 Implicit differentiation

Some curves are given *explicitly*, as $y = f(x)$, for example $y = x^3 + 1$. Others are given *implicitly* by an equation in x and y, such as the circle $x^2 + y^2 = 25$ or the curve $\sin(xy) = 4x$. We do not need to solve for y first. We differentiate both sides with respect to x, treating y as a function of x, so that by the chain rule $\frac{\mathrm{d}}{\mathrm{d}x}[y^n] = ny^{n-1}y'$. Then we solve for y'.

Example 4.12 Find $\frac{\mathrm{d}y}{\mathrm{d}x}$ if $xy^2 + 6y + x = 0$.

Solution. Differentiating term by term (using the product rule on xy^2) gives $y^2 + 2xyy' + 6y' + 1 = 0$. Hence $y'(2xy + 6) = -1 - y^2$, and $y' = -\dfrac{1 + y^2}{2xy + 6}$. ■

Example 4.13 Find y' if $\cos(xy) + \sin x = 1$.

Solution. $-\sin(xy)\,(y + xy') + \cos x = 0$, so $y' = \dfrac{\cos x - y\sin(xy)}{x\sin(xy)}$. ■

In Context: Walking round Lake Bosomtwe

Model the shore of Lake Bosomtwe as the circle $x^2 + y^2 = 16$ (km, centre at the middle of the lake). Implicit differentiation gives $2x + 2yy' = 0$, so $y' = -\frac{x}{y}$. At the point $(2, 2\sqrt{3})$ on the northern shore, the shoreline runs in the direction of gradient $-\frac{1}{\sqrt{3}}$. This is perpendicular to the radius, as Chapter 2 predicts.

Listing 4.5 — Implicit differentiation with `sp.idiff` `ex4_05_implicit.py`

```python
# Listing 4.5: Implicit differentiation with sp.idiff
import sympy as sp

x, y = sp.symbols("x y")

# Worked examples
print("x y^2 + 6y + x = 0   : dy/dx =", sp.simplify(sp.idiff(x*y**2 + 6*y + x, y, x)))
```

```
print("cos(xy) + sin x = 1  : dy/dx =", sp.simplify(sp.idiff(sp.cos(x*y) + sp.sin(x) - 1,
    y, x)))

# The circle x^2 + y^2 = 25: slope of the tangent at (3, 4)
dydx = sp.idiff(x**2 + y**2 - 25, y, x)
print("x^2 + y^2 = 25       : dy/dx =", dydx, "; at (3, 4) it is", dydx.subs({x: 3, y: 4}))


# Second derivative of x^2 + y^2 = 1  (compare the worked example y^3 y'' + 1 = 0)
d2 = sp.idiff(x**2 + y**2 - 1, y, x, 2)
print("x^2 + y^2 = 1        : y'' =", sp.simplify(d2), "-> y^3 y'' =", sp.simplify((y**3*
    d2).subs(x**2, 1 - y**2)))
```

Output

```
x y^2 + 6y + x = 0   : dy/dx = (-y**2 - 1)/(2*(x*y + 3))
cos(xy) + sin x = 1  : dy/dx = (-y + cos(x)/sin(x*y))/x
x^2 + y^2 = 25       : dy/dx = -x/y ; at (3, 4) it is -3/4
x^2 + y^2 = 1        : y'' = (-x**2 - y**2)/y**3 -> y^3 y'' = -1
```

4.6 Exponential and logarithmic functions

Key Concept: Exponential and logarithmic derivatives

$$\frac{\mathrm{d}}{\mathrm{d}x}e^x = e^x \qquad \frac{\mathrm{d}}{\mathrm{d}x}e^{g(x)} = g'(x)e^{g(x)}$$

$$\frac{\mathrm{d}}{\mathrm{d}x}\ln x = \frac{1}{x}\ (x>0) \qquad \frac{\mathrm{d}}{\mathrm{d}x}\ln g(x) = \frac{g'(x)}{g(x)}$$

$$\frac{\mathrm{d}}{\mathrm{d}x}a^x = (\ln a)\,a^x \qquad \frac{\mathrm{d}}{\mathrm{d}x}a^{g(x)} = (\ln a)\,a^{g(x)}g'(x)$$

$$\frac{\mathrm{d}}{\mathrm{d}x}\log_a x = \frac{1}{x\ln a} \qquad \frac{\mathrm{d}}{\mathrm{d}x}\log_a |g(x)| = \frac{g'(x)}{g(x)\ln a}$$

The function e^x is the only function (apart from constant multiples of it) that equals its own derivative. This makes it the natural model for anything that grows at a rate proportional to its size: populations, savings with compound interest, the spread of an epidemic, and (with a negative exponent) radioactive decay and the cooling of a cup of tea. The formula for a^x follows by writing $a^x = e^{x\ln a}$. The formula for $\log_a x$ follows by writing $\log_a x = \frac{\ln x}{\ln a}$.

Example 4.14

(a) $y = e^{x^2} \Rightarrow y' = 2xe^{x^2}$.

(b) $y = e^x \sin 2x + e^{\cos x} \Rightarrow y' = e^x \sin 2x + 2e^x \cos 2x - \sin x\, e^{\cos x}$.

(c) $y = \ln(\sin x) \Rightarrow y' = \dfrac{\cos x}{\sin x} = \cot x$.

(d) $y = \sin(x^2+1)\ln x + \ln(4x^3+3) \Rightarrow y' = \dfrac{\sin(x^2+1)}{x} + 2x\ln x\cos(x^2+1) + \dfrac{12x^2}{4x^3+3}$.

(e) $y = 4^x \Rightarrow y' = (\ln 4)4^x$; $\quad y = 6^{\sin x} \Rightarrow y' = (\ln 6)\,6^{\sin x}\cos x$.

(f) $y = (x^2+1)^{10} + 10^{x^2+1} \Rightarrow y' = 20x(x^2+1)^9 + 2x(\ln 10)10^{x^2+1}$.

(g) $y = \log_4 x \Rightarrow y' = \dfrac{1}{x \ln 4}$.

(h) $f(x) = \log_{10} \sqrt[3]{(2x+5)^2} = \frac{2}{3}\log_{10}|2x+5| \Rightarrow f'(x) = \frac{2}{3}\cdot\dfrac{2}{(2x+5)\ln 10} = \dfrac{4}{(6x+15)\ln 10}$.

4.6.1 Logarithmic differentiation

To differentiate $f(x)^{g(x)}$, where both the base and the exponent vary, or a complicated product or quotient, take logarithms first and then differentiate implicitly.

Example 4.15 Find y' if $y = x^{\sin x}$ $(x > 0)$.

Solution. Taking logarithms, $\ln y = \sin x \ln x$. Differentiating both sides gives

$$\frac{y'}{y} = \cos x \ln x + \frac{\sin x}{x}, \qquad \text{so} \qquad y' = x^{\sin x}\left(\cos x \ln x + \frac{\sin x}{x}\right).$$

■

Watch Out: Power rule or exponential rule?

x^3	variable base, constant power	power rule: $3x^2$
3^x	constant base, variable power	exponential rule: $(\ln 3)3^x$
x^x	variable base *and* power	logarithmic differentiation: $x^x(1+\ln x)$

Listing 4.6 — Exponential and logarithmic derivatives ex4_06_exp_log.py

```
# Listing 4.6: Derivatives of exponential and logarithmic functions
import sympy as sp

x = sp.symbols("x", positive=True)
examples = {
    "e^(x^2)":                    sp.exp(x**2),
    "e^x sin 2x + e^(cos x)":     sp.exp(x)*sp.sin(2*x) + sp.exp(sp.cos(x)),
    "ln(sin x)":                  sp.log(sp.sin(x)),
    "sin(x^2+1) ln x + ln(4x^3+3)": sp.sin(x**2 + 1)*sp.log(x) + sp.log(4*x**3 + 3),
    "4^x":                        4**x,
    "6^(sin x)":                  6**sp.sin(x),
    "log_4 x":                    sp.log(x, 4),
    "x^(sin x)":                  x**sp.sin(x),
}
for name, f in examples.items():
    print(f"d/dx[{name}] = {sp.diff(f, x)}")
```

Output

```
d/dx[e^(x^2)] = 2*x*exp(x**2)
d/dx[e^x sin 2x + e^(cos x)] = exp(x)*sin(2*x) + 2*exp(x)*cos(2*x) - exp(cos(x))*sin(x)
d/dx[ln(sin x)] = cos(x)/sin(x)
d/dx[sin(x^2+1) ln x + ln(4x^3+3)] = 12*x**2/(4*x**3 + 3) + 2*x*log(x)*cos(x**2 + 1) + sin(x
    **2 + 1)/x
d/dx[4^x] = 4**x*log(4)
d/dx[6^(sin x)] = 6**sin(x)*log(6)*cos(x)
```

```
d/dx[log_4 x] = 1/(x*log(4))
d/dx[x^(sin x)] = x**sin(x)*(log(x)*cos(x) + sin(x)/x)
```

In Context: How fast is Ghana growing?

Ghana's Population and Housing Census counted 24 658 823 people in 2010 and 30 832 019 in 2021. Fit the model $P(t) = P_0 e^{rt}$, with t in years after 2010. Then $e^{11r} = \frac{30.83}{24.66}$, so $r = \frac{1}{11}\ln\frac{30.83}{24.66} \approx 0.0203$, a growth rate of about 2% per year. The derivative $P'(t) = rP(t)$ is the number of people added per year. It was about 626 000 per year in 2021: the population of a large city added every single year. This growth is why schools, roads and hospitals must be planned years in advance.

Listing 4.7 — Ghana's population growth rate `ex4_07_population.py`

```python
# Listing 4.7: How fast is Ghana's population growing?
import numpy as np
import matplotlib.pyplot as plt

# Population and Housing Census totals (millions)
P2010, P2021 = 24.658823, 30.832019
r = np.log(P2021 / P2010) / 11          # from P2021 = P2010 * e^(11 r)
print(f"growth rate r = {r:.5f} per year  ({100*r:.2f}% per year)")

P = lambda t: P2010 * np.exp(r*t)       # t = years after 2010
dP = lambda t: r * P(t)                 # derivative of a e^(rt) is r a e^(rt)
for year in [2010, 2021, 2026]:
    t = year - 2010
    print(f"{year}: P = {P(t):6.2f} million,  P'(t) = {1e6*dP(t):,.0f} people per year")

t = np.linspace(0, 20, 100)
plt.figure(figsize=(5.5, 3.4))
plt.plot(2010 + t, P(t), label="model $P(t)=24.66\\,e^{rt}$")
plt.plot([2010, 2021], [P2010, P2021], "ro", label="census")
t0 = 11
plt.plot(2010 + t, P(t0) + dP(t0)*(t - t0), "k--", lw=1, label="tangent at 2021")
plt.xlabel("year"); plt.ylabel("millions"); plt.grid(alpha=0.3); plt.legend()
plt.show()
```

Output

```
growth rate r = 0.02031 per year  (2.03% per year)
2010: P =  24.66 million,  P'(t) = 500,841 people per year
2021: P =  30.83 million,  P'(t) = 626,223 people per year
2026: P =  34.13 million,  P'(t) = 693,160 people per year
```

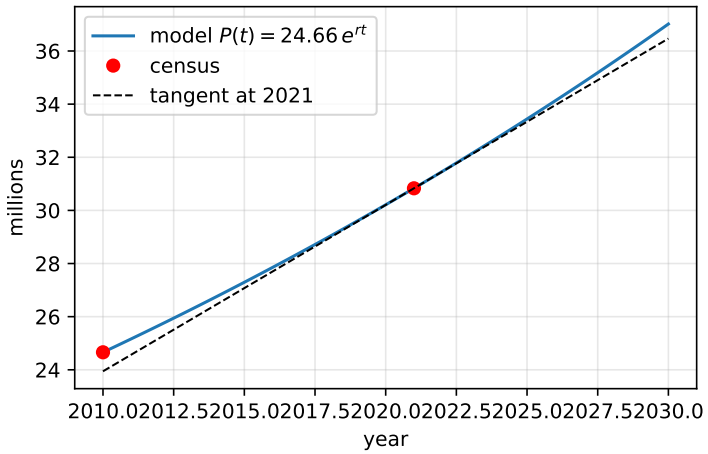


Figure 4.2: Exponential model of Ghana's population and its tangent line at 2021, whose gradient is $P'(11)$.

4.7 Higher-order derivatives

The derivative f' is itself a function, so we can differentiate it again. The *second derivative* is $f'' = (f')'$, the third is f''', and so on:

$$y',\ y'',\ y''',\ \ldots,\ y^{(n)} \qquad \text{or} \qquad \frac{\mathrm{d}y}{\mathrm{d}x},\ \frac{\mathrm{d}^2y}{\mathrm{d}x^2},\ \frac{\mathrm{d}^3y}{\mathrm{d}x^3},\ \ldots,\ \frac{\mathrm{d}^ny}{\mathrm{d}x^n}.$$

Key Concept: Motion along a line

If $s(t)$ is position at time t, then

$$v(t) = s'(t)\ \text{(velocity)}, \qquad a(t) = v'(t) = s''(t)\ \text{(acceleration)}.$$

The car speeds up when v and a have the same sign, and slows down when they have opposite signs.

Example 4.16 If $x^2 + y^2 = 1$, show that $y^3y'' + 1 = 0$.

Solution. Differentiating gives $2x + 2yy' = 0$, so $y' = -\frac{x}{y}$. Then by the quotient rule,

$$y'' = -\frac{y - xy'}{y^2} = -\frac{y + \frac{x^2}{y}}{y^2} = -\frac{y^2 + x^2}{y^3} = -\frac{1}{y^3},$$

since $x^2 + y^2 = 1$. Hence $y^3y'' = -1$. ∎

In Context: A car on the Cape Coast–Takoradi road

A car pulls away from a toll booth. Its position is $s(t) = 1.2t^2 - 0.04t^3$ metres for $0 \le t \le 20$ s. Its velocity is $v = 2.4t - 0.12t^2$ and its acceleration is $a = 2.4 - 0.24t$. The acceleration is zero at $t = 10$ s, and at that moment the speed is greatest: 12 m/s, about 43 km/h. After that the driver eases off. Listing 4.8 finds and plots all three functions.

Listing 4.8 — Position, velocity, acceleration ex4_08_higher_order.py

```python
# Listing 4.8: Position, velocity and acceleration of a car (higher derivatives)
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

t = sp.symbols("t")
s = sp.Rational(6, 5)*t**2 - sp.Rational(1, 25)*t**3     # metres, 0 <= t <= 20 s
v = sp.diff(s, t)
a = sp.diff(s, t, 2)
print("s(t) =", s, "\nv(t) =", v, "\na(t) =", a)
t_max = sp.solve(a, t)[0]
print("maximum speed at t =", t_max, "s:  v =", v.subs(t, t_max), "m/s =",
      float(v.subs(t, t_max))*3.6, "km/h")

T = np.linspace(0, 20, 200)
funcs = [sp.lambdify(t, e) for e in (s, v, a)]
fig, ax = plt.subplots(1, 3, figsize=(9, 2.6))
for axis, f, name in zip(ax, funcs, ["s (m)", "v (m/s)", "a (m/s$^2$)"]):
    axis.plot(T, f(T) * np.ones_like(T)); axis.set_title(name); axis.set_xlabel("t (s)")
    axis.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```

Output

```
s(t) = -t**3/25 + 6*t**2/5
v(t) = -3*t**2/25 + 12*t/5
a(t) = 6*(10 - t)/25
maximum speed at t = 10 s:  v = 12 m/s = 43.2 km/h
```

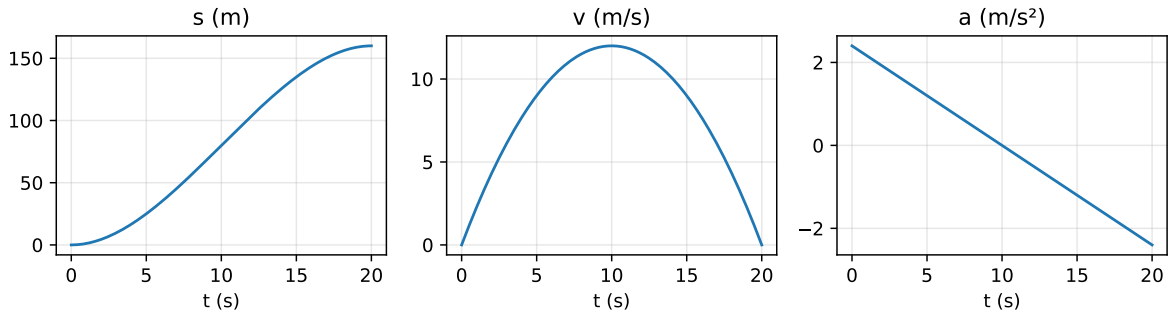


Figure 4.3: The velocity is greatest where the acceleration crosses zero.

4.8 Numerical differentiation

In practice we often know a function only through measured *data*, for example a GPS log, a temperature record or exchange rates. We cannot apply the rules to data, but we can use the

definition (4.1) with a small, fixed h:

$$\text{forward difference:} \quad f'(x) \approx \frac{f(x+h) - f(x)}{h},$$
$$\text{central difference:} \quad f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}.$$

The central difference is much more accurate. Its error shrinks like h^2, while the forward difference error shrinks only like h.

Listing 4.9 — Derivatives from data `ex4_09_numerical_diff.py`

```python
# Listing 4.9: Numerical differentiation - derivatives from data
import numpy as np

# (a) How accurate are difference quotients?  f(x) = sin x at x = 1, exact f'(1) = cos 1
f, x0, exact = np.sin, 1.0, np.cos(1.0)
print(f"{'h':>8} {'forward error':>15} {'central error':>15}")
for h in [0.1, 0.01, 0.001, 0.0001]:
    fwd = (f(x0 + h) - f(x0)) / h
    cen = (f(x0 + h) - f(x0 - h)) / (2*h)
    print(f"{h:>8} {abs(fwd - exact):>15.2e} {abs(cen - exact):>15.2e}")

# (b) Speed from GPS data logged by a ride-hailing app every 10 s (illustrative)
time = np.array([0, 10, 20, 30, 40, 50, 60])            # s
dist = np.array([0, 60, 210, 420, 640, 830, 960])       # m
speed = np.gradient(dist, time)                         # central differences inside
for t, v in zip(time, speed):
    print(f"t = {t:>2} s   speed ~ {v:5.1f} m/s = {3.6*v:5.1f} km/h")
```

Output

```
       h   forward error   central error
     0.1        4.29e-02        9.00e-04
    0.01        4.22e-03        9.00e-06
   0.001        4.21e-04        9.01e-08
  0.0001        4.21e-05        9.00e-10
t =  0 s   speed ~   6.0 m/s =  21.6 km/h
t = 10 s   speed ~  10.5 m/s =  37.8 km/h
t = 20 s   speed ~  18.0 m/s =  64.8 km/h
t = 30 s   speed ~  21.5 m/s =  77.4 km/h
t = 40 s   speed ~  20.5 m/s =  73.8 km/h
t = 50 s   speed ~  16.0 m/s =  57.6 km/h
t = 60 s   speed ~  13.0 m/s =  46.8 km/h
```

Python Tip: `np.gradient`

`np.gradient(y, x)` estimates $\frac{dy}{dx}$ from arrays of data. It uses central differences at interior points and one-sided differences at the two ends.

Chapter Summary

- $f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$: the gradient of the tangent, or the instantaneous rate of change.

- Power, product, quotient and chain rules; derivatives of sin, cos, tan, e^x, $\ln x$, a^x and $\log_a x$.
- Implicit differentiation: differentiate both sides, remembering that $\frac{\mathrm{d}}{\mathrm{d}x}[y^n] = ny^{n-1}y'$.
- Logarithmic differentiation for $f(x)^{g(x)}$ and complicated products.
- $s' = v$ and $s'' = a$. The central difference $\frac{f(x+h)-f(x-h)}{2h}$ estimates f' from data.
- Python: `sp.diff(f, x)`, `sp.diff(f, x, 2)`, `sp.idiff(F, y, x)`, `np.gradient(y, x)`.

4.9 Exercises

A. Skills

1. Find the derivative of $f(x) = ax^2 + bx + c$ from the definition, where a, b and c are non-zero constants.

2. Find the derivative of each function from first principles: (a) $f(x) = (x-1)(x-2)$; (b) $f(t) = \dfrac{4t}{t+1}$; (c) $f(x) = \sin x$ (*hint*: $\sin(x+h) - \sin x = 2\cos\frac{2x+h}{2}\sin\frac{h}{2}$); (d) $f(x) = \sqrt{x}$.

3. Differentiate:

(i) $y = 10^{\frac{1-x}{1+x}}$

(ii) $y = e^{ax}\csc(bx+c)$

(iii) $y = \dfrac{e^{2x}\cos x}{x\sin x}$

(iv) $y = \log_{10}(3x^2+2)^5$

(v) $y = \ln(\sec x + \tan x)$

(vi) $y = \exp\left(\sqrt{\cot x}\right)$

(vii) $y = \ln\dfrac{a+b\tan x}{a-b\tan x}$

(viii) $y = (\tan x)^{\cot x} + (\sec x)^x$

(ix) $y = \dfrac{(4x^2-1)\sqrt{1+x^2}}{x^3(x-7)^{3/4}}$

(x) $y = x^{\ln x}$

4. If $y = x^{x^{x^{\cdot^{\cdot^{\cdot}}}}}$, prove that $\frac{\mathrm{d}y}{\mathrm{d}x} = \dfrac{y^2}{x(1-y\ln x)}$.

5. If $y = (\sin x)^{(\sin x)^{(\sin x)^{\cdots}}}$, show that $\frac{\mathrm{d}y}{\mathrm{d}x} = \dfrac{y^2\cot x}{1-y\ln(\sin x)}$.

6. If $y = \sqrt{\ln x + \sqrt{\ln x + \sqrt{\ln x + \cdots}}}$, show that $\frac{\mathrm{d}y}{\mathrm{d}x} = \dfrac{1}{x(2y-1)}$.

7. If $x^y = e^{x-y}$, prove that $\frac{\mathrm{d}y}{\mathrm{d}x} = \dfrac{\ln x}{(1+\ln x)^2}$.

8. Find the second derivative of (a) $y = \cos(\ln x)$; (b) $y = x^x$; (c) $y = a^x x^a$.

9. If $x^3 - y^3 = 1$, show that $y^5 y'' + 2x = 0$.

10. If $f(x) = \dfrac{1}{x^2+1}$ and $g(x) = \sqrt{x}$, find the derivative of $f(g(x))$. (*Past examination question.*)

11. Find $\frac{\mathrm{d}y}{\mathrm{d}x}$ if $y = \ln(4x^2+2x)\,e^{\cos^2 5x}$.

B. Applications

12. A cocoa farm's yield (tonnes) is modelled by $Y(n) = 2.5\ln(1 + 0.4n)$, where n is the number of bags of fertiliser applied. Find $Y'(n)$ and evaluate it at $n = 5$ and $n = 20$. Interpret the two answers. (This is the law of *diminishing returns*.)

13. The temperature of a cup of hot Milo left on a table in Kumasi satisfies $T(t) = 27 + 63e^{-0.08t}$ (°C, t in minutes). Find $T'(t)$. How fast is the drink cooling at $t = 0$ and at $t = 20$?

14. A trotro's position on the Accra–Tema Motorway is $s(t) = t^3 - 12t^2 + 45t$ metres, for $0 \le t \le 8$ s. Find its velocity and acceleration. When is it momentarily at rest? When is it slowing down?

15. The exchange rate between the cedi and the US dollar is recorded on five consecutive days as 15.10, 15.18, 15.31, 15.35, 15.52 GH₵ per dollar (illustrative). Use central differences to estimate the rate of change on days 2, 3 and 4.

C. Programming

16. Use `sp.diff` to check all your answers to Exercise 3.

17. Write a function `tangent_plot(f, a)` that plots $y = f(x)$ together with its tangent at $x = a$, using `sp.diff` for the gradient and `sp.lambdify` to turn SymPy expressions into NumPy functions.

18. Repeat the error table of Listing 4.9 for $h = 10^{-5}, 10^{-6}, \dots, 10^{-12}$. The error eventually *grows* as h shrinks. Explain why. (*Hint:* computers store only about 16 significant digits.)

19. Use `np.gradient` *twice* on the GPS data of Listing 4.9 to estimate the car's acceleration. When was it braking?

Chapter 5

Applications of the Derivative

Learning Objectives

After working through this chapter you should be able to:

- find the equations of tangents and normals to curves;
- solve related-rates problems;
- find critical numbers, local and absolute extrema, and intervals of increase and decrease;
- use the second derivative to study concavity and points of inflection;
- set up and solve optimisation problems from real situations;
- solve equations with Newton's method and evaluate limits with l'Hôpital's rule;
- use Python to fit models to data, then optimise them.

Historical Note: Fermat's maxima and minima

Long before Newton and Leibniz, Pierre de Fermat, a lawyer in Toulouse who did mathematics in his spare time, found how to locate the greatest and least values of a curve. Around 1636 he observed that near a maximum or minimum, the value of a function barely changes when x changes slightly. In modern language, the tangent there is horizontal: $f'(c) = 0$. Fermat used the same idea to show that light travelling between two points takes the *quickest* route. This was the first "principle of least time". Today optimisation, finding the best design, price, route or schedule, is one of the most widely used parts of mathematics. It is used in engineering, economics, logistics, and in the apps that plan ride-hailing trips across Accra.

5.1 Tangents and normals

The tangent to $y = f(x)$ at $A(x_1, y_1)$ has gradient $f'(x_1)$. The *normal* at A is the line through A perpendicular to the tangent.

Key Concept: Tangent and normal at (x_1, y_1)

$$\text{tangent: } y - y_1 = f'(x_1)(x - x_1), \qquad \text{normal: } y - y_1 = -\frac{1}{f'(x_1)}(x - x_1) \quad (f'(x_1) \neq 0).$$

If $f'(x_1) = 0$, the tangent is horizontal ($y = y_1$) and the normal is vertical ($x = x_1$).

Example 5.1 Find the equations of the tangent and normal to $y = x^2 + 4x - 3$ at $(1, 2)$.

Solution. $y' = 2x + 4$, so the gradient of the tangent is $y'(1) = 6$. The tangent is $y - 2 = 6(x - 1)$, that is $y = 6x - 4$. The normal has gradient $-\frac{1}{6}$: $y - 2 = -\frac{1}{6}(x - 1)$, that is, $x + 6y = 13$. ■

Listing 5.1 — Tangent and normal ex5_01_tangent_normal.py

```python
# Listing 5.1: Tangent and normal lines to a curve
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

x = sp.symbols("x")
f = x**2 + 4*x - 3
a = 1
m = sp.diff(f, x).subs(x, a)              # gradient of the tangent
b = f.subs(x, a)
tangent = sp.expand(b + m*(x - a))
normal = sp.expand(b - (x - a)/m)
print("point:", (a, b), " gradient f'(1) =", m)
print("tangent: y =", tangent)
print("normal : y =", normal, "   i.e. x + 6y = 13")

X = np.linspace(-3, 3, 200)
F, T, N = (sp.lambdify(x, e) for e in (f, tangent, normal))
plt.figure(figsize=(4.8, 4.8))
plt.plot(X, F(X), "k", lw=2, label="$y=x^2+4x-3$")
plt.plot(X, T(X), "C3", label="tangent")
plt.plot(X, N(X), "C0--", label="normal")
plt.plot(a, b, "ko"); plt.ylim(-8, 12)
plt.gca().set_aspect("equal"); plt.grid(alpha=0.3); plt.legend(fontsize=8)
plt.show()
```

Output

```
point: (1, 2)  gradient f'(1) = 6
tangent: y = 6*x - 4
normal : y = 13/6 - x/6    i.e. x + 6y = 13
```

In Context: Skidding off a bend

If a car takes a bend too fast on a wet road, for example on the winding road from Kumasi to Mampong, it skids off along the *tangent* to its path. This is Newton's first law: without the sideways grip of the tyres, it moves in a straight line. Road engineers bank (tilt) curves and post speed limits so that the friction needed stays well within what wet tyres can provide.

5.2 Related rates

In a related-rates problem, two or more quantities change with time t and are linked by an equation. Differentiating the equation with respect to t (using the chain rule) links their rates of change.

Key Concept: Strategy for related rates

1. Draw a diagram and name the variables. Write down the given rates and the rate you want, as derivatives with respect to t.
2. Write an equation linking the variables, valid at *all* times.
3. Differentiate it with respect to t.
4. *Only now* substitute the values at the instant of interest, and solve.

Example 5.2 Air is pumped into a spherical balloon so that its volume increases at $100\,\text{cm}^3/\text{s}$. How fast is the radius increasing when the diameter is $50\,\text{cm}$?

Solution. $V = \frac{4}{3}\pi r^3$, so $\frac{dV}{dt} = 4\pi r^2 \frac{dr}{dt}$. With $\frac{dV}{dt} = 100$ and $r = 25$,

$$\frac{dr}{dt} = \frac{100}{4\pi(25)^2} = \frac{1}{25\pi} \approx 0.0127 \text{ cm/s}.$$

■

In Context: An oil slick off the Western Region coast

Ghana's offshore oil fields lie off the coast of the Western Region. Suppose an accidental spill spreads as a circle whose radius grows at $0.5\,\text{m/min}$. Since $A = \pi r^2$, we have $\frac{dA}{dt} = 2\pi r \frac{dr}{dt}$. When $r = 300\,\text{m}$, $\frac{dA}{dt} = 2\pi(300)(0.5) = 300\pi \approx 942\ \text{m}^2/\text{min}$. The area grows faster and faster even though the radius grows steadily, so a quick response is vital.

Listing 5.2 — Related rates with SymPy `ex5_02_related_rates.py`

```
# Listing 5.2: Related rates - differentiate a relation with respect to time
import sympy as sp

t = sp.symbols("t")
r = sp.Function("r")(t)

# (a) Balloon: V = (4/3) pi r^3, dV/dt = 100 cm^3/s. Find dr/dt when r = 25 cm.
V = sp.Rational(4, 3)*sp.pi*r**3
dVdt = sp.diff(V, t)                                  # 4 pi r^2 dr/dt
drdt = sp.solve(sp.Eq(dVdt, 100), sp.diff(r, t))[0]
print("balloon: dr/dt =", sp.simplify(drdt.subs(r, 25)), "≈", float(drdt.subs(r, 25)), "cm
    /s")

# (b) An oil slick near an offshore field spreads in a circle. Its radius grows at
#     0.5 m/min. How fast is its AREA growing when the radius is 300 m?
A = sp.pi*r**2
dAdt = sp.diff(A, t).subs(sp.diff(r, t), sp.Rational(1, 2)).subs(r, 300)
print("oil slick: dA/dt =", dAdt, "≈", round(float(dAdt), 1), "m^2/min")
```

Output

```
balloon: dr/dt = 1/(25*pi) ≈ 0.012732395447351627 cm/s
oil slick: dA/dt = 300*pi ≈ 942.5 m^2/min
```

Watch Out: Substitute last

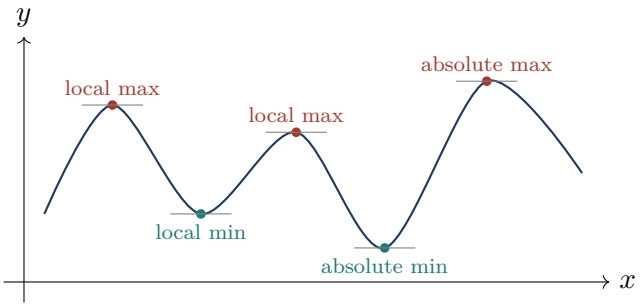
Do not substitute the particular value (such as $r = 25$) *before* differentiating. A quantity that is momentarily 25 is not constant, and its derivative is not 0.

5.3 Maximum and minimum values

Definition 5.1. (Absolute and local extrema)

Let c be in the domain D of f.

- $f(c)$ is the *absolute maximum* of f if $f(c) \geq f(x)$ for all $x \in D$, and the *absolute minimum* if $f(c) \leq f(x)$ for all $x \in D$.
- $f(c)$ is a *local maximum* if $f(c) \geq f(x)$ for all x near c (in some open interval containing c), and a *local minimum* if $f(c) \leq f(x)$ for all x near c.



Theorem 5.1. (Fermat's theorem)

If f has a local maximum or minimum at c, and $f'(c)$ exists, then $f'(c) = 0$.

Definition 5.2. (Critical number)

A number c in the domain of f is a *critical number* if $f'(c) = 0$ or $f'(c)$ does not exist.

Watch Out: $f'(c) = 0$ does not guarantee an extremum

For $f(x) = x^3$, $f'(0) = 0$, but 0 is neither a maximum nor a minimum: the graph flattens out and then keeps rising. Critical numbers are only *candidates*. They must be tested.

Example 5.3 Find the critical numbers of $f(x) = x^{1/3}(8 - x)$.

Solution. By the product rule, $f'(x) = \frac{1}{3}x^{-2/3}(8 - x) - x^{1/3} = \frac{8-x-3x}{3x^{2/3}} = \frac{8-4x}{3x^{2/3}}$. So $f'(x) = 0$ when $x = 2$, and $f'(x)$ does not exist at $x = 0$. The critical numbers are 0 and 2. ■

5.3.1 Absolute extrema on a closed interval

Theorem 5.2. (Extreme Value Theorem)

A function that is continuous on a closed interval $[a, b]$ has both an absolute maximum and an absolute minimum on $[a, b]$.

Key Concept: The closed interval method

To find the absolute extrema of a continuous f on $[a, b]$:

1. find the critical numbers of f in (a, b);
2. evaluate f at each critical number *and* at the endpoints a and b;
3. the largest of these values is the absolute maximum, and the smallest is the absolute minimum.

Example 5.4 Find the absolute extrema of $f(x) = x^3 - 12x$ on $[-3, 5]$.

Solution. $f'(x) = 3x^2 - 12 = 0$ gives $x = \pm 2$. Then $f(-3) = 9$, $f(-2) = 16$, $f(2) = -16$ and $f(5) = 65$. The absolute maximum is $f(5) = 65$ (at an endpoint), and the absolute minimum is $f(2) = -16$. ■

In Context: When should you book a Bolt across Accra?

Ride-hailing trip times depend heavily on when you set off. Here is a *hypothetical* model of the trip time from Madina to Accra Central, departing t hours after 6:00 am ($0 \le t \le 9$, that is, until 3:00 pm):

$$T(t) = 0.4t^3 - 5.4t^2 + 16.8t + 35 \quad \text{minutes.}$$

Then $T'(t) = 1.2t^2 - 10.8t + 16.8 = 1.2(t - 2)(t - 7)$, so the critical numbers are $t = 2$ (8:00 am) and $t = 7$ (1:00 pm). The closed interval method compares:

departure	6:00 am	8:00 am	1:00 pm	3:00 pm
t	0	2	7	9
$T(t)$ (min)	35.0	**50.2**	**25.2**	40.4

The slowest trip starts at 8:00 am, in the morning rush hour. The quickest starts at 1:00 pm, when it takes half as long. The model is invented, but real navigation apps do this kind of calculation constantly, with models fitted to millions of past trips.

Listing 5.3 — Best and worst departure times `ex5_03_bolt_travel_time.py`

```
# Listing 5.3: When is the quickest (and slowest) time to take a Bolt across Accra?
# HYPOTHETICAL model of the trip time from Madina to Accra Central:
#   T(t) = 0.4 t^3 - 5.4 t^2 + 16.8 t + 35  minutes,
#   t = hours after 6:00 am, 0 <= t <= 9 (i.e. 6:00 am to 3:00 pm)
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

```

```
t = sp.symbols("t")
T = sp.Rational(2, 5)*t**3 - sp.Rational(27, 5)*t**2 + sp.Rational(84, 5)*t + 35
dT = sp.factor(sp.diff(T, t))
print("T'(t) =", dT)
crit = sp.solve(sp.diff(T, t), t)
print("critical numbers:", crit)

def clock(h):
    hh = 6 + int(h); mm = round(60*(h - int(h)))
    return f"{hh % 12 or 12}:{mm:02d} {'am' if hh < 12 else 'pm'}"

candidates = [0] + crit + [9]          # closed interval method
for c in candidates:
    print(f"t = {c} ({clock(float(c)):>8}):  T = {float(T.subs(t, c)):.1f} min")
best = min(candidates, key=lambda c: T.subs(t, c))
worst = max(candidates, key=lambda c: T.subs(t, c))
print("quickest trip at", clock(float(best)), "; slowest at", clock(float(worst)))

tt = np.linspace(0, 9, 300)
Tf = sp.lambdify(t, T)
plt.figure(figsize=(6, 3.4))
plt.plot(6 + tt, Tf(tt))
for c in candidates:
    plt.plot(6 + float(c), float(T.subs(t, c)), "o")
plt.xlabel("departure time (hour of day)"); plt.ylabel("trip time T (min)")
plt.grid(alpha=0.3); plt.show()
```

Output

```
T'(t) = 6*(t - 7)*(t - 2)/5
critical numbers: [2, 7]
t = 0 ( 6:00 am):  T = 35.0 min
t = 2 ( 8:00 am):  T = 50.2 min
t = 7 ( 1:00 pm):  T = 25.2 min
t = 9 ( 3:00 pm):  T = 40.4 min
quickest trip at 1:00 pm ; slowest at 8:00 am
```

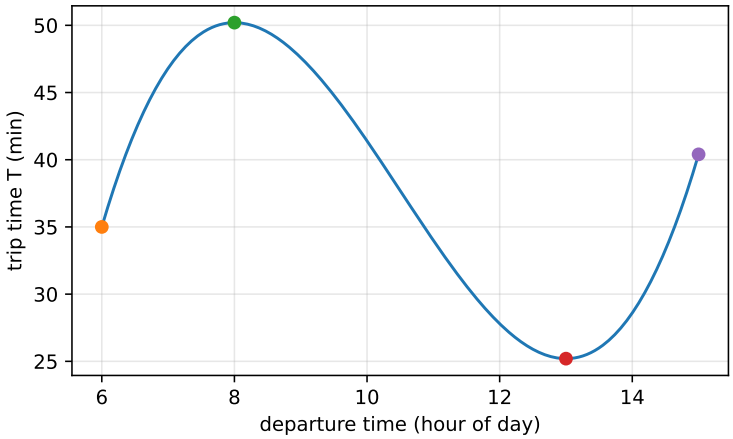


Figure 5.1: The hypothetical Madina–Accra Central trip time. The dots mark the endpoints and the two critical numbers.

Where would such a model come from? In practice we *fit* it to data. Listing 5.4 takes twelve (illustrative) Bolt trip times from UCC to Kotokuraba Market. It fits the best cubic polynomial with `np.polyfit`, and then finds the critical numbers of the fitted model from the roots of its derivative.

Listing 5.4 — Fitting a model to Bolt trip data, then optimising it
`ex5_04_trip_data_fit.py`

```python
# Listing 5.4: From trip data to a model - Bolt trips UCC -> Kotokuraba (illustrative data)

import numpy as np
import matplotlib.pyplot as plt

# departure time (hours after 6:00 am) and observed trip time (minutes)
dep = np.array([0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0])
mins = np.array([14, 17, 22, 25, 26, 24, 21, 17, 15, 15, 17, 21])

coef = np.polyfit(dep, mins, 3)          # best-fitting cubic (least squares)
T = np.poly1d(coef)
dT = T.deriv()                           # derivative of the polynomial
print("model:  T(t) =", np.round(coef, 3))
crit = dT.roots
print("critical numbers:", np.round(crit, 3))
for c in crit:
    kind = "maximum" if T.deriv(2)(c) < 0 else "minimum"
    h = 6 + c
    print(f"local {kind} at about {int(h)}:{int(60*(h % 1)):02d}, T = {T(c):.1f} min")

tt = np.linspace(0, 8, 200)
plt.figure(figsize=(6, 3.4))
plt.plot(6 + dep, mins, "o", label="observed trips")
plt.plot(6 + tt, T(tt), label="cubic model")
plt.xlabel("departure time (hour of day)"); plt.ylabel("minutes")
plt.grid(alpha=0.3); plt.legend(); plt.show()
```

Output

```
model:  T(t) = [ 0.299 -3.651 11.118 13.97 ]
critical numbers: [6.122 2.027]
local minimum at about 12:07, T = 13.7 min
local maximum at about 8:01, T = 24.0 min
```

5.4 Increasing and decreasing functions

Theorem 5.3. (Increasing/decreasing test)

Let f be continuous on $[a, b]$ and differentiable on (a, b).

- If $f'(x) > 0$ on (a, b), then f is increasing on $[a, b]$.
- If $f'(x) < 0$ on (a, b), then f is decreasing on $[a, b]$.

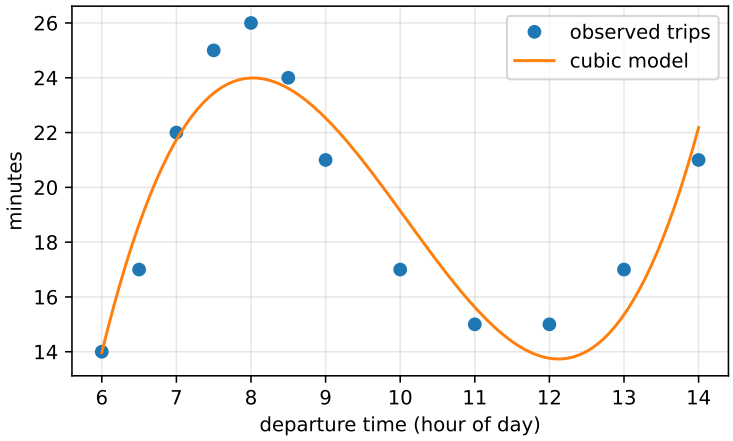


Figure 5.2: Illustrative trip-time data from UCC to Kotokuraba and the fitted cubic. The calculus is done on the model, not on the raw data.

Key Concept: First derivative test

Let c be a critical number of a continuous function f.

- If f' changes from $+$ to $-$ at c, then $f(c)$ is a **local maximum**.
- If f' changes from $-$ to $+$ at c, then $f(c)$ is a **local minimum**.
- If f' does not change sign at c, there is **no** local extremum at c.

Example 5.5 Find the local extrema of $f(x) = x^{1/3}(8 - x)$, and the intervals on which f increases and decreases.

Solution. From the previous example, $f'(x) = \dfrac{8 - 4x}{3x^{2/3}}$, with critical numbers 0 and 2. The denominator $3x^{2/3}$ is positive for all $x \neq 0$.

interval	$(-\infty, 0)$	$(0, 2)$	$(2, \infty)$
sign of $8 - 4x$	$+$	$+$	$-$
sign of $3x^{2/3}$	$+$	$+$	$+$
sign of $f'(x)$	$+$	$+$	$-$
f is	increasing	increasing	decreasing

So f increases on $(-\infty, 2]$ and decreases on $[2, \infty)$. At $x = 2$, f' changes from $+$ to $-$, so $f(2) = 2^{1/3}(6) = 6\sqrt[3]{2} \approx 7.56$ is a local (in fact absolute) maximum. At $x = 0$, f' does *not* change sign, so there is no extremum there. (The graph has a vertical tangent at the origin. The third edition wrongly called $x = 0$ a local maximum.) ■

Listing 5.5 — A sign chart in Python `ex5_05_first_derivative_test.py`

```
# Listing 5.5: The first derivative test for f(x) = x^(1/3) (8 - x)
import sympy as sp

x = sp.symbols("x", positive=True)       # differentiate where x > 0 ...
f = x**sp.Rational(1, 3) * (8 - x)
df = sp.factor(sp.diff(f, x))
```

```python
print("f'(x) =", df)                        # ... the same formula holds for x < 0

def fprime(v):                              # real cube roots, valid for negative v too
    return (8 - 4*v) / (3 * sp.real_root(v, 3)**2)

print("f'(x) = 0 at x =", sp.solve(8 - 4*x, x), "; f'(x) undefined at x = 0")
for lo, hi, test in [("-oo", 0, -1), (0, 2, 1), (2, "oo", 3)]:
    s = fprime(test)
    print(f"on ({lo}, {hi}):  f'({test}) = {float(s):+.3f}  ->",
          "increasing" if s > 0 else "decreasing")
print("local maximum f(2) =", f.subs(x, 2), "≈", round(float(f.subs(x, 2)), 3))
```

Output

```
f'(x) = -4*(x - 2)/(3*x**(2/3))
f'(x) = 0 at x = [2] ; f'(x) undefined at x = 0
on (-oo, 0):  f'(-1) = +4.000  -> increasing
on (0, 2):  f'(1) = +1.333  -> increasing
on (2, oo):  f'(3) = -0.641  -> decreasing
local maximum f(2) = 6*2**(1/3) ≈ 7.56
```

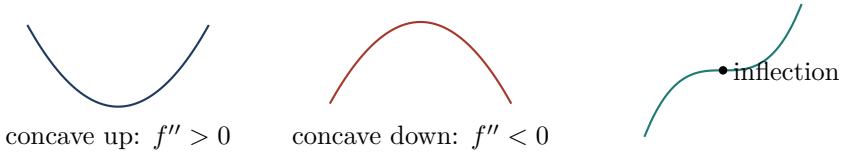
5.5 Concavity and the second derivative

Definition 5.3. (Concavity and inflection)

The graph of f is *concave upward* on an interval if it lies above all its tangents there (it "holds water" $\cup$), and *concave downward* if it lies below its tangents ($\cap$). A *point of inflection* is a point on the graph where the concavity changes.

Key Concept: The second derivative

- **Concavity test:** if $f''(x) > 0$ on an interval, f is concave up there; if $f''(x) < 0$, it is concave down.
- **Second derivative test:** suppose $f'(c) = 0$. If $f''(c) < 0$, then $f(c)$ is a local *maximum* ($\cap$). If $f''(c) > 0$, then $f(c)$ is a local *minimum* ($\cup$). If $f''(c) = 0$, the test fails; use the first derivative test.
- **Inflection points** can occur only where $f''(x) = 0$ or $f''(x)$ does not exist, and only if f'' *changes sign* there.



concave up: $f'' > 0$ concave down: $f'' < 0$

Example 5.6 Let $f(x) = 12 + 2x^2 - x^4$. Use the second derivative test to find the local extrema. Discuss the concavity and find the points of inflection.

Solution. $f'(x) = 4x - 4x^3 = 4x(1-x)(1+x) = 0$ gives $x = 0, \pm 1$. Also $f''(x) = 4 - 12x^2$.

- $f''(0) = 4 > 0$: local minimum $f(0) = 12$.
- $f''(\pm 1) = -8 < 0$: local maxima $f(\pm 1) = 13$.

$f''(x) = 0$ when $x = \pm\frac{1}{\sqrt{3}}$. Testing the sign of f'': $f''(-1) = -8 < 0$, $f''(0) = 4 > 0$ and $f''(1) = -8 < 0$. So f is concave down on $\left(-\infty, -\frac{1}{\sqrt{3}}\right)$, concave up on $\left(-\frac{1}{\sqrt{3}}, \frac{1}{\sqrt{3}}\right)$ and concave down on $\left(\frac{1}{\sqrt{3}}, \infty\right)$. The concavity changes at both points, which are therefore points of inflection: $\left(\pm\frac{1}{\sqrt{3}}, \frac{113}{9}\right)$. (The third edition interchanged "maximum" and "minimum" in this example.) ■

Listing 5.6 — Curve sketching with SymPy ex5_06_curve_sketch.py

```python
# Listing 5.6: Curve sketching with SymPy: f(x) = 12 + 2x^2 - x^4
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

x = sp.symbols("x", real=True)
f = 12 + 2*x**2 - x**4
f1, f2 = sp.diff(f, x), sp.diff(f, x, 2)

for c in sp.solve(f1, x):
    test = f2.subs(x, c)
    kind = "local max" if test < 0 else ("local min" if test > 0 else "test fails")
    print(f"critical x = {c}: f'' = {test} -> {kind}, f = {f.subs(x, c)}")
for p in sp.solve(f2, x):
    print(f"possible inflection at x = {p}, f = {f.subs(x, p)}")

X = np.linspace(-2, 2, 400)
F = sp.lambdify(x, f)
plt.figure(figsize=(5.5, 3.6))
plt.plot(X, F(X), "k")
for c in sp.solve(f1, x):
    plt.plot(float(c), float(f.subs(x, c)), "ro")
for p in sp.solve(f2, x):
    plt.plot(float(p), float(f.subs(x, p)), "gs")
plt.ylim(4, 14); plt.grid(alpha=0.3)
plt.title("red: extrema, green: inflection points")
plt.show()
```

Output

```
critical x = -1: f'' = -8 -> local max, f = 13
critical x = 0: f'' = 4 -> local min, f = 12
critical x = 1: f'' = -8 -> local max, f = 13
possible inflection at x = -sqrt(3)/3, f = 113/9
possible inflection at x = sqrt(3)/3, f = 113/9
```

In Context: The inflection point of a technology

New technologies, such as mobile money, spread along an S-shaped *logistic* curve. Few people use them at first, then adoption is rapid, and finally the market saturates. The inflection point is the moment of *fastest growth*, after which growth slows down. In the



Figure 5.3: $f(x) = 12 + 2x^2 - x^4$: two maxima, one minimum and two inflection points.

illustrative model $N(t) = \dfrac{40}{1 + e^{-0.6(t-8)}}$ (millions of accounts, t years after 2010), the inflection is at $t = 8$, the year 2018. There N is exactly half its limiting value, and N' is greatest. The same S-curve describes the spread of epidemics and the growth of a cassava crop.

Listing 5.7 — Inflection point of a logistic curve `ex5_07_logistic.py`

```python
# Listing 5.7: The inflection point of an S-shaped (logistic) adoption curve
# ILLUSTRATIVE model of active mobile-money accounts in a country (millions):
#   N(t) = 40 / (1 + e^(-0.6 (t - 8))),   t = years after 2010
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

t = sp.symbols("t", real=True)
N = 40 / (1 + sp.exp(-sp.Rational(3, 5)*(t - 8)))
N1, N2 = sp.diff(N, t), sp.diff(N, t, 2)
tp = sp.solve(sp.simplify(N2), t)
print("N''(t) = 0 at t =", tp, "-> year", 2010 + tp[0])
print("N at inflection =", N.subs(t, tp[0]), "million (half of 40)")
print("fastest growth N'(8) =", N1.subs(t, 8), "million per year")

T = np.linspace(0, 16, 300)
fig, ax = plt.subplots(1, 2, figsize=(9, 3))
ax[0].plot(2010 + T, sp.lambdify(t, N)(T)); ax[0].set_title("N(t)")
ax[1].plot(2010 + T, sp.lambdify(t, N1)(T), "C1"); ax[1].set_title("N'(t)")
for a in ax:
    a.axvline(2018, ls="--", color="gray"); a.grid(alpha=0.3)
plt.tight_layout(); plt.show()
```

Output

```
N''(t) = 0 at t = [8] -> year 2018
N at inflection = 20 million (half of 40)
fastest growth N'(8) = 6 million per year
```

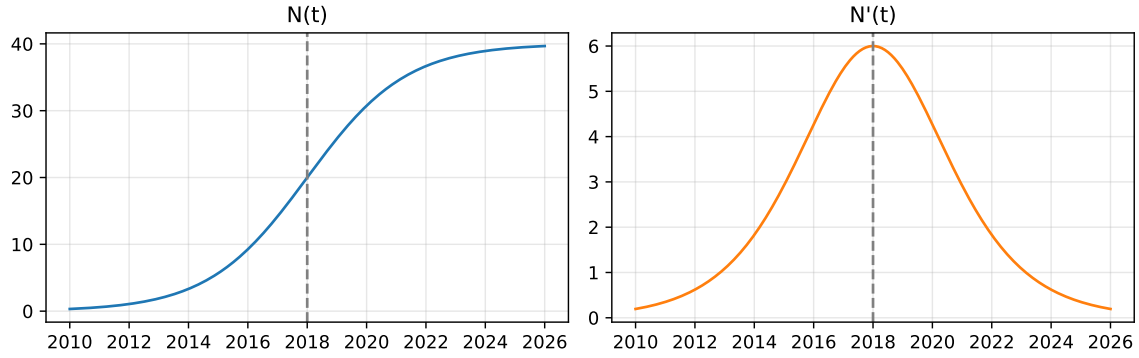


Figure 5.4: Left: the S-shaped adoption curve. Right: its derivative, which peaks at the inflection point.

5.6 Optimisation problems

Key Concept: Steps for solving optimisation problems

1. **Understand** the problem: what is to be maximised or minimised? What is given?
2. **Draw** a diagram and **introduce notation**.
3. Write the quantity to be optimised as a formula (for example $A = xy$).
4. Use the constraint(s) to reduce it to a function of **one** variable, and state its domain.
5. Find the critical numbers, then use the closed interval method or a first/second derivative test.
6. **Answer the question** that was asked, with units, and check that it is sensible.

Example 5.7 A farmer has 2400 m of fencing and wants to fence a rectangular cassava farm that borders the straight bank of the Pra River. No fence is needed along the river. What dimensions give the largest area?

Solution. Let x be the two sides perpendicular to the river and y the side parallel to it. Then $2x + y = 2400$, so $y = 2400 - 2x$, and the area is

$$A(x) = x(2400 - 2x) = 2400x - 2x^2, \qquad 0 \le x \le 1200.$$

$A'(x) = 2400 - 4x = 0$ gives $x = 600$. Then $A(0) = A(1200) = 0$ and $A(600) = 720\,000$. The best farm is 600 m by 1200 m (the long side along the river), with an area of $720\,000\ \text{m}^2 = 72$ hectares. ∎

Example 5.8 A right circular cylinder is inscribed in a sphere of radius R. If the height of the cylinder is $2x$, express its volume in terms of x and R, and find the maximum volume.

Solution. The base radius r satisfies $r^2 = R^2 - x^2$, so $V = \pi r^2 (2x) = 2\pi(R^2 x - x^3)$ for $0 \le x \le R$. Then $V' = 2\pi(R^2 - 3x^2) = 0$ gives $x = \frac{R}{\sqrt{3}}$, and $V'' = -12\pi x < 0$ there, so this is a maximum:

$$V_{\max} = 2\pi\left(\frac{R^3}{\sqrt{3}} - \frac{R^3}{3\sqrt{3}}\right) = \frac{4\pi R^3}{3\sqrt{3}} = \frac{4\sqrt{3}\,\pi R^3}{9}.$$

■

In Context: Designing a tin of tomato paste

A manufacturer in Tema wants a closed cylindrical tin that holds $400\,\text{cm}^3$ and uses the least metal. With radius r and height h, we have $\pi r^2 h = 400$, and the surface area is

$$S = 2\pi r^2 + 2\pi r h = 2\pi r^2 + \frac{800}{r}.$$

$S'(r) = 4\pi r - \frac{800}{r^2} = 0$ gives $r^3 = \frac{200}{\pi}$, so $r \approx 3.99\,\text{cm}$ and $h = 2r$. **The best tin is as tall as it is wide.** Look at the tins in a shop: many are close to this shape, although other factors, such as labels, stacking and seams, also matter.

In Context: Pricing a plate of waakye

A waakye seller near a university hostel finds that at a price of p cedis she sells about $q = 400 - 10p$ plates a day. Her revenue is $R(p) = p(400 - 10p) = 400p - 10p^2$, and $R'(p) = 400 - 20p = 0$ gives $p = 20$. At GH₵ 20 she sells 200 plates for a revenue of GH₵ 4000. Charging more loses too many customers, and charging less earns too little per plate. Ride-hailing "surge pricing" is a more sophisticated version of the same calculation.

Listing 5.8 — Farm, tin and waakye `ex5_08_optimization.py`

```
# Listing 5.8: Three optimisation problems solved with SymPy
import sympy as sp

x, r, p = sp.symbols("x r p", positive=True)

# (a) Farm beside the Pra River: 2400 m of fencing, no fence along the river.
A = x*(2400 - 2*x)
xs = sp.solve(sp.diff(A, x), x)[0]
print(f"(a) x = {xs} m, y = {2400 - 2*xs} m, area = {A.subs(x, xs)} m^2")

# (b) A closed tin can holding 400 cm^3 using the least metal.
h = 400 / (sp.pi*r**2)                      # from the volume constraint
S = 2*sp.pi*r**2 + 2*sp.pi*r*h              # surface area
rs = sp.solve(sp.diff(S, r), r)[0]
print(f"(b) r = {float(rs):.3f} cm, h = {float(h.subs(r, rs)):.3f} cm,"
      f" h/r = {sp.simplify(h.subs(r, rs)/rs)}, S = {float(S.subs(r, rs)):.1f} cm^2")

# (c) A waakye seller sells q = 400 - 10p plates a day at price p (GH¢). Max revenue?
R = p*(400 - 10*p)
ps = sp.solve(sp.diff(R, p), p)[0]
print(f"(c) price p = GH¢ {ps}, plates sold = {400 - 10*ps}, revenue = GH¢ {R.subs(p, ps)}
    ")
```

Output

```
(a) x = 600 m, y = 1200 m, area = 720000 m^2
(b) r = 3.993 cm, h = 7.986 cm, h/r = 2, S = 300.5 cm^2
(c) price p = GH¢ 20, plates sold = 200, revenue = GH¢ 4000
```

In Context: The cheapest speed between Accra and Cape Coast

Driving faster burns more fuel per km, but it saves the driver's time, which also has a value. In the illustrative model

$$C(v) = 0.6 + 0.01v + \frac{40}{v} \qquad \text{GH¢ per km at } v \text{ km/h},$$

$C'(v) = 0.01 - \frac{40}{v^2} = 0$ gives $v^* = \sqrt{4000} \approx 63\,\text{km/h}$. Also $C''(v) = \frac{80}{v^3} > 0$, so this is a minimum. On the 145 km Accra–Cape Coast trip, driving at 100 km/h costs about GH¢ 20 more than driving at 63 km/h.

Listing 5.9 — Optimal cruising speed, by calculus and by SciPy `ex5_09_driver_speed.py`

```python
# Listing 5.9: The cheapest cruising speed for a driver on the Accra-Cape Coast highway
# Illustrative cost per km at speed v km/h:
#   C(v) = 0.6 + 0.01 v + 40 / v   GH¢/km
#   (fixed costs + fuel, which rises with speed + driver's time at GH¢ 40/hour)
import numpy as np
from scipy.optimize import minimize_scalar
import matplotlib.pyplot as plt

C = lambda v: 0.6 + 0.01*v + 40/v

# Calculus: C'(v) = 0.01 - 40/v^2 = 0  ->  v = sqrt(4000)
v_star = np.sqrt(40/0.01)
print(f"calculus : v* = {v_star:.2f} km/h,  C(v*) = GH¢ {C(v_star):.3f} per km")

# Numerical check with SciPy
res = minimize_scalar(C, bounds=(20, 120), method="bounded")
print(f"scipy    : v* = {res.x:.2f} km/h,  C(v*) = GH¢ {res.fun:.3f} per km")
print(f"cost of the 145 km trip at 63 km/h: GH¢ {145*C(63):.2f};  at 100 km/h: GH¢ {145*C
    (100):.2f}")

v = np.linspace(20, 120, 200)
plt.figure(figsize=(5.5, 3.3))
plt.plot(v, C(v)); plt.plot(v_star, C(v_star), "ro")
plt.xlabel("speed v (km/h)"); plt.ylabel("cost (GH¢ per km)"); plt.grid(alpha=0.3)
plt.show()
```

Output

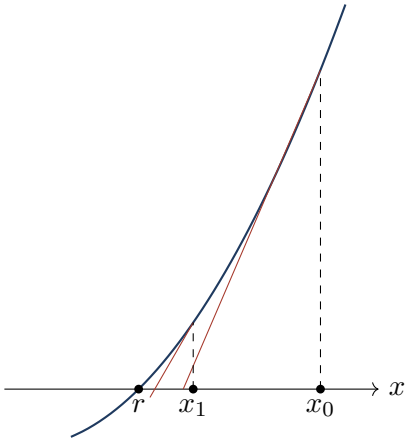
```
calculus : v* = 63.25 km/h,  C(v*) = GH¢ 1.865 per km
scipy    : v* = 63.25 km/h,  C(v*) = GH¢ 1.865 per km
cost of the 145 km trip at 63 km/h: GH¢ 270.41;  at 100 km/h: GH¢ 290.00
```

Python Tip: `scipy.optimize`

`minimize_scalar(f, bounds=(a, b), method="bounded")` finds the minimum of a one-variable function on $[a, b]$ numerically. To *maximise* f, minimise $-f$. Use calculus to understand a problem and SciPy to check the answer, or to solve problems too messy to do by hand.

5.7 Newton's method

To solve $f(x) = 0$, start with a guess x_0. Replace the curve by its tangent at $(x_0, f(x_0))$ and find where the tangent crosses the x-axis: $0 = f(x_0) + f'(x_0)(x - x_0)$. This gives a (usually) better guess x_1. Then repeat.



Key Concept: Newton's method

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \qquad n = 0, 1, 2, \ldots$$

Close to a simple root, the number of correct digits roughly *doubles* at each step. The method can fail if $f'(x_n) \approx 0$ or the first guess is poor, so always check the answer.

Historical Note: Heron of Alexandria and the square root

For $f(x) = x^2 - a$, Newton's formula becomes $x_{n+1} = \frac{1}{2}\left(x_n + \frac{a}{x_n}\right)$: average your guess with a divided by your guess. This is *Heron's method*, described by Heron of Alexandria, Egypt, in the 1st century CE. It was probably known to the Babylonians even earlier. Newton described his general method around 1669, and Joseph Raphson published a simpler form of it in 1690. That is why it is often called the *Newton–Raphson* method. Calculators and computers still find square roots essentially this way.

Listing 5.10 — Newton's method `ex5_10_newton.py`

```
# Listing 5.10: Newton's method x_{n+1} = x_n - f(x_n)/f'(x_n)
import math

def newton(f, df, x0, tol=1e-12, max_steps=50, show=False):
    x = x0
    for n in range(1, max_steps + 1):
```

```
        x_new = x - f(x)/df(x)
        if show:
            print(f"  step {n}: x = {x_new:.15f}")
        if abs(x_new - x) < tol:
            return x_new, n
        x = x_new
    raise RuntimeError("Newton's method did not converge")

# (a) Heron of Alexandria's square root of 2:  f(x) = x^2 - 2
print("sqrt(2):")
root, n = newton(lambda x: x*x - 2, lambda x: 2*x, 1.0, show=True)
print(f"  math.sqrt(2) = {math.sqrt(2):.15f}")

# (b) x^3 - x - 2 = 0 (compare the bisection method of Listing 3.9: 20 steps)
root, n = newton(lambda x: x**3 - x - 2, lambda x: 3*x**2 - 1, 1.5)
print(f"x^3 - x - 2 = 0:  x = {root:.10f} after {n} steps")

# (c) cos x = x
root, n = newton(lambda x: math.cos(x) - x, lambda x: -math.sin(x) - 1, 1.0)
print(f"cos x = x:        x = {root:.10f} after {n} steps")
```

Output

```
sqrt(2):
  step 1: x = 1.500000000000000
  step 2: x = 1.416666666666667
  step 3: x = 1.414215686274510
  step 4: x = 1.414213562374690
  step 5: x = 1.414213562373095
  step 6: x = 1.414213562373095
  math.sqrt(2) = 1.414213562373095
x^3 - x - 2 = 0:  x = 1.5213797068 after 4 steps
cos x = x:        x = 0.7390851332 after 5 steps
```

Compare Newton's 4 steps for $x^3 - x - 2 = 0$ with the 20 halvings that the bisection method needed in Listing 3.9.

5.8 L'Hôpital's rule

Theorem 5.4. (L'Hôpital's rule)

Suppose f and g are differentiable near a, and $\frac{f(x)}{g(x)}$ gives the indeterminate form $\frac{0}{0}$ or $\frac{\infty}{\infty}$ as $x \to a$. Then

$$\lim_{x\to a} \frac{f(x)}{g(x)} = \lim_{x\to a} \frac{f'(x)}{g'(x)},$$

provided the limit on the right exists (or is $\pm\infty$). The rule also holds for one-sided limits and for $a = \pm\infty$.

Example 5.9 $\displaystyle\lim_{x\to 0} \frac{e^x - 1 - x}{x^2} \overset{0/0}{=} \lim_{x\to 0} \frac{e^x - 1}{2x} \overset{0/0}{=} \lim_{x\to 0} \frac{e^x}{2} = \frac{1}{2}.$

Watch Out: Not the quotient rule

L'Hôpital's rule differentiates the top and the bottom *separately*. It is not the quotient rule. Use it only for $\frac{0}{0}$ or $\frac{\infty}{\infty}$. For example, $\lim_{x\to 0} \frac{\cos x}{x+1} = 1$ by substitution, but applying the rule would wrongly give $\lim_{x\to 0} \frac{-\sin x}{1} = 0$.

Historical Note: A textbook bought from its author

The rule is named after the French Marquis Guillaume de l'Hôpital, who in 1696 published *Analyse des infiniment petits*, the first calculus textbook. The rule was in fact discovered by his teacher, Johann Bernoulli, whom l'Hôpital paid for his mathematical discoveries under a private agreement.

Listing 5.11 — L'Hôpital's rule checked with SymPy `ex5_11_lhopital.py`

```
# Listing 5.11: L'Hôpital's rule checked with SymPy
import sympy as sp

x = sp.symbols("x")
cases = [
    ((sp.exp(x) - 1 - x), x**2, 0),
    (sp.log(x), x - 1, 1),
    (x - sp.sin(x), x**3, 0),
    (sp.log(x), x, sp.oo),
    (x**2, sp.exp(x), sp.oo),
]
for f, g, a in cases:
    ratio_of_derivs = sp.diff(f, x) / sp.diff(g, x)
    print(f"lim {str(f/g):<22} x->{str(a):<3} = {str(sp.limit(f/g, x, a)):<4}"
          f"  (f'/g' -> {sp.limit(ratio_of_derivs, x, a)})")
```

Output

```
lim (-x + exp(x) - 1)/x**2 x->0   = 1/2   (f'/g' -> 1/2)
lim log(x)/(x - 1)         x->1   = 1     (f'/g' -> 1)
lim (x - sin(x))/x**3      x->0   = 1/6   (f'/g' -> 1/6)
lim log(x)/x               x->oo  = 0     (f'/g' -> 0)
lim x**2*exp(-x)           x->oo  = 0     (f'/g' -> 0)
```

Chapter Summary

- Tangent: $y - y_1 = f'(x_1)(x - x_1)$. Normal: gradient $-1/f'(x_1)$.
- Related rates: write a relation valid for all t, differentiate with respect to t, and substitute last.
- Critical numbers: $f'(c) = 0$ or $f'(c)$ undefined. Closed interval method for absolute extrema.
- First derivative test: a sign change of f'. Second derivative test: $f''(c) < 0$ gives a maximum and $f''(c) > 0$ a minimum.
- Inflection point: f'' changes sign.
- Optimisation: model, reduce to one variable, differentiate, check and interpret.

- Newton: $x_{n+1} = x_n - f(x_n)/f'(x_n)$. L'Hôpital: for $\frac{0}{0}$ or $\frac{\infty}{\infty}$, use $\frac{f'}{g'}$.
- Python: `np.polyfit` to fit models; `sp.solve(sp.diff(f, x), x)` for critical numbers; `scipy.optimize.minimize_scalar` to check.

5.9 Exercises

A. Tangents, normals and related rates

1. Find the equations of the tangent and normal to the graph of $x^3 - x\ln y + y^3 = 2x + 5$ at the point where $x = 2$.

2. Find the equations of the tangent and normal to $f(x) = x\ln\frac{e}{x}$ at $x = e$.

3. Find the smallest positive value of the constant k such that the graphs of $y = e^x$ and $y = k\sin x$ are tangent to each other, and find the point of tangency.

4. For what non-negative value(s) of b is the line $y = -\frac{1}{12}x + b$ normal to $y = \frac{1}{3}x^3$?

5. The point (a, b) lies on $y = x^3 + x^2 - 9x - 9$, and the tangent there passes through $(4, -1)$. Find all such points (a, b).

6. At what point(s) of $y = x^5 + 4x - 3$ does the tangent pass through $B(0, 1)$?

7. If the line $4x - 9y = 0$ is tangent in the first quadrant to $y = \frac{1}{3}x^3 + c$, find c.

8. Show that the sum of the x- and y-intercepts of every tangent line to the curve $\sqrt{x} + \sqrt{y} = \sqrt{k}$ is equal to k.

9. Find an equation of the tangent to $y = 4x\sqrt{x} - \frac{25x^2}{16}$ at $x = 4$, in the form $ax + by + c = 0$ with integers a, b and c. (*Past examination question.*)

10. An inverted conical water tank has radius 2 m and depth 5 m. Water is pumped in at $0.3\,\text{m}^3/\text{min}$. How fast is the water level rising when the water is 1 m deep?

11. A cylindrical silo of radius 3 m is being filled with maize at $2.83\,\text{m}^3$ per minute. How fast is the depth of maize increasing?

12. A 1.5 m tall student walks towards a 6 m tall street lamp at 1.8 m/s. How fast is the tip of the student's shadow moving?

13. A rocket is launched vertically, and its height after t seconds is $s = 400t - 16t^2$ metres. How fast is its distance from an observer 1800 m from the launch site changing when the rocket is rising and is 2400 m high?

B. Extrema, concavity and curve sketching

14. If $y = xe^x - e^x - 2x^2$, find any local maximum and minimum values. Is there an absolute maximum or minimum?

15. Given $f(x) = x^{3/5}(4 - x)$: (a) find the critical numbers; (b) find the intervals of increase and decrease; (c) find the local extrema.

16. Find the critical numbers of $f(x) = (x-1)^{2/3}$ and decide whether they give extrema or inflection points.

17. Find the absolute extrema on the given interval: (a) $f(x) = \sin x + x$ on $[0, 2\pi]$; (b) $f(x) = \dfrac{2x+5}{(x^2-4)^2}$ on $[-5, -3]$; (c) $f(x) = \cos^2 x + \sin x$ on $[0, \pi]$.

18. For $y = x^3 - 6x^2 + 9x + 2$: (a) find the local extrema; (b) find the point of inflection; (c) sketch the curve.

19. If $g(x) = x(x^2 - 2)$, where does the inflection point occur? (*Past examination question.*)

20. Use l'Hôpital's rule: (a) $\lim_{x\to 0} \frac{\tan x - x}{x^3}$; (b) $\lim_{x\to\infty} \frac{\ln x}{\sqrt{x}}$; (c) $\lim_{x\to 0^+} x \ln x$.

C. Optimisation and applications

21. An open box is made from a $20\,\text{cm} \times 12\,\text{cm}$ sheet of cardboard by cutting equal squares of side x from the corners and folding up the flaps. (a) Express the volume as a function of x. (b) Find the dimensions that give the greatest volume, and that volume to the nearest cm^3.

22. The volume of water in a bird bath is $V = \dfrac{60t + 2}{3t}$ mL at time t hours ($t > 0$). (a) Show that $\frac{dV}{dt} < 0$. (b) At what rate is the water evaporating after 2 hours?

23. A window is a rectangle surmounted by a semicircle of radius $x\,\text{cm}$. The frame, including the base of the semicircle, uses 336 cm of wood. (a) Show that the area is $A = 336x - \frac{1}{2}(4 + \pi)x^2$. (b) Find the width and height that admit the most light. (c) If the width may not exceed 84 cm, what are the best dimensions?

24. A closed cylinder has total surface area $200\pi\,\text{cm}^2$. (a) Express the height h in terms of the radius r. (b) Show that $V = \pi(100r - r^3)$. (c) Show that for maximum volume the height equals the diameter.

25. A right circular cone is inscribed in a sphere of radius 7 cm. (a) Show that the volume of the cone is $V = \frac{\pi}{3}(14h^2 - h^3)$. (b) Find the exact r and h that maximise V, justifying your answer.

26. ⚙ A cocoa cooperative needs a rectangular drying platform of area $600\,\text{m}^2$, fenced on three sides (the fourth side is a warehouse wall). Find the dimensions that use the least fencing.

27. ⚙ A Uber driver in Accra earns GH₵ $E(h) = 60h - 4h^2$ (after costs) in a shift of h hours (illustrative). How long should a shift be to maximise earnings? Explain why earnings fall for very long shifts.

28. ⚙ A fisherman's canoe is 3 km offshore from the nearest point P of a straight beach at Elmina. His home is 8 km along the beach from P. He paddles at 4 km/h and walks at 6 km/h. Where should he land to reach home as quickly as possible?

29. ⚙ In an illustrative model, a ride-hailing driver's earnings per hour depend on the surge multiplier m ($1 \le m \le 2.5$) as $E(m) = m(40 - 12m)$ cedis, because fewer riders accept higher prices. Find the multiplier that maximises E. Why does a platform not simply charge the highest multiplier?

D. Programming

30. Change Listing 5.3 to model *your* usual trip (to campus, to work or to church). Collect about ten real trip times at different departure times, fit a polynomial with `np.polyfit` as in Listing 5.4, and find the best departure time.

31. Write a function `classify_critical_points(f)` that uses SymPy to find all critical numbers of a polynomial f and labels each one as a local maximum, a local minimum or neither (use the second derivative test, falling back to the first derivative test).

32. Use Newton's method to find all three real roots of $x^3 - 6x^2 + 9x - 1 = 0$. Plot the function first to choose good starting points.

33. Use `minimize_scalar` to solve the fisherman problem (Exercise 28) numerically, and compare with your calculus answer.

Chapter 6

Integration

Learning Objectives

After working through this chapter you should be able to:

- find antiderivatives (indefinite integrals) using a table of basic integrals;
- integrate by substitution (change of variable) and by parts;
- approximate areas by Riemann sums and define the definite integral;
- evaluate definite integrals with the Fundamental Theorem of Calculus;
- approximate integrals with the trapezium and Simpson's rules, and with `scipy.integrate.quad`;
- check every integral in Python with `sp.integrate` or by differentiating.

This chapter develops the ideas and techniques of integration. The many uses of the integral (areas, distances, totals, averages, volumes, lengths and probabilities) are the subject of Chapter 7.

Historical Note: From Archimedes to Ibn al-Haytham to Riemann

Integration, the measuring of areas and volumes, is older than differentiation. The Egyptian *Moscow Papyrus* (c. 1850 BCE) gives the correct volume of a truncated pyramid. In *The Quadrature of the Parabola* (c. 250 BCE), Archimedes proved that a parabolic segment has exactly $\frac{4}{3}$ the area of a certain inscribed triangle. He did this by filling the segment with infinitely many ever-smaller triangles.

Around the year 1000, Ibn al-Haytham (Alhazen) worked in **Cairo**, where he also wrote his famous *Book of Optics*. He found formulas for the sums $1^4 + 2^4 + \cdots + n^4$ and used them to find the volume of a paraboloid. In modern terms, he computed $\int x^4\,dx$. The great insight of Newton and Leibniz (and of Isaac Barrow before them) was that integration is the *reverse* of differentiation. This is the Fundamental Theorem of Calculus, which turned a hard geometric problem into a routine calculation. Bernhard Riemann gave the modern definition of the integral as a limit of sums in 1854.

6.1 Antiderivatives and indefinite integrals

Definition 6.1. (Antiderivative)

A function F is an *antiderivative* of f on an interval if $F'(x) = f(x)$ for every x in the interval.

For example, $F(x) = x^2$ is an antiderivative of $f(x) = 2x$. So are $x^2 + 2$, $x^2 - 5$ and $x^2 + \sqrt{3}$. Adding a constant does not change the derivative. In fact *every* antiderivative of $2x$ has the form $x^2 + C$.

Definition 6.2. (Indefinite integral)

The *indefinite integral* of f is the family of all its antiderivatives:

$$\int f(x)\,\mathrm{d}x = F(x) + C, \qquad \text{where } F'(x) = f(x).$$

$f(x)$ is the *integrand* and C is the *constant of integration*.

Key Concept: Table of basic integrals

$$\int k\,\mathrm{d}x = kx + C \qquad \int x^n\,\mathrm{d}x = \frac{x^{n+1}}{n+1} + C \quad (n \neq -1)$$

$$\int \frac{1}{x}\,\mathrm{d}x = \ln|x| + C \qquad \int e^x\,\mathrm{d}x = e^x + C$$

$$\int \cos x\,\mathrm{d}x = \sin x + C \qquad \int \sin x\,\mathrm{d}x = -\cos x + C$$

$$\int \sec^2 x\,\mathrm{d}x = \tan x + C \qquad \int \csc^2 x\,\mathrm{d}x = -\cot x + C$$

$$\int \sec x \tan x\,\mathrm{d}x = \sec x + C \qquad \int \csc x \cot x\,\mathrm{d}x = -\csc x + C$$

Properties: $\displaystyle\int cf(x)\,\mathrm{d}x = c\int f(x)\,\mathrm{d}x$ and $\displaystyle\int [f(x) \pm g(x)]\,\mathrm{d}x = \int f(x)\,\mathrm{d}x \pm \int g(x)\,\mathrm{d}x.$

Example 6.1

(a) $\displaystyle\int x^8\,\mathrm{d}x = \frac{x^9}{9} + C.$

(b) $\displaystyle\int \frac{1}{x^3}\,\mathrm{d}x = \int x^{-3}\,\mathrm{d}x = \frac{x^{-2}}{-2} + C = -\frac{1}{2x^2} + C.$

(c) $\displaystyle\int \sqrt[3]{x^2}\,\mathrm{d}x = \int x^{2/3}\,\mathrm{d}x = \frac{x^{5/3}}{5/3} + C = \frac{3}{5}x^{5/3} + C.$

(d) $\displaystyle\int \frac{\tan x}{\sec x}\,\mathrm{d}x = \int \frac{\sin x}{\cos x}\cos x\,\mathrm{d}x = \int \sin x\,\mathrm{d}x = -\cos x + C.$

Example 6.2

(a) $\displaystyle\int (5x^3 + 2\cos x)\,\mathrm{d}x = \frac{5x^4}{4} + 2\sin x + C.$

(b) $\displaystyle\int \left(8x^3 - 6\sqrt{x} + \frac{1}{x^3}\right)\mathrm{d}x = 2x^4 - 4x^{3/2} - \frac{1}{2x^2} + C.$

(c) $\displaystyle\int \frac{(x^2-1)^2}{x^2}\,dx = \int \left(x^2 - 2 + x^{-2}\right)\,dx = \frac{x^3}{3} - 2x - \frac{1}{x} + C.$

(d) $\displaystyle\int \frac{1}{\cos x \cot x}\,dx = \int \frac{\sin x}{\cos^2 x}\,dx = \int \sec x \tan x\,dx = \sec x + C.$

Key Concept: Always check by differentiating

An integral can always be checked: differentiate your answer, and you must get back the integrand. Listing 6.1 does exactly this for every example.

Listing 6.1 — Indefinite integrals with SymPy `ex6_01_indefinite.py`

```python
# Listing 6.1: Indefinite integrals with sp.integrate (SymPy omits the + C)
import sympy as sp

x = sp.symbols("x", positive=True)
examples = [
    3*x**2, sp.cos(x), x**8, 1/x**3, sp.cbrt(x**2), sp.tan(x)/sp.sec(x),
    5*x**3 + 2*sp.cos(x), 8*x**3 - 6*sp.sqrt(x) + 1/x**3, (x**2 - 1)**2/x**2,
    1/(sp.cos(x)*sp.cot(x)),
]
for f in examples:
    F = sp.integrate(f, x)
    ok = sp.simplify(sp.diff(F, x) - f) == 0      # always check: F' = f
    print(f"∫ {str(f):<28} dx = {str(sp.simplify(F)):<34} check: {ok}")
```

Output

```
∫ 3*x**2                       dx = x**3                                  check: True
∫ cos(x)                       dx = sin(x)                                check: True
∫ x**8                         dx = x**9/9                                check: True
∫ x**(-3)                      dx = -1/(2*x**2)                           check: True
∫ x**(2/3)                     dx = 3*x**(5/3)/5                          check: True
∫ tan(x)/sec(x)                dx = -cos(x)                               check: True
∫ 5*x**3 + 2*cos(x)            dx = 5*x**4/4 + 2*sin(x)                   check: True
∫ -6*sqrt(x) + 8*x**3 + x**(-3) dx = (-8*x**(7/2) + 4*x**6 - 1)/(2*x**2) check: True
∫ (x**2 - 1)**2/x**2           dx = x**3/3 - 2*x - 1/x                    check: True
∫ 1/(cos(x)*cot(x))            dx = 1/cos(x)                              check: True
```

Watch Out: Don't forget $+C$; and $\int \frac{1}{x}$ is special

SymPy leaves out the constant C, but you must not: $\int 2x\,dx = x^2 + C$. The power rule does not work for $n = -1$, because it would divide by zero. Instead, $\int x^{-1}\,dx = \ln|x| + C$.

In Context: Working backwards from speed

If a Bolt car's velocity is $v(t) = 2.4t - 0.12t^2$ m/s (Section 4.7), its position is an antiderivative: $s(t) = \int v\,dt = 1.2t^2 - 0.04t^3 + C$. If the car starts at the toll booth, $s(0) = 0$, so $C = 0$. A condition like this, which fixes the constant, is called an *initial condition*.

6.2 Integration by substitution

The substitution method is the chain rule run backwards. If $u = g(x)$, then $\mathrm{d}u = g'(x)\,\mathrm{d}x$, and

$$\int f(g(x))\,g'(x)\,\mathrm{d}x = \int f(u)\,\mathrm{d}u.$$

Key Concept: Steps for substitution

1. Choose u, usually the "inside" function, whose derivative also appears (up to a constant factor) in the integrand.
2. Compute $\mathrm{d}u = g'(x)\,\mathrm{d}x$, and rewrite the *whole* integral in terms of u.
3. Integrate with respect to u.
4. Substitute back $u = g(x)$.

Example 6.3 Evaluate $\displaystyle\int \sqrt{5x+7}\,\mathrm{d}x$.

Solution. Let $u = 5x+7$, so $\mathrm{d}u = 5\,\mathrm{d}x$, that is $\mathrm{d}x = \frac{1}{5}\,\mathrm{d}u$. Then

$$\int \sqrt{5x+7}\,\mathrm{d}x = \frac{1}{5}\int u^{1/2}\,\mathrm{d}u = \frac{1}{5}\cdot\frac{2}{3}u^{3/2} + C = \frac{2}{15}(5x+7)^{3/2} + C.$$

■

Example 6.4 Evaluate $\displaystyle\int \frac{x^2-1}{(x^3-3x+1)^6}\,\mathrm{d}x$.

Solution. Let $u = x^3 - 3x + 1$, so $\mathrm{d}u = 3(x^2-1)\,\mathrm{d}x$. Then

$$\int \frac{x^2-1}{(x^3-3x+1)^6}\,\mathrm{d}x = \frac{1}{3}\int u^{-6}\,\mathrm{d}u = \frac{1}{3}\cdot\frac{u^{-5}}{-5} + C = -\frac{1}{15(x^3-3x+1)^5} + C.$$

■

Example 6.5

(a) $\displaystyle\int \cos^3 5x \sin 5x\,\mathrm{d}x$: let $u = \cos 5x$, so $\mathrm{d}u = -5\sin 5x\,\mathrm{d}x$, giving $-\frac{1}{5}\int u^3\,\mathrm{d}u = -\frac{\cos^4 5x}{20} + C$.

(b) $\displaystyle\int \frac{e^{\sqrt{x}}}{\sqrt{x}}\,\mathrm{d}x$: let $u = \sqrt{x}$, so $2\,\mathrm{d}u = \frac{\mathrm{d}x}{\sqrt{x}}$, giving $2\int e^u\,\mathrm{d}u = 2e^{\sqrt{x}} + C$.

(c) $\displaystyle\int \frac{\ln x}{x}\,\mathrm{d}x$: let $u = \ln x$, so $\mathrm{d}u = \frac{\mathrm{d}x}{x}$, giving $\int u\,\mathrm{d}u = \frac{(\ln x)^2}{2} + C$. (The third edition misprinted this answer.)

(d) $\displaystyle\int \cos 4x\,\mathrm{d}x = \frac{1}{4}\sin 4x + C$, $\displaystyle\int (2x^3+1)^7 x^2\,\mathrm{d}x = \frac{(2x^3+1)^8}{48} + C$.

Listing 6.2 — Checking substitution answers `ex6_02_substitution.py`

```
# Listing 6.2: Integration by substitution, checked by differentiating
import sympy as sp

```

```
x = sp.symbols("x", positive=True)
hand_answers = {                                   # integrand: our answer by hand
    sp.sqrt(5*x + 7):                 sp.Rational(2, 15)*(5*x + 7)**sp.Rational(3, 2),
    (x**2 - 1)/(x**3 - 3*x + 1)**6:   -1/(15*(x**3 - 3*x + 1)**5),
    sp.cos(5*x)**3*sp.sin(5*x):       -sp.cos(5*x)**4/20,
    sp.exp(sp.sqrt(x))/sp.sqrt(x):    2*sp.exp(sp.sqrt(x)),
    sp.log(x)/x:                      sp.log(x)**2/2,
    sp.cos(4*x):                      sp.sin(4*x)/4,
    (2*x**3 + 1)**7*x**2:             (2*x**3 + 1)**8/48,
}
for f, F in hand_answers.items():
    ok = sp.simplify(sp.diff(F, x) - f) == 0
    print(f"∫ {str(f):<32} dx = {str(F):<30} + C   correct: {ok}")
```

Output

```
∫ sqrt(5*x + 7)                  dx = 2*(5*x + 7)**(3/2)/15          + C   correct: True
∫ (x**2 - 1)/(x**3 - 3*x + 1)**6 dx = -1/(15*(x**3 - 3*x + 1)**5)    + C   correct: True
∫ sin(5*x)*cos(5*x)**3           dx = -cos(5*x)**4/20                + C   correct: True
∫ exp(sqrt(x))/sqrt(x)           dx = 2*exp(sqrt(x))                 + C   correct: True
∫ log(x)/x                       dx = log(x)**2/2                    + C   correct: True
∫ cos(4*x)                       dx = sin(4*x)/4                     + C   correct: True
∫ x**2*(2*x**3 + 1)**7           dx = (2*x**3 + 1)**8/48             + C   correct: True
```

Check Your Understanding

Evaluate (a) $\int x\,e^{x^2}\,dx$; (b) $\int \frac{2x+3}{x^2+3x+5}\,dx$; (c) $\int \sin^2 x \cos x\,dx$.

6.3 Area and Riemann sums

How can we find the area under $y = x^2$ between $x = 0$ and $x = 1$? The region has a curved side, so no school formula applies. Follow Archimedes: approximate it with rectangles, then let the rectangles become thinner and thinner.

Divide $[a, b]$ into n strips of width $\Delta x = \frac{b-a}{n}$, with points $x_i = a + i\,\Delta x$. On each strip, build a rectangle whose height is the value of f at a sample point x_i^* (the left end, the right end or the midpoint). The total area of the rectangles is a **Riemann sum**:

$$R_n = \sum_{i=1}^{n} f(x_i^*)\,\Delta x.$$

Listing 6.3 — Riemann sums ex6_03_riemann_sums.py

```
# Listing 6.3: Riemann sums for the area under y = x^2 on [0, 1]
import numpy as np
import matplotlib.pyplot as plt

f = lambda x: x**2
a, b = 0, 1

```

```python
def riemann(f, a, b, n, rule="left"):
    dx = (b - a) / n
    left = a + dx*np.arange(n)                # left endpoints x_0 ... x_{n-1}
    pts = {"left": left, "right": left + dx, "mid": left + dx/2}[rule]
    return np.sum(f(pts)) * dx

print(f"{'n':>6} {'left':>10} {'right':>10} {'midpoint':>10}")
for n in [4, 10, 100, 1000, 10000]:
    print(f"{n:>6} {riemann(f, a, b, n):>10.6f} {riemann(f, a, b, n, 'right'):>10.6f}"
          f" {riemann(f, a, b, n, 'mid'):>10.6f}")
print("exact area = 1/3 =", 1/3)

# Picture of the right-endpoint rectangles for n = 8
n = 8; dx = (b - a)/n; xr = a + dx*np.arange(1, n + 1)
X = np.linspace(a, b, 200)
plt.figure(figsize=(5, 3.4))
plt.bar(xr - dx, f(xr), width=dx, align="edge", alpha=0.35, edgecolor="k", label="right
    sums, n = 8")
plt.plot(X, f(X), "C3", lw=2, label="$y = x^2$")
plt.legend(); plt.grid(alpha=0.3); plt.show()
```

Output

```
     n       left      right   midpoint
     4   0.218750   0.468750   0.328125
    10   0.285000   0.385000   0.332500
   100   0.328350   0.338350   0.333325
  1000   0.332834   0.333833   0.333333
 10000   0.333283   0.333383   0.333333
exact area = 1/3 = 0.3333333333333333
```

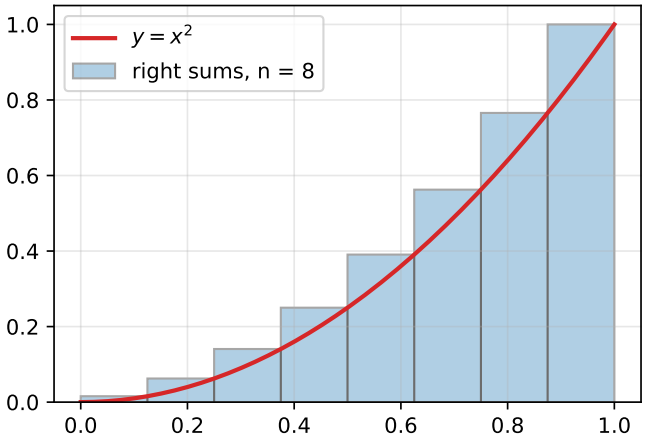


Figure 6.1: Right-endpoint rectangles overestimate the area under the increasing function $y = x^2$. As n grows, the excess shrinks to zero.

For $f(x) = x^2$ on $[0, 1]$ with right endpoints $x_i = \frac{i}{n}$, the sum can be found exactly using $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$:

$$R_n = \sum_{i=1}^{n} \left(\frac{i}{n}\right)^2 \frac{1}{n} = \frac{1}{n^3} \cdot \frac{n(n+1)(2n+1)}{6} = \frac{(1+\frac{1}{n})(2+\frac{1}{n})}{6} \xrightarrow{n\to\infty} \frac{2}{6} = \frac{1}{3}.$$

6.4 The definite integral

Definition 6.3. (Definite integral)

If f is continuous on $[a, b]$, the *definite integral* of f from a to b is

$$\int_a^b f(x)\,\mathrm{d}x = \lim_{n\to\infty} \sum_{i=1}^{n} f(x_i^*)\,\Delta x.$$

The numbers a and b are the lower and upper *limits of integration.*

Key Concept: Meaning and properties of $\int_a^b f(x)\,\mathrm{d}x$

- If $f \geq 0$, the integral is the **area** under the curve. In general it is the **signed area**: area above the x-axis counts as positive, and area below counts as negative.
- $\int_a^a f = 0, \quad \int_b^a f = -\int_a^b f, \quad \int_a^b f + \int_b^c f = \int_a^c f.$
- $\int_a^b [cf \pm g] = c\int_a^b f \pm \int_a^b g.$
- The **average value** of f on $[a, b]$ is $f_{\text{avg}} = \dfrac{1}{b-a}\displaystyle\int_a^b f(x)\,\mathrm{d}x.$

6.5 The Fundamental Theorem of Calculus

Theorem 6.1. (Fundamental Theorem of Calculus)

Let f be continuous on $[a, b]$.

1. The function $G(x) = \int_a^x f(t)\,\mathrm{d}t$ is differentiable, and $G'(x) = f(x)$.
2. If F is *any* antiderivative of f, then

$$\int_a^b f(x)\,\mathrm{d}x = F(b) - F(a) = \Big[F(x)\Big]_a^b.$$

Key Concept: Why the FTC is so important

Part 2 replaces the limit of a sum, which is hard, by the difference of two values of an antiderivative, which is easy. Part 1 says that differentiation and integration are *inverse* processes. Together they connect the two halves of calculus.

Example 6.6 $\displaystyle\int_1^2 x\,\mathrm{d}x = \left[\frac{x^2}{2}\right]_1^2 = \frac{4}{2} - \frac{1}{2} = \frac{3}{2}.$ $\displaystyle\int_0^1 x^2\,\mathrm{d}x = \left[\frac{x^3}{3}\right]_0^1 = \frac{1}{3}$, which agrees with the Riemann sums above.

Example 6.7 Evaluate $\displaystyle\int_2^{10} \frac{3}{\sqrt{5x-1}}\,\mathrm{d}x.$

Solution. Let $u = 5x - 1$, so $\mathrm{d}x = \frac{1}{5}\,\mathrm{d}u$. **Change the limits as well**: when $x = 2$, $u = 9$, and

when $x = 10$, $u = 49$. Then

$$\int_2^{10} \frac{3}{\sqrt{5x-1}}\,\mathrm{d}x = \frac{3}{5}\int_9^{49} u^{-1/2}\,\mathrm{d}u = \frac{3}{5}\Big[2u^{1/2}\Big]_9^{49} = \frac{6}{5}(7-3) = \frac{24}{5}.$$

■

Watch Out: Change the limits too

In a definite integral by substitution, either change the limits to u-values (as above), or substitute back to x before using the original limits. Never mix u-limits with an x-antiderivative.

Listing 6.4 — Definite integrals and the FTC ex6_04_ftc.py

```
# Listing 6.4: Definite integrals and the Fundamental Theorem of Calculus
import sympy as sp

x, t = sp.symbols("x t")
print("∫_1^2 x dx                =", sp.integrate(x, (x, 1, 2)))
print("∫_2^10 3/sqrt(5x-1) dx    =", sp.integrate(3/sp.sqrt(5*x - 1), (x, 2, 10)))
print("∫_0^pi sin x dx           =", sp.integrate(sp.sin(x), (x, 0, sp.pi)))
print("∫_0^2pi sin x dx          =", sp.integrate(sp.sin(x), (x, 0, 2*sp.pi)), " (signed
    area!)")

# FTC part 1: d/dx of ∫_0^x f(t) dt is f(x)
G = sp.integrate(t**3 + 1, (t, 0, x))
print("G(x) = ∫_0^x (t^3 + 1) dt =", G, ";  G'(x) =", sp.diff(G, x))

# Archimedes: the parabolic segment cut from y = x^2 by y = 1 has area 4/3 of
# the triangle with the same base and vertex.
segment = sp.integrate(1 - x**2, (x, -1, 1))
triangle = sp.Rational(1, 2) * 2 * 1
print("segment =", segment, ", triangle =", triangle, ", ratio =", segment/triangle)
```

Output

```
∫_1^2 x dx                = 3/2
∫_2^10 3/sqrt(5x-1) dx    = 24/5
∫_0^pi sin x dx           = 2
∫_0^2pi sin x dx          = 0  (signed area!)
G(x) = ∫_0^x (t^3 + 1) dt = x**4/4 + x ;  G'(x) = x**3 + 1
segment = 4/3 , triangle = 1 , ratio = 4/3
```

The last line of the output checks Archimedes' theorem. The region between $y = x^2$ and $y = 1$ has area $\frac{4}{3}$, which is exactly $\frac{4}{3}$ of the triangle with vertices $(-1, 1)$, $(1, 1)$ and $(0, 0)$.

6.6 Integration by parts

Substitution reverses the chain rule. *Integration by parts* reverses the product rule. Integrating $\frac{\mathrm{d}}{\mathrm{d}x}[uv] = u\,v' + v\,u'$ gives $uv = \int u\,v'\,\mathrm{d}x + \int v\,u'\,\mathrm{d}x$. Rearranging gives the following formula.

Key Concept: Integration by parts

$$\int u\,dv = uv - \int v\,du, \qquad \text{and} \qquad \int_a^b u\,dv = \Big[uv\Big]_a^b - \int_a^b v\,du.$$

Choosing u: let u be the factor that becomes *simpler* when differentiated. A useful order of preference is **LIATE**: **L**ogarithms, **I**nverse trigonometric functions, **A**lgebraic (powers of x), **T**rigonometric, **E**xponential.

Example 6.8 Evaluate $\displaystyle\int xe^x\,dx$.

Solution. Let $u = x$ and $dv = e^x\,dx$. Then $du = dx$ and $v = e^x$, so

$$\int xe^x\,dx = xe^x - \int e^x\,dx = xe^x - e^x + C = (x-1)e^x + C.$$

■

Example 6.9

(a) $\displaystyle\int x\cos x\,dx$: with $u = x$, $dv = \cos x\,dx$, we get $x\sin x - \int \sin x\,dx = x\sin x + \cos x + C$.

(b) $\displaystyle\int \ln x\,dx$: with $u = \ln x$, $dv = dx$, we get $x\ln x - \int x\cdot\frac{1}{x}\,dx = x\ln x - x + C$.

(c) $\displaystyle\int x^2e^x\,dx$: two applications give $(x^2 - 2x + 2)e^x + C$.

Example 6.10 Evaluate $\displaystyle I = \int e^x\sin x\,dx$.

Solution. Integrating by parts twice, with $u = \sin x$ and then $u = \cos x$:

$$I = e^x\sin x - \int e^x\cos x\,dx = e^x\sin x - \Big(e^x\cos x + \int e^x\sin x\,dx\Big) = e^x(\sin x - \cos x) - I.$$

The original integral has reappeared. Solving for I gives $I = \frac{1}{2}e^x(\sin x - \cos x) + C$. ■

Listing 6.5 — Integration by parts, checked `ex6_05_by_parts.py`

```python
# Listing 6.5: Integration by parts, checked with SymPy
import sympy as sp

x = sp.symbols("x", positive=True)

# Our answers by hand, using  ∫ u dv = u v - ∫ v du
hand_answers = {
    x*sp.exp(x):        (x - 1)*sp.exp(x),
    x*sp.cos(x):        x*sp.sin(x) + sp.cos(x),
    sp.log(x):          x*sp.log(x) - x,
    x**2*sp.exp(x):     (x**2 - 2*x + 2)*sp.exp(x),
    sp.exp(x)*sp.sin(x): sp.exp(x)*(sp.sin(x) - sp.cos(x))/2,
}
for f, F in hand_answers.items():
    ok = sp.simplify(sp.diff(F, x) - f) == 0
    print(f"∫ {str(f):<16} dx = {str(F):<32} + C   correct: {ok}")

```

```python
# A definite integral by parts
print("∫_0^pi x sin x dx =", sp.integrate(x*sp.sin(x), (x, 0, sp.pi)))
```

Output

```
∫ x*exp(x)        dx = (x - 1)*exp(x)                  + C   correct: True
∫ x*cos(x)        dx = x*sin(x) + cos(x)               + C   correct: True
∫ log(x)          dx = x*log(x) - x                    + C   correct: True
∫ x**2*exp(x)     dx = (x**2 - 2*x + 2)*exp(x)         + C   correct: True
∫ exp(x)*sin(x)   dx = (sin(x) - cos(x))*exp(x)/2      + C   correct: True
∫_0^pi x sin x dx = pi
```

Watch Out: A poor choice of u

If you choose $u = e^x$ and $\mathrm{d}v = x\,\mathrm{d}x$ in $\int xe^x\,\mathrm{d}x$, the new integral $\int \frac{x^2}{2}e^x\,\mathrm{d}x$ is *harder* than the original. If that happens, swap your choices.

6.7 Numerical integration

Many integrals cannot be done with formulas. Either the integrand is known only from data, or it has no elementary antiderivative, like e^{-x^2} or $\frac{\sin x}{x}$. We then approximate the integral. Divide $[a, b]$ into n strips of width $h = \frac{b-a}{n}$ and write $f_i = f(a + ih)$.

Key Concept: Trapezium and Simpson's rules

Trapezium rule: join neighbouring points with straight lines.

$$\int_a^b f(x)\,\mathrm{d}x \approx \frac{h}{2}\Big[f_0 + 2f_1 + 2f_2 + \cdots + 2f_{n-1} + f_n\Big].$$

Simpson's rule (n even): join groups of three points with parabolas.

$$\int_a^b f(x)\,\mathrm{d}x \approx \frac{h}{3}\Big[f_0 + 4f_1 + 2f_2 + 4f_3 + \cdots + 2f_{n-2} + 4f_{n-1} + f_n\Big].$$

Doubling n divides the error of the trapezium rule by about 4, and the error of Simpson's rule by about 16.

Listing 6.6 — Trapezium versus Simpson `ex6_06_numerical_rules.py`

```python
# Listing 6.6: Trapezium rule versus Simpson's rule on ∫_0^pi sin x dx = 2
import numpy as np

def trapezium(f, a, b, n):
    x = np.linspace(a, b, n + 1)
    h = (b - a) / n
    y = f(x)
    return h/2 * (y[0] + 2*y[1:-1].sum() + y[-1])

def simpson(f, a, b, n):                # n must be even
```

```
    x = np.linspace(a, b, n + 1)
    h = (b - a) / n
    y = f(x)
    return h/3 * (y[0] + 4*y[1:-1:2].sum() + 2*y[2:-1:2].sum() + y[-1])

print(f"{'n':>4} {'trapezium':>12} {'error':>10} {'Simpson':>12} {'error':>10}")
for n in [2, 4, 8, 16, 32]:
    T, S = trapezium(np.sin, 0, np.pi, n), simpson(np.sin, 0, np.pi, n)
    print(f"{n:>4} {T:>12.8f} {abs(T - 2):>10.2e} {S:>12.8f} {abs(S - 2):>10.2e}")
```

Output

```
   n    trapezium      error      Simpson      error
   2   1.57079633   4.29e-01   2.09439510   9.44e-02
   4   1.89611890   1.04e-01   2.00455975   4.56e-03
   8   1.97423160   2.58e-02   2.00026917   2.69e-04
  16   1.99357034   6.43e-03   2.00001659   1.66e-05
  32   1.99839336   1.61e-03   2.00000103   1.03e-06
```

The table confirms the error rates: each doubling of n cuts the trapezium error by a factor of about 4 and the Simpson error by a factor of about 16. Simpson's rule with only 8 strips is more accurate than the trapezium rule with 32.

Python Tip: Three ways to integrate in Python

- `sp.integrate(f, (x, a, b))`: exact, when a formula exists.
- `scipy.integrate.quad(f, a, b)`: very accurate numerical answer for a function you can evaluate anywhere, including infinite limits.
- `np.trapz(y, x)` or `scipy.integrate.simpson(y, x=x)`: for data at fixed points. (In NumPy 2.0 and later, `np.trapz` is renamed `np.trapezoid`.)

Listing 6.7 — Integrals with no formula ex6_07_quad.py

```
# Listing 6.7: Integrals with no formula - scipy.integrate.quad
import numpy as np
from scipy.integrate import quad

# exp(-x^2) has no elementary antiderivative, but its integral is easy to compute
val, err = quad(lambda x: np.exp(-x**2), 0, 1)
print(f"∫_0^1 exp(-x^2) dx = {val:.10f}  (error estimate {err:.1e})")

# quad also handles infinite limits
val, _ = quad(lambda x: np.exp(-x**2), -np.inf, np.inf)
print(f"∫ over all x       = {val:.10f};  sqrt(pi) = {np.sqrt(np.pi):.10f}")

# and integrands such as sin(x)/x
val, _ = quad(lambda x: np.sinc(x/np.pi), 0, 10)       # np.sinc(t) = sin(pi t)/(pi t)
print(f"∫_0^10 sin(x)/x dx = {val:.10f}")
```

Output

```
∫_0^1 exp(-x^2) dx = 0.7468241328  (error estimate 8.3e-15)
∫ over all x       = 1.7724538509;  sqrt(pi) = 1.7724538509
∫_0^10 sin(x)/x dx = 1.6583475942
```

Numerical integration is used throughout Chapter 7, where integrals are computed from measured data such as speedometer readings, rainfall and land surveys.

Chapter Summary

- $\int f(x)\,\mathrm{d}x = F(x) + C$ where $F' = f$. Check by differentiating.
- Substitution: $u = g(x)$, $\mathrm{d}u = g'(x)\,\mathrm{d}x$. For definite integrals, change the limits.
- $\int_a^b f(x)\,\mathrm{d}x = \lim_{n\to\infty} \sum f(x_i^*)\Delta x$: the signed area.
- FTC: $\int_a^b f(x)\,\mathrm{d}x = F(b) - F(a)$, and $\frac{\mathrm{d}}{\mathrm{d}x}\int_a^x f(t)\,\mathrm{d}t = f(x)$.
- By parts: $\int u\,\mathrm{d}v = uv - \int v\,\mathrm{d}u$ (choose u by LIATE).
- Trapezium and Simpson's rules for numerical integration. In Python: `sp.integrate`, `quad`, `np.trapz`, `simpson`.

6.8 Exercises

A. Skills

1. Evaluate: (a) $\int (4x^3 - 3x^2 + 2)\,\mathrm{d}x$; (b) $\int \left(\sqrt{x} + \frac{1}{\sqrt{x}}\right)^2 \mathrm{d}x$; (c) $\int \frac{x^3 - 2x + 5}{x}\,\mathrm{d}x$; (d) $\int (3\sec^2 x - 2e^x)\,\mathrm{d}x$; (e) $\int \frac{\sin x}{\cos^2 x}\,\mathrm{d}x$.

2. Evaluate $\int \frac{\mathrm{d}}{\mathrm{d}x}\left(x^2 \sin x\right)\,\mathrm{d}x$.

3. Find $\int \sec^2 5x\,\mathrm{d}x$. (*Past examination question.*)

4. If $\frac{\mathrm{d}^2 y}{\mathrm{d}x^2} = 10 - 3x$, find $\frac{\mathrm{d}y}{\mathrm{d}x}$. (*Past examination question.*)

5. Evaluate by substitution:

(a) $\int x(x^2 + 4)^{10}\,\mathrm{d}x$

(b) $\int \frac{x}{\sqrt{1 - x^2}}\,\mathrm{d}x$

(c) $\int \tan x\,\mathrm{d}x$

(d) $\int \frac{(\ln x)^3}{x}\,\mathrm{d}x$

(e) $\int \frac{e^x}{1 + e^x}\,\mathrm{d}x$

(f) $\int \sin^4 x \cos x\,\mathrm{d}x$

(g) $\int \frac{\cos\sqrt{x}}{\sqrt{x}}\,\mathrm{d}x$

(h) $\int x^2 e^{x^3}\,\mathrm{d}x$

6. Evaluate: (a) $\int_0^3 (x^2 - 2x)\,\mathrm{d}x$; (b) $\int_0^{\pi/2} \cos x\,\mathrm{d}x$; (c) $\int_1^e \frac{\mathrm{d}x}{x}$; (d) $\int_0^1 x e^{x^2}\,\mathrm{d}x$; (e) $\int_0^4 \frac{x}{\sqrt{x^2 + 9}}\,\mathrm{d}x$.

7. Find $\frac{\mathrm{d}}{\mathrm{d}x}\int_1^x \sqrt{1 + t^3}\,\mathrm{d}t$ and $\frac{\mathrm{d}}{\mathrm{d}x}\int_0^{x^2} \cos t\,\mathrm{d}t$.

8. Use the formula for $\sum_{i=1}^n i$ and right-endpoint Riemann sums to show that $\int_0^2 x\,\mathrm{d}x = 2$.

9. Integrate by parts: (a) $\int x e^{2x}\,\mathrm{d}x$; (b) $\int x \sin x\,\mathrm{d}x$; (c) $\int x^2 \ln x\,\mathrm{d}x$; (d) $\int_0^1 x e^{-x}\,\mathrm{d}x$.

10. Estimate $\int_0^1 e^{-x^2}\,\mathrm{d}x$ using (a) the trapezium rule and (b) Simpson's rule, each with $n = 4$. Compare with the value 0.746824.

B. Applications

11. A car starts from rest and its acceleration is $a(t) = 6 - 1.2t\ \mathrm{m/s^2}$ for $0 \le t \le 5\ \mathrm{s}$. Find its velocity $v(t)$ and its position $s(t)$, given $v(0) = 0$ and $s(0) = 0$.

12. The marginal cost of producing q bags of gari is $C'(q) = 12 - 0.02q$ cedis per bag, and the fixed cost is GH₵ 500. Find the cost function $C(q)$.

C. Programming

13. Without using SciPy, write a function `midpoint_rule(f, a, b, n)`. Add a column for it to the table of Listing 6.6. How does its error compare with that of the trapezium rule?

14. Use `plt.bar` to draw left, right and midpoint rectangles for $\int_0^2 e^{-x^2}\,\mathrm{d}x$ with $n = 6$, and compare each sum with the value from `quad`.

15. Use `sp.integrate` to check all your answers to Exercises 5, 6 and 9.

Chapter 7

Applications of Integration

Learning Objectives

After working through this chapter you should be able to:

- find the area of a region between two curves;
- use the integral of a rate to find a total change: distance, rainfall, volume of flow, cost;
- find the average value of a function on an interval;
- find volumes of solids of revolution by the disc and washer methods;
- find the length of a curve;
- compute areas and totals from measured data, and probabilities from a density;
- carry out all these calculations in Python, exactly or numerically.

Chapter 6 showed *how* to integrate. This chapter shows *why* integrals are so useful. The idea is always the same. To find a total, cut it into many small pieces, approximate each piece, add the pieces up, and let the pieces shrink. The limit of the sum is an integral:

$$\text{total} = \lim_{n\to\infty} \sum (\text{small piece}) = \int_a^b (\text{density of the quantity})\,\mathrm{d}x.$$

Historical Note: Spheres, wine barrels and curved lengths

Archimedes regarded his discovery that a sphere has exactly two-thirds of the volume of the cylinder that encloses it as his finest result. According to the Roman writer Cicero, a sphere and a cylinder were carved on his tomb. In 1615 Johannes Kepler, after watching a wine merchant measure barrels with a dipstick, wrote a book on the volumes of barrels in which he sliced solids into thin discs, the method of Section 7.4. For centuries many mathematicians believed that the length of a curve could never be found exactly. Then in 1657 the young English mathematician William Neile found the exact length of an arc of the curve $y^2 = x^3$, a curve of the same kind as the one in Example 7.6.

7.1 Area between curves

If $f(x) \geq g(x)$ on $[a, b]$, cut the region between the curves into thin vertical strips. A strip at x of width $\mathrm{d}x$ has height $f(x) - g(x)$, so

Key Concept: Area between two curves

$$A = \int_a^b \big[f(x) - g(x)\big]\,\mathrm{d}x \qquad \text{(top minus bottom)}.$$

Find a and b by solving $f(x) = g(x)$ for the points where the curves meet. If the curves cross inside $[a, b]$, split the integral at the crossing points.

Example 7.1 Find the area enclosed by $y = 2x$ and $y = x^2$.

Solution. The curves meet where $x^2 = 2x$, that is at $x = 0$ and $x = 2$. On $[0, 2]$ the line is on top, so $A = \int_0^2 (2x - x^2)\,\mathrm{d}x = \left[x^2 - \frac{x^3}{3}\right]_0^2 = 4 - \frac{8}{3} = \frac{4}{3}$. ■

Listing 7.1 — Area between two curves ex7_01_area_between.py

```python
# Listing 7.1: Area between two curves: y = 2x and y = x^2
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

x = sp.symbols("x")
top, bottom = 2*x, x**2
a, b = sp.solve(sp.Eq(top, bottom), x)          # intersection points
area = sp.integrate(top - bottom, (x, a, b))
print("curves meet at x =", a, "and x =", b, ";  area =", area)

X = np.linspace(-0.3, 2.3, 200)
plt.figure(figsize=(4.5, 3.6))
plt.plot(X, 2*X, label="$y = 2x$"); plt.plot(X, X**2, label="$y = x^2$")
Xf = np.linspace(0, 2, 100)
plt.fill_between(Xf, Xf**2, 2*Xf, alpha=0.3, label=f"area = {area}")
plt.legend(); plt.grid(alpha=0.3); plt.show()
```

Output

```
curves meet at x = 0 and x = 2 ;  area = 4/3
```

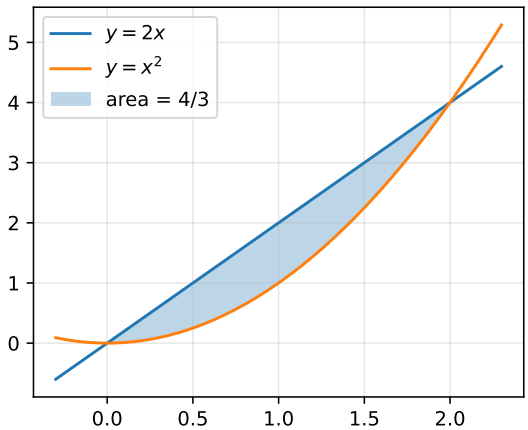


Figure 7.1: The region between $y = 2x$ and $y = x^2$.

Watch Out: Area is never negative

$\int_a^b f(x)\,dx$ counts area below the x-axis as negative. To find the *geometric* area between $y = f(x)$ and the x-axis, integrate $|f(x)|$, that is, split the interval where f changes sign. For example, the area between $y = \sin x$ and the axis for $0 \le x \le 2\pi$ is 4, not $\int_0^{2\pi} \sin x\,dx = 0$.

7.2 Accumulation: distance, rainfall and more

Key Concept: The integral of a rate is a total change

If $Q'(t)$ is the rate of change of a quantity Q, then

$$\int_a^b Q'(t)\,dt = Q(b) - Q(a) = \text{total change in } Q \text{ from } t = a \text{ to } t = b.$$

Integrating velocity gives displacement. Integrating rainfall intensity (mm/h) gives total rainfall (mm). Integrating a flow rate (litres/s) gives a volume (litres). Integrating marginal cost gives a change in cost.

Key Concept: Displacement and distance

For a body moving along a line with velocity $v(t)$:

$$\text{displacement} = \int_a^b v(t)\,dt, \qquad \text{distance travelled} = \int_a^b |v(t)|\,dt.$$

They differ when the body reverses direction (v changes sign).

Example 7.2 A particle moves with $v(t) = t^2 - 4t + 3 = (t-1)(t-3)$ m/s. Find its displacement and the distance it travels for $0 \le t \le 4$.

Solution. Displacement: $\int_0^4 (t^2 - 4t + 3)\,dt = \left[\frac{t^3}{3} - 2t^2 + 3t\right]_0^4 = \frac{4}{3}$ m. The velocity is negative on $(1, 3)$, so the distance is

$$\int_0^1 v\,dt - \int_1^3 v\,dt + \int_3^4 v\,dt = \frac{4}{3} + \frac{4}{3} + \frac{4}{3} = 4 \text{ m}.$$

■

When the rate is known only from *measurements*, we integrate numerically with the trapezium or Simpson's rule (Section 6.7).

In Context: How far did the trotro go?

A passenger reads a trotro's speedometer every 5 minutes on the Kasoa–Accra road, through the usual stop-start traffic. The distance travelled is $\int v\,dt$, the area under the speed–time graph. The trapezium rule gives about 19 km in 45 minutes, an average speed of about 25 km/h.

Listing 7.2 — Distance from speedometer readings `ex7_02_distance_from_speed.py`

```python
# Listing 7.2: Distance travelled from speedometer readings (trapezium rule)
# A trotro's speed (km/h) read every 5 minutes on the Kasoa - Accra road (illustrative)
import numpy as np

t_min = np.array([0, 5, 10, 15, 20, 25, 30, 35, 40, 45])
speed = np.array([0, 18, 35, 12, 8, 30, 52, 47, 25, 0])     # km/h
t_hr = t_min / 60

distance = np.trapz(speed, t_hr)                            # ∫ v dt
print(f"distance travelled ≈ {distance:.2f} km in {t_min[-1]} minutes")
print(f"average speed     ≈ {distance/t_hr[-1]:.1f} km/h")

# Same thing 'by hand': each strip is a trapezium with area h*(v_i + v_{i+1})/2
h = 5/60
by_hand = sum(h*(speed[i] + speed[i + 1])/2 for i in range(len(speed) - 1))
print(f"by hand           ≈ {by_hand:.2f} km")
```

Output

```
distance travelled ≈ 18.92 km in 45 minutes
average speed     ≈ 25.2 km/h
by hand           ≈ 18.92 km
```

In Context: An afternoon storm in Kumasi

During a storm, an automatic rain gauge records the rainfall *intensity* (mm per hour) every 10 minutes. The total rainfall is the integral of the intensity. Over a roof, each millimetre of rain on each square metre collects one litre of water, which helps when designing a rainwater-harvesting tank.

Listing 7.3 — Total rainfall from intensity data `ex7_03_rainfall.py`

```python
# Listing 7.3: Total rainfall from rainfall intensity (an accumulation problem)
# Intensity r(t) in mm/hour during an afternoon storm in Kumasi, every 10 min (
    illustrative)
import numpy as np

t = np.arange(0, 121, 10) / 60                      # hours: 0, 1/6, ..., 2
r = np.array([0, 4, 15, 42, 60, 38, 25, 18, 10, 6, 3, 1, 0])   # mm/h

total = np.trapz(r, t)                              # ∫ r(t) dt  (mm)
print(f"total rainfall ≈ {total:.1f} mm over {t[-1]:.0f} hours")
print(f"average intensity ≈ {total/t[-1]:.1f} mm/h;  peak = {r.max()} mm/h")

# Volume of water falling on a 12 m x 10 m roof (1 mm on 1 m^2 = 1 litre)
roof = 12 * 10
print(f"rainwater on a {roof} m^2 roof ≈ {total*roof:,.0f} litres")
```

Output

```
total rainfall ≈ 37.0 mm over 2 hours
average intensity ≈ 18.5 mm/h;  peak = 60 mm/h
rainwater on a 120 m^2 roof ≈ 4,440 litres
```

7.3 Average value of a function

The average of n numbers is their sum divided by n. For a function on $[a, b]$, the "sum" becomes an integral and n becomes the length of the interval.

Key Concept: Average value

$$f_{\text{avg}} = \frac{1}{b-a}\int_a^b f(x)\,dx.$$

Mean value theorem for integrals: if f is continuous on $[a, b]$, there is at least one c in $[a, b]$ with $f(c) = f_{\text{avg}}$. Geometrically, the rectangle of height f_{avg} on $[a, b]$ has the same area as the region under the curve.

Example 7.3 Find the average value of $f(x) = x^2$ on $[0, 3]$.

Solution. $f_{\text{avg}} = \frac{1}{3}\int_0^3 x^2\,dx = \frac{1}{3}\cdot 9 = 3$. The value 3 is taken at $c = \sqrt{3} \in [0, 3]$. ■

In Context: The average temperature of a day

A simple model of the temperature on a hot day in Tamale is $T(t) = 30 + 6\sin\frac{\pi(t-9)}{12}$ °C, with t in hours after midnight (an illustrative model). Over a whole day the sine term averages to zero, so the average temperature is 30°C. Over working hours, from 8 am to 6 pm, the average is higher, about 33.8°C. The temperature passes through that average at about 11:40 am.

Listing 7.4 — Average temperature `ex7_04_average_value.py`

```python
# Listing 7.4: Average value of a function - the average temperature of a day
# Illustrative model of the temperature in Tamale on a hot day (°C), t = hours after midnight:
#   T(t) = 30 + 6 sin(pi (t - 9) / 12)
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

t = sp.symbols("t", real=True)
T = 30 + 6*sp.sin(sp.pi*(t - 9)/12)

avg_day = sp.integrate(T, (t, 0, 24)) / 24
avg_work = sp.integrate(T, (t, 8, 18)) / (18 - 8)
print("average over the whole day       =", avg_day, "°C")
print("average from 8 am to 6 pm        ≈", round(float(avg_work), 2), "°C")

# Mean value theorem for integrals: T(c) equals the average at some time c
```

```
cs = sp.solve(sp.Eq(T, avg_work), t)
print("times when T equals the 8-6 average:", [round(float(c), 2) for c in cs if 8 <= c <=
    18])

tt = np.linspace(0, 24, 300)
Tf = sp.lambdify(t, T)
plt.figure(figsize=(6, 3.2))
plt.plot(tt, Tf(tt), label="T(t)")
plt.axhline(float(avg_day), color="C1", ls="--", label="daily average")
plt.xlabel("hour of the day"); plt.ylabel("°C"); plt.grid(alpha=0.3); plt.legend()
plt.show()
```

Output

```
average over the whole day       = 30 °C
average from 8 am to 6 pm        ≈ 33.83 °C
times when T equals the 8-6 average: [11.65]
```

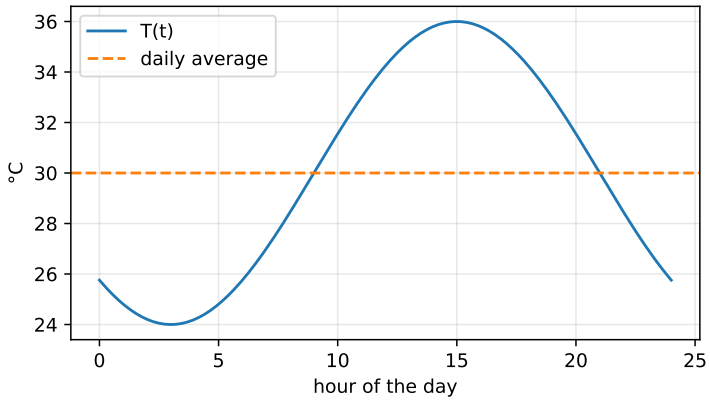
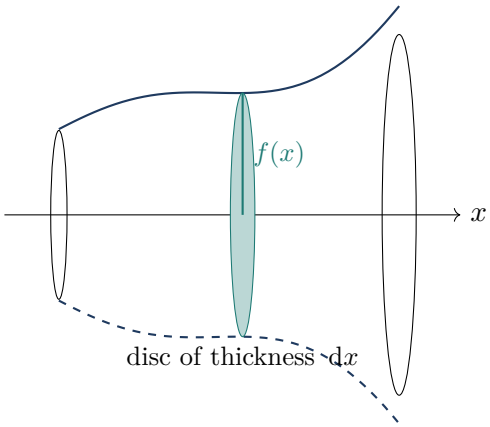


Figure 7.2: The temperature model and its daily average.

7.4 Volumes of solids of revolution

Rotate the region under $y = f(x)$, $a \le x \le b$, about the x-axis. The result is a *solid of revolution*. Slice it perpendicular to the axis. Each slice is a thin disc of radius $f(x)$ and thickness $\mathrm{d}x$, with volume $\pi[f(x)]^2\,\mathrm{d}x$.



Key Concept: Disc and washer methods

Disc method (rotation about the x-axis):

$$V = \int_a^b \pi[f(x)]^2 \,\mathrm{d}x.$$

Washer method: if the region lies between an outer curve $y = R(x)$ and an inner curve $y = r(x)$, each slice is a washer (a disc with a hole):

$$V = \int_a^b \pi\Big([R(x)]^2 - [r(x)]^2\Big) \,\mathrm{d}x.$$

For rotation about the y-axis, write the radius as a function of y and integrate with respect to y.

Example 7.4 Show that a sphere of radius r has volume $\frac{4}{3}\pi r^3$.

Solution. Rotate the semicircle $y = \sqrt{r^2 - x^2}$, $-r \le x \le r$, about the x-axis:

$$V = \int_{-r}^{r} \pi(r^2 - x^2)\,\mathrm{d}x = \pi\left[r^2 x - \frac{x^3}{3}\right]_{-r}^{r} = \pi\left(\frac{2r^3}{3} + \frac{2r^3}{3}\right) = \frac{4}{3}\pi r^3.$$

■

Example 7.5 The region between $y = 2x$ and $y = x^2$ is rotated about the x-axis. Find the volume.

Solution. On $[0, 2]$ the outer radius is $2x$ and the inner radius is x^2, so $V = \pi\int_0^2 (4x^2 - x^4)\,\mathrm{d}x = \pi\left(\frac{32}{3} - \frac{32}{5}\right) = \frac{64\pi}{15}$. ■

In Context: How much water does the pot hold?

Clay water pots, made by potters in many towns across West Africa, keep drinking water cool. Their shape is close to a solid of revolution. Suppose the inside radius of a pot at height y cm above the base is $R(y) = 12 + 8\sin\frac{\pi y}{40}$, for $0 \le y \le 40$ (an illustrative profile). Rotating this profile about the vertical axis, the volume is $V = \int_0^{40} \pi[R(y)]^2\,\mathrm{d}y \approx 37\,500$ cm^3, about 37.5 litres.

Listing 7.5 — Volumes of revolution ex7_05_volume.py

```python
# Listing 7.5: Volumes of revolution (disc and washer methods)
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

x, y, r, h = sp.symbols("x y r h", positive=True)

# (a) Sphere: rotate y = sqrt(r^2 - x^2) about the x-axis
print("sphere :", sp.simplify(sp.integrate(sp.pi*(r**2 - x**2), (x, -r, r))))
# (b) Cone: rotate y = (r/h) x, 0 <= x <= h, about the x-axis
print("cone   :", sp.integrate(sp.pi*(r*x/h)**2, (x, 0, h)))
# (c) Washer: region between y = 2x and y = x^2 rotated about the x-axis
print("washer :", sp.integrate(sp.pi*((2*x)**2 - (x**2)**2), (x, 0, 2)))

# (d) A clay water pot. Its inside radius (cm) at height y above the base is
#     R(y) = 12 + 8 sin(pi y / 40),  0 <= y <= 40   (illustrative shape)
R = 12 + 8*sp.sin(sp.pi*y/40)
V = sp.integrate(sp.pi*R**2, (y, 0, 40))
print("pot    :", sp.simplify(V), "≈", round(float(V)), "cm^3 ≈", round(float(V)/1000, 1),
      "litres")

Y = np.linspace(0, 40, 200)
Rf = sp.lambdify(y, R)
plt.figure(figsize=(4, 4))
plt.plot(Rf(Y), Y, "k"); plt.plot(-Rf(Y), Y, "k")
plt.axvline(0, ls="--", color="gray")
plt.gca().set_aspect("equal"); plt.title("profile of the pot (cm)"); plt.grid(alpha=0.3)
plt.show()
```

Output

```
sphere : 4*pi*r**3/3
cone   : pi*h*r**2/3
washer : 64*pi/15
pot    : 15360 + 7040*pi ≈ 37477 cm^3 ≈ 37.5 litres
```

7.5 Arc length

To measure the length of a curve $y = f(x)$ from $x = a$ to $x = b$, approximate it by many short straight segments. A segment spanning a small change Δx rises by about $f'(x)\Delta x$, so by Pythagoras its length is $\sqrt{(\Delta x)^2 + (f'(x)\Delta x)^2} = \sqrt{1 + [f'(x)]^2}\,\Delta x$.

Key Concept: Arc length

$$L = \int_a^b \sqrt{1 + \left(\frac{\mathrm{d}y}{\mathrm{d}x}\right)^2}\,\mathrm{d}x.$$

The square root usually prevents an exact answer, so arc lengths are often computed numerically with quad.

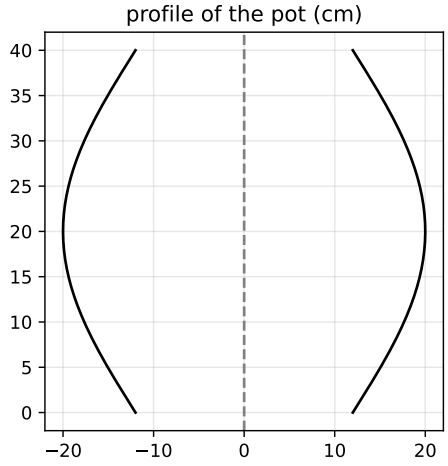


Figure 7.3: The profile of the pot. Rotating it about the dashed axis generates the pot.

Example 7.6 Find the length of the curve $y = \frac{2}{3}x^{3/2}$ from $x = 0$ to $x = 3$.

Solution. $y' = x^{1/2}$, so $1 + (y')^2 = 1 + x$ and $L = \int_0^3 \sqrt{1+x}\,dx = \left[\frac{2}{3}(1+x)^{3/2}\right]_0^3 = \frac{2}{3}(8-1) = \frac{14}{3}$. ■

In Context: How long is the arch?

A parabolic steel arch spans 240 m and rises 50 m, so it follows $y = 50 - \frac{50}{120^2}x^2$ (illustrative figures, compare Section 9.1). The engineers ordering the steel need the length of the arch, not its span. Listing 7.6 finds it to be about 265 m.

Listing 7.6 — Arc length, exact and numerical `ex7_06_arc_length.py`

```
# Listing 7.6: Arc length L = ∫ sqrt(1 + (dy/dx)^2) dx
import numpy as np
import sympy as sp
from scipy.integrate import quad

x = sp.symbols("x", positive=True)

# (a) An exact example: y = (2/3) x^(3/2) from x = 0 to x = 3
y = sp.Rational(2, 3)*x**sp.Rational(3, 2)
L = sp.integrate(sp.sqrt(1 + sp.diff(y, x)**2), (x, 0, 3))
print("y = (2/3)x^(3/2), 0 <= x <= 3:  L =", L)

# (b) A parabolic arch spanning 240 m and rising 50 m: y = 50 - k x^2
k = 50 / 120**2
dydx = lambda s: -2*k*s
L, _ = quad(lambda s: np.sqrt(1 + dydx(s)**2), -120, 120)
print(f"length of the arch = {L:.2f} m  (span 240 m, so {L - 240:.2f} m longer)")

# (c) A curve whose arc-length integral has no formula: y = sin x, 0 <= x <= pi
L, _ = quad(lambda s: np.sqrt(1 + np.cos(s)**2), 0, np.pi)
print(f"one arch of y = sin x has length {L:.6f}")
```

Output

```
y = (2/3)x^(3/2), 0 <= x <= 3:  L = 14/3
length of the arch = 265.43 m  (span 240 m, so 25.43 m longer)
one arch of y = sin x has length 3.820198
```

7.6 Areas from survey data

In Context: Surveying an irregular farm

Farm boundaries often follow streams, footpaths and old boundary trees rather than straight lines. A surveyor can measure the area of such a farm by running a straight baseline along the road edge and measuring perpendicular *offsets* to the far boundary at regular intervals. The area is $\int w(x)\,\mathrm{d}x$, computed from the data by the trapezium or Simpson's rule. Here the two rules agree to within about 2%.

Listing 7.7 — Area of an irregular farm `ex7_07_farm_area.py`

```
# Listing 7.7: Area of an irregular farm from a surveyor's offsets
# A straight baseline runs along the road edge of a farm. Every 20 m along it the
# surveyor measures the perpendicular width (offset) to the far boundary (illustrative).
import numpy as np
from scipy.integrate import simpson
import matplotlib.pyplot as plt

x = np.arange(0, 201, 20)                                  # 0, 20, ..., 200 m
w = np.array([0, 38, 52, 61, 70, 74, 69, 63, 55, 41, 0])   # offsets (m)

A_trap = np.trapz(w, x)
A_simp = simpson(w, x=x)
print(f"trapezium rule: {A_trap:8.0f} m^2 = {A_trap/10000:.3f} ha")
print(f"Simpson's rule: {A_simp:8.0f} m^2 = {A_simp/10000:.3f} ha")

plt.figure(figsize=(6, 2.8))
plt.fill_between(x, 0, w, alpha=0.3, color="green")
plt.plot(x, w, "go-"); plt.axhline(0, color="k", lw=3)
plt.text(80, -12, "road (baseline)")
plt.xlabel("distance along road (m)"); plt.ylabel("offset (m)")
plt.ylim(-20, 85); plt.grid(alpha=0.3); plt.show()
```

Output

```
trapezium rule:    10460 m^2 = 1.046 ha
Simpson's rule:    10667 m^2 = 1.067 ha
```

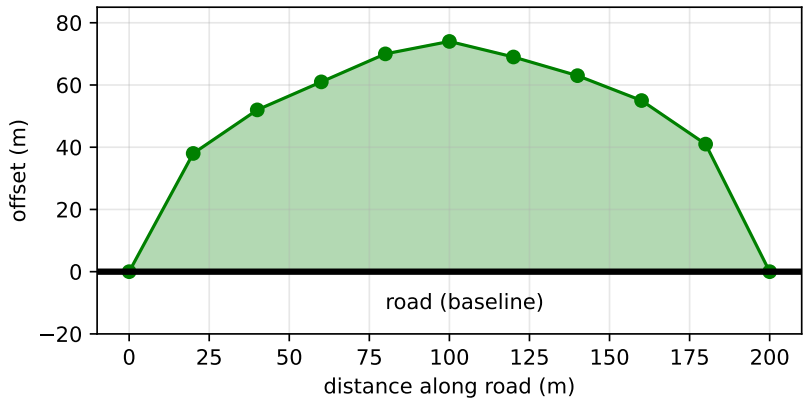


Figure 7.4: Surveyor's offsets from a straight road to the irregular back boundary of a farm.

7.7 Probability

Many quantities, such as heights, examination marks and measurement errors, are described by a *probability density function* $p(x) \geq 0$. Probabilities are areas under its graph.

Key Concept: Probability as an integral

$$P(a \leq X \leq b) = \int_a^b p(x)\,\mathrm{d}x, \qquad \int_{-\infty}^{\infty} p(x)\,\mathrm{d}x = 1.$$

The *normal* density with mean μ and standard deviation σ is $p(x) = \dfrac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)}$. It has no elementary antiderivative, so normal probabilities are computed numerically.

In Context: Examination marks

Suppose the marks in a large examination follow a normal density with mean 55 and standard deviation 12 (illustrative). About 79% of candidates score between 40 and 70, and about 11% score 70 or more.

Listing 7.8 — Probability as area `ex7_08_probability.py`

```python
# Listing 7.8: Probability as area under a density curve
import numpy as np
from scipy.integrate import quad
import matplotlib.pyplot as plt

# Examination marks modelled by a normal density with mean 55, standard deviation 12
mu, sigma = 55, 12
pdf = lambda x: np.exp(-((x - mu)/sigma)**2 / 2) / (sigma*np.sqrt(2*np.pi))

total, _ = quad(pdf, -np.inf, np.inf)
print(f"total area under the density = {total:.6f}")
for lo, hi in [(40, 70), (70, 100), (0, 40)]:
    p, _ = quad(pdf, lo, hi)
    print(f"P({lo} <= mark <= {hi}) ≈ {p:.3f}")

```

```
X = np.linspace(10, 100, 300)
plt.figure(figsize=(6, 3.2))
plt.plot(X, pdf(X), "k")
Xs = np.linspace(70, 100, 100)
plt.fill_between(Xs, pdf(Xs), alpha=0.4, label="P(mark ≥ 70)")
plt.xlabel("mark"); plt.grid(alpha=0.3); plt.legend()
plt.show()
```

Output

```
total area under the density = 1.000000
P(40 <= mark <= 70) ≈ 0.789
P(70 <= mark <= 100) ≈ 0.106
P(0 <= mark <= 40) ≈ 0.106
```

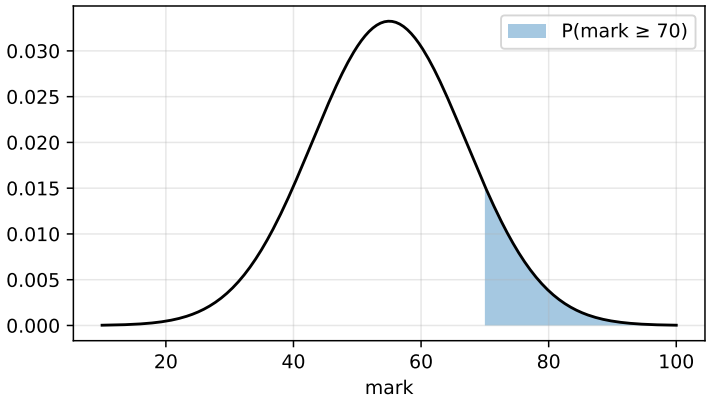


Figure 7.5: The shaded area is the probability of a mark of 70 or more.

Chapter Summary

- Area between curves: $\int_a^b (\text{top} - \text{bottom})\,dx$.
- The integral of a rate is a total change. Displacement is $\int v\,dt$, and distance is $\int |v|\,dt$.
- Average value: $\frac{1}{b-a}\int_a^b f(x)\,dx$.
- Volumes of revolution: discs $\int \pi f^2\,dx$; washers $\int \pi(R^2 - r^2)\,dx$.
- Arc length: $\int_a^b \sqrt{1 + (y')^2}\,dx$.
- Probability: $P(a \le X \le b) = \int_a^b p(x)\,dx$.
- Python: `sp.integrate` when a formula exists; `quad`, `np.trapz` and `simpson` otherwise.

7.8 Exercises

A. Skills

1. Find the area enclosed between (a) $y = x^2$ and $y = x + 2$; (b) $y = \sin x$ and the x-axis for $0 \le x \le \pi$; (c) $y = x^3$ and $y = x$.

2. Find the average value of (a) $f(x) = 3x^2 - 2x$ on $[1, 4]$; (b) $f(x) = \sin x$ on $[0, \pi]$.

3. Find the volume generated when the region under the curve is rotated about the x-axis: (a) $y = \sqrt{x}$, $0 \le x \le 4$; (b) $y = e^x$, $0 \le x \le 1$; (c) the region between $y = x$ and $y = x^2$.

4. Use the disc method to show that a cone of base radius r and height h has volume $\tfrac{1}{3}\pi r^2 h$.

5. Find the length of the curve (a) $y = x^{3/2}$ for $0 \le x \le 4$; (b) $y = \tfrac{x^3}{6} + \tfrac{1}{2x}$ for $1 \le x \le 2$.

B. Applications

6. A car's velocity is $v(t) = 3t^2 - 12t + 9$ m/s. Find (a) its displacement and (b) the total distance it travels between $t = 0$ and $t = 4$ s.

7. Water flows into a reservoir behind a dam at a rate of $R(t) = 200 + 50\sin\left(\tfrac{\pi t}{6}\right)$ million cubic metres per month, where t is in months (illustrative). Find the total inflow in one year.

8. The marginal cost of producing q bags of gari is $C'(q) = 12 - 0.02q$ cedis per bag. Find the extra cost of increasing production from 100 to 300 bags.

9. The width of a fish pond, measured every 4 m along its 32 m length, is 0, 9.5, 12.0, 13.4, 14.0, 13.1, 11.2, 7.8, 0 metres. Estimate its surface area with (a) the trapezium rule; (b) Simpson's rule.

10. The rate at which mobile-money transactions are processed on a busy afternoon is modelled by $r(t) = 1200t\,e^{-0.5t}$ transactions per hour, for $0 \le t \le 8$ hours (illustrative). Estimate the total number of transactions.

11. A bowl is made by rotating the curve $y = \tfrac{x^2}{8}$, $0 \le x \le 8$ (in cm), about the y-axis. How many litres of water does the bowl hold when full?

12. Using the examination-mark model of Section 7.7, find the probability that a candidate scores between 50 and 60.

C. Programming

13. Write a function `arc_length(f, a, b)` that takes a SymPy expression and returns its arc length on $[a, b]$, using `sp.diff`, `sp.lambdify` and `quad`. Test it on Exercise 5.

14. Download (or invent) a table of daily rainfall for one month in your home town. Use `np.cumsum` to plot the *accumulated* rainfall through the month. How is this related to the integral?

15. Draw the pot of Listing 7.5 in three dimensions with `plot_surface`. (*Hint:* use the parameters y and an angle ϕ, with $x = R(y)\cos\phi$ and $z = R(y)\sin\phi$.)

Chapter 8

Parametric Equations and Polar Coordinates

Learning Objectives

After working through this chapter you should be able to:

- describe curves by parametric equations, and eliminate the parameter;
- model the motion of a projectile with parametric equations;
- find tangents, second derivatives and arc lengths of parametric curves;
- convert between polar and Cartesian coordinates;
- sketch and recognise standard polar curves (circles, cardioids, roses, spirals);
- find slopes, areas and lengths in polar coordinates;
- plot parametric and polar curves in Python.

So far every curve has been described by one equation relating x and y. Some curves are easier to describe in other ways. The path of a football is most naturally given by *where the ball is at each time* t: two functions $x(t)$ and $y(t)$. A radar screen locates an aircraft by its *distance and direction* from the radar, not by x and y. This chapter studies both descriptions, *parametric equations* and *polar coordinates*, and prepares for the conic sections of Chapter 9.

Historical Note: Spirals, cycloids and the fastest slide

In *On Spirals* (c. 225 BCE) Archimedes studied the curve traced by a point moving steadily outwards along a line that turns steadily about a fixed point. Today we write it in polar coordinates as $r = a\theta$. The *cycloid*, the path of a point on the rim of a rolling wheel, fascinated 17th-century mathematicians. In 1696 Johann Bernoulli challenged the mathematicians of Europe to find the curve down which a bead slides from one point to another *in the least time* (the *brachistochrone*). Newton, Leibniz, Jacob Bernoulli and l'Hôpital all found the answer: an upside-down cycloid. Jacob Bernoulli also made systematic use of polar coordinates in the 1690s. Leonhard Euler later developed them into the form we use.

8.1 Parametric equations

Definition 8.1. (Parametric curve)

A *parametric curve* is the set of points $(x, y) = \big(f(t), g(t)\big)$ as the *parameter* t runs through an interval. The equations $x = f(t)$, $y = g(t)$ are *parametric equations* of the curve.

The parameter often represents time, and the curve is then the path of a moving point. Unlike a graph $y = f(x)$, a parametric curve can loop, cross itself and run vertically.

Key Concept: Standard parametrisations

line through (x_0, y_0) in direction (a, b)	$x = x_0 + at,\ \ y = y_0 + bt$
circle, centre (h, k), radius r	$x = h + r\cos t,\ \ y = k + r\sin t$
ellipse $\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$	$x = a\cos t,\ \ y = b\sin t$
cycloid (wheel of radius r)	$x = r(t - \sin t),\ \ y = r(1 - \cos t)$
graph $y = f(x)$	$x = t,\ \ y = f(t)$

Example 8.1 Eliminate the parameter: (a) $x = 2\cos t$, $y = 3\sin t$; (b) $x = t^2$, $y = t^3$.

Solution. (a) $\cos t = \frac{x}{2}$ and $\sin t = \frac{y}{3}$. Since $\cos^2 t + \sin^2 t = 1$, $\frac{x^2}{4} + \frac{y^2}{9} = 1$: an ellipse. (b) $y^2 = t^6 = (t^2)^3 = x^3$, so the curve is $y^2 = x^3$ (Neile's semicubical parabola). ■

Listing 8.1 — Plotting parametric curves `ex8_01_parametric_curves.py`

```
# Listing 8.1: Plotting parametric curves x = f(t), y = g(t)
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

t = np.linspace(0, 2*np.pi, 400)
fig, ax = plt.subplots(1, 4, figsize=(10, 2.8))
ax[0].plot(np.cos(t), np.sin(t));           ax[0].set_title("circle")
ax[1].plot(2*np.cos(t), 3*np.sin(t), "C1");  ax[1].set_title("ellipse")
T = np.linspace(0, 4*np.pi, 400)
ax[2].plot(T - np.sin(T), 1 - np.cos(T), "C2"); ax[2].set_title("cycloid")
ax[3].plot(np.sin(3*t), np.sin(2*t), "C3");  ax[3].set_title("Lissajous")
for a in ax:
    a.set_aspect("equal"); a.grid(alpha=0.3)
plt.tight_layout(); plt.show()

# Eliminating the parameter with SymPy: x = 2 cos t, y = 3 sin t
s = sp.symbols("t")
x, y = 2*sp.cos(s), 3*sp.sin(s)
print("(x/2)^2 + (y/3)^2 =", sp.simplify((x/2)**2 + (y/3)**2))
```

Output

```
(x/2)^2 + (y/3)^2 = 1
```

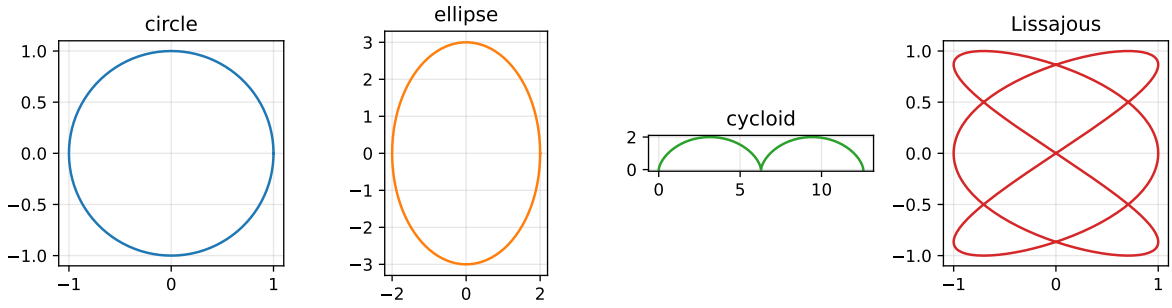


Figure 8.1: A circle, an ellipse, two arches of a cycloid, and a Lissajous figure $x = \sin 3t$, $y = \sin 2t$.

Watch Out: A parametric curve has a direction

The parametrisations $x = \cos t,\ y = \sin t$ and $x = \cos t,\ y = -\sin t$ trace the same circle, but in opposite directions. Eliminating the parameter loses the direction and the speed of the motion, and sometimes part of the domain. For example, $x = t^2$, $y = t^2$ gives only the half-line $y = x$, $x \geq 0$.

8.1.1 Projectile motion

If a ball is launched from the origin with speed v at an angle α above the horizontal, and air resistance is ignored, then after t seconds

$$x = (v\cos\alpha)\,t, \qquad y = (v\sin\alpha)\,t - \tfrac{1}{2}gt^2,$$

where $g \approx 9.8\ \text{m/s}^2$. Eliminating t gives a parabola. The ball lands when $y = 0$, at time $T = \frac{2v\sin\alpha}{g}$, and the range is $R = \frac{v^2\sin 2\alpha}{g}$, which is greatest when $\alpha = 45°$.

In Context: A free kick

A footballer kicks the ball at 25 m/s. At 45° it would travel about 64 m. A free kick 20 m from goal at 30° reaches the goal line after 0.92 s, still 7.4 m high, far over the 2.44 m crossbar. That is why free-kick takers strike the ball lower, or use spin and air resistance (which our simple model ignores) to bring it down.

Listing 8.2 — Projectile paths `ex8_02_projectile.py`

```
# Listing 8.2: The path of a kicked football (no air resistance)
#   x = v cos(a) t,   y = v sin(a) t - g t^2 / 2
import numpy as np
import matplotlib.pyplot as plt

g, v = 9.8, 25.0                    # m/s^2, kick speed in m/s
plt.figure(figsize=(6.5, 3))
for deg in [20, 30, 45, 60]:
    a = np.radians(deg)
    T = 2*v*np.sin(a)/g             # time of flight (y = 0 again)
    t = np.linspace(0, T, 200)
    x, y = v*np.cos(a)*t, v*np.sin(a)*t - g*t**2/2
    print(f"angle {deg:>2}°: flight {T:4.2f} s, range {x[-1]:6.2f} m, "
          f"max height {(v*np.sin(a))**2/(2*g):5.2f} m")
```

```
    plt.plot(x, y, label=f"{deg}°")
plt.xlabel("x (m)"); plt.ylabel("y (m)"); plt.gca().set_aspect("equal")
plt.grid(alpha=0.3); plt.legend(fontsize=8); plt.show()

# A free kick 20 m from goal at 30°: how high is the ball when it reaches the goal line?
a = np.radians(30)
t_goal = 20 / (v*np.cos(a))
print(f"at the goal line: t = {t_goal:.3f} s, height = {v*np.sin(a)*t_goal - g*t_goal
    **2/2:.2f} m"
      " (crossbar 2.44 m)")
```

Output

```
angle 20°: flight 1.75 s, range  40.99 m, max height  3.73 m
angle 30°: flight 2.55 s, range  55.23 m, max height  7.97 m
angle 45°: flight 3.61 s, range  63.78 m, max height 15.94 m
angle 60°: flight 4.42 s, range  55.23 m, max height 23.92 m
at the goal line: t = 0.924 s, height = 7.37 m (crossbar 2.44 m)
```

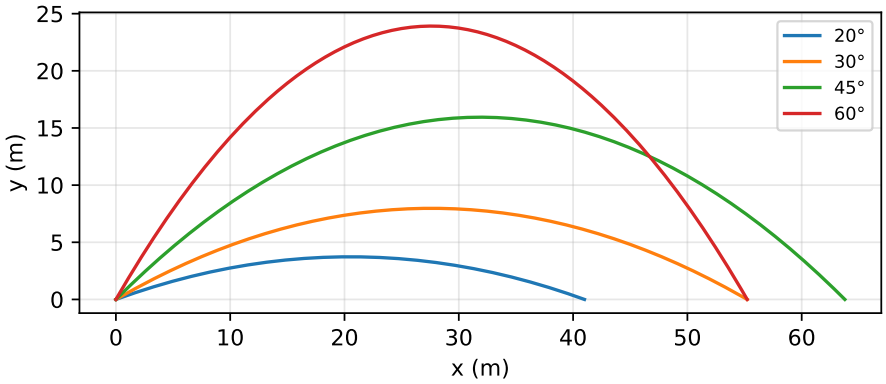


Figure 8.2: Paths of a ball kicked at 25 m/s at four angles. The 30° and 60° paths have the same range.

8.2 Calculus with parametric curves

If $x = f(t)$ and $y = g(t)$, the chain rule gives $\frac{dy}{dt} = \frac{dy}{dx}\frac{dx}{dt}$, so:

Key Concept: Derivatives and length of a parametric curve

$$\frac{dy}{dx} = \frac{dy/dt}{dx/dt}, \qquad \frac{d^2y}{dx^2} = \frac{\frac{d}{dt}\left(\frac{dy}{dx}\right)}{dx/dt}, \qquad L = \int_\alpha^\beta \sqrt{\left(\frac{dx}{dt}\right)^2 + \left(\frac{dy}{dt}\right)^2}\,dt.$$

The tangent is horizontal where $\frac{dy}{dt} = 0$ (and $\frac{dx}{dt} \neq 0$), and vertical where $\frac{dx}{dt} = 0$ (and $\frac{dy}{dt} \neq 0$).

Watch Out: The second derivative

$\dfrac{\mathrm{d}^2y}{\mathrm{d}x^2}$ is *not* $\dfrac{\mathrm{d}^2y/\mathrm{d}t^2}{\mathrm{d}^2x/\mathrm{d}t^2}$. Differentiate $\frac{\mathrm{d}y}{\mathrm{d}x}$ with respect to t, then divide by $\frac{\mathrm{d}x}{\mathrm{d}t}$.

Example 8.2 For the cycloid $x = r(t - \sin t)$, $y = r(1 - \cos t)$, find $\frac{\mathrm{d}y}{\mathrm{d}x}$, the tangent at $t = \frac{\pi}{2}$ when $r = 1$, and the length of one arch.

Solution. $\frac{\mathrm{d}x}{\mathrm{d}t} = r(1 - \cos t)$ and $\frac{\mathrm{d}y}{\mathrm{d}t} = r\sin t$, so $\dfrac{\mathrm{d}y}{\mathrm{d}x} = \dfrac{\sin t}{1 - \cos t}$. At $t = \frac{\pi}{2}$ the slope is 1 and the point is $\left(\frac{\pi}{2} - 1,\ 1\right)$, so the tangent is $y = x - \frac{\pi}{2} + 2$.

For the length, $\left(\frac{\mathrm{d}x}{\mathrm{d}t}\right)^2 + \left(\frac{\mathrm{d}y}{\mathrm{d}t}\right)^2 = r^2(2 - 2\cos t) = 4r^2\sin^2\frac{t}{2}$. Hence

$$L = \int_0^{2\pi} 2r\sin\frac{t}{2}\,\mathrm{d}t = \left[-4r\cos\frac{t}{2}\right]_0^{2\pi} = 8r.$$

One arch of the cycloid is exactly four times the diameter of the wheel, a result found by Christopher Wren in 1658. ∎

Listing 8.3 — Calculus of the cycloid `ex8_03_parametric_calculus.py`

```
# Listing 8.3: Calculus of parametric curves - the cycloid x = r(t - sin t), y = r(1 - cos t)
import sympy as sp

t = sp.symbols("t", real=True)
r = sp.symbols("r", positive=True)
x, y = r*(t - sp.sin(t)), r*(1 - sp.cos(t))

dydx = sp.simplify(sp.diff(y, t) / sp.diff(x, t))
print("dy/dx =", dydx)
print("slope at t = pi/2:", dydx.subs(t, sp.pi/2))
d2 = sp.simplify(sp.diff(dydx, t) / sp.diff(x, t))
print("d2y/dx2 =", d2)

# Tangent line at t = pi/2 when r = 1
P = (x.subs({t: sp.pi/2, r: 1}), y.subs({t: sp.pi/2, r: 1}))
X = sp.symbols("X")
print("tangent at t = pi/2 (r = 1): y =", sp.expand(P[1] + dydx.subs(t, sp.pi/2)*(X - P[0])))

# Arc length of one arch: ∫_0^{2 pi} sqrt(x'^2 + y'^2) dt
speed = sp.sqrt(sp.diff(x, t)**2 + sp.diff(y, t)**2)
speed = 2*r*sp.sin(t/2)          # simplification of speed, valid for 0 <= t <= 2 pi
print("check speed formula:", sp.simplify((sp.diff(x, t)**2 + sp.diff(y, t)**2) - speed**2))
print("length of one arch =", sp.integrate(speed, (t, 0, 2*sp.pi)))
```

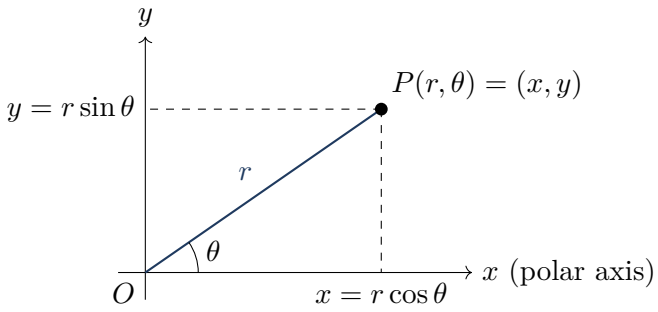
Output

```
dy/dx = -sin(t)/(cos(t) - 1)
slope at t = pi/2: 1
d2y/dx2 = -1/(r*(cos(t) - 1)**2)
tangent at t = pi/2 (r = 1): y = X - pi/2 + 2
```

```
check speed formula: 0
length of one arch = 8*r
```

8.3 Polar coordinates

Choose a point O (the *pole*) and a ray from it (the *polar axis*, normally the positive x-axis). A point P has *polar coordinates* (r, θ), where r is its distance from O and θ is the angle from the polar axis to OP, measured anticlockwise.



Key Concept: Polar and Cartesian coordinates

$$x = r\cos\theta, \quad y = r\sin\theta; \qquad r^2 = x^2 + y^2, \quad \tan\theta = \frac{y}{x}.$$

A point has many polar coordinates: (r, θ), $(r, \theta + 2\pi)$ and $(-r, \theta + \pi)$ all describe the same point.

Example 8.3

(a) $(r, \theta) = \left(4, \frac{3\pi}{4}\right)$ gives $(x, y) = \left(4\cos\frac{3\pi}{4}, 4\sin\frac{3\pi}{4}\right) = (-2\sqrt{2},\ 2\sqrt{2})$.

(b) $(x, y) = (-1, 1)$ gives $r = \sqrt{2}$. Since $\tan\theta = -1$ and the point is in the *second* quadrant, $\theta = \frac{3\pi}{4}$, not $-\frac{\pi}{4}$.

(c) The circle $x^2 + y^2 = 4y$ becomes $r^2 = 4r\sin\theta$, that is, $r = 4\sin\theta$.

Watch Out: Check the quadrant

$\tan^{-1}\frac{y}{x}$ only returns angles between $-\frac{\pi}{2}$ and $\frac{\pi}{2}$, so it gives the wrong answer for points with $x < 0$. In Python, use `np.arctan2(y, x)`, which always returns the correct quadrant.

In Context: Radar and bearings

An airport radar reports each aircraft by its *range* (distance) and *bearing*. Navigators measure bearings *clockwise from north*, while mathematicians measure θ *anticlockwise from east*, so $\theta = 90° - \text{bearing}$. An aircraft at range 40 km on a bearing of 120° has $\theta = -30°$. It is about 34.6 km east and 20 km south of the radar.

Listing 8.4 — Converting coordinates ex8_04_polar_convert.py

```python
# Listing 8.4: Converting between polar and Cartesian coordinates
import numpy as np

def to_cartesian(r, theta):
    return r*np.cos(theta), r*np.sin(theta)

def to_polar(x, y):
    return np.hypot(x, y), np.arctan2(y, x)       # arctan2 picks the correct quadrant

for r, deg in [(2, 30), (4, 135), (3, 270)]:
    x, y = np.round(to_cartesian(r, np.radians(deg)), 4) + 0.0   # + 0.0 removes "-0.0"
    print(f"(r, θ) = ({r}, {deg}°)  ->  (x, y) = ({x:.4f}, {y:.4f})")
for x, y in [(1, 1), (-1, 1), (-3, -4)]:
    r, th = to_polar(x, y)
    print(f"(x, y) = ({x}, {y})  ->  r = {r:.4f}, θ = {np.degrees(th):.2f}°  "
          f"(arctan(y/x) alone gives {np.degrees(np.arctan(y/x)):.2f}°)")

# An airport radar reports an aircraft at range 40 km on a BEARING of 120°
# (bearings are measured clockwise from north). Polar angle = 90° - bearing.
rng, bearing = 40, 120
x, y = to_cartesian(rng, np.radians(90 - bearing))
print(f"(x, y) = ({x:.2f}, {y:.2f}): the aircraft is {x:.2f} km east and "
      f"{-y:.2f} km south of the radar")
```

Output

```
(r, θ) = (2, 30°)  ->  (x, y) = (1.7321, 1.0000)
(r, θ) = (4, 135°)  ->  (x, y) = (-2.8284, 2.8284)
(r, θ) = (3, 270°)  ->  (x, y) = (0.0000, -3.0000)
(x, y) = (1, 1)  ->  r = 1.4142, θ = 45.00°  (arctan(y/x) alone gives 45.00°)
(x, y) = (-1, 1)  ->  r = 1.4142, θ = 135.00°  (arctan(y/x) alone gives -45.00°)
(x, y) = (-3, -4)  ->  r = 5.0000, θ = -126.87°  (arctan(y/x) alone gives 53.13°)
(x, y) = (34.64, -20.00): the aircraft is 34.64 km east and 20.00 km south of the radar
```

8.4 Polar curves

A polar equation $r = f(\theta)$ describes the curve of all points whose polar coordinates satisfy it. Some curves that are awkward in Cartesian form are very simple in polar form.

Key Concept: Standard polar curves

$r = a$	circle of radius a, centred at the pole
$\theta = \alpha$	straight line through the pole
$r = 2a\cos\theta$, $r = 2a\sin\theta$	circles of radius a through the pole
$r = a(1 \pm \cos\theta)$	cardioid ("heart shape")
$r = a\cos n\theta$	rose with n petals if n is odd, $2n$ petals if n is even
$r = a\theta$	spiral of Archimedes
$r^2 = a^2\cos 2\theta$	lemniscate (figure of eight)
$r = \dfrac{ed}{1 + e\cos\theta}$	conic with eccentricity e (Section 9.5)

Listing 8.5 — A gallery of polar curves `ex8_05_polar_gallery.py`

```python
# Listing 8.5: A gallery of polar curves r = f(theta)
import numpy as np
import matplotlib.pyplot as plt

th = np.linspace(0, 2*np.pi, 1000)
curves = {
    "r = 2":                  2 + 0*th,
    "r = 2 cos θ":            2*np.cos(th),
    "r = 1 + cos θ (cardioid)": 1 + np.cos(th),
    "r = cos 3θ (3 petals)":  np.cos(3*th),
    "r = cos 2θ (4 petals)":  np.cos(2*th),
    "r = θ/π (spiral)":       th/np.pi,
}
fig, ax = plt.subplots(2, 3, figsize=(9, 6), subplot_kw={"projection": "polar"})
for a, (name, r) in zip(ax.flat, curves.items()):
    a.plot(th, r)                    # matplotlib's polar axes accept negative r
    a.set_title(name, fontsize=9)
    a.set_yticklabels([])
plt.tight_layout(); plt.show()
```

In Context: The cardioid microphone

Microphones used on stage and in radio studios are often *cardioid* microphones. Their sensitivity in the direction θ is proportional to $1 + \cos\theta$. They pick up sound from the front ($\theta = 0$) strongly, from the sides half as strongly, and from behind ($\theta = \pi$) not at all. This reduces feedback from the loudspeakers facing the performer.

8.5 Calculus in polar coordinates

A polar curve $r = f(\theta)$ is a parametric curve with parameter θ: $x = f(\theta)\cos\theta$, $y = f(\theta)\sin\theta$. So the results of Section 8.2 apply. For area, cut the region into thin sectors. A sector of radius r and small angle $\mathrm{d}\theta$ has area $\frac{1}{2}r^2\,\mathrm{d}\theta$.

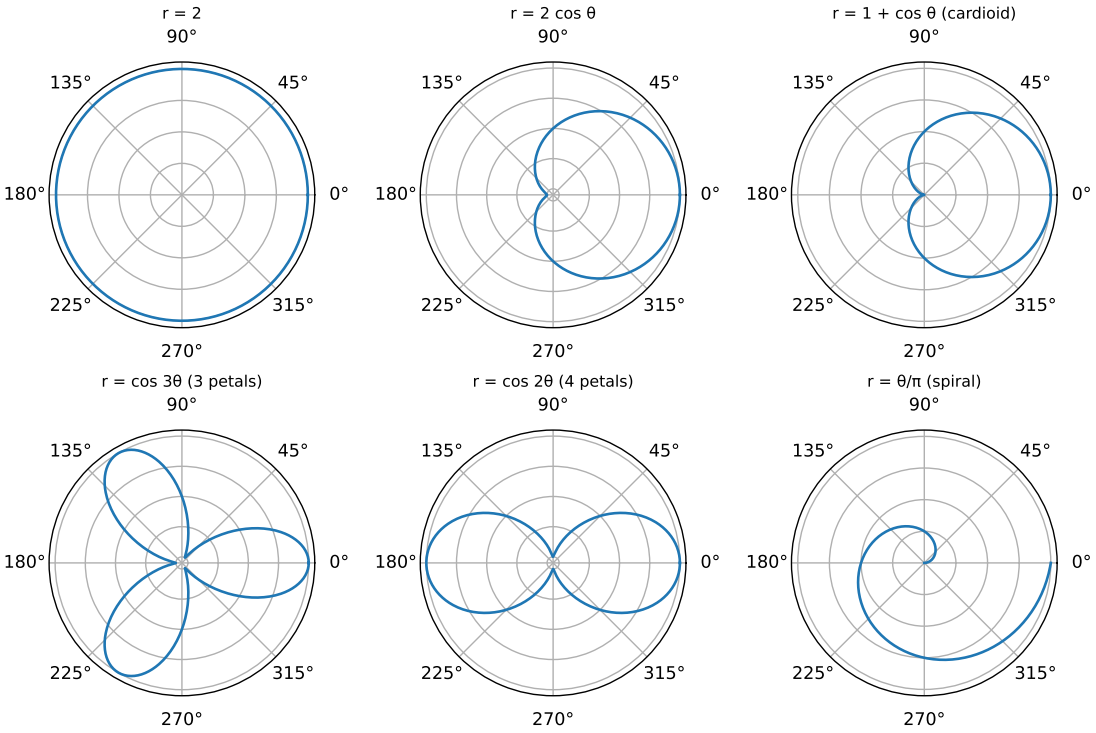


Figure 8.3: Polar curves plotted on Matplotlib's polar axes.

Key Concept: Slope, area and length in polar coordinates

$$\frac{\mathrm{d}y}{\mathrm{d}x} = \frac{r'\sin\theta + r\cos\theta}{r'\cos\theta - r\sin\theta}, \qquad A = \frac{1}{2}\int_\alpha^\beta r^2\,\mathrm{d}\theta, \qquad L = \int_\alpha^\beta \sqrt{r^2 + (r')^2}\,\mathrm{d}\theta,$$

where $r' = \frac{\mathrm{d}r}{\mathrm{d}\theta}$. The area formula gives the region swept out by the ray from the pole as θ runs from α to β.

Example 8.4 Find the area enclosed by the cardioid $r = a(1+\cos\theta)$.

Solution.

$$A = \frac{1}{2}\int_0^{2\pi} a^2(1+\cos\theta)^2\,\mathrm{d}\theta = \frac{a^2}{2}\int_0^{2\pi}\left(1 + 2\cos\theta + \frac{1+\cos 2\theta}{2}\right)\mathrm{d}\theta = \frac{a^2}{2}\cdot 3\pi = \frac{3\pi a^2}{2}.$$

■

Example 8.5 Find the area of one petal of the rose $r = \cos 2\theta$.

Solution. One petal is traced as θ runs from $-\frac{\pi}{4}$ to $\frac{\pi}{4}$ (where $r = 0$): $A = \frac{1}{2}\int_{-\pi/4}^{\pi/4}\cos^2 2\theta\,\mathrm{d}\theta = \frac{1}{2}\cdot\frac{\pi}{4} = \frac{\pi}{8}$. ■

Example 8.6 Find the area inside $r = 3\sin\theta$ and outside $r = 1 + \sin\theta$.

Solution. The curves meet where $3\sin\theta = 1 + \sin\theta$, that is $\sin\theta = \frac{1}{2}$, so at $\theta = \frac{\pi}{6}$ and $\frac{5\pi}{6}$. The area is

$$\frac{1}{2}\int_{\pi/6}^{5\pi/6}\left[(3\sin\theta)^2 - (1+\sin\theta)^2\right]\mathrm{d}\theta = \pi.$$

■

Listing 8.6 — Slopes and areas in polar coordinates `ex8_06_polar_calculus.py`

```python
# Listing 8.6: Slopes and areas in polar coordinates
import sympy as sp

th = sp.symbols("theta", real=True)
a = sp.symbols("a", positive=True)

# Slope of r = f(theta):  dy/dx = (r' sin + r cos) / (r' cos - r sin)
r = 1 + sp.cos(th)                                  # cardioid
rp = sp.diff(r, th)
slope = sp.simplify((rp*sp.sin(th) + r*sp.cos(th)) / (rp*sp.cos(th) - r*sp.sin(th)))
print("cardioid slope at theta = pi/2:", slope.subs(th, sp.pi/2))

# Area A = (1/2) ∫ r^2 dtheta
print("area of cardioid r = a(1 + cos θ):", sp.integrate((a*(1 + sp.cos(th)))**2/2, (th, 0,
      2*sp.pi)))
print("one petal of r = cos 2θ          :", sp.integrate(sp.cos(2*th)**2/2, (th, -sp.pi/4,
      sp.pi/4)))
print("whole circle r = 2a cos θ        :", sp.integrate((2*a*sp.cos(th))**2/2, (th, -sp.
    pi/2, sp.pi/2)))

# Inside r = 3 sin θ but outside r = 1 + sin θ: the curves meet where sin θ = 1/2
area = sp.integrate((9*sp.sin(th)**2 - (1 + sp.sin(th))**2)/2, (th, sp.pi/6, 5*sp.pi/6))
print("inside r = 3 sin θ, outside r = 1 + sin θ:", sp.simplify(area))

# Arc length in polar form: L = ∫ sqrt(r^2 + r'^2) dθ for the cardioid r = 1 + cos θ
L = sp.integrate(2*sp.cos(th/2), (th, 0, sp.pi)) * 2      # sqrt(r^2 + r'^2) = 2|cos(θ/2)|
print("length of the cardioid r = 1 + cos θ:", L)
```

Output

```
cardioid slope at theta = pi/2: 1
area of cardioid r = a(1 + cos θ): 3*pi*a**2/2
one petal of r = cos 2θ          : pi/8
whole circle r = 2a cos θ        : pi*a**2
inside r = 3 sin θ, outside r = 1 + sin θ: pi
length of the cardioid r = 1 + cos θ: 8
```

Watch Out: Find the right limits

In polar area problems the main difficulty is choosing α and β. Sketch the curve first (Python makes this easy), and check that θ from α to β traces the region exactly once. Integrating $\frac{1}{2}\int_0^{2\pi}\cos^2 2\theta\,d\theta$ gives the area of all four petals of $r=\cos 2\theta$, not one.

Chapter Summary

- Parametric curve: $x=f(t)$, $y=g(t)$. Eliminate t to get a Cartesian equation.
- Projectile: $x=(v\cos\alpha)t$, $y=(v\sin\alpha)t-\frac{1}{2}gt^2$.
- $\frac{dy}{dx}=\frac{\dot y}{\dot x}$ and $L=\int\sqrt{\dot x^2+\dot y^2}\,dt$.
- Polar: $x=r\cos\theta$, $y=r\sin\theta$, $r^2=x^2+y^2$. Use `np.arctan2` to find θ.

- Polar area: $\frac{1}{2}\int r^2\,\mathrm{d}\theta$. Polar length: $\int \sqrt{r^2 + r'^2}\,\mathrm{d}\theta$.
- Python: plot parametric curves with `plt.plot(x(t), y(t))`, and polar curves with `subplot_kw={"projection": "polar"}`.

8.6 Exercises

A. Skills

1. Eliminate the parameter and sketch the curve: (a) $x = 3t - 1$, $y = 2t + 4$; (b) $x = 1 + 2\cos t$, $y = -2 + 2\sin t$; (c) $x = e^t$, $y = e^{2t}$; (d) $x = \sec t$, $y = \tan t$.

2. Find $\frac{\mathrm{d}y}{\mathrm{d}x}$ and $\frac{\mathrm{d}^2y}{\mathrm{d}x^2}$ for $x = t^2 + 1$, $y = t^3 - 3t$. Where is the tangent horizontal? Where is it vertical?

3. Find the equation of the tangent to $x = 2\cos t$, $y = 3\sin t$ at $t = \frac{\pi}{4}$.

4. Find the length of the curve $x = t^2$, $y = \frac{2}{3}t^3$ for $0 \le t \le \sqrt{3}$.

5. Convert to Cartesian coordinates: (a) $(2, \frac{\pi}{3})$; (b) $(5, -\frac{\pi}{2})$; (c) $(-3, \frac{\pi}{4})$.

6. Convert to polar coordinates with $r > 0$ and $0 \le \theta < 2\pi$: (a) $(3, 3)$; (b) $(-1, \sqrt{3})$; (c) $(0, -4)$; (d) $(-2, -2\sqrt{3})$.

7. Write in polar form: (a) $x^2 + y^2 = 9$; (b) $y = x$; (c) $x^2 + y^2 = 6x$; (d) $xy = 1$.

8. Write in Cartesian form and identify: (a) $r = 5$; (b) $r = 4\cos\theta$; (c) $r\sin\theta = 3$; (d) $r = \frac{2}{1 - \cos\theta}$.

9. Find the area enclosed by (a) the circle $r = 4\cos\theta$; (b) one petal of $r = \sin 3\theta$; (c) the cardioid $r = 2 - 2\cos\theta$.

10. Find the slope of the tangent to $r = 2\sin\theta$ at $\theta = \frac{\pi}{6}$.

B. Applications

11. ⚙ A ball is kicked from the ground at 20 m/s at 35° above the horizontal. Find (a) its time of flight; (b) its range; (c) its greatest height. (Use $g = 9.8$ m/s^2.)

12. ⚙ A goalkeeper's goal kick must clear a defender 2 m tall standing 15 m away. If the ball is kicked at 18 m/s, what is the smallest launch angle that succeeds?

13. ⚙ A ship is at range 12 km on a bearing of 045° from a lighthouse, and a second ship is at range 8 km on a bearing of 300°. How far apart are the two ships?

14. ⚙ A lawn sprinkler waters the region $r \le 4 + 2\cos\theta$ (metres). What area does it water?

C. Programming

15. 🐍 Plot the roses $r = \cos n\theta$ for $n = 2, 3, 4, 5$. Check the rule for the number of petals.

16. 🐍 Simulate a ball kicked at 25 m/s with air resistance: at each small time step Δt, update the velocity by $\Delta\mathbf{v} = (-k\,|\mathbf{v}|\,\mathbf{v} - g\mathbf{j})\Delta t$ with $k = 0.01$. Compare the path with the parabola of Listing 8.2.

17. Write a function `polar_area(r, a, b)` that uses `quad` to compute $\frac{1}{2}\int_a^b r(\theta)^2\,\mathrm{d}\theta$. Use it to check Exercises 9 and 14.

18. Animate a point moving along a cycloid together with the rolling wheel that generates it (use a loop of `plt.pause` calls or `matplotlib.animation.FuncAnimation`).

Chapter 9

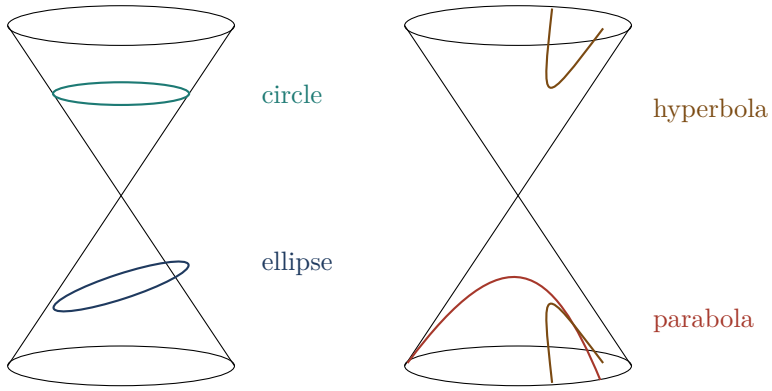
Conic Sections

Learning Objectives

After working through this chapter you should be able to:

- describe the parabola, ellipse and hyperbola as sections of a cone, and by their focus–directrix and distance properties;
- find the vertex, foci, directrix, axes and asymptotes from an equation, completing the square when necessary;
- find the equation of a conic from given geometric data;
- classify a general second-degree equation by its discriminant;
- apply conics to dishes, bridges, orbits and positioning systems, and plot them in Python.

Slice a double cone with a flat plane. Depending on the angle of the cut, the edge of the slice is a **circle**, an **ellipse**, a **parabola** or a **hyperbola**. These are the *conic sections*. They are all described by equations of the second degree in x and y, so they are the natural next step after straight lines (Chapter 1) and circles (Chapter 2).



Historical Note: Menaechmus, Apollonius and Hypatia

The conic sections were discovered by Menaechmus (c. 350 BCE), a student of Plato, while he was trying to solve the famous problem of "doubling the cube". Apollonius of Perga (c. 262–190 BCE), who studied in **Alexandria**, wrote the eight books of the *Conics*. He gave the curves the names we still use: *ellipse* ("falling short"), *parabola* ("placed alongside") and *hyperbola* ("thrown beyond").

Around 400 CE Hypatia of Alexandria, the most celebrated woman mathematician of the ancient world, taught mathematics and astronomy and wrote a commentary on Apollonius' *Conics*. She was murdered by a mob in 415 CE, and her writings have not survived. Much of the knowledge she and her father Theon preserved was later carried into Arabic science.
The conics turned out to describe the heavens. In 1609 Johannes Kepler announced that the planets move in *ellipses* with the Sun at one focus. In 1687 Newton proved, using calculus, that *any* body moving under gravity follows a conic: an ellipse for planets and satellites, and a parabola or hyperbola for some comets.

9.1 The parabola

Definition 9.1. (Parabola)

A *parabola* is the set of all points in a plane that are equidistant from a fixed point, the *focus*, and a fixed line, the *directrix*.

The line through the focus perpendicular to the directrix is the *axis* of the parabola. The point of the parabola nearest to the directrix (halfway between focus and directrix) is the *vertex*.

Take the focus $F(0,c)$ and the directrix $y=-c$, where $c>0$. A point $P(x,y)$ is on the parabola when $|PF|$ equals the distance from P to the directrix:

$$\sqrt{x^2+(y-c)^2}=y+c \implies x^2+y^2-2cy+c^2=y^2+2cy+c^2 \implies x^2=4cy.$$

Key Concept: Standard parabolas with vertex $(0,0)$

opens	equation	focus	directrix
upward	$x^2=4cy$	$(0,c)$	$y=-c$
downward	$x^2=-4cy$	$(0,-c)$	$y=c$
right	$y^2=4cx$	$(c,0)$	$x=-c$
left	$y^2=-4cx$	$(-c,0)$	$x=c$

($c>0$ is the distance from the vertex to the focus.) With vertex (h,k), replace x by $x-h$ and y by $y-k$. For example, $(x-h)^2=4c(y-k)$ has focus $(h,k+c)$ and directrix $y=k-c$.

Example 9.1

(a) Find the focus of $x^2=8y$. *Solution:* $4c=8$, so $c=2$. The focus is $(0,2)$ and the directrix is $y=-2$.

(b) Find the parabola with focus $(0,-4)$ and directrix $y=4$. *Solution:* it opens downward with $c=4$, so $x^2=-16y$.

(c) Find the focus and directrix of $3y^2+20x=0$. *Solution:* $y^2=-\frac{20}{3}x$, so the parabola opens left with $4c=\frac{20}{3}$, $c=\frac{5}{3}$. The focus is $\left(-\frac{5}{3},0\right)$ and the directrix is $x=\frac{5}{3}$, that is $3x-5=0$. (The third edition gave the sign of the directrix wrongly.)

(d) Find the focus and directrix of $x^2=15y$. *Solution:* $c=\frac{15}{4}$, focus $\left(0,\frac{15}{4}\right)$, directrix $4y+15=0$.

(e) Find the parabola with focus $\left(\frac{7}{2}, 0\right)$ and directrix $2x + 7 = 0$. *Solution:* it opens right with $c = \frac{7}{2}$, so $y^2 = 14x$.

Example 9.2 Find the vertex, focus and directrix of $x^2 + 4y + 8x + 4 = 0$.

Solution. Completing the square: $x^2 + 8x + 16 = -4y - 4 + 16$, so $(x+4)^2 = -4(y-3)$. This is a parabola opening downward with vertex $(-4, 3)$ and $4c = 4$, so $c = 1$. The focus is $(-4, 2)$ and the directrix is $y = 4$. ■

Example 9.3 Find the equation of the parabola with focus $Q(4, 3)$ and directrix $x - y = 11$.

Solution. Let $R(x, y)$ be on the parabola. By Theorem 1.1, its distance from the directrix is $\frac{|x-y-11|}{\sqrt{2}}$. Setting $|RQ|^2$ equal to the square of this distance gives

$$(x-4)^2 + (y-3)^2 = \frac{(x-y-11)^2}{2} \implies x^2 + 2xy + y^2 + 6x - 34y - 71 = 0.$$

The axis of this parabola is not parallel to a coordinate axis, so the equation has an xy term. ■

⚠ Watch Out: A focus on the directrix gives no parabola

The third edition solved this example with the directrix $x - y = 1$, which passes *through* the focus $(4, 3)$. In that case the "parabola" collapses: the equation becomes $(x + y - 7)^2 = 0$, a single straight line. Always check that the focus is not on the directrix.

Listing 9.1 — Checking the focus–directrix property ex9_01_parabola.py

```python
# Listing 9.1: A parabola is the set of points equidistant from a focus and a directrix
import numpy as np
import matplotlib.pyplot as plt

c = 2                                   # x^2 = 4cy = 8y: focus (0, 2), directrix y = -2
x = np.linspace(-6, 6, 13)
y = x**2 / (4*c)
d_focus = np.sqrt(x**2 + (y - c)**2)
d_directrix = y + c
print(" x      y   dist to focus   dist to directrix")
for row in zip(x[::3], y[::3], d_focus[::3], d_directrix[::3]):
    print("{:>3.0f} {:>6.2f} {:>15.4f} {:>19.4f}".format(*row))

X = np.linspace(-6, 6, 200)
plt.figure(figsize=(5, 3.8))
plt.plot(X, X**2/(4*c), "k", lw=2, label="$x^2 = 8y$")
plt.axhline(-c, color="C3", ls="--", label="directrix $y=-2$")
plt.plot(0, c, "C3*", ms=12, label="focus (0, 2)")
P = (4, 2)                              # a sample point on the parabola
plt.plot([P[0], 0], [P[1], c], "C0"); plt.plot([P[0], P[0]], [P[1], -c], "C0")
plt.plot(*P, "C0o")
plt.gca().set_aspect("equal"); plt.ylim(-3, 6); plt.grid(alpha=0.3); plt.legend(fontsize=8)

plt.show()
```

Output

```
 x      y   dist to focus   dist to directrix
-6   4.50          6.5000              6.5000
-3   1.12          3.1250              3.1250
 0   0.00          2.0000              2.0000
 3   1.12          3.1250              3.1250
 6   4.50          6.5000              6.5000
```

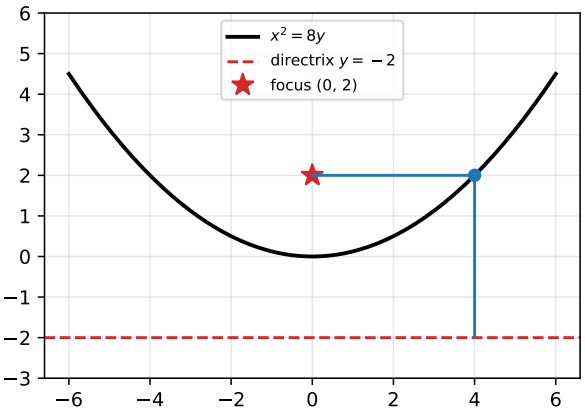


Figure 9.1: Every point of $x^2 = 8y$ is equally far from the focus and the directrix.

9.1.1 The reflective property

A ray travelling parallel to the axis of a parabola is reflected through the focus. Conversely, light from a lamp at the focus is reflected into a parallel beam.

In Context: Satellite dishes and Ghana's radio telescope

Satellite TV dishes on rooftops across Ghana are paraboloids, the 3-D shape formed by spinning a parabola about its axis. The weak signal from the satellite arrives in parallel rays and is concentrated at the focus, where the receiver (the LNB) sits. For a dish of diameter D and depth d, the cross-section $x^2 = 4cy$ passes through $\left(\frac{D}{2}, d\right)$, so the focal length is $c = \frac{D^2}{16d}$.
The same principle, on a much larger scale, is used at Kuntunse near Accra. There a 32-metre dish, converted from an old satellite-communications antenna, now serves as Ghana's radio telescope (the Ghana Radio Astronomy Observatory), part of a growing network of African radio telescopes.

Listing 9.2 — Focal length of a satellite dish `ex9_02_dish.py`

```python
# Listing 9.2: Where should the receiver of a satellite dish go?
import numpy as np
import matplotlib.pyplot as plt

D, depth = 90.0, 12.0                  # dish diameter and depth (cm)
# Dish cross-section: x^2 = 4cy passes through (D/2, depth)
```

```
c = (D/2)**2 / (4*depth)
print(f"focal length c = D^2/(16 depth) = {c:.2f} cm above the centre of the dish")

# Reflect vertical incoming rays off y = x^2/(4c): they all pass through (0, c)
X = np.linspace(-D/2, D/2, 200)
plt.figure(figsize=(5, 4))
plt.plot(X, X**2/(4*c), "k", lw=3)
for x0 in np.linspace(-40, 40, 9):
    y0 = x0**2/(4*c)
    plt.plot([x0, x0], [60, y0], "C1", lw=0.8)            # incoming ray
    plt.plot([x0, 0], [y0, c], "C0", lw=0.8)              # reflected ray
plt.plot(0, c, "r*", ms=14)
plt.gca().set_aspect("equal"); plt.ylim(-2, 62); plt.grid(alpha=0.3)
plt.title("all reflected rays meet at the focus")
plt.show()
```

Output

```
focal length c = D^2/(16 depth) = 42.19 cm above the centre of the dish
```

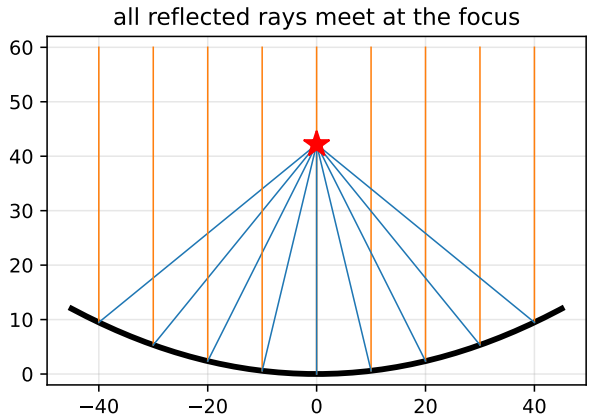


Figure 9.2: Rays parallel to the axis all reflect through the focus.

In Context: Arch bridges and football

The Adomi Bridge over the Volta River at Atimpoku, opened in 1957, carries the road from Accra towards the Volta Region beneath a great steel arch. An arch carrying a uniform load is close to a parabola. Suppose an arch spans 240 m and rises 50 m (these are illustrative figures). With the vertex at $(0, 50)$, the arch is $y = 50 - \frac{50}{120^2}x^2$. A football kicked by a Black Stars player also follows a parabola, if air resistance is ignored.

Listing 9.3 — A parabolic arch `ex9_03_arch.py`

```
# Listing 9.3: A parabolic arch bridge (illustrative dimensions)
# The arch spans 240 m and rises 50 m above its springing points.
# Put the vertex at (0, 50): the arch is y = 50 - k x^2 with y = 0 at x = +/-120.
k = 50 / 120**2
print(f"arch: y = 50 - {k:.6f} x^2   (i.e. x^2 = -4c(y - 50) with c = {1/(4*k):.1f} m)")
```

```python
for x in [0, 30, 60, 90, 110, 120]:
    print(f"x = {x:>3} m from the centre: height of arch = {50 - k*x**2:6.2f} m")
```

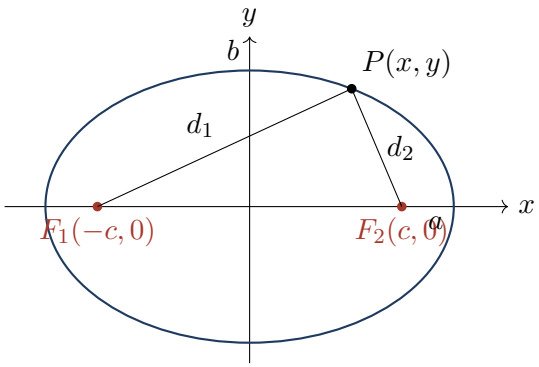
Output

```
arch: y = 50 - 0.003472 x^2   (i.e. x^2 = -4c(y - 50) with c = 72.0 m)
x =   0 m from the centre: height of arch =  50.00 m
x =  30 m from the centre: height of arch =  46.88 m
x =  60 m from the centre: height of arch =  37.50 m
x =  90 m from the centre: height of arch =  21.88 m
x = 110 m from the centre: height of arch =   7.99 m
x = 120 m from the centre: height of arch =   0.00 m
```

9.2 The ellipse

Definition 9.2. (Ellipse)

An *ellipse* is the set of all points in a plane for which the *sum* of the distances from two fixed points F_1 and F_2, the *foci*, is a constant $2a$.



$$d_1 + d_2 = 2a, \qquad b^2 = a^2 - c^2$$

Place the foci at $(\pm c, 0)$. Then $\sqrt{(x+c)^2+y^2} + \sqrt{(x-c)^2+y^2} = 2a$. Isolating one root, squaring, simplifying and squaring again gives $(a^2-c^2)x^2 + a^2y^2 = a^2(a^2-c^2)$. Writing $b^2 = a^2-c^2$ and dividing by a^2b^2:

Key Concept: The ellipse

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1 \qquad (a > b > 0), \qquad c^2 = a^2 - b^2.$$

- The **centre** is $(0,0)$. The **vertices** (ends of the major axis) are $(\pm a, 0)$, and the ends of the minor axis are $(0, \pm b)$.
- The **foci** are $(\pm c, 0)$. The semi-major axis is a and the semi-minor axis is b.
- The **eccentricity** $e = \dfrac{c}{a} = \sqrt{1 - \dfrac{b^2}{a^2}}$ satisfies $0 \le e < 1$. When $e = 0$ the ellipse is a circle, and as $e \to 1$ it becomes long and thin.

- For a vertical major axis: $\dfrac{x^2}{b^2}+\dfrac{y^2}{a^2}=1$, with foci $(0,\pm c)$.
- With centre (h,k): $\dfrac{(x-h)^2}{a^2}+\dfrac{(y-k)^2}{b^2}=1$, with foci $(h\pm c,k)$.

Example 9.4 For $\dfrac{x^2}{9}+\dfrac{y^2}{4}=1$: $a=3$ and $b=2$, so the vertices are $(\pm3,0)$ and the ends of the minor axis are $(0,\pm2)$. Also $c^2=9-4=5$, so the foci are $(\pm\sqrt{5},0)$ and $e=\frac{\sqrt{5}}{3}\approx 0.75$.

Example 9.5 For $\dfrac{(x-3)^2}{25}+\dfrac{(y-1)^2}{16}=1$: the centre is $(3,1)$, $a=5$, $b=4$ and $c=3$. The foci are $(6,1)$ and $(0,1)$.

Example 9.6 Find the centre, vertices and foci of $3x^2+4y^2+24x-16y+52=0$.

Solution. Group the terms and complete the squares:

$$3(x^2+8x+16)+4(y^2-4y+4)=-52+48+16$$
$$\implies 3(x+4)^2+4(y-2)^2=12 \implies \frac{(x+4)^2}{4}+\frac{(y-2)^2}{3}=1.$$

The centre is $(-4,2)$, with $a=2$ and $b=\sqrt{3}$. The vertices are $(-6,2)$ and $(-2,2)$. Since $c^2=4-3=1$, the foci are $(-5,2)$ and $(-3,2)$, and $e=\frac{1}{2}$. ■

Example 9.7 Find the ellipse with vertices $(3,2)$ and $(3,-4)$ and foci $(3,-1\pm\sqrt{5})$.

Solution. The centre is the midpoint $(3,-1)$, and the major axis is vertical with $a=3$ and $c=\sqrt{5}$. Then $b^2=a^2-c^2=4$, so $\dfrac{(y+1)^2}{9}+\dfrac{(x-3)^2}{4}=1$. ■

Listing 9.4 — Analysing an ellipse `ex9_04_ellipse.py`

```python
# Listing 9.4: Analysing the ellipse 3x^2 + 4y^2 + 24x - 16y + 52 = 0
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

x, y = sp.symbols("x y")
E = 3*x**2 + 4*y**2 + 24*x - 16*y + 52
# Completing the square: 3(x+4)^2 + 4(y-2)^2 = 12
print("check:", sp.expand(3*(x + 4)**2 + 4*(y - 2)**2 - 12 - E) == 0)
h, k, a, b = -4, 2, 2, np.sqrt(3)
c = np.sqrt(a**2 - b**2)
print(f"centre ({h}, {k}), a = {a}, b = {b:.4f}, c = {c:.0f}, e = c/a = {c/a}")
F1, F2 = np.array([h - c, k]), np.array([h + c, k])
print("foci:", F1, F2)

t = np.linspace(0, 2*np.pi, 7)[:-1]
for P in zip(h + a*np.cos(t), k + b*np.sin(t)):          # sample points on the ellipse
    P = np.array(P)
    print(f"P = ({P[0]:6.3f}, {P[1]:6.3f}):  PF1 + PF2 = "
          f"{np.linalg.norm(P - F1) + np.linalg.norm(P - F2):.6f}")

```

```
T = np.linspace(0, 2*np.pi, 300)
plt.figure(figsize=(5, 3.4))
plt.plot(h + a*np.cos(T), k + b*np.sin(T), "k")
plt.plot(*F1, "r*", ms=10); plt.plot(*F2, "r*", ms=10)
plt.gca().set_aspect("equal"); plt.grid(alpha=0.3); plt.show()
```

Output

```
check: True
centre (-4, 2), a = 2, b = 1.7321, c = 1, e = c/a = 0.5000000000000001
foci: [-5.  2.] [-3.  2.]
P = (-2.000,  2.000):  PF1 + PF2 = 4.000000
P = (-3.000,  3.500):  PF1 + PF2 = 4.000000
P = (-5.000,  3.500):  PF1 + PF2 = 4.000000
P = (-6.000,  2.000):  PF1 + PF2 = 4.000000
P = (-5.000,  0.500):  PF1 + PF2 = 4.000000
P = (-3.000,  0.500):  PF1 + PF2 = 4.000000
```

The output confirms the defining property: for every sampled point P, $PF_1 + PF_2 = 4 = 2a$.

In Context: Orbits: from the Earth to GhanaSat-1

The Earth's orbit is an ellipse with the Sun at one focus. The Earth's closest distance to the Sun (about 147.1 million km, in early January) and its furthest distance (about 152.1 million km, in early July) give a and c. The eccentricity is only about 0.017, so the orbit is very nearly a circle. (The seasons are caused by the tilt of the Earth's axis, not by the changing distance.)

In July 2017 **GhanaSat-1**, Ghana's first satellite, was released into orbit from the International Space Station. It was a small "CubeSat" built by students at All Nations University in Koforidua. Its orbit, about 400 km above the Earth, was a nearly circular ellipse with the centre of the Earth at a focus. Kepler's third law, $T = 2\pi\sqrt{a^3/GM}$, gives its period: about 92 minutes, or roughly 15 orbits a day.

Listing 9.5 — The Earth's orbit and GhanaSat-1 `ex9_05_orbits.py`

```
# Listing 9.5: Elliptical orbits - the Earth, and Ghana's first satellite
import numpy as np
from scipy.integrate import quad

# Earth's orbit: closest to the Sun 147.1 million km, furthest 152.1 million km
peri, aph = 147.1, 152.1
a = (peri + aph) / 2                  # semi-major axis
c = a - peri                          # centre-to-focus distance (Sun at a focus)
e = c / a
b = np.sqrt(a**2 - c**2)
print(f"a = {a:.2f}, b = {b:.3f} (million km), e = {e:.4f}")

# Length of the orbit: arc length of x = a cos t, y = b sin t (an integral, Ch. 6)
L, _ = quad(lambda t: np.sqrt((a*np.sin(t))**2 + (b*np.cos(t))**2), 0, 2*np.pi)
print(f"orbit length = {L:.2f} million km;  2*pi*a = {2*np.pi*a:.2f}")
print(f"average orbital speed = {L*1e6/(365.256*24*3600):.1f} km/s")

```

```
# GhanaSat-1 orbited about 400 km above the Earth (radius 6371 km).
# Kepler's third law: period T = 2 pi sqrt(a^3 / GM), GM = 398600 km^3/s^2
GM, a_sat = 398600, 6371 + 400
T = 2*np.pi*np.sqrt(a_sat**3 / GM)
print(f"period at 400 km altitude = {T/60:.1f} minutes, i.e. {24*3600/T:.1f} orbits per
    day")
```

Output

```
a = 149.60, b = 149.579 (million km), e = 0.0167
orbit length = 939.90 million km;  2*pi*a = 939.96
average orbital speed = 29.8 km/s
period at 400 km altitude = 92.4 minutes, i.e. 15.6 orbits per day
```

Python Tip: An integral with no formula

The perimeter of an ellipse has no elementary formula. Listing 9.5 finds it as the arc length of the parametric curve $x = a\cos t$, $y = b\sin t$ (Section 8.2), computed with `quad`.

In Context: Whispering galleries and kidney stones

A sound or shock wave sent out from one focus of an elliptical reflector is reflected to the other focus. In an elliptical "whispering gallery", a whisper at one focus can be heard clearly at the other. Doctors use the same property in *lithotripsy*: shock waves produced at one focus of an elliptical reflector converge on a kidney stone placed at the other focus and break it up without surgery.

9.3 The hyperbola

Definition 9.3. (Hyperbola)

A *hyperbola* is the set of all points in a plane for which the *difference* of the distances from two fixed points (the foci) is a constant, $\pm 2a$.

With foci $(\pm c, 0)$, the condition $|d_1 - d_2| = 2a$ leads, by the same algebra as for the ellipse, to $\frac{x^2}{a^2} - \frac{y^2}{c^2-a^2} = 1$. Here $c > a$, and we write $b^2 = c^2 - a^2$.

Key Concept: The hyperbola

$$\frac{x^2}{a^2} - \frac{y^2}{b^2} = 1, \qquad c^2 = a^2 + b^2.$$

- The **vertices** are $(\pm a, 0)$ and the **foci** are $(\pm c, 0)$. The **transverse axis**, of length $2a$, joins the vertices.
- The **asymptotes** are $y = \pm\dfrac{b}{a}x$. The branches approach these lines as $|x| \to \infty$.
- The **eccentricity** $e = \dfrac{c}{a} > 1$.

- With vertical transverse axis: $\dfrac{y^2}{a^2} - \dfrac{x^2}{b^2} = 1$, with foci $(0, \pm c)$ and asymptotes $y = \pm \dfrac{a}{b}x$.
- With centre (h, k): replace x by $x - h$ and y by $y - k$.

Watch Out: Ellipse or hyperbola: which c?

For an ellipse $c^2 = a^2 - b^2$ (the foci are *inside*, so $c < a$). For a hyperbola $c^2 = a^2 + b^2$ (the foci are *beyond* the vertices, so $c > a$). Also, in a hyperbola a need not be larger than b. The term with the *plus* sign tells you the direction of the transverse axis.

Why the asymptotes? Solving $\frac{x^2}{a^2} - \frac{y^2}{b^2} = 1$ for y gives

$$y = \pm \frac{b}{a}\sqrt{x^2 - a^2} = \pm \frac{b}{a}x\sqrt{1 - \frac{a^2}{x^2}}.$$

As $|x| \to \infty$, $\sqrt{1 - a^2/x^2} \to 1$ (a limit from Chapter 3), so $y \approx \pm \frac{b}{a}x$.

Example 9.8 Find the centre, vertices, foci and asymptotes of $16x^2 - 9y^2 - 128x - 18y + 103 = 0$.

Solution. $16(x^2 - 8x + 16) - 9(y^2 + 2y + 1) = -103 + 256 - 9$, so $16(x-4)^2 - 9(y+1)^2 = 144$, that is,

$$\frac{(x-4)^2}{9} - \frac{(y+1)^2}{16} = 1.$$

The centre is $(4, -1)$, with $a = 3$, $b = 4$ and $c = \sqrt{9 + 16} = 5$. The vertices are $(1, -1)$ and $(7, -1)$. The foci are $(-1, -1)$ and $(9, -1)$. The asymptotes are $y + 1 = \pm \frac{4}{3}(x - 4)$, and $e = \frac{5}{3}$. ■

Example 9.9 Find the hyperbola with vertices $(2, 4)$ and $(2, -2)$ and foci $(2, 6)$ and $(2, -4)$.

Solution. The centre is the midpoint $(2, 1)$, and the transverse axis is vertical. Then $a = 3$ and $c = 5$, so $b^2 = 25 - 9 = 16$. The hyperbola is $\dfrac{(y-1)^2}{9} - \dfrac{(x-2)^2}{16} = 1$. ■

Example 9.10 Find the hyperbola with foci $(-3, 2)$ and $(5, 2)$ and eccentricity $\frac{3}{2}$.

Solution. The centre is $(1, 2)$ and $c = 4$. From $e = \frac{c}{a} = \frac{3}{2}$ we get $a = \frac{8}{3}$. Then $b^2 = c^2 - a^2 = 16 - \frac{64}{9} = \frac{80}{9}$. The hyperbola is $\dfrac{9(x-1)^2}{64} - \dfrac{9(y-2)^2}{80} = 1$. ■

Listing 9.6 — A hyperbola and its asymptotes `ex9_06_hyperbola.py`

```
# Listing 9.6: The hyperbola 16x^2 - 9y^2 - 128x - 18y + 103 = 0 and its asymptotes
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

x, y = sp.symbols("x y")
H = 16*x**2 - 9*y**2 - 128*x - 18*y + 103
standard = 16*(x - 4)**2 - 9*(y + 1)**2 - 144
print("16(x-4)^2 - 9(y+1)^2 = 144 is the same curve:", sp.expand(standard - H) == 0)
h, k, a, b = 4, -1, 3, 4
```

```python
c = np.sqrt(a**2 + b**2)
print(f"centre ({h}, {k}), a = {a}, b = {b}, c = {c:.0f}, e = {c/a:.4f}")
print(f"foci ({h - c:.0f}, {k}) and ({h + c:.0f}, {k}); asymptotes y + 1 = ±(4/3)(x - 4)")

t = np.linspace(-2, 2, 200)                       # x = h ± a cosh t, y = k + b sinh t
plt.figure(figsize=(5, 4))
plt.plot(h + a*np.cosh(t), k + b*np.sinh(t), "k")
plt.plot(h - a*np.cosh(t), k + b*np.sinh(t), "k")
X = np.linspace(-6, 14, 2)
plt.plot(X, k + b/a*(X - h), "C3--", lw=1); plt.plot(X, k - b/a*(X - h), "C3--", lw=1)
plt.plot([h - c, h + c], [k, k], "b*", ms=10)
plt.xlim(-6, 14); plt.ylim(-12, 10); plt.gca().set_aspect("equal"); plt.grid(alpha=0.3)
plt.show()
```

Output

```
16(x-4)^2 - 9(y+1)^2 = 144 is the same curve: True
centre (4, -1), a = 3, b = 4, c = 5, e = 1.6667
foci (-1, -1) and (9, -1); asymptotes y + 1 = ±(4/3)(x - 4)
```

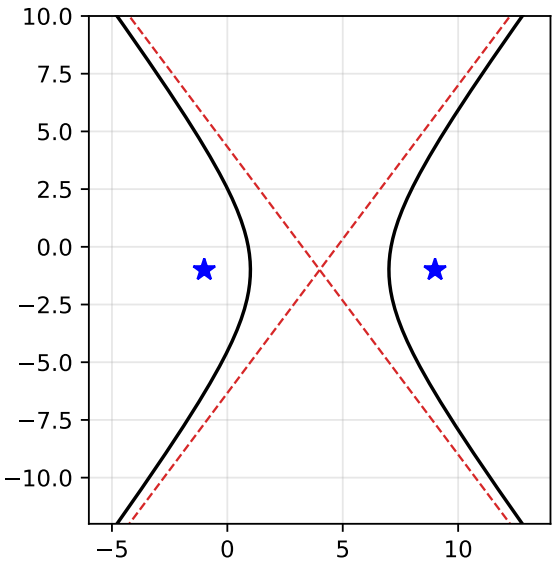


Figure 9.3: Both branches of the hyperbola, its asymptotes (dashed) and its foci.

In Context: Where did the lightning strike?

Thunderstorms are common in Ghana's rainy season. A lightning strike gives off a radio pulse that reaches different weather stations at slightly different times. If the pulse reaches station 1 a time Δt after station 2, then the strike is $v\,\Delta t$ further from station 1 than from station 2. The difference of its distances to two fixed points is constant, so it lies on a *hyperbola* with those stations as foci. A third station gives a second hyperbola, and the strike is where the two meet. The same principle, called *time difference of arrival*, was used by the LORAN ship-navigation system and is used to locate mobile phones in emergencies.

Listing 9.7 — Locating a lightning strike ex9_07_lightning.py

```python
# Listing 9.7: Locating a lightning strike from arrival-time differences (hyperbolas)
import numpy as np
from scipy.optimize import fsolve

v = 3.0e5                                          # radio waves travel at 300 000 km/s
S = np.array([[0.0, 0.0], [40.0, 0.0], [10.0, 35.0]])   # three weather stations (km)

true = np.array([25.0, 18.0])                      # used only to simulate the data
arrival = np.linalg.norm(S - true, axis=1) / v     # arrival times (s)
dt12, dt13 = arrival[0] - arrival[1], arrival[0] - arrival[2]
print(f"time differences: {dt12*1e6:.3f} and {dt13*1e6:.3f} microseconds")

# Each time difference puts the strike on one branch of a hyperbola with foci at
# two stations:  |PS1| - |PS2| = v * dt12  and  |PS1| - |PS3| = v * dt13
def equations(P):
    d = np.linalg.norm(S - P, axis=1)
    return [d[0] - d[1] - v*dt12, d[0] - d[2] - v*dt13]

estimate = fsolve(equations, x0=[20, 20])
print("estimated strike position:", np.round(estimate, 4), "km")
```

Output

```
time differences: 24.584 and 27.114 microseconds
estimated strike position: [25. 18.] km
```

9.4 The general second-degree equation

Every conic can be written as

$$Ax^2 + Bxy + Cy^2 + Dx + Ey + F = 0.$$

The Bxy term appears when the axes of the conic are tilted. Apart from degenerate cases (a point, a line, a pair of lines, or no points at all), the type is decided by the *discriminant*.

Key Concept: Classifying conics

$B^2 - 4AC < 0$	ellipse (a circle if also $A = C$ and $B = 0$)
$B^2 - 4AC = 0$	parabola
$B^2 - 4AC > 0$	hyperbola

Listing 9.8 — Classifying and plotting conics ex9_08_classify.py

```python
# Listing 9.8: Classifying Ax^2 + Bxy + Cy^2 + Dx + Ey + F = 0 and plotting it
import numpy as np
import matplotlib.pyplot as plt

def classify(A, B, C):
    disc = B**2 - 4*A*C
```

```
    if disc < 0:
        return "circle" if (A == C and B == 0) else "ellipse"
    return "parabola" if disc == 0 else "hyperbola"

curves = {                                      # (A, B, C, D, E, F)
    "x²+y²+3x−6y−1=0":          (1, 0, 1, 3, -6, -1),
    "3x²+4y²+24x−16y+52=0":     (3, 0, 4, 24, -16, 52),
    "x²+4y+8x+4=0":             (1, 0, 0, 8, 4, 4),
    "16x²−9y²−128x−18y+103=0": (16, 0, -9, -128, -18, 103),
    "x²+2xy+y²+6x−34y−71=0":    (1, 2, 1, 6, -34, -71),
}
X, Y = np.meshgrid(np.linspace(-12, 16, 600), np.linspace(-12, 16, 600))
plt.figure(figsize=(5.5, 5.5))
for i, (name, (A, B, C, D, E, F)) in enumerate(curves.items()):
    kind = classify(A, B, C)
    print(f"{name:<26} B²−4AC = {B**2 - 4*A*C:>5}  -> {kind}")
    Z = A*X**2 + B*X*Y + C*Y**2 + D*X + E*Y + F
    plt.contour(X, Y, Z, levels=[0], colors=f"C{i}")
    plt.plot([], [], color=f"C{i}", label=f"{kind}")
plt.gca().set_aspect("equal"); plt.grid(alpha=0.3); plt.legend(fontsize=8, loc="lower left
    ")
plt.show()
```

Output

```
x²+y²+3x−6y−1=0             B²−4AC =   -4  -> circle
3x²+4y²+24x−16y+52=0        B²−4AC =  -48  -> ellipse
x²+4y+8x+4=0                B²−4AC =    0  -> parabola
16x²−9y²−128x−18y+103=0     B²−4AC =  576  -> hyperbola
x²+2xy+y²+6x−34y−71=0       B²−4AC =    0  -> parabola
```

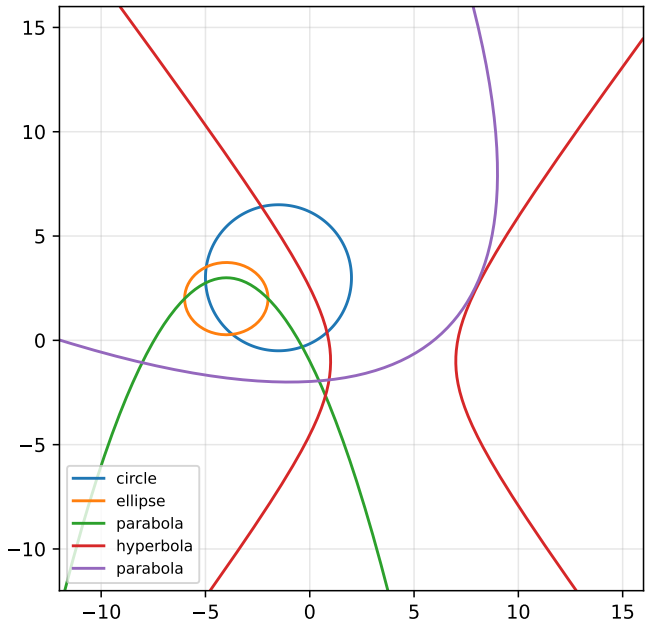


Figure 9.4: The five conics from the examples of this book, drawn as the zero contour of $Ax^2 + Bxy + Cy^2 + Dx + Ey + F$.

Python Tip: Plotting an implicit curve

To draw a curve given by $G(x, y) = 0$, without solving for y, evaluate G on a grid made by `np.meshgrid`, and draw its zero level with `plt.contour(X, Y, G, levels=[0])`.

9.5 Eccentricity: one definition for all conics

The focus–directrix definition of the parabola extends to all conics. With the focus at the pole, the polar coordinates of Chapter 8 give a single equation for every conic.

Theorem 9.1. (Focus–directrix property)

Let F be a fixed point (the focus), ℓ a fixed line (the directrix) and $e > 0$ a constant. The set of points P with

$$\frac{|PF|}{\text{distance from } P \text{ to } \ell} = e$$

is an ellipse if $e < 1$, a parabola if $e = 1$ and a hyperbola if $e > 1$. In polar coordinates, with the focus at the origin and the directrix $x = d$, it is $r = \dfrac{ed}{1 + e\cos\theta}$.

Listing 9.9 — A family of conics with the same focus `ex9_09_eccentricity.py`

```
# Listing 9.9: One equation for all conics: r = e d / (1 + e cos(theta))
import numpy as np
import matplotlib.pyplot as plt

d = 1.0                                          # distance from the focus to the directrix
theta = np.linspace(0, 2*np.pi, 2000)
plt.figure(figsize=(5.5, 4.5))
for e, style in [(0.5, "C0"), (0.8, "C2"), (1.0, "C3"), (1.5, "C1")]:
    r = e*d / (1 + e*np.cos(theta))
    r[(1 + e*np.cos(theta)) <= 0.02] = np.nan         # skip points at infinity
    plt.plot(r*np.cos(theta), r*np.sin(theta), style, label=f"e = {e}")
plt.axvline(d, color="k", ls="--", lw=1, label="directrix")
plt.plot(0, 0, "k*", ms=10, label="focus")
plt.xlim(-4, 1.5); plt.ylim(-3, 3); plt.gca().set_aspect("equal")
plt.grid(alpha=0.3); plt.legend(fontsize=8)
plt.show()
```

Chapter Summary

- Parabola: $|PF|$ equals the distance to the directrix. $x^2 = 4cy$ has focus $(0, c)$ and directrix $y = -c$. Reflective property.
- Ellipse: $d_1 + d_2 = 2a$. $\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$ with $c^2 = a^2 - b^2$ and $e = \frac{c}{a} < 1$.
- Hyperbola: $|d_1 - d_2| = 2a$. $\frac{x^2}{a^2} - \frac{y^2}{b^2} = 1$ with $c^2 = a^2 + b^2$, asymptotes $y = \pm\frac{b}{a}x$ and $e > 1$.
- Complete the square to find the centre or vertex. Classify by $B^2 - 4AC$.
- Python: parametric plots ($a\cos t$, $b\sin t$ and $a\cosh t$, $b\sinh t$), `plt.contour` for im-

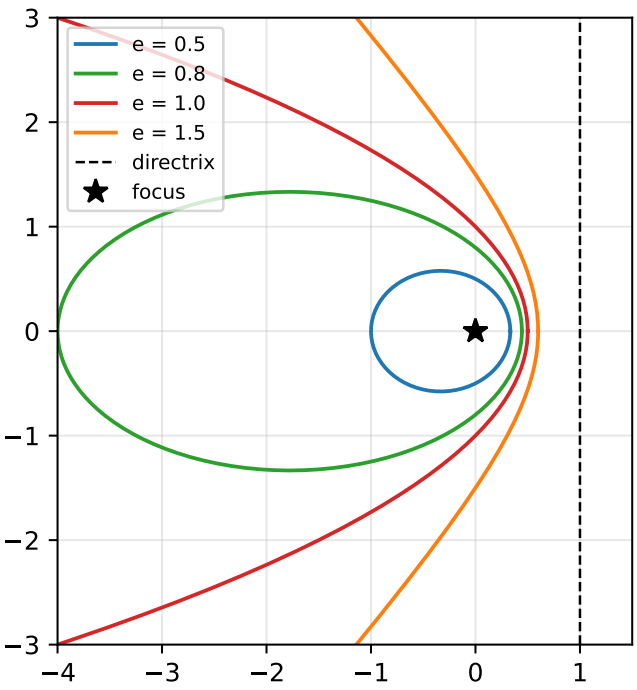


Figure 9.5: As e increases through 1, the closed ellipse opens out into a parabola and then a hyperbola.

plicit curves, and `fsolve` for intersecting conics.

9.6 Exercises

A. Skills

1. Find the vertex, focus and directrix, and sketch: (a) $y^2 = 12x$; (b) $x^2 = -6y$; (c) $y^2 - 4y - 8x + 28 = 0$; (d) $x^2 + 6x + 4y + 1 = 0$.

2. Find the equation of the parabola with (a) focus $(0, 3)$ and directrix $y = -3$; (b) vertex $(2, -1)$ and focus $(2, 1)$; (c) focus $(1, 1)$ and directrix $x + y = 0$.

3. Find the centre, vertices, foci and eccentricity: (a) $\frac{x^2}{25} + \frac{y^2}{9} = 1$; (b) $4x^2 + y^2 = 16$; (c) $9x^2 + 4y^2 - 18x + 16y - 11 = 0$.

4. Find the equation of the ellipse with (a) foci $(\pm 4, 0)$ and vertices $(\pm 5, 0)$; (b) centre $(1, 2)$, a focus at $(1, 5)$ and $a = 5$; (c) vertices $(0, \pm 6)$ and eccentricity $\frac{1}{3}$.

5. Find the centre, vertices, foci and asymptotes: (a) $\frac{x^2}{16} - \frac{y^2}{9} = 1$; (b) $y^2 - 4x^2 = 4$; (c) $4x^2 - y^2 - 24x - 4y + 28 = 0$.

6. Find the equation of the hyperbola with (a) vertices $(\pm 3, 0)$ and foci $(\pm 5, 0)$; (b) foci $(0, \pm 10)$ and asymptotes $y = \pm \frac{3}{4}x$.

7. Classify each conic and find its centre or vertex: (a) $x^2 + y^2 - 6x + 2y + 6 = 0$; (b) $x^2 - 4y^2 - 2x + 16y - 19 = 0$; (c) $2y^2 + x - 4y = 0$; (d) $25x^2 + 9y^2 + 50x - 36y - 164 = 0$.

8. Show that the line $y = mx + c$ is a tangent to the ellipse $\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$ if and only if $c^2 = a^2m^2 + b^2$.

B. Applications

9. ⚙ A satellite-TV dish is 60 cm across and 8 cm deep. How far from the vertex should the receiver be placed?

10. ⚙ A footbridge over a stream at a university campus has a parabolic arch that spans 30 m and rises 6 m. A canoe 2 m tall and 3 m wide must pass under it. How far from the centre of the stream may the canoe safely pass?

11. ⚙ An elliptical running track at a sports stadium has a major axis of 120 m and a minor axis of 80 m. Find its eccentricity and the distance between its foci. Use `quad` to estimate the length of one lap.

12. ⚙ Two radio stations, 100 km apart on a straight coastline, transmit synchronised pulses. A fishing boat receives the pulse from station A 200 microseconds before the pulse from station B. Using $v = 300\,000$ km/s, show that the boat lies on a hyperbola, and find its equation, taking A and B at $(\mp 50, 0)$.

C. Programming

13. 🐍 Write a function `conic_info(A, C, D, E, F)` for $Ax^2 + Cy^2 + Dx + Ey + F = 0$ (with no xy term). It should complete the squares, decide the type of conic, and print the centre or vertex, the foci and the eccentricity.

14. 🐍 Animate (or plot as a sequence of frames) the ellipse $\frac{x^2}{a^2} + \frac{y^2}{1} = 1$ as a increases from 1 to 5, marking the foci. What happens to the eccentricity?

15. 🐍 Modify Listing 9.7 by adding a random timing error of about 0.1 microseconds to each station. Repeat 500 times and plot the estimated strike positions. How accurate is the method?

16. 🐍 Kepler's second law says that the line from the Sun to a planet sweeps out equal areas in equal times. Use `scipy.integrate.solve_ivp` (look it up!) to simulate an orbit with $e = 0.5$, and check the law numerically.

Chapter

A

Python Quick Reference

Getting started

`import numpy as np`	numerical arrays and functions
`import sympy as sp`	exact (symbolic) mathematics
`import matplotlib.pyplot as plt`	graphs
`from scipy.optimize import minimize_scalar, fsolve`	numerical optimisation and equation solving
`from scipy.integrate import quad, simpson`	numerical integration

Core Python

`x = 3.5`	assign a value to a variable
`+ - * / ** // %`	add, subtract, multiply, divide, power, whole-number division, remainder
`print(f"x = {x:.3f}")`	formatted output (3 decimal places)
`L = [1, 4, 9]; L[0]; len(L)`	list, first item, length
`for i in range(1, 11):`	loop over $1, 2, \ldots, 10$
`if x > 0: ... elif ...: ... else: ...`	decisions
`def f(x): return x**2`	define a function
`f = lambda x: x**2`	one-line function

NumPy

`np.linspace(a, b, n)`	n equally spaced points from a to b
`np.arange(a, b, h)`	$a, a+h, \ldots$ (stopping before b)
`np.sin, np.cos, np.tan, np.exp, np.log, np.sqrt`	functions on arrays (ln is `np.log`; radians)
`np.pi, np.e`	π and e
`np.radians(d), np.degrees(r)`	convert angles
`a.sum(), a.mean(), a.max()`	summaries of an array
`np.polyfit(x, y, k)`; `np.poly1d(c)`	best-fit polynomial of degree k; make it a function
`np.gradient(y, x)`	numerical derivative from data
`np.trapz(y, x)`	trapezium rule (`np.trapezoid` in NumPy $\geq$ 2.0)
`np.linalg.solve(A, b)`	solve the linear system $A\mathbf{x} = \mathbf{b}$

SymPy

`x = sp.symbols("x")`	create a symbol
`sp.expand, sp.factor, sp.simplify`	algebra
`sp.solve(eq, x)`	solve an equation (or a list of equations)
`expr.subs(x, 2)`	substitute a value
`sp.limit(f, x, a)`; `dir="+"`	limit (one-sided with `dir`)
`sp.diff(f, x)`; `sp.diff(f, x, 2)`	first and second derivatives
`sp.idiff(F, y, x)`	implicit differentiation of $F(x, y) = 0$
`sp.integrate(f, x)`; `sp.integrate(f, (x, a, b))`	indefinite and definite integrals
`sp.Rational(1, 3)`; `sp.sqrt(2)`; `sp.pi`; `sp.oo`	exact numbers and ∞
`sp.N(expr, 6)`	decimal value to 6 significant figures
`sp.lambdify(x, expr)`	turn a formula into a fast NumPy function
`sp.Point, sp.Line`	coordinate geometry objects

Matplotlib

`plt.plot(x, y, "r-", label="...")`	line plot (colour/style string optional)
`plt.plot(a, b, "o")`	mark a single point
`plt.xlabel, plt.ylabel, plt.title, plt.legend()`	labels
`plt.grid(alpha=0.3)`	light grid
`plt.gca().set_aspect("equal")`	equal scales (circles look round)
`plt.fill_between(x, y1, y2)`	shade the region between two curves
`plt.contour(X, Y, Z, levels=[0])`	draw the implicit curve $Z(x, y) = 0$
`fig, ax = plt.subplots(1, 3)`	several plots side by side
`plt.show()`	display the figure

Common error messages

`IndentationError`	the lines of a block are not indented consistently
`NameError: name 'np' is not defined`	you forgot `import numpy as np` (or did not run that cell)
`SyntaxError`	a missing bracket, colon or quotation mark; check the line *before* the one reported
`TypeError: can't multiply sequence`	you wrote `2x` instead of `2*x`, or used a list where an array was needed
`ZeroDivisionError`	dividing by zero, e.g. evaluating $\frac{\sin x}{x}$ at $x = 0$
`RuntimeWarning: invalid value`	NumPy met $\sqrt{-1}$ or $\frac{0}{0}$ and produced `nan`

Watch Out: `^` is not a power

In Python, `2^3` is *not* 2^3. It is a bitwise operation that gives 1. Always write `2**3`.

Chapter B

Answers to Selected Exercises

These answers were checked with SymPy and SciPy (the script `tools/check_answers.py` reproduces them). Where a "show that" exercise or a programming exercise has no single answer, no answer is given. Use Python to check your own working.

Chapter 0

1. (a) 13 841 287 201; (b) 6; (c) 3.146264; (d) both 0.5, but remember `np.sin(np.radians(30))`. **5.** $x = 3$ or $x = \frac{1}{2}$. **6.** $d = \frac{40}{3} \approx 13.3\,\text{km}$. **7.** After 12 months: GH₵ 563.41.

Chapter 1

1. (a) 17; (b) $\left(1, -\frac{5}{2}\right)$; (c) $\left(-\frac{5}{7}, \frac{5}{7}\right)$; (d) $\left(9, -\frac{35}{2}\right)$; (e) $-\frac{15}{8}$; (f) $15x+8y+5=0$; (g) $16x-30y-91=0$; (h) $\frac{67}{17}$. **2.** $r < 1$ or $r > 2$. **6.** $y = -2$. **7.** $y = -x + 5$ (through $(-8, 13)$). **9.** (a) $\tan\theta = \frac{2}{3}$, so $\theta \approx 33.69°$; (b) no. **12.** $21x - 77y + 106 = 0$ and $99x + 27y - 76 = 0$. **13.** 6.41 km; midpoint (1.45, −1.2). **14.** (a) GH₵ 0.60 per km; (b) GH₵ 4; (c) GH₵ 16. **15.** 4.5%; 2.58°. **16.** $\frac{3}{\sqrt{5}} \approx 1.34$ hundred metres (134 m); nearest point (5.6, 4.8). **17.** $(14, 8)$.

Chapter 2

2. $k = \pm 1$. **3.** Outside ($9+25 > 9$). **4.** $k = 36$. **5.** $x^2+y^2-x-2y=0$. **6.** $2x-3y+6=0$. **7.** $x^2 + y^2 - 8x - 6y = 0$. **8.** $(2, 0)$ and $(0, 2)$. **9.** $(x+2)^2 + (y+2)^2 = 25$. **10.** (a) $(-3, 5)$, $\sqrt{43}$; (b) $(0, -2)$, 2; (c) $(4, 0)$, 4; (d) $(-4, 11)$, 3; (e) $(5, 1)$, 2. **11.** $x^2 + y^2 - 2x - 6y - 15 = 0$. **12.** $3x + 2y = 0$. **13.** Chord $3x + 3y - 10 = 0$; length $\frac{\sqrt{178}}{3} \approx 4.45$. **15.** Centre $(20, 10)$, radius $10\sqrt{5} \approx 22.4\,\text{m}$. **16.** About 96.3 km. **17.** Centre $(30, 40)$, radius 20 m, area $1257\,\text{m}^2 \approx 0.126\,\text{ha}$. **18.** Yes ($1 < d = 7 < 9$); common chord $x = \frac{50}{7}$.

Chapter 3

1. (i) 4; (ii) 0; (iii) 12; (iv) 1; (v) does not exist (left -1, right 1); (vi) $\frac{4}{9}$; (vii) $\frac{\sqrt{2}}{4}$; (viii) 1; (ix) does not exist (left $-\frac{3}{2}$, right $\frac{3}{2}$); (x) 0; (xi) -3; (xii) 38. **2.** $\lim_{x\to -1} f(x) = -1$; $\lim_{x\to 1} f(x)$ does not exist (left 1, right -1). **3.** $c = \frac{1}{3}$. **4.** 16. **5.** $a = 6$. **6.** -7. **7.** $\frac{1}{2}$. **8.** $a = b = \frac{1}{2}$. **9.** (a) $\frac{5}{2}$; (b) 0; (c) e^2. **11.** $f(0) = -1 < 0 < 1 = f(1)$; apply the IVT.

12. Average speed $0.7 + 0.02h$ km/min; instantaneous speed 0.7 km/min $= 42$ km/h. **13.** Discontinuous at $t = 1, 2, 3, 4$; $\lim_{t\to 2^-} C = 15$ and $\lim_{t\to 2^+} C = 20$. **14.** $r \approx 0.0203$; the limit is about 0.626 million people per year, the growth rate of the population in 2021. **15.** About GH¢ 2.29.

Chapter 4

1. $2ax + b$. **2.** (a) $2x - 3$; (b) $\frac{4}{(t+1)^2}$; (c) $\cos x$; (d) $\frac{1}{2\sqrt{x}}$. **3.** (iv) $\frac{30x}{(3x^2+2)\ln 10}$; (v) $\sec x$; (vii) $\frac{2ab\sec^2 x}{a^2-b^2\tan^2 x}$; (x) $2x^{\ln x - 1}\ln x$. **8.** (a) $\frac{\sin(\ln x)-\cos(\ln x)}{x^2}$; (b) $x^x\left[(1+\ln x)^2 + \frac{1}{x}\right]$. **10.** $-\frac{1}{(x+1)^2}$. **11.** $\left[\frac{8x+2}{4x^2+2x} - 10\ln(4x^2+2x)\sin 5x\cos 5x\right]e^{\cos^2 5x}$. **12.** $Y'(n) = \frac{1}{1+0.4n}$; $Y'(5) = \frac{1}{3}$ and $Y'(20) = \frac{1}{9}$ tonne per bag: each extra bag helps less. **13.** $T'(t) = -5.04e^{-0.08t}$; -5.04°C/min at $t = 0$ and about -1.02°C/min at $t = 20$. **14.** $v = 3(t-3)(t-5)$ and $a = 6(t-4)$; at rest at $t = 3$ and $t = 5$; slowing down on $(0, 3)$ and $(4, 5)$. **15.** 0.105, 0.085 and 0.105 GH¢ per dollar per day.

Chapter 5

1. The point is $(2, 1)$ and $y' = -10$; tangent $y = -10x + 21$, normal $x - 10y + 8 = 0$. **2.** Tangent $y = -x + e$; normal $y = x - e$. **3.** $k = \sqrt{2}\,e^{\pi/4}$, at $\left(\frac{\pi}{4}, e^{\pi/4}\right)$. **4.** $b = \frac{49\sqrt{3}}{6}$. **5.** $(-2, 5)$, $(2, -15)$ and $\left(\frac{11}{2}, \frac{1105}{8}\right)$. **6.** $(-1, -8)$. **7.** $c = \frac{16}{81}$. **9.** $x + 2y - 18 = 0$. **10.** $\frac{7.5}{4\pi} \approx 0.597$ m/min. **11.** About 0.100 m/min. **12.** 2.4 m/s. **13.** 64 m/s. **14.** Local maximum $y(0) = -1$; local minimum $y(\ln 4) \approx -2.298$; no absolute extrema. **15.** (a) 0 and $\frac{3}{2}$; (b) increasing on $\left(-\infty, \frac{3}{2}\right]$, decreasing on $\left[\frac{3}{2}, \infty\right)$; (c) local maximum $f\left(\frac{3}{2}\right) \approx 3.19$. **16.** $x = 1$ gives a minimum (a cusp); no inflection point. **17.** (a) min 0, max 2π; (b) max $f(-5) = -\frac{5}{441}$, min $f(-3) = -\frac{1}{25}$; (c) max $\frac{5}{4}$ (at $\frac{\pi}{6}$ and $\frac{5\pi}{6}$), min 1. **18.** Local max $(1, 6)$, local min $(3, 2)$, inflection point $(2, 4)$. **19.** $x = 0$. **20.** (a) $\frac{1}{3}$; (b) 0; (c) 0. **21.** (a) $V = x(20-2x)(12-2x)$; (b) $x \approx 2.43$ cm, giving a box $15.15 \times 7.15 \times 2.43$ cm with $V \approx 263$ cm^3. **22.** (b) $-\frac{1}{6}$ mL per hour. **23.** (b) Width ≈ 94.1 cm, height ≈ 47.0 cm; (c) width 84 cm, height ≈ 60.0 cm. **25.** $h = \frac{28}{3}$ cm, $r = \frac{14\sqrt{2}}{3}$ cm. **26.** 17.3 m by 34.6 m (long side parallel to the wall); 69.3 m of fencing. **27.** 7.5 hours. **28.** Land $\sqrt{7.2} \approx 2.68$ km from P. **29.** $m = \frac{5}{3} \approx 1.67$.

Chapter 6

1. (a) $x^4 - x^3 + 2x + C$; (b) $\frac{x^2}{2} + 2x + \ln|x| + C$; (c) $\frac{x^3}{3} - 2x + 5\ln|x| + C$; (d) $3\tan x - 2e^x + C$; (e) $\sec x + C$. **2.** $x^2\sin x + C$. **3.** $\frac{1}{5}\tan 5x + C$. **4.** $10x - \frac{3}{2}x^2 + C$. **5.** (a) $\frac{(x^2+4)^{11}}{22}$; (b) $-\sqrt{1-x^2}$; (c) $-\ln|\cos x|$; (d) $\frac{(\ln x)^4}{4}$; (e) $\ln(1+e^x)$; (f) $\frac{\sin^5 x}{5}$; (g) $2\sin\sqrt{x}$; (h) $\frac{e^{x^3}}{3}$ (all $+C$). **6.** (a) 0; (b) 1; (c) 1; (d) $\frac{e-1}{2}$; (e) 2. **7.** $\sqrt{1+x^3}$; $2x\cos(x^2)$. **9.** (a) $\frac{(2x-1)e^{2x}}{4} + C$; (b) $\sin x - x\cos x + C$; (c) $\frac{x^3\ln x}{3} - \frac{x^3}{9} + C$; (d) $1 - \frac{2}{e}$. **10.** (a) 0.742984; (b) 0.746855. **11.** $v(t) = 6t - 0.6t^2$ m/s; $s(t) = 3t^2 - 0.2t^3$ m. **12.** $C(q) = 500 + 12q - 0.01q^2$.

Chapter 7

1. (a) $\frac{9}{2}$; (b) 2; (c) $\frac{1}{2}$. **2.** (a) 16; (b) $\frac{2}{\pi}$. **3.** (a) 8π; (b) $\frac{\pi(e^2-1)}{2}$; (c) $\frac{2\pi}{15}$. **5.** (a) $\frac{8}{27}\left(10\sqrt{10}-1\right) \approx 9.07$; (b) $\frac{17}{12}$. **6.** (a) 4 m; (b) 12 m. **7.** 2400 million m^3. **8.** GH¢ 1600. **9.** (a) $324\,\mathrm{m}^2$; (b) about $333\,\mathrm{m}^2$. **10.** About 4360 transactions. **11.** $256\pi \approx 804\,\mathrm{cm}^3$, about 0.80 litres. **12.** About 0.323.

Chapter 8

1. (a) $2x - 3y + 14 = 0$; (b) $(x-1)^2 + (y+2)^2 = 4$; (c) $y = x^2$, $x > 0$; (d) $x^2 - y^2 = 1$. **2.** $\frac{\mathrm{d}y}{\mathrm{d}x} = \frac{3(t^2-1)}{2t}$, $\frac{\mathrm{d}^2y}{\mathrm{d}x^2} = \frac{3(t^2+1)}{4t^3}$; horizontal at $(2, \mp 2)$ (where $t = \pm 1$), vertical at $(1, 0)$ ($t = 0$). **3.** $y = -\frac{3}{2}x + 3\sqrt{2}$. **4.** $\frac{14}{3}$. **5.** (a) $(1, \sqrt{3})$; (b) $(0, -5)$; (c) $\left(-\frac{3}{\sqrt{2}}, -\frac{3}{\sqrt{2}}\right)$. **6.** (a) $\left(3\sqrt{2}, \frac{\pi}{4}\right)$; (b) $\left(2, \frac{2\pi}{3}\right)$; (c) $\left(4, \frac{3\pi}{2}\right)$; (d) $\left(4, \frac{4\pi}{3}\right)$. **7.** (a) $r = 3$; (b) $\theta = \frac{\pi}{4}$; (c) $r = 6\cos\theta$; (d) $r^2 = \frac{2}{\sin 2\theta}$. **8.** (a) circle $x^2 + y^2 = 25$; (b) circle $(x-2)^2 + y^2 = 4$; (c) line $y = 3$; (d) parabola $y^2 = 4x + 4$. **9.** (a) 4π; (b) $\frac{\pi}{12}$; (c) 6π. **10.** $\sqrt{3}$. **11.** (a) 2.34 s; (b) 38.4 m; (c) 6.71 m. **12.** About 21.6°. **13.** About 16.1 km. **14.** $18\pi \approx 56.5\,\mathrm{m}^2$.

Chapter 9

1. (a) vertex $(0, 0)$, focus $(3, 0)$, directrix $x = -3$; (b) focus $\left(0, -\frac{3}{2}\right)$, directrix $y = \frac{3}{2}$; (c) vertex $(3, 2)$, focus $(5, 2)$, directrix $x = 1$; (d) vertex $(-3, 2)$, focus $(-3, 1)$, directrix $y = 3$. **2.** (a) $x^2 = 12y$; (b) $(x-2)^2 = 8(y+1)$; (c) $x^2 - 2xy + y^2 - 4x - 4y + 4 = 0$. **3.** (a) vertices $(\pm 5, 0)$, foci $(\pm 4, 0)$, $e = \frac{4}{5}$; (b) vertices $(0, \pm 4)$, foci $(0, \pm 2\sqrt{3})$, $e = \frac{\sqrt{3}}{2}$; (c) centre $(1, -2)$, vertices $(1, 1)$ and $(1, -5)$, foci $(1, -2 \pm \sqrt{5})$, $e = \frac{\sqrt{5}}{3}$. **4.** (a) $\frac{x^2}{25} + \frac{y^2}{9} = 1$; (b) $\frac{(x-1)^2}{16} + \frac{(y-2)^2}{25} = 1$; (c) $\frac{x^2}{32} + \frac{y^2}{36} = 1$. **5.** (a) vertices $(\pm 4, 0)$, foci $(\pm 5, 0)$, asymptotes $y = \pm\frac{3}{4}x$; (b) vertices $(0, \pm 2)$, foci $(0, \pm\sqrt{5})$, asymptotes $y = \pm 2x$; (c) centre $(3, -2)$, vertices $(2, -2)$ and $(4, -2)$, foci $(3 \pm \sqrt{5}, -2)$, asymptotes $y + 2 = \pm 2(x - 3)$. **6.** (a) $\frac{x^2}{9} - \frac{y^2}{16} = 1$; (b) $\frac{y^2}{36} - \frac{x^2}{64} = 1$. **7.** (a) circle, centre $(3, -1)$; (b) hyperbola, centre $(1, 2)$; (c) parabola, vertex $(2, 1)$; (d) ellipse, centre $(-1, 2)$. **9.** 28.1 cm. **10.** The centre of the canoe must stay within about 10.7 m of the centre of the stream. **11.** $e \approx 0.745$; foci 89.4 m apart; one lap ≈ 317 m. **12.** $\frac{x^2}{900} - \frac{y^2}{1600} = 1$, the branch with $x < 0$ (nearer to A).

References and Further Reading

Calculus and analytic geometry

[1] S. E. Moore, *Lecture Notes on Analytic Geometry and Calculus*, 3rd edition, Department of Mathematics, University of Cape Coast, 2024.

[2] J. Stewart, D. Clegg and S. Watson, *Calculus: Early Transcendentals*, 9th edition, Cengage Learning, 2020.

[3] J. Hass, C. Heil and M. D. Weir, *Thomas' Calculus*, 14th edition, Pearson, 2018.

[4] H. Anton, I. Bivens and S. Davis, *Calculus*, 10th edition, Wiley, 2012.

[5] G. Strang and E. Herman, *Calculus, Volume 1*, OpenStax, Rice University, 2016. Free online at `https://openstax.org`.

Python and scientific computing

[6] Q. Kong, T. Siauw and A. Bayen, *Python Programming and Numerical Methods: A Guide for Engineers and Scientists*, Academic Press, 2020.

[7] J. Sundnes, *Introduction to Scientific Programming with Python*, Springer (open access), 2020.

[8] H. P. Langtangen, *A Primer on Scientific Programming with Python*, 5th edition, Springer, 2016.

[9] J. VanderPlas, *Python Data Science Handbook*, O'Reilly, 2016.

[10] C. R. Harris et al., "Array programming with NumPy", *Nature* 585 (2020), 357–362.

[11] A. Meurer et al., "SymPy: symbolic computing in Python", *PeerJ Computer Science* 3 (2017), e103.

[12] P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python", *Nature Methods* 17 (2020), 261–272.

[13] J. D. Hunter, "Matplotlib: a 2D graphics environment", *Computing in Science & Engineering* 9 (2007), 90–95.

History of mathematics, with an African perspective

[14] G. G. Joseph, *The Crest of the Peacock: Non-European Roots of Mathematics*, 3rd edition, Princeton University Press, 2011.

[15] C. Zaslavsky, *Africa Counts: Number and Pattern in African Cultures*, 3rd edition, Lawrence Hill Books, 1999.

[16] P. Gerdes, *Geometry from Africa: Mathematical and Educational Explorations*, Mathematical Association of America, 1999.

[17] R. J. Gillings, *Mathematics in the Time of the Pharaohs*, MIT Press, 1972 (Dover reprint, 1982).

[18] V. J. Katz, *A History of Mathematics: An Introduction*, 3rd edition, Addison-Wesley, 2009.

[19] C. B. Boyer, *The History of the Calculus and its Conceptual Development*, Dover, 1959.

[20] C. H. Edwards, *The Historical Development of the Calculus*, Springer, 1979.

Data sources

[21] Ghana Statistical Service, *Ghana 2021 Population and Housing Census: General Report*, Accra, 2021–2022 (and the 2010 Census report).

Except where a source is given, the numerical data in the “In Context” boxes (fares, trip times, tariffs, rainfall, survey offsets and so on) are realistic but *invented for teaching*, and must not be quoted as real figures.